

回転機構のレーザー受光と楽器間無線 通信による同期・輪唱システム

情報工学科 2 年

24G1059

佐藤 涼菜

—— チーム 40 ——

宇井 祐真 (24G1021) 梅野 大誠 (24G1026)

山口 侑希 (24G1136) 薄井 悠希 (25G1020)

菊池 侑人 (25G1041)

2026 年 7 月 16 日

1 はじめに

1.1 社会的背景・課題・本稿の目的

近年、動画配信サイトやストリーミングサービスの普及によって、誰もが容易に音楽コンテンツを視聴できる環境が整っている。特に日本では、ゲームやアニメなどのサブカルチャーを起点とした音楽コンテンツが数多く生み出されており、独自の文化として定着している。一方で、このようなデジタル配信の消費形態は、実際に会場へ足を運んで演奏を体感するライブイベントやコンサートとの相乗効果を生み、市場規模の拡大にも寄与している [1][2]。たとえば、博報堂の谷口による分析では、ストリーミングサービスの利用者がサービス内で新たに好みのアーティストを複数発掘し、そのアーティストたちを一度に鑑賞できるライブやフェスへ参加するという行動の流れが形成されつつあり、さらにライブでの体験を通じて別のアーティストと出会うことで、再びストリーミングでの視聴に戻るという音楽消費サイクルが生まれていると指摘されている [4]。すなわち、デジタル配信とライブイベントは互いの顧客を送客し合う関係にあり、この循環が音楽市場全体の拡大を後押ししていると考えられる。実際に、一般社団法人コンサートプロモーターズ協会が実施したライブ市場調査によれば、2025 年におけるライブ市場の年間売上高は 6443 億円、入場者数は 5999 万人にのぼり、デジタル配信の普及以降もライブ市場は堅調に拡大を続けている [3]。図 1 に示すように、国内のライブ・コンサート市場における年間売上額は 2010 年の 1280 億円から 2019 年の 3665 億円まで、10 年間でおよそ 2.9 倍に拡大している。2020 年には新型コロナウイルス感染症の影響により、会場に人を集めること自体が困難となった結果、年間売上額は 779 億円まで急落した。しかし、行動制限の緩和とともに市場は急速に回復し、2022 年には 3984 億円、2023 年には 5140 億円、そして 2025 年には過去最高となる 6443 億円を記録している。この推移から、ライブ会場に足を運ぶという体験に対する需要は、一時的な外的要因によって落ち込むことはあっても、長期的には一貫して拡大する傾向にあると言える。

デジタル配信が普及していく中で、なぜデジタル配信では代替できない価値がライブ会場に存在するのだろうか。その理由の一つとして、ライブ演出に用いられる音響や照明、ライブ参加者自身が感じる振動といった複数の物理的刺激が挙げられる。これらの刺激は、観客一人ひとりに個別に届けられるのではなく、同じ空間にいる観客全員に同時に共有されるという性質を持つ。その結果、観客同士が同じ体験を共有しているという感覚、すなわち空間全体としての一体感が形成される。換言すれば、ライブ会場における没入感とは、ユーザが音楽を聴くだけにとどまらず、物理的、視覚的な技術と同じ空間で同時に受け取ることによって生まれる体験だと考えられる。この仮説は、ライブ演奏とデジタル配信を直接比較した実証研究によっても裏付けられている。Kreuzer ら (2025) は、同一のクラシック音楽コンサートを、会場で聴取する条件と映像配信で視聴する条件とで比較する実験を行った。その結果、没入感や社会的なつながりの感覚を含む複数の評価軸において、ライブ条件の方が配信条件よりも有意に高い評価を得ることを示した [5]。すなわち、同じ演奏内容・同じ音質であっても、同じ空間を他の観客と共有しているという感覚そのものが、没

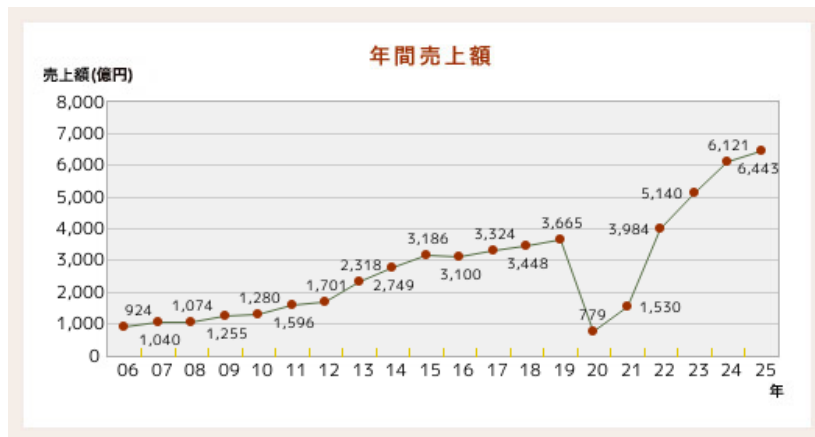


図1 国内ライブ・コンサート市場における年間売上額の推移（一般社団法人コンサートプロモーターズ協会「基礎調査推移表」より）

入感を左右する独立した要因であり、これは録画・配信技術の高度化だけでは補えないことを意味する。したがって、音そのものの再現性だけでなく、ライブ体験に固有の空間的フィードバックを組み込むことで没入感の高い演奏体験の実現に有効であると考ええる。

また、Natalia Cooper ら（2018）は、仮想現実（VR）環境における体験の再現度を高めるため、視覚情報のみでは表現が難しい感覚を、音や触覚など別の感覚を用いた代替フィードバックとして提示する実験を行った。その結果、視覚刺激のみを提示した場合と比較して、音や触覚といった複数の異なる刺激を組み合わせると提示した場合の方が、利用者の没入感・関与感が有意に高まることを明らかにした [6]。つまり、複数の感覚情報を同時に提示する多感覚的フィードバックは、ユーザの体験における没入感の向上に有効であると考ええる。また、Human-Computer Interaction (HCI) 分野の知見も同様の傾向を支持している。Martin ら（2024）は、ライブコンサート環境を対象とした研究において、観客が演奏を受動的に聴取するだけでなく、能動的に体験へ参加することが、会場全体の活気や観客の感情の高まりに寄与すると結論づけた [7]。よって、ユーザ自身が能動的に関与できる仕組みもまた、システムの没入感を高める要因として重要であると考ええる。

一方、音楽を自動で演奏する自動演奏技術の分野では、デジタル技術の発展により高精度な同期や楽曲再現が可能になっている。たとえば、Yamaha が提供する Disklavier ENSPIRE シリーズは、光学センサとサーボモータを用いて演奏情報を高精度に再現するとともに、鍵盤が実際に動作する様子をユーザへ視覚的にフィードバックを行う機能を備えている [8]。しかし、この視覚的フィードバックは、ピアノの鍵盤という限られた範囲でのみ提示されるものであり、その場に居合わせたユーザ1人のみが確認できる情報にとどまる。したがって、複数人が同じ空間にいる状況を想定すると、自動演奏技術によって提示される視覚的フィードバックは、空間全体には行き渡らないという限界を抱えていると言える。

以上を踏まえると、現状の自動演奏技術は、演奏そのものの精度は高い一方で、前述した「多感覚的フィードバック」と「空間全体への共有」という、没入感を高めるうえで重要な要素を十分に満たせていないという課題が生じる。特に、Kreuzer らの知見が示すとおり、同じ場を共有して

いるという空間的なフィードバックは、音質や演奏精度をいくら高めても代替できない、ライブ体験に固有の要素である。したがって、デジタル配信や高精度な自動演奏技術がどれほど発展しても、この空間的フィードバックを欠いたままでは、ライブ会場に匹敵する高い没入感を得ることはできないと結論づけられる。そこで本稿では、自動演奏技術に、ライブ体験に共通するこの空間的フィードバックを付加した楽器間通信システム「回転機構のレーザー受光と楽器間無線通信による同期・輪唱システム」を提案する。本システムは、ユーザの動作を検知する指揮デバイス、指揮デバイスからの情報に応じ回転する観覧車型の中継機デバイス、そして中継機からのレーザー光を受光し演奏を行う楽器デバイスによって構成される。指揮デバイスによるユーザの能動的な動作、観覧車型デバイスによる空間全体への視覚的なフィードバックの共有、そして高精度な音色の演奏による聴覚的な刺激を組み合わせることで、従来の自動演奏技術では実現が難しかった、会場全体を巻き込む没入感の高い演奏体験の提供を目指す。

本システムの実装にあたっては、レーザー受光間隔からの BPM 推定と楽器デバイス間の UDP 通信による同期補正、加速度センサの振り検知によるリアルタイムな BPM・音量制御、および加算合成に基づく 6 種の楽器音の生成といった技術的工夫を行った。評価の結果として、結合テストでは、振り動作による BPM・音量変更の反映成功率 100%、ボタン押下の成功率 98.8%を確認し、複数の楽器デバイスをハース効果の許容限界である 50 ms 未満の演奏ズレで協調動作させた。また、参加者共通アンケートでは「演出の華やかさ」および「楽しさ・高揚感」が全参加チームの平均を統計的に有意に上回る評価を得た。一方、同期開始条件を厳しくした場合に同期確立までの時間が長くなること、および没入感・一体感については統計的な有意差が確認されなかったことから、同期確立の高速化・安定化が今後の課題であるという知見を得た。これらの評価方法と結果の詳細は 4 節および 5 節で述べる。

1.2 類似技術・先行研究

本節では、本システムの独自性を明確にするため、音楽に視覚的なフィードバックを付加する既存技術を整理し、それぞれの特徴と限界を検討する。

第一に、DMX512 信号（以下、DMX 信号と表記する）を用いた照明制御システムが挙げられる。DMX 信号とは、舞台照明の操作卓と調光機との間でデータをやり取りするための通信規格であり、現在では舞台演出やコンサートにおいて、音楽に合わせた光量制御を行う目的で広く利用されている [9]。このシステムは、音楽という聴覚的な刺激に加えて光という視覚的な刺激を提示することで、観客の没入感を高めることができる。しかし、DMX 信号による照明制御は、あらかじめプログラムされた演出を再生する仕組みであるため、観客はその光の変化を一方向的に受け取る、すなわち受動的に認識する立場にとどまる。前節で述べたとおり、能動的な関与が没入感を高める要因の一つであることを踏まえると、観客が演奏そのものに関与できない DMX 信号による演出には限界があると言える。この課題に対し、本提案では指揮デバイスを導入し、利用者が指揮という身体動作を通じて演奏そのものを能動的に制御できるようにすることで、身体的な関与に基づく没入感の向上を狙う。

第二に、物理的な操作によって音楽を制御するタンジブル・インターフェースの例として、

ReacTable が挙げられる。ReacTable は、卓型のディスプレイ上に複数の物理モジュールを配置し、その配置や動きに応じて音を合成するシンセサイザの一種であり、利用者は身体的な物理操作を通じて演奏を制御するとともに、ディスプレイに表示される視覚的な情報によって演奏状態のフィードバックを受け取ることができる [10]。しかし、この視覚的フィードバックはディスプレイという限られた表示領域内で完結しており、操作を行う利用者本人以外の第三者や、会場全体へフィードバックを共有する仕組みとしては設計されていない。そこで本提案では、指揮デバイスに加えて観覧車型の中継機デバイスを導入する。観覧車型デバイスが演奏情報に応じて回転する様子を、利用者本人だけでなく周囲にいる第三者も視認できるようにすることで、演奏状態の把握や共有を空間全体に広げ、多感覚的なフィードバックを実現することを狙う。

以上のように、DMX 信号による照明制御は観客の能動的な関与を欠き、ReacTable のような既存のタンジブル・インターフェースは視覚的フィードバックが個人の操作範囲内にとどまるという、それぞれ異なる限界を抱えている。本提案は、指揮デバイスによる能動的な演奏制御という既存研究の知見と、観覧車型デバイスによる空間共有性という要素を組み合わせることで、これら二つの課題を同時に解決し、空間全体への多感覚的なフィードバックを通じて、利用者の没入感および関与感を高めることを目指す。

2 作成したシステムの概要

本節では、作成したシステムの概要について説明する。本システムは、ユーザの能動的な動作と連動した観覧車型中継機デバイスの回転およびレーザの発光を通じて、演奏空間全体に多感覚なフィードバックを与えることで、従来の自動演奏技術における演奏インターフェースでは得られない高い没入感をユーザに提供することを目的として設計されている。まず、本システムの全体の構成図を図 2 に示す。本システムは、指揮デバイス、観覧車型の中継機デバイス、楽器デバイスという 3 つのデバイスから構成される。ユーザが指揮デバイスに取り付けられたボタンを押下すると、同期開始情報が無線通信 (UDP) によって中継機デバイスおよび楽器デバイスへ送信され、中継機デバイスの回転が開始する。中継機デバイスは常にレーザー光を照射しており、回転するレーザー光を楽器デバイスの CdS セルが受光することで楽器間で同期通信が開始される。楽器間の同期通信では、まず各楽器デバイスがレーザー光の受光間隔から BPM を推定し、その推定値が安定するまで待機する。推定が安定すると、ドラムを担当する楽器デバイスをマスター、ピアノ・ストリングス・フルートを担当する楽器デバイスをスレーブとした UDP 通信により、スレーブがマスターへ参加を通知したうえで両者の発音タイミングのずれを相互に送受信し合い、発音タイミングを補正する。全ての楽器デバイス間でこのずれが閾値内に収まり同期が確立すると、各楽器デバイスは自身が保持する楽譜データに基づき、発音タイミングごとに USB シリアル通信で音程や音量等の発音情報を PC へ送信する。PC 上で動作する Processing は、受信した発音情報に従って実際の演奏音を生成・出力することで、演奏が開始される。なお、この同期通信および発音処理の詳細については 2.3.3 項（楽器デバイスの内部設計）で説明する。また、演奏開始後にて楽曲の BPM を変更したい場合、ユーザがスライド式可変抵抗器を操作したうえで指揮デバイスを振ると、加速度セン

サが振り動作を検知し、中継機デバイスへ UDP 通信がなされ回転速度が変化する。そして、この回転速度に伴い変化した受光間隔によって各楽器デバイスが USB シリアル通信を行うことで、接続された PC から演奏されている楽曲の BPM が変更される。最後に、演奏開始後に楽曲の音量を変更したい場合、ユーザがスライド式可変抵抗器を操作したうえで指揮デバイスを振ると、加速度センサが振り動作を検知し、直接楽器デバイスへ UDP 通信を行い楽曲の音量を変更する。ここからは各デバイスの概要について説明する。

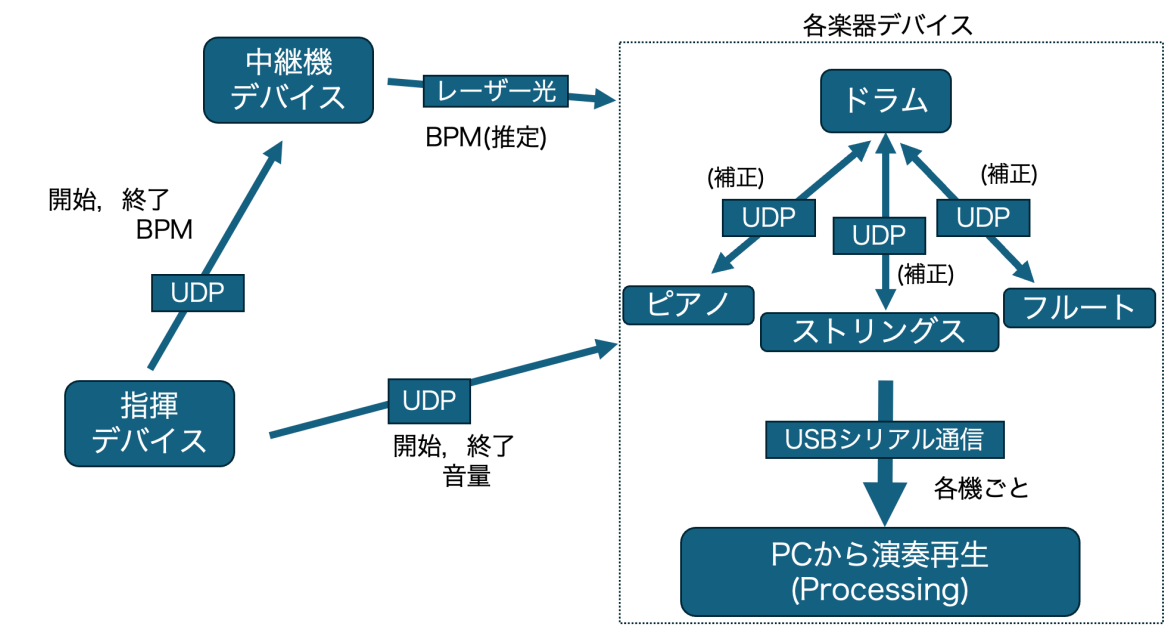


図 2 全体の構成図

2.1 指揮デバイス

2.1.1 概要

まず、指揮デバイスについて説明する。指揮デバイスのシステム構成図を図4に示す。本デバイスは、演奏全体を統括するデバイスであり、主に以下の3つの機能を有する。

第一の機能は、演奏開始トリガーの送信である。本機能の設計について図5に示す。ユーザが指揮デバイスに搭載されたタクトスイッチを押下すると、中継機デバイスと楽器デバイスへ演奏開始トリガーを送信する。さらに、中継機デバイスの回転が開始される。

第二の機能は、楽曲の BPM 変更である。本機能の設計について図6に示す。ユーザがスライド式可変抵抗器を操作した後、デバイスを振ると新しい設定 BPM 情報が中継機デバイスへ送信される。これにより、中継機デバイスのモータ回転速度がこの値に追従して変化する。そして、その回

転に伴うレーザーの通過光を介して各楽器デバイスに情報が伝達され、BPM の変更される。また、楽曲の BPM は 5 段階に分けて変更できる。

第三の機能は、楽曲の音量変更である。本機能の設計について図 7 に示す。ユーザがスライド式可変抵抗器を操作した後、デバイスを振ると音量情報が各楽器デバイスへマルチキャスト送信されることで、音量が変更される。また、BPM と同様に音量も 5 段階に分けて変更できる。以上の機能を含む、演奏開始から終了までのシステム全体の処理の流れをフローチャートとして図 3 に示す。指揮デバイスのボタン押下を起点として、中継機デバイスの回転とレーザー光を介した楽器デバイスの同期が確立した後に演奏が開始され、演奏中は振り動作による BPM・音量変更を受け付け、終了ボタンの押下によって全デバイスが停止する。ここからは、外部設計と内部設計について説明をする。

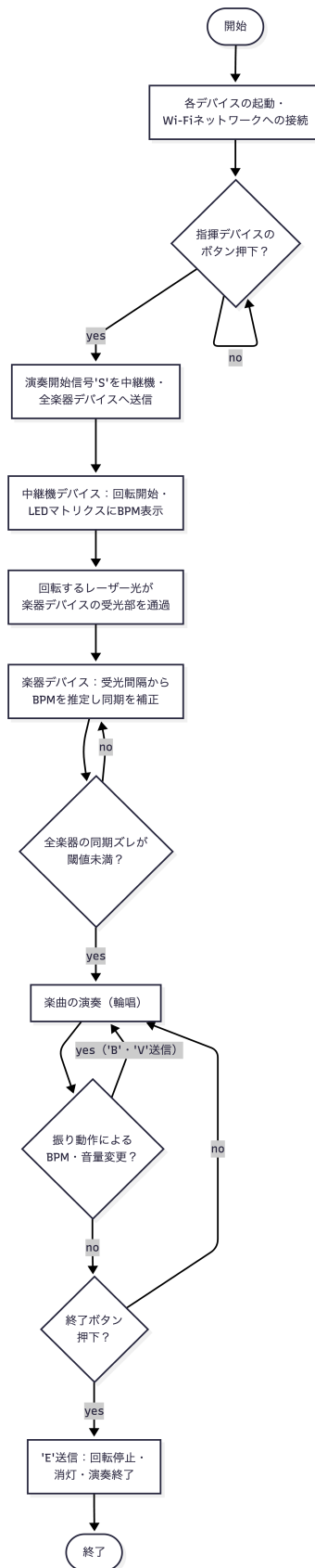


図3 システム全体の処理フローチャート

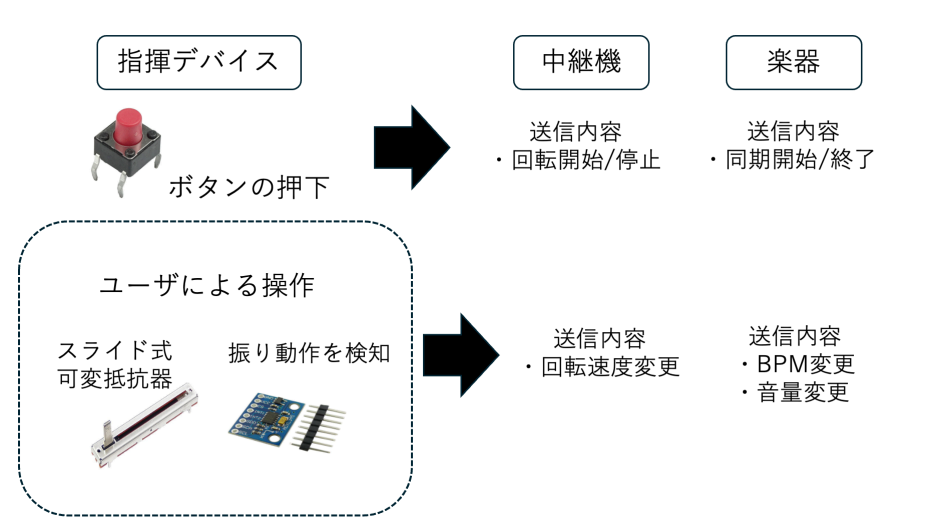


図 4 指揮デバイスのシステム構成図

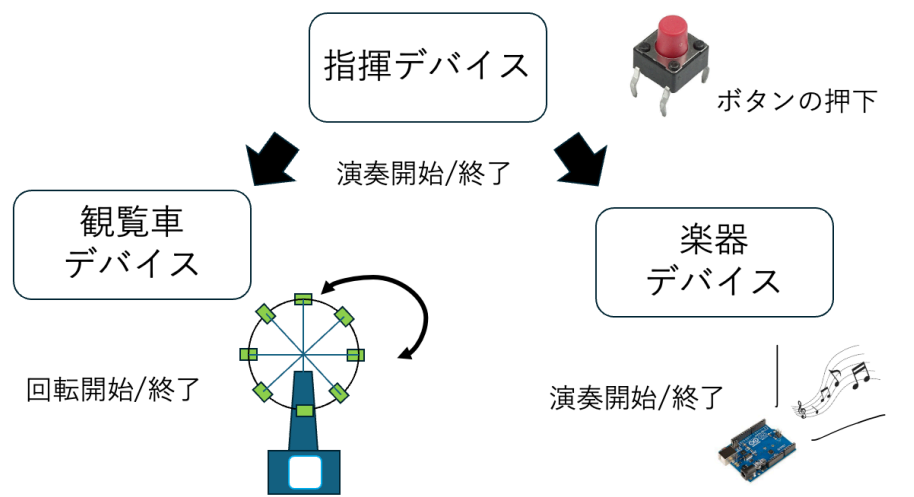


図 5 演奏開始トリガー送信機能設計

2.1.2 外部設計

本節では指揮デバイスの外部設計について説明する。まず、本デバイスを構成する部品について表 1 に示す。デジタル 3 軸加速度センサは指揮デバイスの動きの検知に使用し、スライド式可変抵抗器は BPM および音量変更に使用される。当初はデバイスの電力供給源として 9V のアルカリ電池を外部電源として使用することを検討していたが、電池なしでも正常動作が確認できたため、軽量化のため PC との USB 接続から電力を供給するように変更した。Arduino 内蔵の電圧レギュレータにより 5V および 3.3V を生成し、各モジュールへ安定した電力を供給する。また、指揮デバイスの回路図を図 8 に示す。加速度センサには GY-291 (ADXL345) を用いており、I2C 通信により

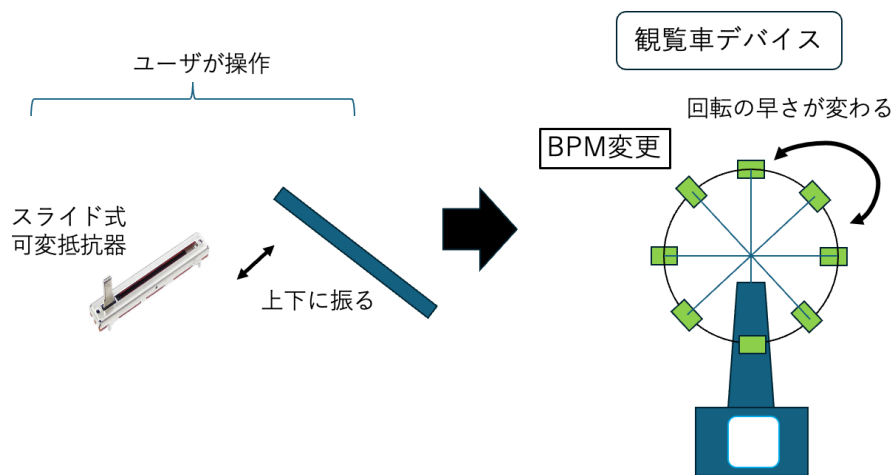


図 6 BPM 変更機能設計

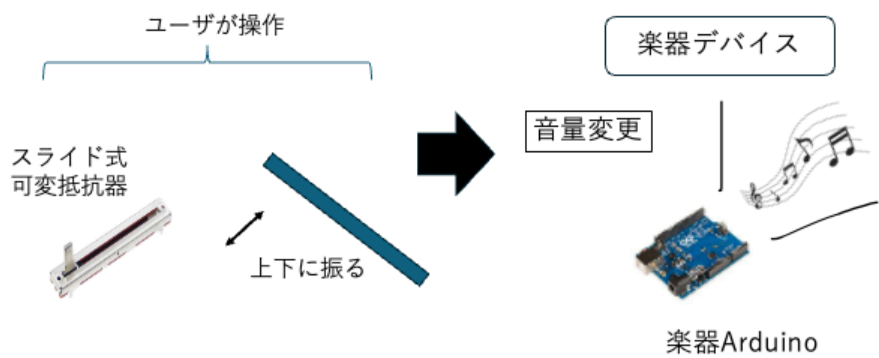


図 7 音量変更機能設計

Arduino と接続する。CS ピンを Arduino の 3.3 V に接続することで I2C モードに設定し、さらに SDO ピンを GND に接続することで I2C アドレスを 0x53 に固定している。また、SDA ピンおよび SCL ピンはそれぞれデータ通信線およびクロック信号線として Arduino の A4、A5 ピンに接続している。加えて、加速度が内部設定した閾値を超過した場合には、INT1 ピンから Arduino の D2 ピンへ割り込み信号が出力される。これにより、指揮デバイスの振り動作を検知し、モータ制御用 Arduino との UDP 通信を開始する。

さらに、演奏開始および終了信号の送信に用いるボタン入力判定回路を実装する。ボタンの誤判定を防止するため 10 k Ω のプルアップ抵抗を用いた回路を構成する。ボタンが押されていない状態では、入力ピン D3 はプルアップ抵抗を介して 5V に接続されるため HIGH 状態となる。一方、ボタン押下時には回路が GND に短絡され、入力電位が LOW に遷移する。指揮デバイスのプログラムでは D3 ピンの状態変化を常時監視することで、演奏開始および終了操作を判定する。

表 1 指揮デバイスの構成部品

機材	品番	個数 [個]	素子番号
Arduino UNO R4 WiFi	-	1	A ₁
スライド式可変抵抗器 10 k Ω	SL4515G-B103L15CM	2	R ₁ , R ₂
デジタル 3 軸加速度センサ	GY-291	1	U ₁
タクトスイッチ	DTS-63	1	SW ₁
抵抗器 10 k Ω	-	1	R ₃
ジャンパーワイヤー	-	適宜	-

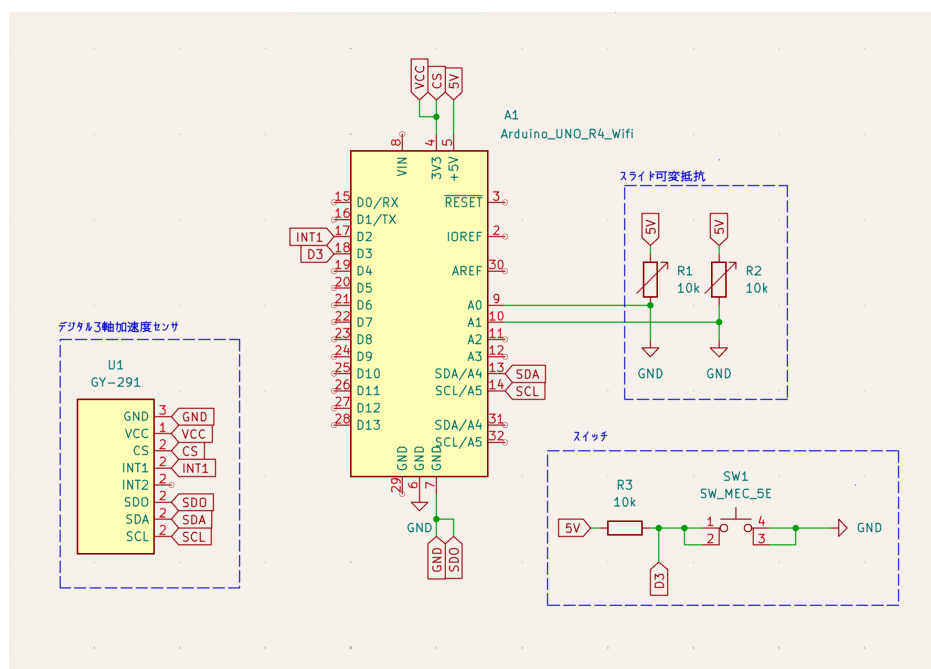


図 8 指揮デバイスの回路図

実際に作成した指揮デバイスの内部および完成状態は、3.2.1 項の図 30 および図 31 に示す。実際の指揮棒のようにユーザが片手で振って操作できるよう、細長い筒状の外装を採用した。また、内部にはブレッドボードに加速度センサ、スライド式可変抵抗器、タクトスイッチを設置する。デバイスのユーザビリティを向上させるため、ユーザが操作するスライダー部分やボタンは外部へ露出させる構造を採用し、さらにデバイスの先端部に加速度センサを設置することで操作時の重心を安定させる。

2.1.3 内部設計

本節では、指揮デバイスの内部設計について説明する。まず、指揮デバイス全体の内部動作を明確にするため、シーケンス図を図 9 に示す。ユーザが指揮デバイスを上下に振ると加速度センサが動作を検知し Arduino に送信される。この時、値が閾値を超過した場合、中継機デバイスへ演奏情報が UDP 通信により送信され、中継機デバイスはこの情報を基に回転する。

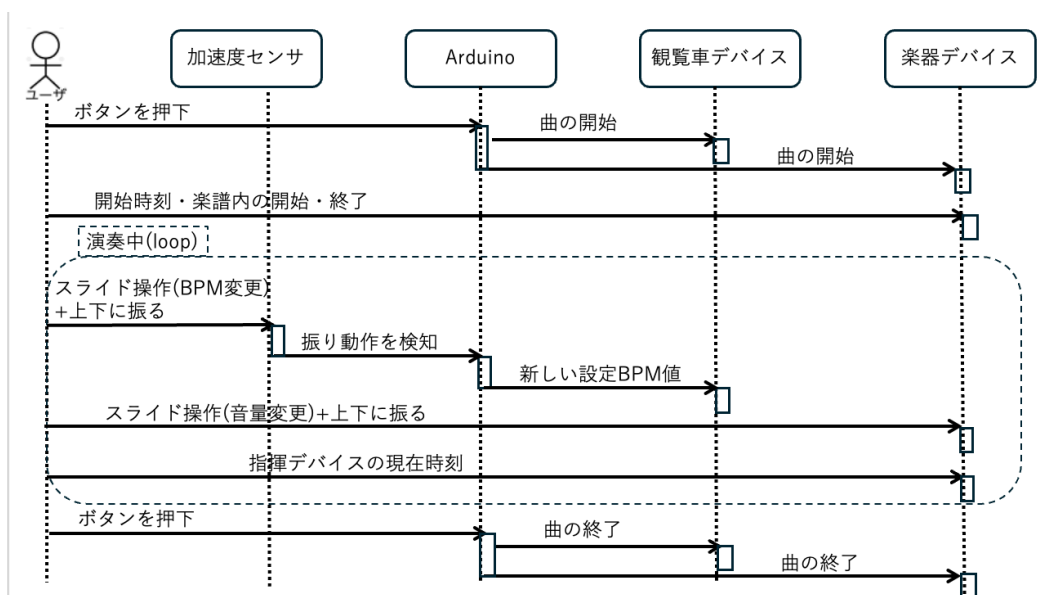


図 9 指揮デバイスのシーケンス図

次に、指揮デバイスの制御プログラムのファイル構成を示す。表 2 の通り、各ファイルは機能ごとにクラス化されている。

表 2 指揮デバイスのファイル構成

ファイル名	概要
Controller_simultaneous.ino	setup/loop による全体制御, ボタン入力 of 判定
AccManager.h / AccManager.cpp	加速度センサ of 初期化, 振り動作 of 検知, I2C バス復旧処理
PotManager.h / PotManager.cpp	スライド式可変抵抗器 2 系統 of 移動平均処理および BPM・音量段階 of 判定
SyncManager.h / SyncManager.cpp	Wi-Fi/UDP 通信 of 初期化, 各デバイスへの冗長送信処理

次に、本デバイスの処理内容についてボタン入力判定、加速度センサの制御、可変抵抗器を用いた BPM・音量読み取り機能、各デバイスへの無線送信機能の順に説明する。

まず、指揮デバイスにおけるボタン入力判定処理について説明する。本デバイスではボタンの誤判定を防ぐため、プルアップ抵抗を用いた回路を構成しており、ボタンが押されていない状態ではデジタルピンに 5V が印加され、押下時には回路が GND に短絡されることでピンの電位が LOW へ遷移する。指揮デバイスのプログラムでは、このデジタルピンの電圧状態の変化を常時監視することでボタンの押下を判定する。さらに、チャタリングによる誤判定や冗長パケットによる誤作動を防止するため、押下検知後に 300ms の待機処理を設けるとともに、演奏状態管理フラグを用いたトグル制御を導入している。これにより、同一の物理ボタンのみで演奏の開始と終了を交互に切り替えることが可能となる。押下を検知した際は、停止中であれば現在設定されている BPM 段階を付与した演奏開始の合図を、演奏中であれば演奏終了の合図を、後述する無線送信機能を介して中継機デバイスおよび全楽器デバイスへ送信する。

次に、指揮デバイスにおける加速度センサの制御処理について説明する。指揮デバイスは、加速度センサを用いてユーザが指揮デバイスを振ったことを検知し、演奏情報の更新を行う。センサ値の急激な変化を読み取ることで動作を判定し、また誤検知防止機能を設けることで安定した動作を実現している。処理の流れは、差分算出と判定、多重検知の防止、状態の反転、事後処理の 4 段階に分類される。まず差分算出と判定では、一定周期 (20ms) ごとに 3 軸の測定値から合成加速度を算出し、前回値との差の絶対値が検知閾値を超過したかを判定する。次に多重検知の防止では、一度振る動作を検知してから一定時間 (750ms) が経過するまで新たな検知を無効とすることで、一連の振る動作が複数回の操作として誤検知されることを防ぐ。状態の反転では、振る動作を検知するたびに演奏状態を示すフラグを反転させる。最後に事後処理として、今回の測定値を次の比較に用いる前回値として保存する。また、振る動作の確定時には、可変抵抗器側で設定値の変更が検知されている場合に限り、変更後の BPM・音量の段階番号を無線送信する。このため、実際に無線送信が発生するのは、可変抵抗器を操作したうえでデバイスを振った場合に限られる。さらに、センサとの I2C 通信に異常を検知した場合には、バスの初期化とセンサの再設定による自動復旧を行い、復旧が完了するまでは直前の測定値を代用することで誤検知を防いでいる。

次に、指揮デバイスにおけるスライド式可変抵抗器を用いた BPM・音量読み取り機能について説明する。指揮デバイスでは、演奏の要素である BPM と音量を、ユーザが操作できるようにスライド式可変抵抗器を用いて 5 段階で調整する。本プログラムは、アナログ入力値の特性を考慮したデータ処理を行うことで、安定した制御環境を構築している。アルゴリズムは、サンプリングと移動平均化、範囲分割、パラメータの抽出と出力の 3 段階に分けられる。まずサンプリングと移動平均化では、一定周期ごとに BPM 用・音量用の 2 系統のアナログ入力値を読み取り、系統ごとに直近 10 回分の値を保持してその平均を求めることで、アナログ入力に含まれるノイズを平滑化する。次に範囲分割では、得られた移動平均値を 4 つの閾値と比較することで 1~5 の段階番号へ離散化する。閾値の間隔を均等にしていないのは、可変抵抗器の実測特性に合わせて各段階の操作のしやすさを調整するためである。最後にパラメータの抽出と出力では、算出した段階番号が直前の値から変化した場合にのみ、変更があったことを示すフラグを記録する。このフラグは、前述の振る動

作の確定時に参照され、変更後の段階番号が無線送信される。なお、音量用の可変抵抗器は配線の都合上、操作方向とセンサ値の増減が逆になるため、段階番号を反転して扱うことで操作方向と音量の増減を一致させている。

最後に、各デバイスへの無線送信機能について説明する。指揮デバイスは、Wi-Fi 経由の UDP 通信により、演奏開始・演奏終了・BPM 変更・音量変更の 4 種類の指令を各デバイスへ送信する。いずれの指令も、種類を表すヘッダ 1 バイトと段階番号などの値 1 バイトからなる固定長 2 バイトの packet として送信される。送信先は指令の種類に応じて異なり、演奏開始・終了は中継機デバイスと全楽器デバイスへ、BPM 変更は中継機デバイスへ、音量変更は全楽器デバイスへ、それぞれあらかじめ定めた固定 IP アドレス宛てのユニキャストで送信される。また、UDP 通信には packet 消失時の再送機構がないため、本機能では同一の指令を繰り返し送信する冗長化を行っている。具体的には、対象となる全デバイスへ 1 回ずつ送信する処理を 1 周とし、20 ms の間隔を空けて 3 周繰り返す。デバイスごとに連続して複数回送信するのではなく周単位で繰り返す方式としたのは、前者では送信順が後方のデバイスほど 1 回目の指令の到達が遅れ、デバイス間で反映タイミングに差が生じるためである。なお、これらの指令が中継機デバイスおよび楽器デバイス側でどのように受信・反映されるかについては、それぞれ 2.2.3 項および 2.3.3 項で説明する。

2.2 中継機デバイス

2.2.1 概要

次に、中継機デバイスについて説明する。中継機デバイスの構成図を図 10 に示す。本デバイスは、指揮デバイスからの情報を受け取り中継機デバイスの回転を作動させることに加え、現在の BPM を表示するデバイスであり、主に以下の 2 つの機能を有する。

第一の機能は、上記で述べた BPM に合わせて回転し、受光する間隔の変化によって楽曲の BPM を制御する機能である。本デバイスは、レーザー光が常に照射されており、回転に追従するレーザー光を後述する楽器デバイスの受光部である CdS セルによって受光検知することで、受光間隔から BPM を推定し、楽曲の BPM を変更できるようにする。

第二の機能は、Arduino Uno R4 WiFi に付属している LEDMatrix に BPM を表示する機能である。ここに指揮デバイスから受け取った BPM を表示し、視覚的に現在流れている楽曲の BPM を理解できるようにする。ここからは、外部設計と内部設計について説明する。

2.2.2 外部設計

本節では中継機デバイスの外部設計について説明する。まず、本デバイスを構成する部分について表 3 に示す。この内、個別で購入する必要があるものはハイパワーギヤーボックス、ブレッドボード用 DC ジャック DIP 化キット、ボタン電池、ボタン電池ホルダー、QI ケーブル、有極コンデンサである。このうち、ルーターは、指揮デバイス・中継機デバイス・各楽器デバイスに搭載された Arduino Uno R4 WiFi が共通して接続するローカルな Wi-Fi ネットワークを構築するために使用する。各デバイスはこのネットワークに参加したうえで、UDP 通信によって演奏開始・終了や

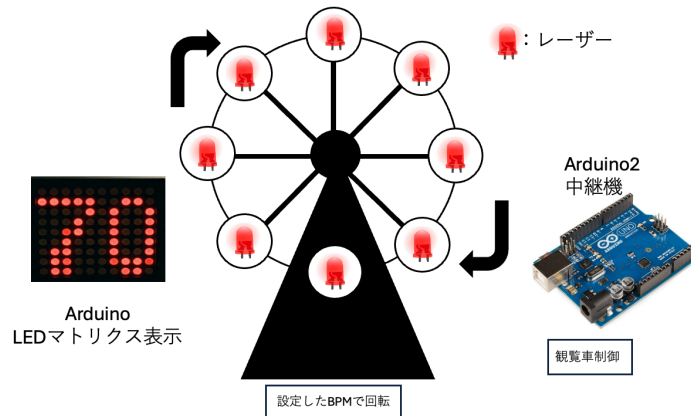


図 10 中継機デバイスの構成図

BPM、音量等の情報をやり取りする。また、スライドスイッチは、観覧車のレーザー基板ごとに実装され、ボタン電池からレーザーモジュールへの給電を ON/OFF する役割を持つ。レーザーは演奏中常時点灯させる必要があるが、非使用時にはこのスイッチによって電源を遮断できるようにすることで、ボタン電池の消耗を抑えている。

また、中継機の回路図を図 11 に示す。モータードライバーに Arduino の D5 ピンを接続し、PWM 制御を行えるようにする。モータードライバーへの電力供給は当初、外部電源として 9V のアルカリ電池を検討していたが、Arduino の 5V からの電力供給と DC ジャックからの AC アダプター 5V1A による外部給電により、アルカリ電池が不要であることが実装時に確認されたため除外した。また、モータードライバー及び DC ジャックはピンヘッダーのはんだ付けを行う。加えて、ギアボックスは $0.01\mu\text{F}$ のコンデンサーと直接はんだ付けを行う。ギアボックスと並列に接続された $0.01\mu\text{F}$ のコンデンサは、モータのブラシ接点で発生する電気ノイズを吸収し、制御信号や周辺回路への影響を低減する役割を果たしている。残りの受動部品についても、いずれもノイズ対策および電源安定化を目的として実装している。抵抗器 10Ω とコンデンサー $0.047\mu\text{F}$ は、モータードライバーの出力端子間に直列接続することで RC スナバ回路を構成している。スナバ回路とは、スイッチング素子が電流を遮断する際に、配線路のインダクタンス成分に起因して発生する過大な電圧であるサージ電圧を抑制するための緩衝回路である [13]。インダクタンス L を含む回路において、スイッチが高速にオフすると、電流の変化率 di/dt に比例した電圧 $L di/dt$ が発生し、これが半導体素子の耐電圧を超過したり、電磁雑音の放射を増大させたりする恐れがある。この問題に対して、スナバ回路は遮断された電流にたいしてバイパス経路を提供することで電流変化率を緩和し、サージ電圧を抑制する役割を果たす。本デバイスにおいては、DC モータの内部にコイルが内蔵されているためこの回路が有効である。実際には、モータードライバーの出力端子間に抵抗 $R_1 = 10\Omega$ とコンデンサ $C_1 = 0.047\mu\text{F}$ を直列接続しており、モータ駆動時のスイッチングノイズを吸収している。これは、スイッチ素子自体の保護よりも、モータのインダクタンスに起因するノイズが電源ラインや信号ラインへ伝搬することを抑制する目的に重点を置いた構成と言える。また、有極性コンデンサー $100\mu\text{F}$ は、DC ジャックからの電源供給ライン上に並列接続する平滑用コンデ

表 3 中継機デバイスを構成する部品

機材	品番	個数 [個]	素子番号
Arduino Uno R4 WiFi	-	1	A_1
ルーター	-	1	-
ブレッドボード	-	1	-
赤色ドットレーザーモジュール	FU650AD5-C6	8	-
ボタン電池基板取付用ホルダー (CR2477 用)	CH031-2477LF	4	-
ボタン電池	CR2477	4	-
スライドスイッチ	SS-12D00-G5	4	-
モータードライバモジュール	AE-DRV8835-S	1	U_2
ハイパワーギヤーボックス HE	72003	1	M_1
ジャンパーワイヤー	-	適宜	
ブレッドボード用 2.1 mm 標準 DC ジャック DIP 化キット	AE-DC-POWER-JACK- DIP	1	J_1
超小型スイッチング AC アダプター 5 V 1 A	-	1	-
QI ケーブル 2P-1Px2	311-262	1	-
抵抗器 10 Ω	-	1	R_1
コンデンサー 0.047 μF	-	1	C_1
コンデンサー 0.01 μF	-	1	C_2
有極性コンデンサー 100 μF	-	1	C_3

ンサであり、モータの起動・停止に伴う電流変動による電圧降下を抑制し、Arduino およびモータドライバへの電源供給を安定させる役割を持つ。

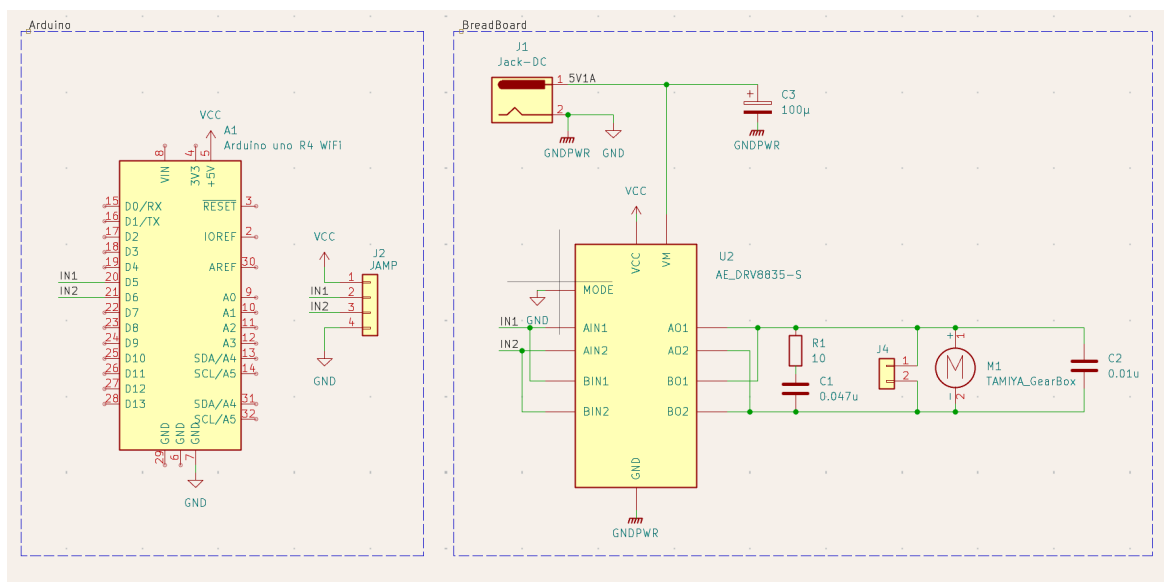


図 11 観覧車型中継機デバイス回路図

次に、観覧車の設計について説明する。作成するために、3D プリンタを用いて作成する。実際に設計に使用するモデルデータを図 12 から図 17 に示している。観覧車の直径は約 16cm とし、ボタン電池付きレーザーを 45 度間隔で配置する。そして、観覧車を支える左右の支柱および土台を設計する。観覧車本体は、以下のパーツに分割して設計を行う。

- ・土台 観覧車全体を支える部分であり、内部を空洞化させる。また、支柱を土台に差し込み、固定する役割も持つ。実際に設計する部品は図 12 の通りである。
- ・支柱 観覧車の回転軸を左右から支える部品である。回転軸を安定して保持し、観覧車が滑らかに回転できるようにする。回転軸を上から挿入する形であり、軸が折れないようギアボックスモータに繋ぐ側の支柱の支える穴を大きくしている。実際に設計する部品は図 13 の通りである。
- ・回転軸 ギアボックスモータの回転を観覧車本体へ伝達する部品である。モータと接続し、観覧車全体を回転させる役割を持つ。支柱に固定できるように、支柱の位置の軸部をへこませて固定できるようにしている。実際に設計する部品は図 14 の通りである。
- ・回転リング 観覧車の円形部分であり、レーザー側とその反対側で 2 個作成する。モータによって回転し、レーザーによる光信号送信を行う部分でもある。角度 45 度おきにレーザーを設置する穴を用意し、そこにレーザーをはめ込み固定する。もう一つの円盤に電池基板を設置し電力供給する。実際に設計する部品は図 15、16 の通りである。
- ・受光部 レーザーの延長線上に設置する照度センサーの受光部である。回転リングから平行な位置に円盤を設置し、中央にギアボックスモータを設置する台を設ける。90 度おきに円盤に

くぼみを作り，センサーを設置できるようにした．受光部の円盤によりレーザー光が後ろに漏れないという設計となっている．実際に設計する部品は図 17 の通りである．

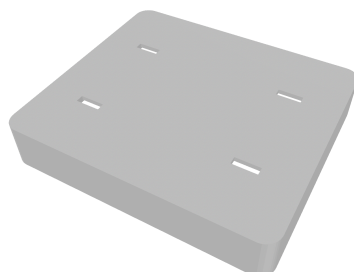


図 12 設計に使用するモデルデータ (土台部)



図 13 設計に使用するモデルデータ (支柱部)

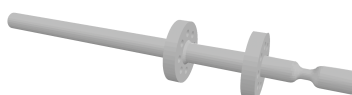


図 14 設計に使用するモデルデータ (回転軸部)

実際に作成した中継機デバイスの全体は，3.2.2 項の図 34 に示す．また，回転リングおよび受光部の実装の詳細は 3.2.2 項および 3.2.3 項で述べる．

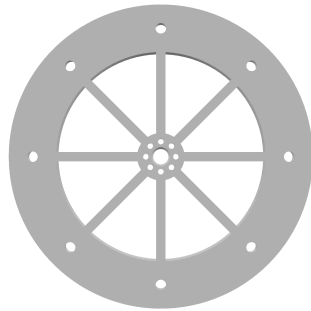


図 15 設計に使用するモデルデータ (回転リング部 1)

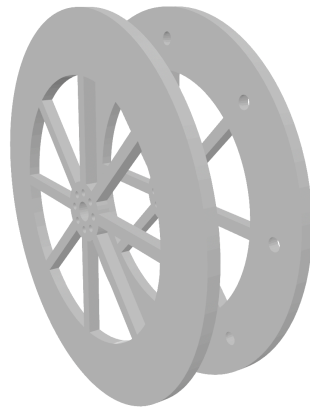


図 16 設計に使用するモデルデータ (回転リング部 2)

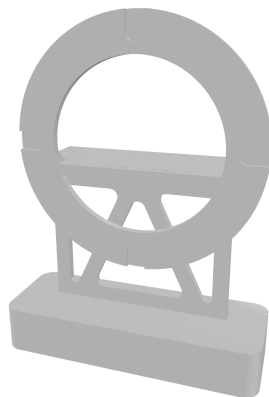


図 17 設計に使用するモデルデータ (受光部)

表 4 中継機デバイスのファイル構成

ファイル名	概要
FerrisWheel.ino	WiFi 接続・UDP 受信の管理，受信ヘッダに応じたモータ制御・表示処理の呼び出し
Display.cpp	表示文字の独自フォント定義，LED マトリクスへの描画処理
Display.h	定数定義，関数宣言，外部参照
MotorControl.cpp	PWM 出力によるモータ回転制御，回転速度テーブルの生成
MotorControl.h	モータ制御用の定数・変数定義，関数宣言

2.2.3 内部設計

本節では中継機デバイスの内部設計について説明する。本デバイスでは，モータ制御と現在の BPM を表示する LEDMatrix のプログラムによって構成される。また，中継機デバイスの制御プログラムのファイル構成を示す。表 4 の通り，各ファイルは機能ごとに分割されている。

本デバイスの全体の処理の流れを図 18 に示す。起動時に，Wi-Fi ネットワークへの接続と UDP 受信用ポートの開放，およびモータ制御・LED マトリクス表示の初期化を行う。その後は，指揮デバイスから送信される固定長 2 バイトのパケットを常時監視し，先頭のヘッダに応じて処理を分岐する。演奏開始の指令を受信すると，パケットに付与された BPM 段階番号に対応する速度でモータの回転を開始し，LED マトリクスに該当する BPM 値を表示する。演奏中に BPM 変更の指令を受信した場合は，回転速度と表示を新しい段階に更新する。なお，BPM 変更の指令は演奏中のみ受け付け，停止中は無視する。演奏終了の指令を受信すると，モータを停止させ，LED マトリクスを消灯する。いずれの処理も完了後は受信待ちへ戻り，次の指令に備える。

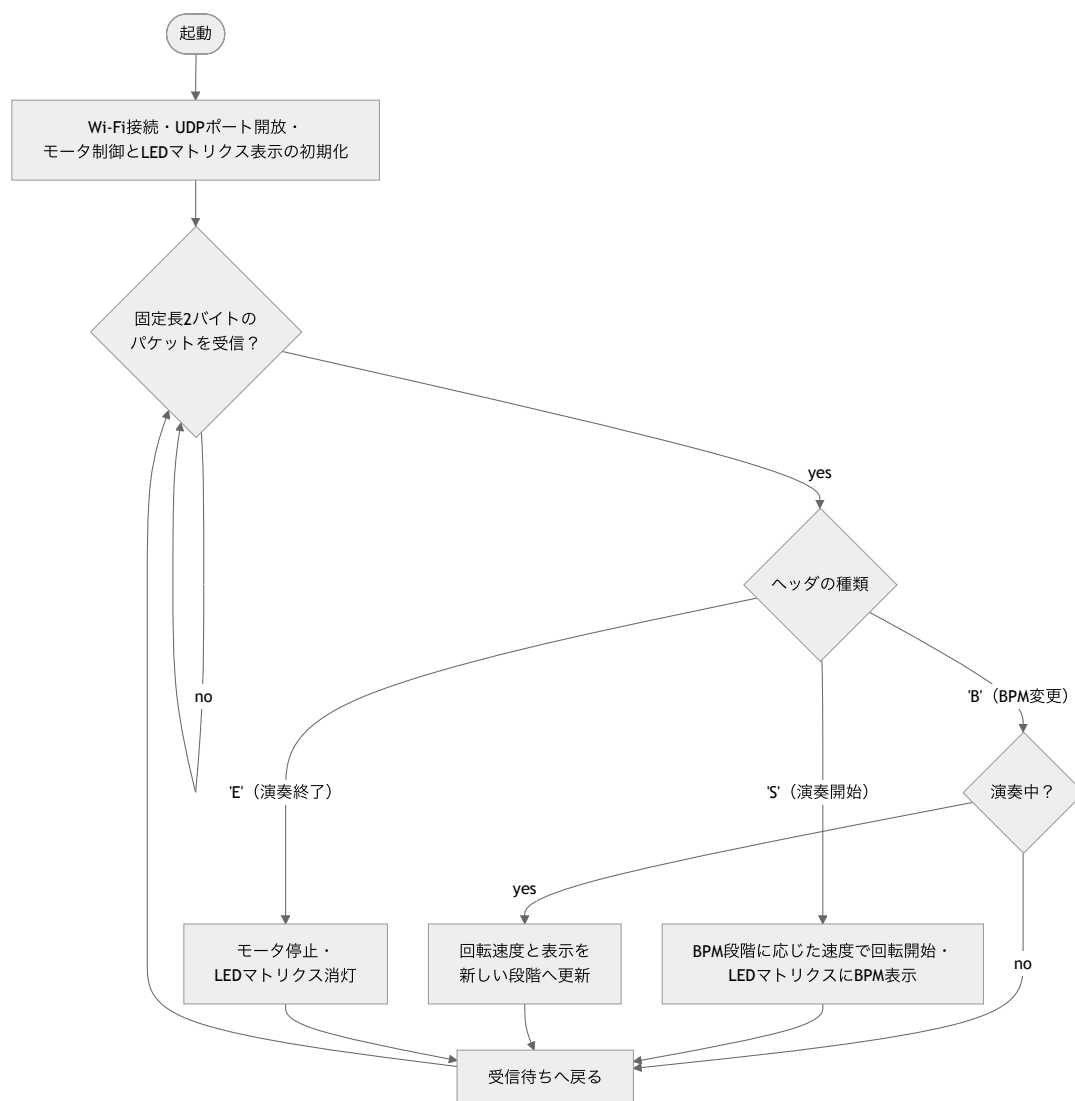


図 18 中継機デバイス全体の処理フローチャート

ここからは、モータの制御、LEDMatrix での BPM 表示、回転速度実測プログラム (LaserIntervalTest) の順に説明する。

まず、中継機デバイスにおけるモータの制御処理について説明する。本デバイスのギアボックスモータの回転速度を制御するために、Arduino のパルス幅変調機能 (PWM) を用いてモータの平均印加電圧を制御する。PWM とは、デジタル出力ピンから出力される矩形波の HIGH 状態と LOW 状態の時間比率 (デューティ比) を変化させることで、擬似的にアナログ電圧を生成する制御方式である。Arduino のデジタル出力ピンは 5V の HIGH か 0V の LOW のいずれかしか出力できないが、HIGH と LOW を高速に切り替えてデューティ比を調整することで、負荷に対して見かけ上の連続的な電圧を印加することが可能となる。

本デバイスは、Arduino の D5 ピンをモータドライバモジュールに接続し、PWM 制御を行う構成としている。Arduino の analogWrite() は、引数として 0 から 255 の整数値を受け取り、この値に応じたデューティ比の矩形波を出力する。このとき引数が 0 のときデューティ比は常時 LOW であり、255 のときデューティ比は常時 HIGH となる。中間値では、引数に比例したデューティ比の矩形波が生成される。この関係を式で表すと、供給電圧を V_{cc} (定数 VCC)、プログラム上の出力値を $pwmValue$ とすると、モータに加わる平均電圧 $V_{average}$ は (1) 式で決定される。なお、実装上の都合により実際のプログラムにおいては分母を 256 とする近似演算を用いている。

$$V_{average} = V_{cc} \times \frac{pwmValue}{255} \quad (1)$$

このとき、モータの巻線が持つインダクタンスとギアボックスの機械的慣性が電氣的・機械的なローパスフィルタとして機能するため、モータはパルスの平均値に応じた一定速度で回転する。

次に、ルックアップテーブルによる速度制御について説明する。一般的な DC モータの回転速度は供給電圧に比例する性質があるが [11]、実際には摩擦や機構の負荷などによる非線形な挙動を示す事が多い。そのため、当初検討していた「4 拍で 90 度回転する」といった幾何学的な理論式による制御ではなく、より確実な速度追従を実現するため、幾何学的な縛りを廃止して実測値に基づくルックアップテーブル方式を採用する。本デバイスでは、表 5 に示す離散的な 5 段階の BPM 設定値と、あらかじめ実測により求めた PWM 出力値を、コード内の配列 bpmTable・pwmTable として対応させ、受信した段階番号に応じた pwmValue を即座に選択・出力する構成とする。表 5 に示す実測周期は、後述する回転速度実測プログラム (LaserIntervalTest) を用いて計測した値に基づいて決定している。

表 5 BPM に対する実測周期と出力 PWM 値

段階	目標 BPM	実測周期 [ms]	出力 PWM 値
1	60	230	26
2	90	180	28
3	120	140	30
4	150	110	32
5	180	80	37

また、モータの起動・停止時には、PWM 出力値を一定周期で少しずつ増減させる段階的な制御を行っている。演奏開始時には出力値を 0 から目標値まで徐々に増加させることで急激な起動による機構への負荷を避け、演奏終了時には徐々に減少させて滑らかに停止させる。一方、演奏中の BPM 変更時には、対応する PWM 出力値へ直ちに切り替えることで、操作に対する応答性を確保している。

次に、中継機デバイスにおける LEDMatrix 表示処理について説明する。本システムでは表示用のマトリクスとして、Arduino Uno R4 WiFi 付属のマトリクスを使用する。このマトリクスを用いて、受信した BPM 段階番号を数値変換し、独自定義したフォント配列を用いて表示を行う。ここで、今回利用する Arduino のマトリクスは縦 8 横 12 の範囲で構成されている。この時、通常で定義されている文字列では文字の間隔が短くなることや、それに付随した可読性の低下が考えられる。よって、今回は 1 文字を縦 5 横 3 で再定義することにより文字同士の間隔を開け、可読性の担保を目指す。例として数字 0 のドットパターンをコード 1 に示す。

コード 1 font3x5 配列における数字 0 のドットパターン定義

```

1 {1,1,1,
2 1,0,1,
3 1,0,1,
4 1,0,1,
5 1,1,1}
```

また、LEDMatrix の描画処理は、受信した段階番号から BPM 値を参照し、整数を各桁に分割した後、独自定義した 3 × 5 のフォント配列を用いてフレームバッファに書き込むことで行う。

続いて、LEDMatrix 表示で使用する変数・定数の仕様を表 6 に示す。

表 6 LEDMatrix 表示で使用する変数・定数の仕様一覧

変数名	型	説明
BPM_STEP_1～5	#define	BPM5 段階 (60,90,120,150,180) の各値を定義
FONT_WIDTH	#define	1 文字の横ドット数 (3) を定義
FONT_HEIGHT	#define	1 文字の縦ドット数 (5) を定義
MATRIX_ROWS	#define	LED マトリクスの行数 (8) を定義
MATRIX_COLS	#define	LED マトリクスの列数 (12) を定義
font3x5[10][15]	const uint8_t	独自定義した 0～9 の 3 × 5 ドットパターンフォント配列
matrix	ArduinoLEDMatrix	LED マトリクスのインスタンス
bpmTable[5]	static const int	段階番号 (1～5) から実際の BPM 値へ変換するための対応表
frameBuffer[8][12]	static uint8_t	描画用の共有フレームバッファ
currentBPM	static int	現在表示中の BPM 値を保持する内部状態変数。clearDisplay 呼び出し時に初期値 (-1) へリセットされる

続いて、LEDMatrix 表示で使用する関数の仕様を表 7 に示す。

表 7 LEDMatrix 表示で使用する関数の仕様一覧

関数名	戻り値	引数	説明
displaySetup	void	なし	LED マトリクスの初期化を行う
displayBPM	void	uint8_t stepNumber	受信した BPM 段階番号を LED マトリクスに数値表示する
drawDigit	void	int digit, int x_offset	1 桁の数字をフレームバッファの指定位置に描画する
clearDisplay	void	なし	LED マトリクスを全消灯する

LEDMatrix プログラムのクラス図は以下の図 19 の通りである。WiFi 接続・UDP 受信および各処理の呼び出しを行う FerrisWheel.ino, LED マトリクスへの描画処理を行う Display.cpp, および関数宣言やフォント定義を行う Display.h で構成される。各モジュール間の関係について、FerrisWheel.ino から Display.cpp への関係は、表示処理を利用する依存関係である。また、Display.cpp から Display.h への関係は、関数宣言およびフォント定義を参照する依存関係である。

以下では、表 7 に示した順に、各関数の処理内容を説明する。はじめに、displaySetup 関数について説明する。この関数は、matrix.begin() を呼び出した後、消灯フレームを一度描画してから 200 ms 待機する。これは、begin() 直後の最初の loadFrame() は反映されない場合があるための対策である。displaySetup 関数の処理の流れを図 20 に示す。図 20 に示すように、本関数は分岐を持た

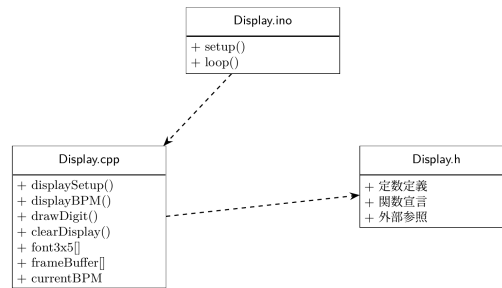


図 19 LEDMatrix のクラス図

ない直列の初期化処理であり，マトリクス of 初期化，消灯フレームの描画，待機の 3 ステップを順に実行する．また，displaySetup 関数をコード 2 に示す．

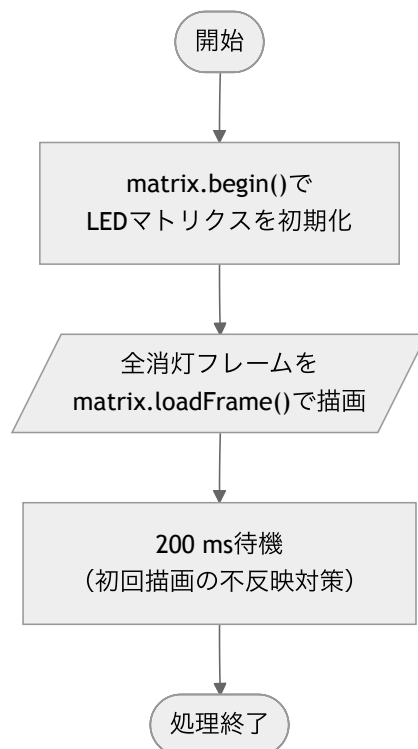


図 20 displaySetup 関数の処理フローチャート

コード 2 displaySetup 関数

```

1 void displaySetup() {
2     matrix.begin();
3     uint32_t warmUp[3] = {0, 0, 0};
4     matrix.loadFrame(warmUp);
5     delay(200);
6 }
  
```

次に、displayBPM() について説明する。この関数は、loop() から受け取った BPM 段階番号 (1~5) を bpmTable[] で参照して実際の BPM 値に変換したうえで各桁ごとに分割し、drawDigit() を用いてフレームバッファに描画する。そして、uint32_t[3] 形式にビットパックした後、matrix.loadFrame() で描画を行う。displayBPM() の処理の流れを図 21 に示す。図 21 に示すように、本関数は最初に段階番号の範囲確認を行い、範囲外であれば何も表示せずに処理を終了することで、不正な受信値による誤表示を防いでいる。範囲内であれば、BPM 値への変換、フレームバッファの初期化、桁分割、中央揃えの開始位置算出、桁ごとの描画、ビットパックと表示の順に処理を進める。また、displayBPM() をコード 3 に示す。

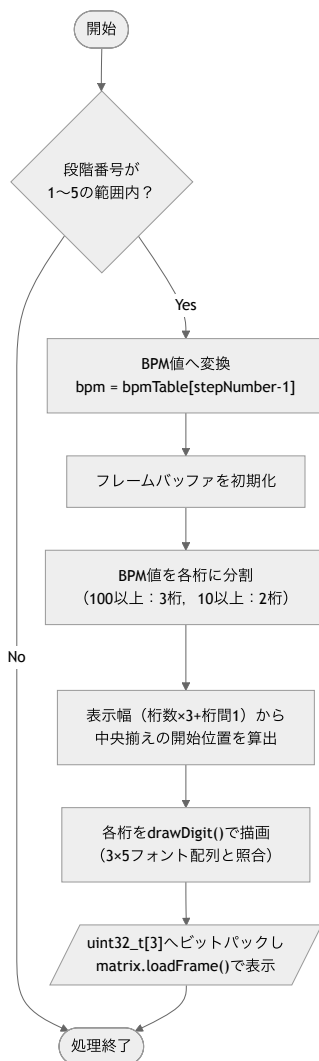


図 21 displayBPM 関数の処理フローチャート

コード 3 displayBPM 関数

```
1 void displayBPM(uint8_t stepNumber) {
```

```

2   if (stepNumber < 1 || stepNumber > 5) return;
3   uint8_t bpm = bpmTable[stepNumber - 1];
4
5   memset(frameBuffer, 0, sizeof(frameBuffer));
6
7   int digits[3];
8   int numDigits;
9   if (bpm >= 100) {
10      numDigits = 3;
11      digits[0] = bpm / 100;
12      digits[1] = (bpm / 10) % 10;
13      digits[2] = bpm % 10;
14  } else if (bpm >= 10) {
15      numDigits = 2;
16      digits[0] = bpm / 10;
17      digits[1] = bpm % 10;
18  } else {
19      numDigits = 1;
20      digits[0] = bpm;
21  }
22
23  int totalWidth = numDigits * FONT_WIDTH + (numDigits - 1) * 1;
24  int startX     = (MATRIX_COLS - totalWidth) / 2;
25
26  for (int i = 0; i < numDigits; i++) {
27      drawDigit(digits[i], startX + i * (FONT_WIDTH + 1));
28  }
29
30  uint32_t bitmap[3] = {0, 0, 0};
31  for (int row = 0; row < MATRIX_ROWS; row++) {
32      for (int col = 0; col < MATRIX_COLS; col++) {
33          if (frameBuffer[row][col]) {
34              int n = row * MATRIX_COLS + col;
35              bitmap[n / 32] |= (1UL << (31 - (n % 32)));
36          }
37      }
38  }
39  matrix.loadFrame(bitmap);
40 }

```

次に、drawDigit() について説明する。この関数は、引数の数字が 0 から 9 の範囲内であるかを確認した後、縦方向の中央寄せオフセットを算出し、独自フォント配列から該当する数字のドットパターンを 1 ドットずつ読み取ってフレームバッファの指定位置に書き込む。drawDigit() の処理の流れを図 22 に示す。図 22 に示すように、本関数も最初に引数の範囲確認を行い、範囲外であれば描画を行わない。範囲内であれば、 3×5 の 15 ドット分の繰り返し処理により、フォント配列の各ドットをフレームバッファへ書き込む。このとき、書き込み先の座標がマトリクスの範囲内である

ことをドットごとに確認しており、画面端での配列範囲外への書き込みを防止している。また、drawDigit() をコード 4 に示す。

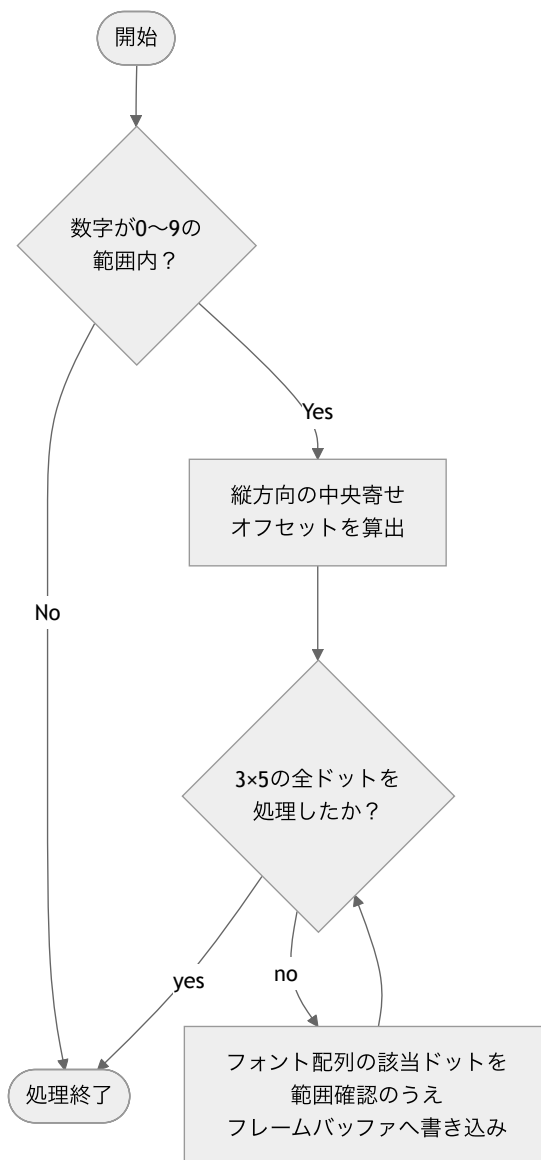


図 22 drawDigit 関数の処理フローチャート

コード 4 drawDigit 関数

```
1 void drawDigit(int digit, int x_offset) {  
2     if (digit < 0 || digit > 9) return;  
3     const int y_offset = (MATRIX_ROWS - FONT_HEIGHT) / 2;  
4     for (int y = 0; y < FONT_HEIGHT; y++) {  
5         for (int x = 0; x < FONT_WIDTH; x++) {  
6             int col = x_offset + x;
```

```

7      int row = y_offset + y;
8      if (col >= 0 && col < MATRIX_COLS && row >= 0 && row < MATRIX_ROWS) {
9          framebuffer[row][col] = font3x5[digit][y * FONT_WIDTH + x];
10     }
11 }
12 }
13 }

```

次に、clearDisplay() について説明する。この関数は、BPM 値を初期値 (-1) にリセットし、全消灯フレームを matrix.loadFrame() で描画して表示のリセットを行う。clearDisplay() の処理の流れを図 23 に示す。図 23 に示すように、本関数は表示状態を保持する変数のリセットと全消灯フレームの描画のみを行う単純な構成であり、演奏終了の指令を受信した際に呼び出される。表示状態の変数を初期値へ戻すことで、次の演奏開始時に同じ BPM が指定された場合でも確実に再描画が行われる。また、clearDisplay() をコード 5 に示す。

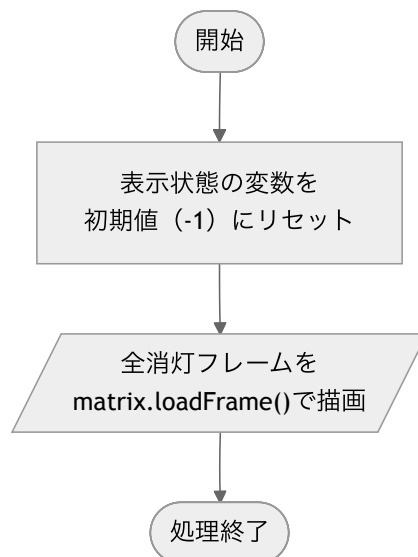


図 23 clearDisplay 関数の処理フローチャート

コード 5 clearDisplay 関数

```

1 void clearDisplay() {
2     currentBPM = -1;
3     uint32_t blank[3] = {0, 0, 0};
4     matrix.loadFrame(blank);
5 }

```

最後に、中継機デバイスのモータ回転速度を実測するために用いたプログラムである LaserIntervalTest について説明する。このプログラムは中継機デバイス本体には搭載せず、開発段階で観覧車に設置したレーザーと CdS セルを用いて、各 PWM 出力値における実際の回転周期を計測するために使用したものであり、表 5 に示した実測周期の値はこのプログラムによって得られて

いる。本プログラムは、CdS セルの出力値を監視し、レーザー光の通過を検知するたびに直前のパルス検知からの経過時間（interval）をシリアルモニタへ出力する。周囲光量やレーザーの取り付け誤差による検知レベルのばらつきに対応するため、固定閾値ではなく、起動直後のキャリブレーションと直近のピーク値履歴に基づいて検知閾値を継続的に調整する適応閾値方式を採用している。

2.3 楽器デバイス

2.3.1 概要

最後に、楽器デバイスについて説明する。楽器デバイスの構成図を図 24 に示す。本デバイスは、レーザー光を受光するための CdS セルと Arduino、そして Processing により楽曲を再生するための PC で構成され、楽器を再現するデバイスであり、主に以下の 4 つの機能を有する。第一の機能は、レーザー光の受光による BPM の推定である。本機能のシーケンス図を図 25 に示す。中継機デバイスの回転に伴い照射されるレーザー光を CdS セルが検知すると、そのデジタル信号が Arduino へ送られ、レーザーの通過間隔から楽曲の BPM が検出される。

第二の機能は、楽器デバイス間の UDP 通信による同期補正である。ドラムを担当する楽器デバイスをマスター、ピアノ・ストリングス・フルートを担当する楽器デバイスをスレーブとし、各機で検出した発音タイミングのずれを相互に送受信することで、4 台の楽器デバイス間の同期を補正する。

第三の機能は、演奏の開始・終了および音量変更の受信である。これらの情報は、指揮デバイスからの UDP 通信によって各楽器デバイスへ直接送信される。

第四の機能は、楽曲の発音である。各楽器デバイスの Arduino に接続された PC 上で動作する Processing が、Arduino からの USB シリアル通信で受け取った発音指示（音程・音量等）をもとに波形を合成することで再生される。本システムでは、ドラム、フルート、ストリングス、ピアノの 4 種類の楽器を再現する。

ここからは、外部設計と内部設計について説明する。

2.3.2 外部設計

本節では、楽器デバイスの外部設計について説明する。まず、本デバイスを構成する部品について表 8 に示す。本デバイスでは、光センサモジュールとして CdS セルを使用する。CdS セルにレーザー光が照射された際の出力を Arduino のアナログピンへ接続することで、中継機デバイスのレーザー光の通過を検知する。また、今回はドラム、ピアノ、ストリングス、フルートの 4 種類の楽器の音色を再現するため、それぞれに Arduino を使用し、CdS セルおよびブレッドボードも 4 台用意する。

次に、楽器デバイスの受光部の回路図を図 26 に示す。電源仕様は、PC との USB 接続から電力を供給する。光センサは Arduino の 5V ピンから給電されるため、外部電源は不要である。また、光センサが光を受光した瞬間に A0 のアナログピンにセンサ値を出力する。このセンサ値の変化を

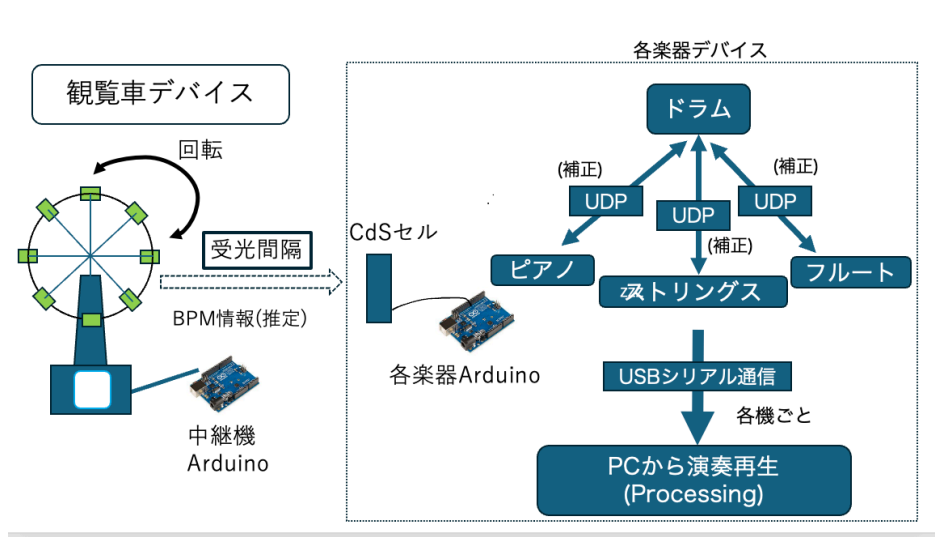


図 24 楽器デバイスのシステム構成図

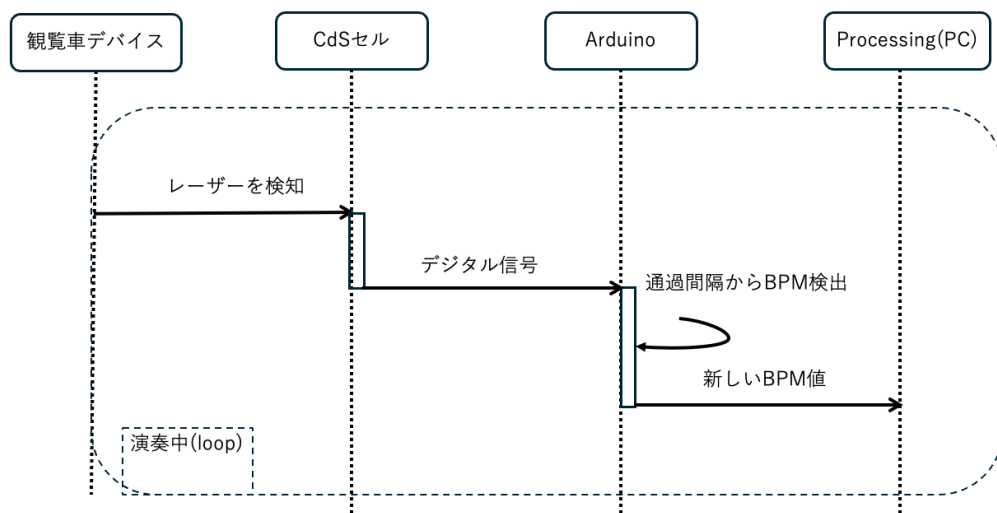


図 25 レーザー光受光による BPM 検出のシーケンス図

表 8 楽器デバイスに必要な機器

機材	品番	個数 [個]	素子番号
Arduino UNO R4	-	4	A_1
CdS セル	GL5516	4	R_{photo}
抵抗器 10 k Ω	-	4	R_2
ブレッドボード	-	4	
ジャンパーワイヤー	-	適宜	

Arduino で処理することで光を検知する仕様である。本回路における固定抵抗 ($R_2 = 10 \text{ k}\Omega$) の役割は、CdS セル (受光素子: R_{photo}) における抵抗値の変化を、Arduino の A/D コンバータ (ADC) で読み取り可能な電圧変動へと変換することである。Arduino の A0 ピンは電圧を直接測定する仕様であり、センサ単体の抵抗値を直接計測することはできない。そこで、電源の電圧 ($V_{CC} = 5 \text{ V}$) とグランド (GND) の間に CdS セルと固定抵抗を直列に接続し、分圧回路を構成する。A0 ピンに入力される出力電圧 (V_{out}) は、式 2 のように示される。

$$V_{\text{out}} = V_{cc} \times \frac{R_2}{R_{\text{photo}} + R_2} \quad (2)$$

中継機デバイスのレーザー光が受光部を通過する際、CdS セルの抵抗値 R_{photo} はレーザー光の照射によって低下する。この抵抗値の変化が式 2 の分母に作用することで、結果としてアナログ信号へと変換され、A0 ピンに印加される。したがって、当該抵抗器 ($10 \text{ k}\Omega$) は単なる過電流防止の目的ではなく、CdS セルが捉えたレーザー光の照射の有無を Arduino 側で信号処理可能な電圧として抽出するための信号変換として機能している。

2.3.3 内部設計

本節では、楽器デバイスの内部設計について説明する。楽器デバイスのソフトウェアは、PC 上で音色表現を担う Processing プログラムと、レーザー受光の検知および楽器デバイス間の同期を担う Arduino プログラム (douki_saigo) の 2 つで構成される。

両プログラムを含む楽器デバイス全体の処理の流れを図 27 に示す。図 27 に示すように、各楽器デバイスの Arduino は、起動後に指揮デバイスからの演奏開始指示を待機し、指示を受けるとレーザー受光間隔からの BPM 推定と楽器デバイス間の同期補正を行う。全楽器デバイスのずれが閾値以内に収まって同期が確立すると、楽譜データに基づく発音情報を PC ヘシリアル送信し、PC 上の Processing が波形を合成して演奏音を出力する。この送信と発音は、演奏終了指示を受信して停止状態へ移行するまで繰り返される。まず前者の Processing プログラムについて説明する。

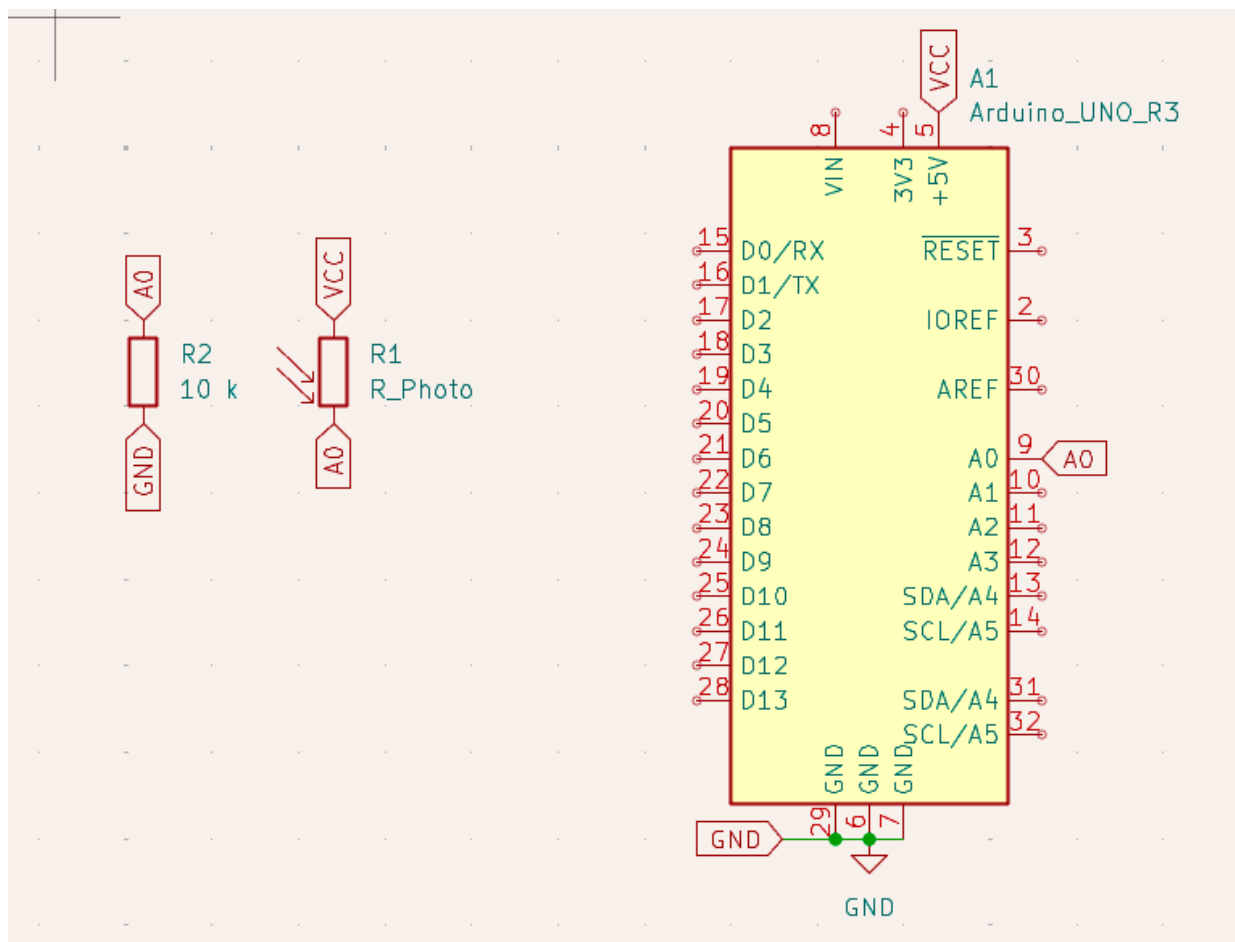


図 26 楽器デバイスの受光部の回路図

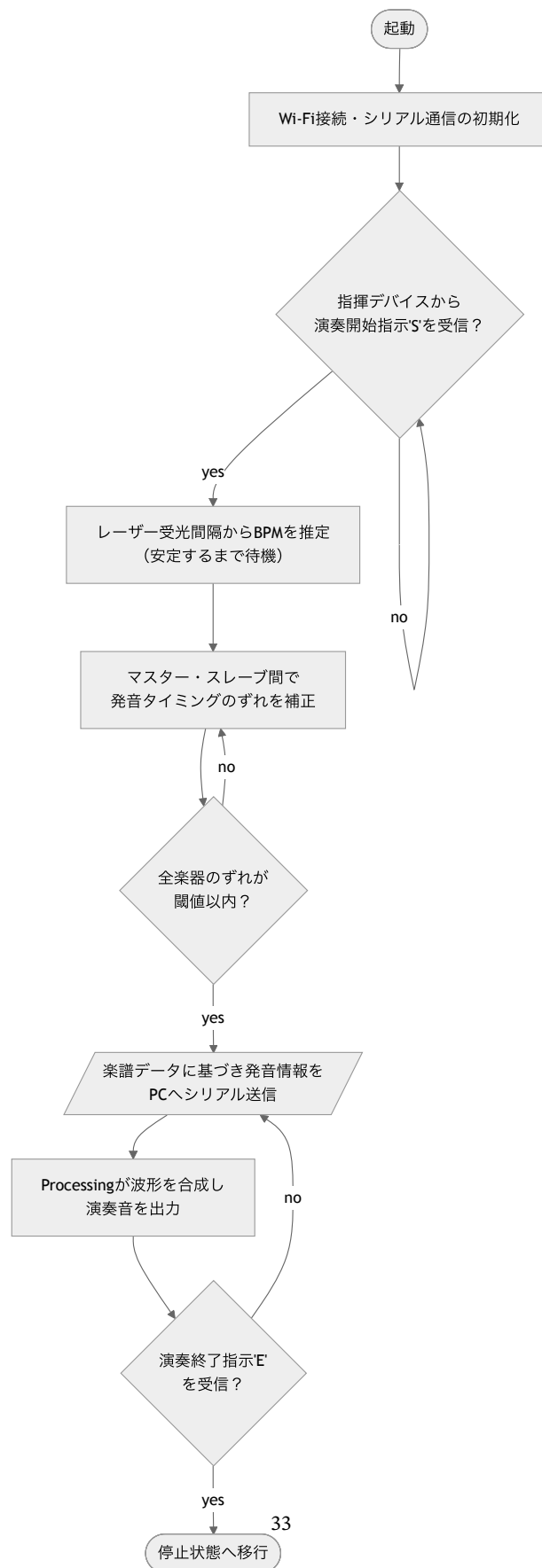


図 27 楽器デバイス全体の処理フローチャート

楽器デバイスは、Arduino から受信した発音指示をもとに、Processing 上で波形を合成して音を鳴らす構成となっている。合成方式は、あらかじめ実在の楽器音を FFT 解析して得た周波数・振幅データ（ルックアップテーブル、以下 LUT）をもとに多数の正弦波を重ね合わせる加算合成を基本とし、これに息や弦の摩擦音を模したノイズ成分を加えることで音色を再現している。

続いて、Processing プログラムのファイル構成を示す。表 9 の通り、各ファイルは機能ごとに分割されている。

表 9 Processing プログラムのファイル構成

ファイル名	概要
SoundProcessingCode.pde	setup/draw/serialEvent による全体制御，シリアル通信の受信処理，デバッグ用のキー入力処理
Utils.pde	LUT データの読み込みと保持，ピッチ・ベロシティに応じた倍音プロファイルの生成を行う DataUtils クラス
InstrumentPlayer.pde	発音命令を各楽器クラスへ委譲する InstrumentPlayer クラス
NoteSampler.pde	Arduino 未接続でも単体動作確認できる自動演奏機能，および内蔵の試奏用楽譜データ
AddEffect.pde	残響音（リバーブ）エフェクトを付加する AddReverb クラス
Flute.pde	フルートの音色合成を行う FluteInstrument クラス
Strings.pde	ストリングス（弦楽器群）の音色合成を行う StringsInstrument クラス
Piano.pde	ピアノの音色合成を行う PianoInstrument クラス
Drum.pde	キック・スネア・ハイハットの音色合成を行う KickInstrument・SnareInstrument・HiHatInstrument クラス
Horn.pde	ホルンの音色合成を行う HornInstrument クラス
Trumpet.pde	トランペットの音色合成を行う TrumpetInstrument クラス

各楽器クラスは、Minim ライブラリの提供する Instrument インタフェースを実装しており、コンストラクタで音色を構築したうえで、noteOn 関数で発音を、noteOff 関数で消音を行う共通の枠組みに従っている。発音命令は InstrumentPlayer クラスを介して各楽器固有のクラスへ委譲され、波形生成に必要な LUT データは DataUtils クラスに集約されている。最終的な音声信号は、楽器の種類に応じて AddReverb による残響エフェクトを通したうえで AudioOutput へ出力される。プログラム全文は付録に示す。

ここからは、シリアル通信による Arduino との連携、LUT データの管理（DataUtils）、発音命令の委譲（InstrumentPlayer）、残響エフェクト（AddEffect）、デバッグ用自動演奏（NoteSampler）、各楽器クラスによる音色合成の順に説明する。まず、Arduino とのシリアル通信および発音ディスパッチについて説明する。本デバイスは 1 台の PC につき 1 種類の楽器を担当する構成となっており、プログラム内で定義した楽器番号の定数によって、この Processing インスタンスがどの楽器と

して動作するかを切り替える。Arduino からのシリアル通信は、改行で区切られたテキストメッセージとして受信され、カンマで分割したうえで先頭のヘッダにより処理を分岐する。発音指示のメッセージには、音程、音の長さ、発音開始時と終了時の強さ、音色の種別の組が1 音符分以上含まれており、これを解析して担当楽器のクラスへ発音命令として委譲する。このほか、BPM 変更のメッセージを受信した場合は演奏テンポを、音量変更のメッセージを受信した場合は出力音量を、それぞれ受信値に応じて更新する。

次に、楽器デバイスにおける LUT データの管理 (DataUtils クラス) について説明する。計画書当初は、倍音の周波数と振幅を数式によって機械的に算出し合成する設計を検討していたが、低音域や高音域において実在の楽器との音色の再現度が低下するという問題が生じたため、実際はアイオワ大学等が公開する実楽器の音声データを FFT 解析し、各音程・各強弱 (pp/mf/ff) ごとの「周波数と振幅のペア」を抽出して JSON 形式の LUT として事前に用意しておく設計に変更した。

次に、発音命令の委譲を行う InstrumentPlayer クラスについて説明する。このクラスは、AudioOutput への参照を保持し、各 PlayXxx 関数が呼び出されるたびに、対応する楽器クラスのインスタンスをその場で生成して out.playNote 関数へ渡すという単純な委譲処理のみを行う。

次に、残響エフェクトを付加する AddEffect.pde の AddReverb クラスについて説明する。このクラスは Minim ライブラリの UGen を継承しており、講義で扱われたたたみ込み積分を、指数関数的に減衰するランダムノイズを疑似的なインパルス応答として用いることで実装している。左右のチャンネルの記憶バッファ (historyBufferL・historyBufferR) を独立させることで、ステレオ音場に対応している。

次に、デバッグ用の自動演奏機能を提供する NoteSampler.pde について説明する。本機能は、指揮デバイスや中継機デバイスが未接続の状態でも PC 単体で音色の検証を行えるようにするためのものであり、Processing の標準描画ループ (draw) に依存すると波形描画の負荷でタイミングがずれるため、音楽専用のスレッド (audioThread) を用いて高精度にタイミングを管理している。

最後に、各楽器クラスによる音色合成について説明する。いずれの楽器クラスも、コンストラクタ内で DataUtils から取得した倍音プロファイルをもとに正弦波を加算合成し、ADSR (Attack, Decay, Sustain, Release) エンベロープおよび Line による時間変化を付加したうえでミキサーへ結線し、noteOn 関数でエンベロープと Line を一斉に起動する、という共通の設計に従っている。

次に、各楽器デバイスの同期方式を担う Arduino プログラムについて説明する。楽器デバイスは、指揮デバイスからの演奏開始指示を受けた後、中継機デバイスの回転に伴うレーザー光を CdS セルで検知することでおおまかな BPM を推定し、さらに楽器デバイス同士の UDP 通信によって発音タイミングを細かく補正することで、複数台の Arduino 間での同期を実現している。この同期方式では、ドラムを担当する Arduino を基準時刻源 (マスター) とし、ピアノ・ストリングス・フルートを担当する 3 台の Arduino をスレーブとするマスタースレーブ方式が採用されている。また、各 Arduino は、1 拍を 24 分割した Tick と呼ばれる時間単位で内部時刻を刻んでおり、起動からの通算 Tick 数 (以下、globalTick) を共有の基準時刻として同期に用いる。

続いて、同期プログラムのファイル構成を示す。表 10 の通り、各ファイルは担当する楽器ごとに用意されている。

表 10 同期プログラム (douki_saigo) のファイル構成

ファイル名	概要
drum_0706.ino	ドラム担当機のプログラム。同期のマスターとして動作し、全楽器デバイスの発音タイミングを管理する
flute_0706.ino	フルート担当機のプログラム。同期のスレーブとして動作する
piano_0706.ino	ピアノ担当機のプログラム。同期のスレーブとして動作する
strings_0706.ino	ストリングス担当機のプログラム。同期のスレーブとして動作する

flute_0706.ino・piano_0706.ino・strings_0706.ino の 3 ファイルは、IP アドレス・楽器 ID・CdS 検知閾値・埋め込みの楽譜データが異なるのみで、通信・同期ロジックの構造は完全に同一である。そのため、以下ではスレーブ側の代表例として flute_0706.ino を用いて説明し、マスター側は drum_0706.ino を用いて説明する。

次に、各 Arduino 間の同期に用いられる通信ヘッダの一覧を示す。表 11 の通り、指揮デバイスからの一斉指示に加え、マスター・スレーブ間で複数種類のパケットがやり取りされる。

表 11 楽器デバイス間の同期通信で使用するヘッダの一覧

ヘッダ	送信元 → 送信先	概要
'S'	指揮デバイス → 全楽器デバイス	演奏開始指示。受信した各機は状態を初期化する
'E'	指揮デバイス → 全楽器デバイス	演奏終了指示。受信した各機は状態を停止状態に戻す
'V'	指揮デバイス → 全楽器デバイス	音量段階の通知
'J'	スレーブ → マスター	同期への参加予告
'M'	スレーブ → マスター	レーザー検知による BPM 提案
'N'	マスター → 各スレーブ	多数決により決定した確定 BPM のブロードキャスト
'T'	スレーブ → マスター	自身の globalTick の報告（ずれ算出の依頼）
'R'	マスター → スレーブ	'T' 受信時点での両者の globalTick の差（以下、diff）の返信
'O'	マスター → スレーブ	演奏を開始すべき globalTick 値の指示

続いて、レーザー光の検知処理について説明する。各 Arduino が中継機デバイスのレーザー光を検知したパルス間隔から求めた実測 BPM は、マスターでは自身の BPM 推定値として、スレーブでは 'M' 通信による BPM 提案として用いられる。

次に、マスターにおける同期状態管理について説明する。マスターは、state という状態変数によって管理される 5 段階の状態遷移に沿って動作する。具体的には、BPM の安定を待つ待機状態、各スレーブとのずれの補正を行う同期状態、同期完了後の演奏開始準備状態、演奏中の状態、および停止状態の 5 段階である。

まず、'T' 通信の受信処理について説明する。この処理は、スレーブの globalTick とマスター自身の globalTick との差である diff を算出し、'R' 通信で即座に返信する。

次に、スレーブにおける同期処理について説明する。スレーブは、state が 4 段階であり、具体的には BPM 安定待ち、マスターと同期中、演奏中、停止中である。'R' 通信で受け取った diff の大きさに応じて自身の Tick の周期を一時的に伸縮させることでマスターとの同期を保つ。

最後に、'O' 通信の受信処理について説明する。この処理は、指示された演奏開始時刻 (globalTick 値) と自身の globalTick を比較し、まだ到達していなければ開始待ちの状態で待機し、すでに超過していれば超過分だけ楽譜の再生位置を進めて即座に演奏へ合流する。プログラム全文は付録に示す。

3 システム実装

3.1 チーム構成と役割分担

本プロジェクトは 6 名で実施した。役割分担は、作業分解構造 (WBS) に基づき、楽曲表現系 (ハードウェア全般) を山口・佐藤、楽器系 (音色合成・発音機能) を梅野・薄井、通信系 (通信同期制御) を宇井・菊池がそれぞれペアで担当する構成とした。このうち報告者 (佐藤) は、ハードウェア面については指揮デバイスの作成、楽器デバイスの受光部回路の作成、および結合テスト時における全体的な調整を担当した。また、ソフトウェア面では、中継機デバイスの LEDMatrix に現在の BPM を表示する Display モジュール (Display.h, Display.cpp) の実装を担当した。Display モジュールのプログラム全文は付録に示す。

また、計画当初のガントチャートを図 28、最終的な実績のガントチャートを図 29 に示す。図 29 は、WBS の各タスクについて、実際に作業を行った期間と作業達成率を日単位で示したものである。ハードウェア面については、部品の変更や仕様の変更などによって大幅に作業が遅延した。この遅延により、プロジェクト全体として遅延傾向になったと考える。

図 28 計画書時点のガントチャート（上段：実装、下段：テスト）

図 29 最終的な実績のガントチャート（上段：実装，下段：テスト）

3.2 報告者の担当範囲と技術的課題

本節では、報告者が担当したハードウェアの実装について、採用した技術の選定理由を述べたうえで、実装の過程で直面した技術的課題とその解決のために行った技術的貢献を、各デバイスごとに説明する。ハードウェア実装における共通の課題は、設計上は動作するはずの回路や機構を、実環境において安定して動作させることであった。具体的には、回転体の質量バランスや軸の摩擦といった機械的な問題、モータの特性変動といった電気的な問題、およびレーザー光の受光精度という光学的な問題が、実装段階ではじめて顕在化した。以下では、これらの課題に対して行った対応を含めて述べる。

3.2.1 指揮デバイス

はじめに、操作入力部品の選定理由について述べる。BPM・音量の調整には、スライド式可変抵抗器を採用した。回転式可変抵抗器と比較して選定した理由は次の2点である。第一に、つまみの物理的な位置が設定値と対応するため、ユーザが現在の BPM・音量の段階を、表示装置を介さずつまみの位置のみから確認できるためである。第二に、本デバイスは「指揮棒を振っている」というユーザの操作によって演奏が変化することで没入感を高めることが目的であり、回転式と比較してスライド式はデバイスを握ったまま親指のみで容易に操作が可能であり、演奏中に持ち替えを必要としないためである。また、抵抗値には $10\text{k}\Omega$ を選定した。これは、分圧回路として Arduino のアナログ入力へ接続する際に、消費電流を抑えつつ、アナログ/デジタル変換の入力インピーダンスに対して十分に低い出力インピーダンスを保てる標準的な値である。

次に、振り動作の検知に用いる加速度センサの選定理由について述べる。振り動作の検知には、デジタル3軸加速度センサ ADXL345 を採用した。選定理由は次の3点である。第一に、3軸の加速度を同時に計測できるため、3軸のベクトル合成値を判定に用いることで、ユーザがデバイスを振る方向に依存しない検知を実現できる。第二に、このモジュールは I2C 通信に対応しており、信号線2本 (SDA・SCL) で Arduino と接続できるため、限られた実装空間における配線数を削減できる。第三に、測定レンジを $\pm 2\text{g}$ から $\pm 16\text{g}$ まで設定でき、振り動作で生じる加速度に対して測定値の飽和を避けた計測ができる。加えて、割り込み信号の出力ピン (INT1) を備えており、閾値超過の検知をセンサ側で行い Arduino へ通知する構成に対応できる。

次に、実装における技術的貢献について述べる。実装した指揮デバイスの内部を図 30 に、外装を含む完成状態を図 31 に示す。内部は、ブレッドボード上にタクトスイッチと加速度センサを実装し、2系統のスライド式可変抵抗器はユニバーサル基板へはんだ付けして固定した。外装の設計にあたっては、ユーザビリティの向上を目的として、ユーザが操作するスライド式可変抵抗器のスライダー部分およびタクトスイッチの部分を外装から露出させる構造とした。なお、本稿ではユーザが操作方法の説明を受けなくても、操作箇所を特定し目的的操作を行える度合いをユーザビリティと定義している。これにより、ユーザは外装を開けることなく、露出したつまみとボタンのみで演奏の開始・終了と BPM・音量の全操作が行える。

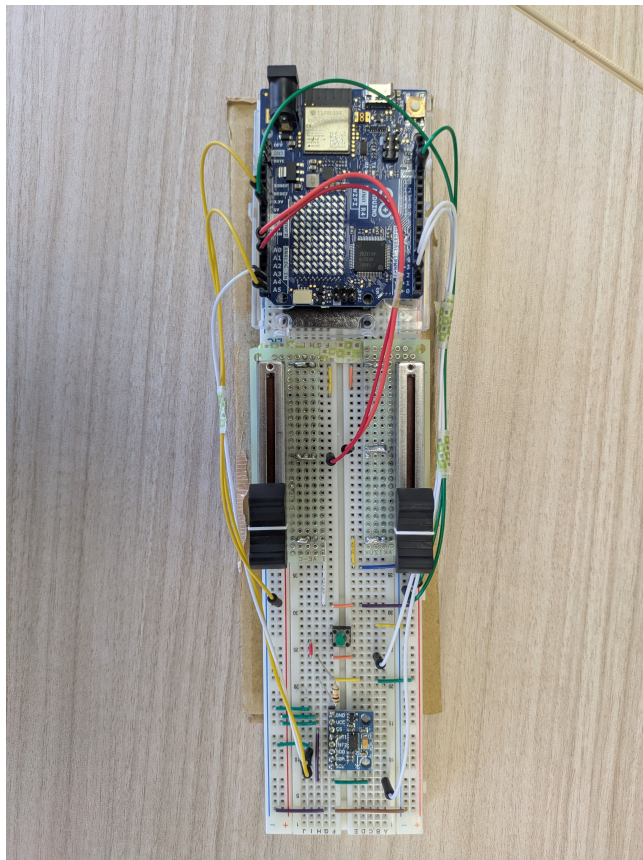


図 30 指揮デバイスの内部実装

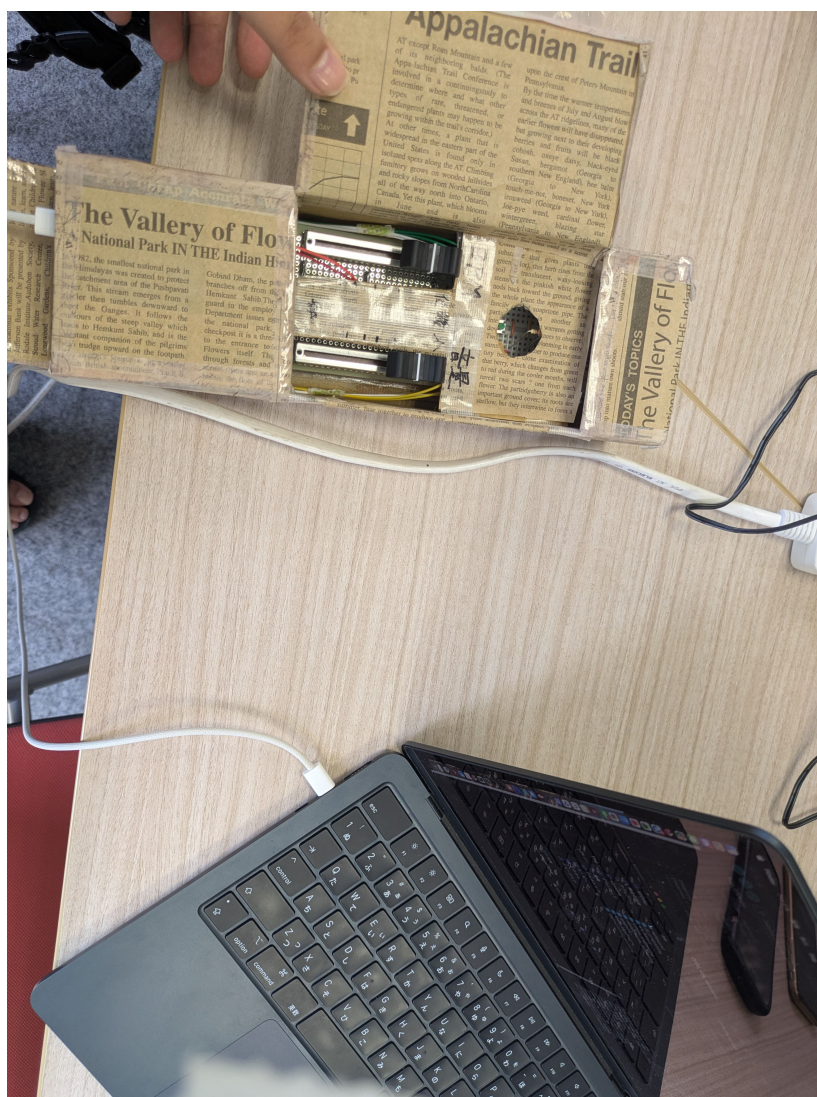


図 31 作成した指揮デバイス

3.2.2 観覧車型中継機デバイス

はじめに、主要部品の選定理由について述べる。第一に、回転リング上のレーザーモジュールの電源には、ボタン電池を採用した。レーザーモジュールは回転リングに搭載されて本体ごと回転するため、外部から配線で給電すると回転に伴い配線が捻れ、回転体へ電力を伝えるための回転接点部品であるスリップリング等の追加機構が必要となる。そこで、電源を回転体上で完結させる方式とし、回転体に搭載する電源として小型・軽量で質量バランスへの影響が小さいボタン電池を選定した。型番には、ボタン電池の中でも大容量な CR2477 を選定した。採用したレーザーモジュールは 3V 駆動かつ定電流回路を内蔵しているため、公称電圧 3V で外部抵抗なしに直結でき、回路を簡素化できる [12]。また、基板取付用ホルダーを介して実装することで、電池の消耗時に交換が可能である。実際に作成したレーザーによる光信号送信基盤を図 32 に示す。図 32 の通り、ユニバーサル基盤にボタン電池ホルダーとレーザーモジュールをはんだ付けしている。これにより、ボタンをオンにすると回転リングの穴からレーザー光が照射される。

第二に、モータの駆動には、モータドライバモジュールを採用した。Arduino のデジタル出力ピンはモータの駆動に必要な電流を直接供給できないため、Arduino からモータへ電力を供給するためのモータドライバが必要となる。DRV8835 を選定した理由は次の 3 点である。第一に、H ブリッジという 4 つのスイッチング素子を組み合わせ、モータへ流す電流の向きを切り替えられる回路を内蔵し、PWM 信号の入力によって回転速度の制御が可能であるからだ。第二に、モータ用電源として最大 11V・1 チャンネルあたり最大 1.5A の駆動に対応しており、AC アダプタからの 5V 単一電源系でギヤボックスモータを駆動できる。第三に、ブレッドボードに直接実装できる小型の DIP 化モジュールであり、試作段階においてははんだ付けを行わずに配線を変更できる。実装したモータドライバ周辺の回路を図 33 に示す。

次に、実装における技術的貢献について説明する。実装した中継機デバイスの全体を図 34 に示す。

まず、回転部の機械的な安定化について述べる。回転する部品を支える回転軸は、安定性を担保するために左右の支柱で軸受け穴の大きさを非対称とし、ギヤボックスモータに接続する側の穴を大きくしている。これは、モータ側の軸受けに回転トルクによる負荷が集中し、穴の径が小さいと軸が折れるおそれがあるためである。軸まわりの部品形状を図 13、図 14 に示す。計画書当初は同一の大きさに軸を設計しており、実際に 3D プリンターにて実装をしたところ耐久性に乏しく折れてしまったが、現在の仕様に変更したところ実験中に軸が折れることはなく、安定性が担保された。さらに、軸受け部にはグリス（半固体状の潤滑剤）を塗布し、軸と軸受け間の摩擦を低減させた。また、回転軸をモータへ差し込む接続部にはネジロック剤（振動による締結部の緩みを防止する接着剤）を塗布し、回転の振動による軸の緩み・脱落を防止した。

次に、回転体の質量バランスの調整について述べる。ボタン、電池、ホルダー、そしてレーザーモジュールで構成される、レーザーによる光信号送信基盤を回転リングへ取り付ける際、当初はリング内に収まることを優先して両面テープで貼り付けていた。しかし、この配置では回転体の質量分布に偏りが生じ、回転速度にムラが発生した。そこで、4 枚の電池基板を中心軸を囲うように十字状の対称配置へ変更することで偏心を抑え、回転速度のムラを解消した。さらに、両面テープで

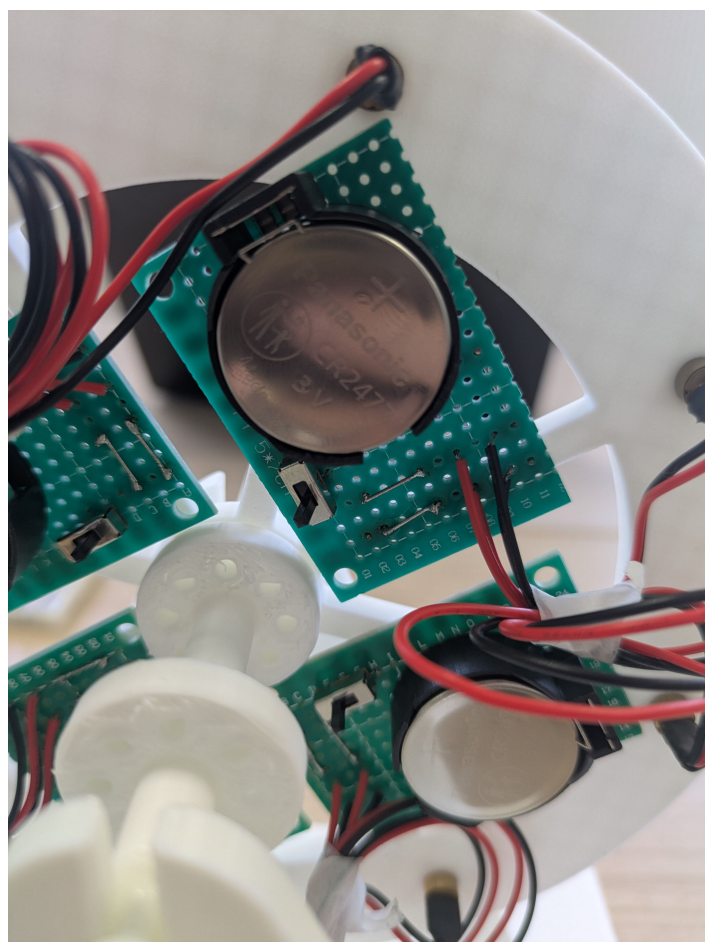


図 32 実際に作成したレーザーによる光信号送信基盤

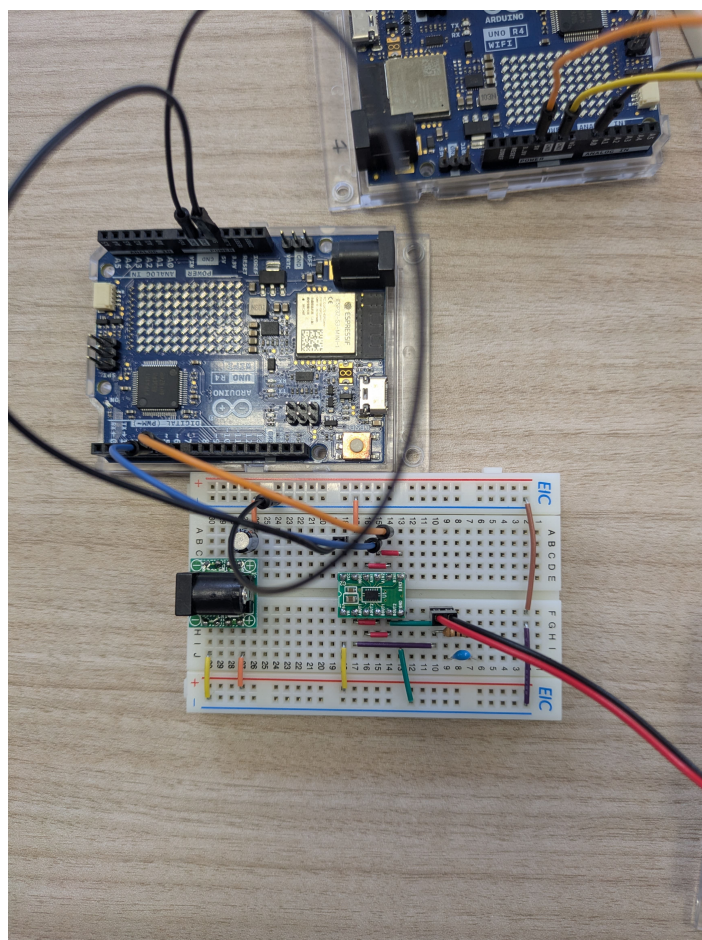


図 33 モータドライバ周辺の回路実装

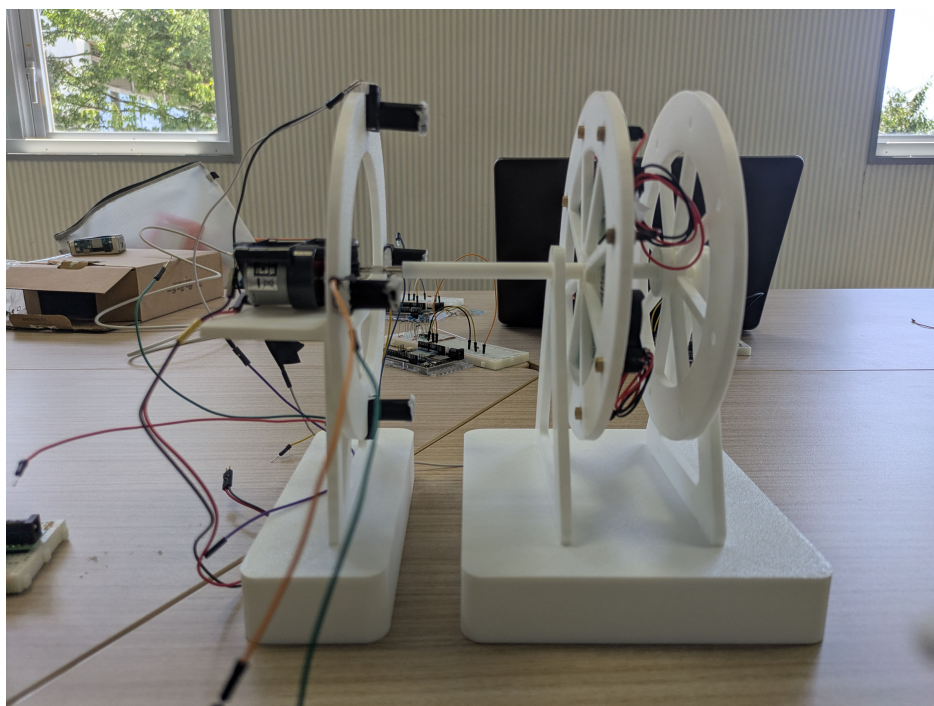


図 34 実装した中継機デバイスの全体（左：受光部リング側，右：回転リング側）

は回転の振動により固定が緩むため、瞬間接着剤による固定へ変更し、安定性を向上させた。実装した回転リングの表面・裏面を図 35 に示す。

また、動作試験の過程で、回転の反動によりモータ部が土台から浮いて動いてしまい、レーザー光の照射位置がずれて受光精度が低下する事象が確認された。この対策として、使用済みのボタン電池を土台部に載せておもりとし、モータ部の浮き上がりを抑えた。廃棄予定の電池を再利用することで、追加部品なしに質量を確保している。

さらに、動作試験を繰り返す中で、同一の PWM 設定値に対するモータの回転速度が試行ごとに一定にならない事象が確認された。この要因としては、電源の起電力が試行ごとに変動していたことが考えられる。モータの回転速度は印加電圧に依存するため、電源電圧の変動やブラシ接触抵抗の変化が回転速度の変動として現れたと考えられ、5 節で述べる同期確立時間の変動要因の一つとなった可能性がある。

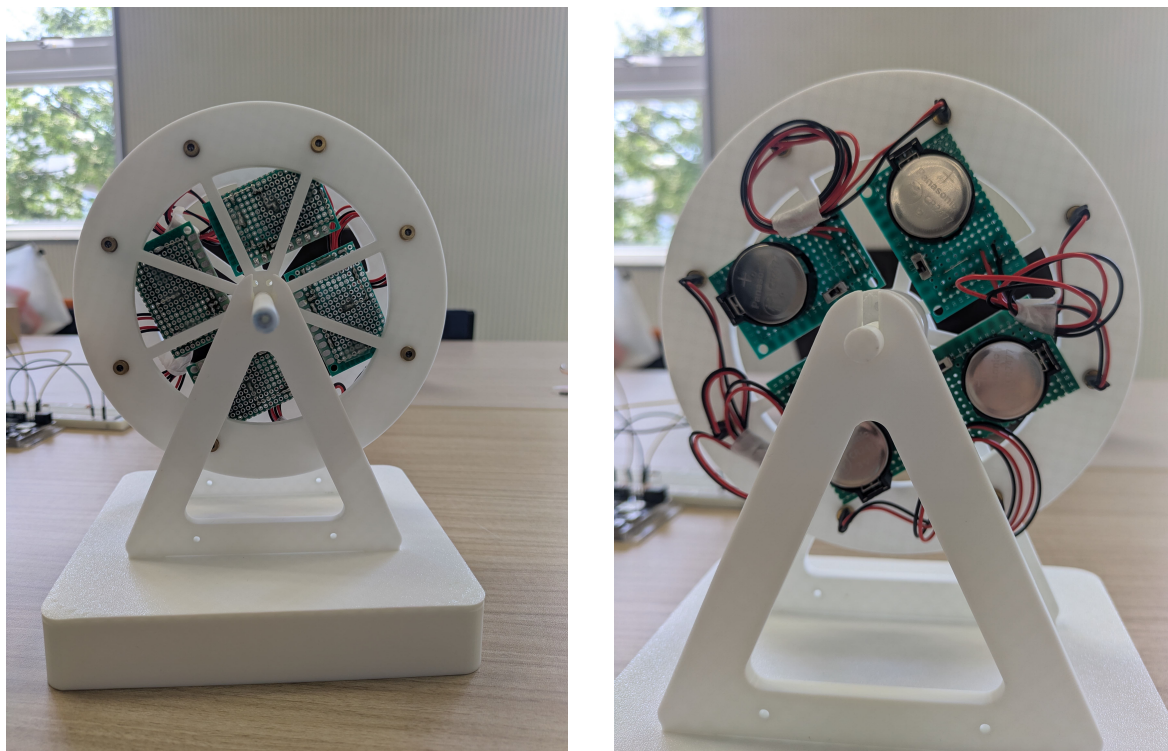


図 35 回転リングへのレーザーによる光信号送信基板の実装（左：正面，右：背面）

3.2.3 楽器デバイス受光部装置

はじめに，受光素子の選定理由について述べる．レーザー光の検知には CdS セルを採用した．CdS セルとは，硫化カドミウムを用いた，受光量に応じて抵抗値が変化する光導電素子である．CdS セルは，フォトダイオード等の他の受光素子と比較して応答速度は遅いものの，抵抗値の変化を分圧回路で読み取るだけの簡素な回路で使用でき，可視光域に感度を持つためレーザーモジュール（波長 650 nm の赤色光）の検知に適する．本システムの受光イベントは最低でも数百 m 秒であるモータの回転周期で発生するため，CdS セルの応答速度で十分であると判断した．

次に，実装における技術的貢献について述べる．まず，レーザー光を使用する際における安全対策について述べる．レーザー光は直接目に入ると危険であるため，回転する部品と同じ大きさの障害部品（遮光板）を製作し，レーザー光が受光部の後方へ漏れて観客の目に入ることを防止した．受光部の円盤形状は図 17 に示す通りである．

次に，CdS セルの設置方法について述べる．受光部の遮光板には，図 17 の通り CdS セルを設置するためのくぼみを 90 度間隔に設けている．CdS セルをこの受光部の遮光板へ固定するにあたり，当初は CdS セルを遮光板へ直接固定することを検討したが，デバイスの組み立て方やレーザー光の機微な方向の違いによって受光しやすい位置が変わり受光精度が変動するという問題があった．また，本デバイスは 3D プリンターで制作したため，ポリ乳酸 (PLA) という素材特性の観点からも設置が困難である．PLA は硬度が高く寸法安定性に優れる一方で，表面が滑らかで接着剤が定着しにくいという特性を持つ．以上の観点から，CdS セルを遮光板に直接接着した場合，十分な固定

強度が得られず受光精度が不安定となる問題があった。そこで、小さく切った発泡スチロールに CdS セルの素子部分を折り曲げて沿わせ、テープで固定したうえで、この発泡スチロールごとくぼみに設置する方式とした。これにより、発泡スチロールを動かすことで CdS セルの位置を微調整でき、組み立て後にレーザー光の照射位置へ合わせ込むことが可能となった。なお、この方式で実験を続けたところ、固定していたテープが浮いてしまう事象が生じたため、テープの重ね貼りによる補強を行った。

最後に、受光精度の向上について述べる。CdS セルは受光面が小さく、回転するレーザー光の細いビームを直接受光することが困難であったため、受光精度を段階的に改善した。まず、黒いストローで CdS セルを覆った。これは、ストローの筒内に入射したレーザー光が内部で反射して CdS セルへ届きやすくなるとともに、筒が周囲の環境光を遮ることで、受光時と非受光時の明暗差を大きくする狙いである。しかし、一般的な細さのストローでは開口部が小さく、依然として受光が困難であったため、開口径の大きいタピオカ用の黒いストローへ変更したところ、受光精度が向上した。さらに、ストローの先端をティッシュペーパーで薄く覆うことで、レーザー光が通過する際に拡散され、ビームの位置が多少ずれても拡散光として CdS セルへ届くようにした。加えて、ストローと CdS セルの間を瞬間接着剤で補強することで、隙間からの環境光の混入を防ぎ遮光性を高めた。実装した受光部回路を図 36 に、遮光板リング全体と受光部を図 37 に示す。図 37 では、各受光部に黒いストローとティッシュペーパーによる拡散部が取り付けられ、土台部におもりとして使用済みボタン電池（黒いテープで絶縁したもの）が載せられている様子が確認できる。

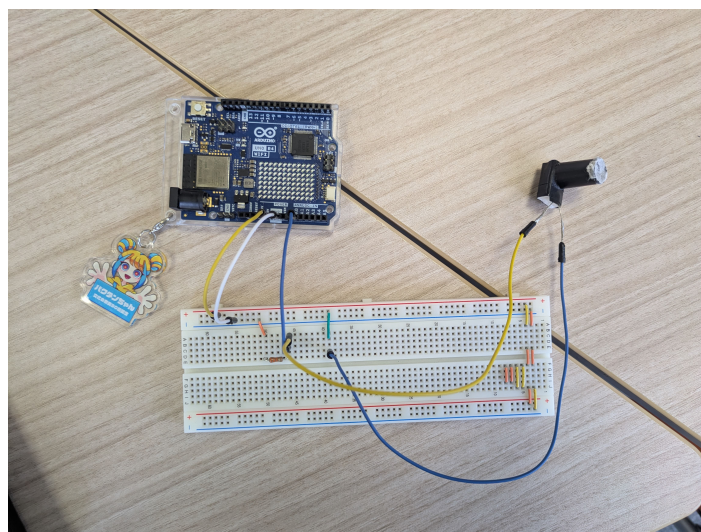


図 36 受光部回路の実装

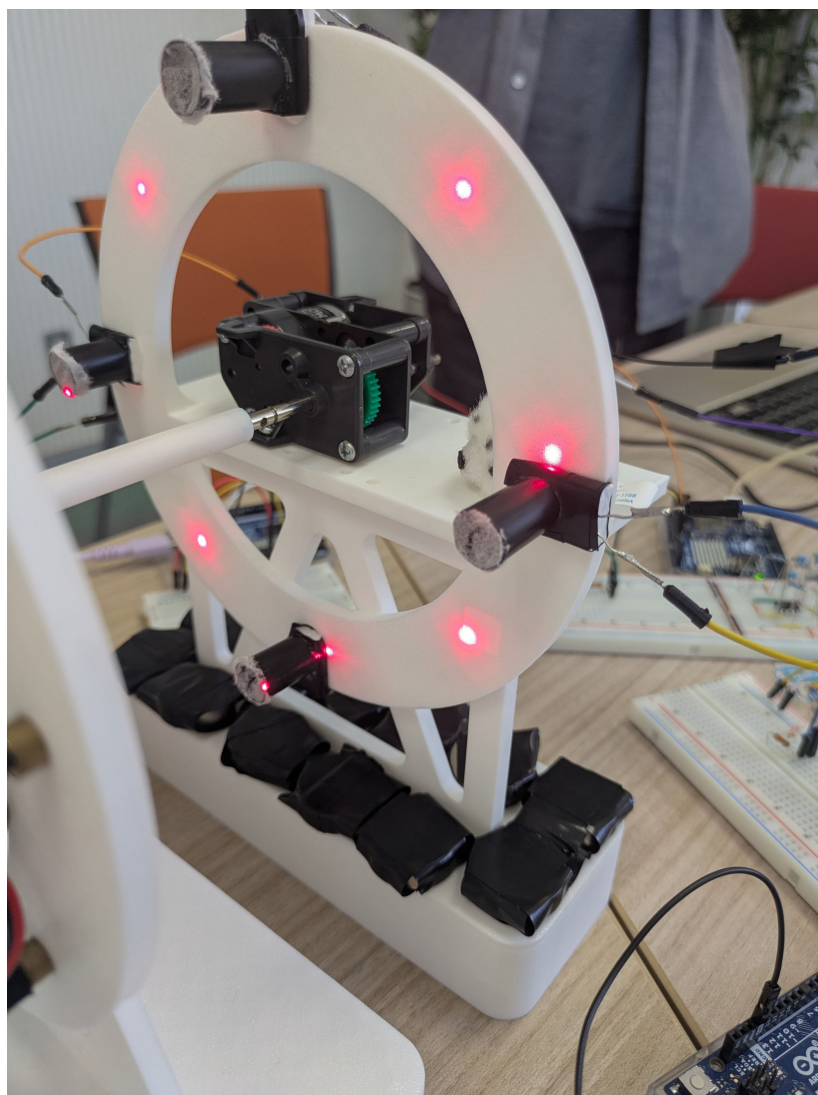


図 37 遮光板リングと受光部の実装

3.2.4 LEDMatrix 表示プログラム

本節では、ソフトウェア面で担当した中継機デバイスの LEDMatrix に現在の BPM を表示する Display モジュール (Display.h, Display.cpp) の実装について、採用した技術の選定理由、実装の過程で直面した技術的課題、およびその解決のために行った技術的貢献を説明する。

はじめに、採用した技術とその選定理由について述べる。第一に、数字の表示には 2.2.3 項で述べた独自定義の 3 × 5 ドットフォントを採用した。これは、8 行 12 列という限られた表示領域に最大 3 桁の BPM 値を収めたうえで、桁間に 1 列の間隔を確保して可読性を保つためであり、汎用フォントライブラリを用いる方式では文字幅が大きく 3 桁を表示できないためである。第二に、フレームバッファの描画方式には、設計当初に予定していた 8 行 12 列の 2 次元配列を直接渡す方式ではなく、96 ビット分の表示内容を 3 個の 32 ビット整数ヘビットパックして転送する方式を採用した。これは、後者の方が描画 API へ渡すデータ量が小さくメモリ効率が高いためである。第三に、受信データの読み込みには、設計当初に予定していた文字列としての読み込みではなく、2 バイトのバイナリ形式のまま読み取り、1 バイト目をヘッダ文字、2 バイト目を段階番号として処理する方式を採用した。これは、2.1.3 項で述べた通信プロトコル自体が固定長 2 バイトのバイナリ形式で定義されており、プロトコル仕様に忠実な実装とするためである。

次に、実装の過程で直面した技術的課題とその解決について述べる。最大の課題は、コンパイルおよび書き込みは正常に完了するにもかかわらず、LEDMatrix に何も表示されないという不具合であった。この問題の切り分けのため、起動時の点灯テストや診断用の描画処理を一時的に追加し、どの段階で表示が失われるかを段階的に確認した。その結果、原因は使用した LEDMatrix 制御ライブラリの内部構造にあることが判明した。本ライブラリの表示バッファはヘッダファイル内で static 変数として定義されているため、ヘッダを取り込んだソースファイルごとに別々の実体生成される。このため、メインスケッチ側で初期化を行い、Display モジュール側で描画を行う構成では、初期化と描画が互いに異なる実体に対して行われ、表示が反映されなかった。この解決として、初期化と描画を含むマトリクスへの全操作を Display.cpp 内へ集約し、初期化用の関数 (displaySetup) を新設してメインスケッチからはこれと呼び出す構成へ変更した。さらに、初期化直後の最初の描画が反映されない場合がある事象に対しては、初期化時に消灯フレームを一度描画してから 200 ms 待機するウォームアップ処理を設けることで、以降の描画を確実に反映させた。この一連の対応により、ライブラリ内部の実装に依存する不具合を、モジュール構成の見直しによって解決した。

また、設計段階の関数構成に対して、実装段階で次の機能追加を行った。演奏終了の指令 (ヘッダ'E') を受信した際に LEDMatrix を全消灯する処理は、通信プロトコル上はヘッダが定義されていたものの、設計段階の表示側関数一覧には対応する処理の記載がなかったため、消灯用の関数 (clearDisplay) を追加して対応した。あわせて、桁単位の描画を行う内部関数 (drawDigit) を分離することで、表示処理の見通しを改善した。

開発の経過は次の通りである。まず表示機能の骨格を作成し、設計書に沿って BPM 表示機能を実装した。この際、通信部を一時的に無効化し、与えた BPM 値の描画のみを単体で検証できる状態とした。その後、前述の無表示問題の切り分けと修正を行い、表示の安定化を確認した。続い

て、演奏終了時の消灯処理を追加するとともに通信部を有効化し、最後に、指揮デバイスの送信先アドレスと一致させるための静的 IP アドレス設定を追加して、結合テストへ移行した。なお、開発段階では表示モジュール単体での検証のために受信処理を仮実装していたが、結合にあたっては、UDP 受信を中継機デバイス本体のプログラム側で行い、Display モジュールは描画専用とする構成に整理した。

3.3 開発支援ツール（生成 AI を含む）の活用と検証

本節では、開発過程で利用した支援ツールおよび情報源と、その利用目的・検証方法について述べる。まず、技術情報源として、Arduino 公式リファレンス、および使用した各部品のデータシートを参照した。部品の定格・接続仕様はデータシートの記載に基づいて決定し、回路実装前に確認した。次に、生成 AI（Anthropic 社の Claude）を、次の 3 つの目的で利用した。第一に、3.2.4 節で述べた LEDMatrix 表示プログラムの実装補助である。第二に、評価データの統計処理およびグラフ生成用 Python スクリプトの作成である。第三に、本稿の LaTeX 原稿の草稿作成・推敲およびレイアウト調整の補助である。LEDMatrix 表示プログラムの実装では、生成 AI を次の場面で利用した。実装の初期段階では、LEDMatrix 制御ライブラリの利用方法や関数の仕様について質問し、実装の方針検討に用いた。ただし、回答に含まれるライブラリ名や関数名が実際の開発環境に存在しないものである場合があったため、回答をそのまま採用せず、Arduino 公式リファレンスと照合して実在する名称・仕様へ修正したうえで、妥当と判断した部分のみを採用した。次に、前述の LEDMatrix に何も表示されない不具合の調査では、症状と構成を提示して原因の候補と切り分け手順の提案を受け、提案された診断手順を実機で順に実行することで、表示バッファの実体がソースファイルごとに分かれるという原因の特定に至った。このとき提案された「マトリクスへの操作を一つのソースファイルへ集約する」という修正方針は、原因の説明として整合することをコード上で確認したうえで採用した。また、演奏終了指令の受信時に全消灯を行う処理の実装方針についても提案を受け、コンパイルと実機での表示確認により動作を検証したうえで採用した。生成 AI の出力は、次の方法で検証したうえで採否を判断した。プログラムに関する出力については、コンパイルの成否、実機での動作、およびシリアルモニタでの出力値の確認により妥当性を検証し、さらにライブラリや関数の仕様に関する記述は Arduino 公式リファレンスと照合して、意図と異なる動作をする生成結果や事実と異なる記述は修正または破棄した。統計処理については、算出された統計量を教員から配布された集計シートの値と照合し、一致することを確認した。また、グラフおよび表の数値は生データからの再計算によって確認した。LaTeX 原稿については、コンパイル結果の出力および図表の配置を確認するとともに、記載内容が実際の実装・測定結果と一致しているかを照合し、事実と異なる記述や根拠のない表現は修正または削除した。いずれの利用においても、生成結果をそのまま採用するのではなく、内容を理解したうえで最終的な採否と修正を判断した。

4 評価方法と結果

4.1 単体テスト

4.1.1 評価方法

単体テストでは、各デバイスを結合する前の段階で、デバイス単位・機能単位での基礎的な処理精度を個別に評価する。はじめに、指揮デバイスの単体テストについて説明する。指揮デバイスは、ボタン判定の制御テストと加速度センサの制御テスト、そして BPM・音量の読み取りテストを実施する。ボタン判定の制御テストとは、タクトスイッチを押下してから中継機デバイスが動作するか確認するテストであり、複数回ボタンを押下し、タクトスイッチを押下してから中継機デバイスが反応し動作するまでの時間を計測することで、ボタン入力から動作開始までの応答遅延が許容範囲内であるか評価する。この時、Nielsen はユーザインタフェースにおける応答時間の閾値として、0.1 秒（即時応答の知覚限界）、1.0 秒（思考の流れを維持できる限界）、10 秒（タスクへの注意を保持できる限界）の 3 段階を提示している [17]。Nielsen によると 10 秒を超える遅延に対しては、ユーザはタスクから注意を逸らし、別の作業を行おうとする傾向があるとされる。この知見に基づき、ボタン押下から中継機デバイスの動作開始までの応答遅延が 10 秒以内である場合を許容範囲内と定義する。さらに、連打や長押しをした場合、意図しない動作をしないか確認する。なお、ボタン押下に対して中継機デバイスが動作する成功率、およびボタン押下から楽器デバイスの楽曲再生開始までの応答遅延は、全デバイスを結合した状態での評価となるため、4.2.1 項で評価する。

次に加速度センサの制御テストについて説明する。加速度センサの制御テストとは、指揮デバイスにおける振り動作の判定に関するテストである。2.1.3 項で述べた通り、加速度センサの制御処理は差分算出、多重検知防止、状態の反転という段階的な検知ロジックを実装している。そこで、検知閾値が正しく設定されているかを確認する。この閾値が正しく設定されていないと、一度の振る動作で複数回検知されたり、振っても検知されないことが起きる。評価方法はシリアルモニタを用いて実測加速度値と判定結果（演奏状態フラグの反転タイミング）を出力し、意図した振り動作に対して正しく検知できているか、また多重検知や停止状態での誤反応が生じていないかを確認する。また、指揮デバイスを振ったと判断されるまでの遅延を計測し、体感として許容範囲であるか評価する。ここで計測する時間とは、指揮デバイスを振ってから中継機デバイスの動作に反映されるまでの応答遅延時間である。この許容範囲についても、ボタン判定の制御テストと同様に Nielsen の示す応答時間の閾値 [17] に基づき、中継機デバイスの動作開始までの応答遅延が 10s 以内である場合を許容範囲内と定義する。

最後に、スライド式可変抵抗器による BPM・音量読み取りテストについて説明する。スライド式可変抵抗器による BPM・音量読み取りテストとは、指揮デバイスのスライド式可変抵抗器を操作し、変更された情報が正確に中継機デバイスへ伝達・反映されるかを確認するテストである。2.1.3 項で述べた通り、BPM・音量読み取り機能は、移動平均処理によりスライド式可変抵抗器の

アナログ値を平滑化したうえで、5段階のBPM・音量の段階番号へ変換する処理を行っている。本テストでは、指揮デバイスを操作して振ってから、中継機デバイスの回転速度が変化するまでの時間を計測するとともに、変更した段階が正しく反映されているかをシリアルモニタで確認する。なお、結合状態での操作反映の成功率は、4.2.1項で評価する。

次に、中継機デバイスの単体テストについて説明する。中継機デバイスは、デバイスの回転制御テストを実施する。これは、最遅BPMであるBPM60の場合と、最早BPMであるBPM180の場合において、モータが止まらずに稼働するかを確認する。加えて、中継機デバイスの物理負荷テストとして、BPM180の場合においても長時間稼働し続けられるか確認する。この時、長時間とは中継機デバイスが回転を開始してから同期をし、一曲演奏し終わるまでの時間と定義する。また、中継機デバイスのArduinoに搭載されているLEDMatrixの表示が現在のBPMを反映しているか確認する。

最後に、楽器デバイスの単体テストについて説明する。楽器デバイスは、レーザー光受光の検知テストと各楽器の音色再現度テストを実施する。2.3.3項で述べた通り、楽器デバイスの受光部はCdSセルの暗状態における出力の推定値を継続的に追従させながら、検知閾値を実測値に基づいて動的に更新する適応閾値方式を採用している。レーザー光受光検知テストでは、レーザー光の点灯・消灯を手動で切り替え、検知パルスの立ち上がりないし立ち下がりタイミングをシリアルモニタで確認することで、安定して受光を検知できているかを評価する。

また、各楽器の音色再現度のテストとは、作成した音源(ストリングス、フルート、ピアノ、ハイハット、キックドラム、スネアドラム)と実際の楽器音との類似度をアンケート調査によって評価するテストである。アンケート方式は、計画書当初は各楽器について3種類の音色パターンを提示し比較する方式を検討していたが、回答者1人あたりの回答時間が長くなり十分なサンプル数を確保することが難しいという問題があったため、実際は提示する実楽器音(参考音)と自作音を1対1で比較する方式に変更し、回答1件あたりの所要時間を短縮した。被験者100名に対し、参考音と自作音を1対1で提示し、両者の類似度を1~5の5段階リッカート尺度(1: 全く似ていない, 2: あまり似ていない, 3: どちらともいえない, 4: ある程度似ている, 5: 非常に似ている)で回答を得た。ここでリッカート尺度とは、質問に対する評価や同意の程度を、あらかじめ段階付けされた選択肢の中から回答者に選択させて数値化する、アンケート調査で広く用いられる回答形式である。

本テストの評価指標には、MOS (Mean Opinion Score) 評価を採用する。MOS評価とは、ITU-T勧告P.800で標準化された主観品質評価手法であり、被験者が音の品質を5段階の絶対範疇尺度(1: Bad~5: Excellent)で評定し、その代表値によって品質を判定するものである。MOS評価は音声・音響分野における主観品質の標準的な評価手法として広く用いられており、本テストのように「合成音の実楽器音にどれだけ近いか」という聴感上の品質を被験者の評定によって定量化する目的に整合する。MOS評価において評定値4は「Good (良い)」に相当し、実用上十分な品質と判断される水準である[20]。

MOP (Measure of Performance : 性能指標) とは、システムが要求される性能をどの程度満たしているか定量的に評価するための指標である。本プロジェクトでは、まずシステムの成果物が目的に対してどのような効果を発揮すべきかを示す上位の定性的な指標として、MOE(Measure of

Effectiveness：効果指標)を定義している。音色再現に関する MOE は、「音色の再現ができていないか」であり、MOP はこの MOE を客観的に判定するために設定した具体的な数値基準である。本テストでは、MOS 評価にならない、MOP の目標値を「回答分布の中央値および最頻値がともに 4.0 以上」とする。なお、リッカート尺度は順序尺度であり、評定値間の間隔が等しいことは保証されないため、回答分布の代表値としては平均値ではなく中央値と最頻値を用いる。また、回答分布の散らばりを評価するため、補間を行わない経験分布関数に基づく四分位数（第 1 四分位数 Q_1 、第 3 四分位数 Q_3 ）および四分位範囲（IQR）を併せて算出する。ここで経験分布関数とは、理論上の分布を仮定せず、得られた標本のみから各値以下の回答が占める割合として定義される分布関数である。四分位数とは、回答を小さい順に並べたときに下位から 25%の位置にある値を第 1 四分位数 Q_1 、75%の位置にある値を第 3 四分位数 Q_3 と呼ぶものであり、四分位範囲は Q_3 と Q_1 の差として回答の中央 50%が収まる幅を表す。

単体テストで評価する項目と MOP の目標値を表 12 に示す。

表 12 単体テストで評価する性能指標 (MOP)

テスト項目	評価方法と MOP 目標値
指揮デバイス：ボタン判定の制御テスト	ボタン押下から中継機デバイスの動作開始までの応答遅延が 10s 以内 [17] であること、連打・長押し時の誤動作がないこと
指揮デバイス：加速度センサの制御テスト	シリアルモニタによる実測加速度値・判定結果（演奏状態フラグの反転タイミング）の確認、多重検知や停止状態での誤反応がないこと、振ってから中継機デバイスの動作に反映されるまでの応答遅延が 10s 以内 [17] であること
指揮デバイス：BPM・音量読み取りテスト	シリアルモニタによるアナログ値・変換後段階番号の確認、スライド式可変抵抗器操作から中継機デバイスの回転速度変化までの時間の計測および段階反映の確認
中継機デバイス：回転制御テスト	最遅 BPM (BPM60) および最早 BPM (BPM180) のいずれにおいてもモータが停止せず稼働すること
中継機デバイス：物理負荷テスト	BPM180 において、回転開始から同期・1 曲演奏終了までの間、稼働を継続できること
中継機デバイス：LEDMatrix 表示テスト	表示内容が現在の BPM と一致していること
楽器デバイス：レーザー光受光の検知テスト	レーザー光の点灯・消灯に対するパルス検知タイミングの確認（定性的な動作確認であり数値目標は設定していない）
楽器デバイス：音色再現度テスト	被験者アンケート（1～5 の 5 段階評価）に対する MOS 評価 [20] に基づき、回答分布の中央値および最頻値がともに 4.0 以上であること

4.1.2 結果

はじめに、指揮デバイスの単体テストの結果について説明する。まず、ボタン判定の制御テストの結果を述べる。各 BPM 設定において複数回のボタン押下を行い、ボタン押下から中継機デバイスが回転を開始するまでの応答時間を計測した結果を表 13 に示す。全 BPM 設定を通じて、計測された応答時間は最小 510 ms、最大 2030 ms の範囲に収まり、全試行が約 2.1 s 以内で応答した。全体の平均応答時間は 1005.8 ms（標準誤差 30.0 ms）であった。これは Nielsen が提示する応答時間の閾値 10s [17] の約 10 分の 1 であり、全試行がこの基準を満たした。また、連打や長押しを行った場合においても、中継機デバイスが意図しない動作を行う事象は確認されなかった。なお、表 13

および以降の結果に示す標準誤差とは、標本から求めた平均値が母集団の真の平均値に対してどの程度ばらつき得るかを表す推定精度の指標であり、標本の標準偏差 s と試行回数 n を用いて式 3 で算出される。

$$SE = \frac{s}{\sqrt{n}} \quad (3)$$

表 13 BPM 別ボタン押下から回転開始までの応答時間

BPM	試行回数	平均値 [ms]	標準誤差 [ms]	中央値 [ms]	最小値 [ms]	最大値 [ms]
60	16	1231.9	81.6	1155.0	840	1950
90	18	914.4	36.4	900.0	660	1140
120	23	884.8	37.6	890.0	510	1220
150	14	1053.6	103.4	885.0	730	2030
180	14	1015.7	49.1	990.0	740	1410
全体	85	1005.8	30.0	950.0	510	2030

次に、加速度センサの制御テストの結果を述べる。シリアルモニタへ実測加速度値と判定結果（演奏状態フラグの反転タイミング）を出力して確認したところ、意図した振り動作に対して検知が行われ、一度の振り動作に対する多重検知や、停止状態での誤反応は確認されなかった。

最後に、BPM・音量読み取りテストの結果を述べる。スライド式可変抵抗器の操作に対するアナログ値と変換後の段階番号をシリアルモニタで確認したところ、全ての試行において、操作した段階が正しく読み取られ、中継機デバイスの回転速度へ反映されることが確認された。

以上の指揮デバイスの単体テストの結果と、表 12 に示した MOP 目標値に基づく判定結果を表 14 にまとめる。指揮デバイスの単体テストで評価した全項目について、MOP を満たすことが確認された。

表 14 指揮デバイス単体テストの結果

テスト項目	MOP 目標値	結果	判定
ボタン判定：応答遅延	中継機デバイスの動作開始まで 10s 以内 [17]	全 85 試行が 2030 ms 以内（平均 1005.8 ms）	達成
ボタン判定：連打・長押し	意図しない動作をしないこと	誤動作は確認されず	達成
加速度センサ：振り動作の検知	多重検知・停止状態での誤反応がないこと	多重検知・誤反応は確認されず	達成
BPM・音量読み取り	変更した段階が正しく反映されること	全試行で意図した段階を読み取り、回転速度へ反映	達成

次に、中継機デバイスの単体テストの結果について説明する。回転制御テストでは、最遅 BPM である BPM60 および最早 BPM である BPM180 のいずれにおいても、モータが停止することなく安定して稼働することが確認された。また、物理負荷テストにおいても、BPM180 の条件下で回転開始から同期・1 曲演奏終了までの間、中継機デバイスが継続して稼働できることが確認された。さらに、LEDMatrix 表示テストでは、表示内容が現在の BPM と一致していることが確認された。以上より、中継機デバイスの単体テストで評価した全項目について、MOP を満たすことが確認された。

続いて、楽器デバイスの単体テストのうち、レーザー光受光の検知テストの結果について説明する。レーザー光の点灯・消灯を手動で切り替え、検知パルスの立ち上がり・立ち下りのタイミングをシリアルモニタ上で確認したところ、暗状態の出力の推定値に基づく適応閾値方式により、安定して受光を検知できることが確認された。

続いて、各楽器の音色再現度テストの結果について説明する。被験者 100 名に対する回答の度数分布を表 15 に示す。また、回答割合を視覚化した 100%積み上げ棒グラフを図 38 に、各楽器・各評価スコアの回答人数を示すヒストグラムを図 39 に示す。

表 15 各楽器に対する回答の度数分布 (N = 100)

楽器	1	2	3	4	5
	全く似ていない	あまり似ていない	どちらともいえない	ある程度似ている	非常に似ている
ストリングス	0	5	5	42	48
フルート	0	11	5	36	48
ピアノ	3	13	13	43	28
ハイハット	1	5	5	27	62
キックドラム	3	11	12	49	25
スネアドラム	1	5	15	49	30

図 38, 図 39 の通り、ハイハットは最も良い評価 5 の回答が 62% と最も高い割合を示し、評価 4 以上の回答が合計 89% を占めた。ストリングスおよびフルートにおいても評価 4 以上の割合はそれぞれ 90%, 84% であり、高い類似度が認められた。一方、キックドラムでは評価 3 以下の回答が 26% を占め、6 楽器中で最も低い類似度の分布を示した。ピアノについても評価 3 以下の割合が 29% と比較的高い値を示した。

各楽器の中央値・最頻値および四分位数 (Q_1 , Q_3 , IQR) と、MOP の目標値 (中央値および最頻値がともに 4.0 以上) に基づく判定結果を表 16 に示す。また、中央値・最頻値を MOP 目標値とともに視覚化した棒グラフを図 40 に示す。

表 16 に示す通り、全 6 楽器において中央値および最頻値はともに 4.0 以上であり、MOS 評価に基づく MOP は全楽器で達成された。特にハイハットは中央値・最頻値がともに 5 であり、6 楽器中で最も高い評価を得た。ストリングスおよびフルートは最頻値が 5 を示し、高い再現性が確認された。一方、回答分布の散らばりに着目すると、ピアノおよびキックドラムは Q_1 が 3 であり、回答の 4 分の 1 以上が評価 3 以下に分布している。特にピアノは IQR が 2 と 6 楽器中で最も大きく、

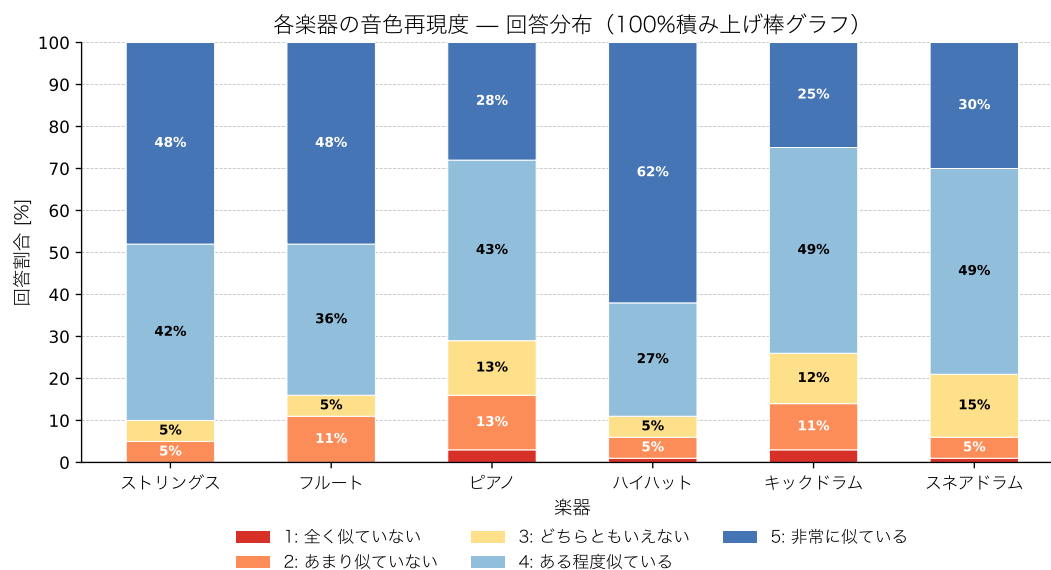


図 38 各楽器の音色再現度に対する回答分布（100%積み上げ棒グラフ）

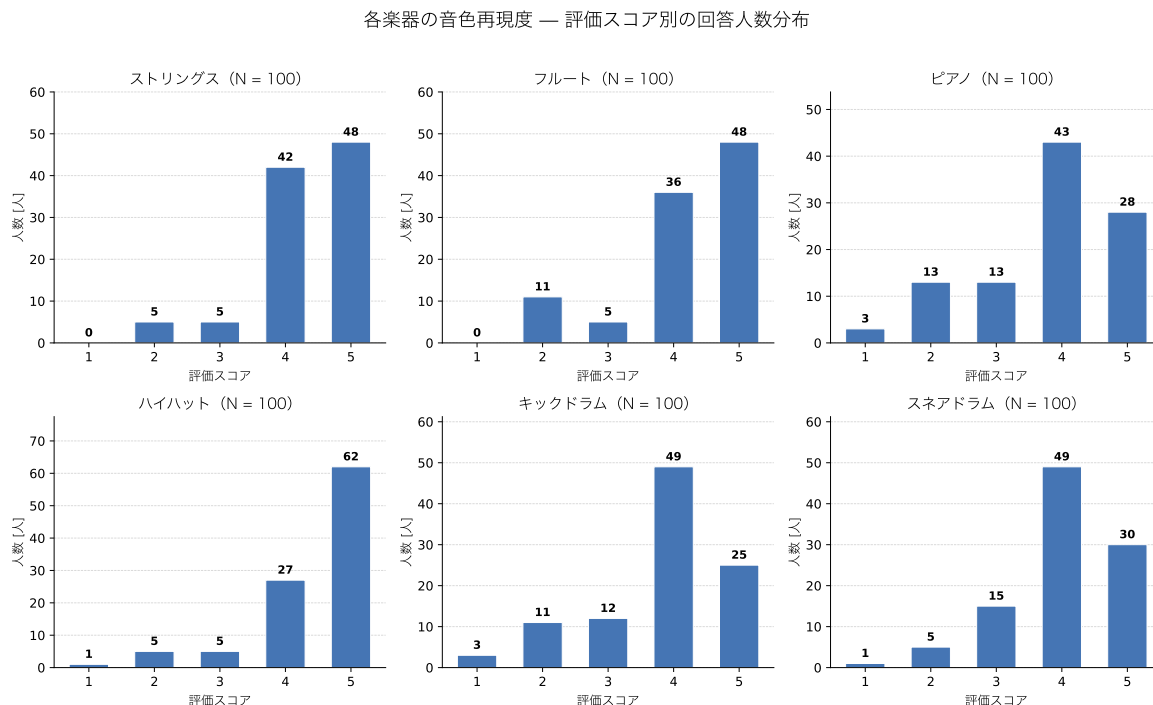


図 39 各楽器の評価スコア別の回答人数分布

表 16 各楽器の MOS 評価と MOP 判定 (N = 100)

楽器	中央値	最頻値	Q ₁	Q ₃	IQR	MOP 判定
ストリングス	4	5	4	5	1	達成
フルート	4	5	4	5	1	達成
ピアノ	4	4	3	5	2	達成
ハイハット	5	5	4	5	1	達成
キックドラム	4	4	3	4	1	達成
スネアドラム	4	4	4	5	1	達成

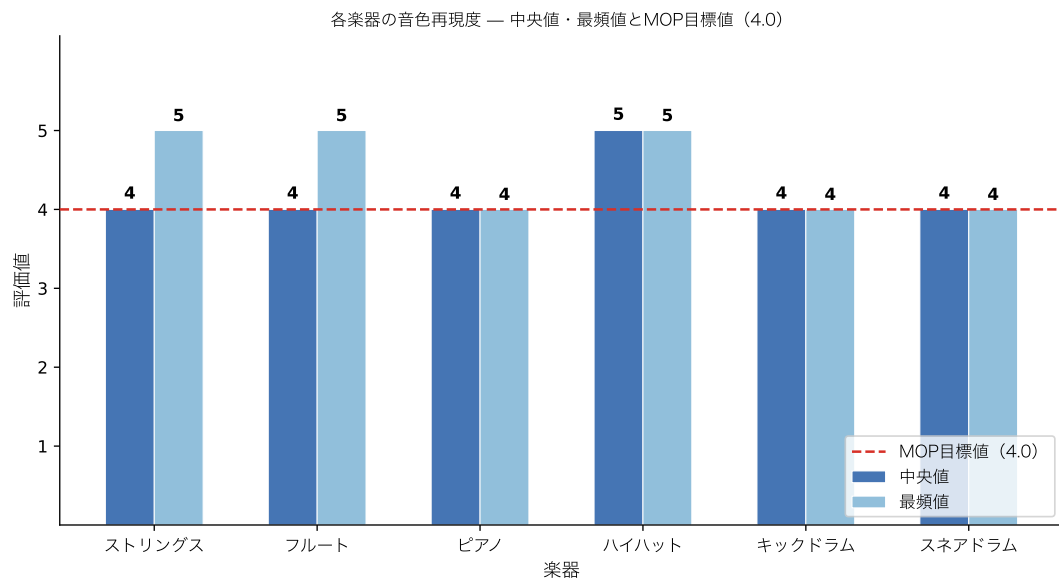


図 40 各楽器の中央値・最頻値と MOP 目標値の比較

被験者間で評価が分かれたことを示している。すなわち、MOP の目標値は全楽器で達成されたものの、ピアノおよびキックドラムについては他の 4 楽器と比較して低評価側への分布の広がり認められた。

4.2 結合テスト

4.2.1 評価方法

結合テストでは、中継機デバイス本体のハードウェア実装と回転制御プログラム、楽器デバイスの受光部と楽曲表現プログラム、および指揮デバイスからの BPM・音量指令を行う通信部を結合した段階で評価を行う。本システムは、指揮デバイスと観覧車型の中継機・楽器デバイスを用いた空間的なフィードバックにより、利用者に高い没入感を与えることを目的として設計されている。そのため、結合テストでは、利用者がこの空間的フィードバックを違和感なく体験できているかという知覚・体験の観点から、「輪唱のタイミングに対して不快に感じる程度の遅延がないか」、「ユーザの操作が正しく反映されるか」、「エンターテインメント性があるか」の 3 点を MOE として定める。これらは、本システムが技術的に動作するだけでなく、利用者の体験として意図した効果を実際に発揮しているかを確認するために設定したものである。

まず、輪唱のタイミングに対して不快に感じる程度の遅延がないかについて評価する。これは、各楽器デバイスの発音タイミングにズレが生じた際、利用者がそれを違和感なく 1 つの輪唱として聴取できるかを確認するものである。MOP の目標値は、ハース効果の許容限界である 50 ms 未満とする。ハース効果とは、時間差を伴う類似した 2 つの音を、一定時間内であれば聴覚が 1 つの音として知覚する現象である [14]。

なお、2.3.3 項で述べた同期通信である、演奏開始時刻を事前に指定して伝える 'O' 通信の冗長送信等は、この MOP を満たすことを目的として実装されたものであり結合テストではこの機構が実際に機能しているかを合わせて確認する。

また、これに関連する評価として、指揮デバイスのボタンを押下してから実際に楽器デバイスの PC から楽曲が再生されるまでの応答遅延時間を計測し、許容範囲内であるか評価する。演奏開始は、中継機デバイスの回転開始後に実行されるレーザー受光による BPM 推定や楽器デバイス間の同期補正といった処理を経た最終応答であり、全デバイスを結合した状態ではじめて評価できる項目である。ユーザは中継機が回転している様子を見ることでシステムが処理中であることを視覚的に認知するため、待機に対する許容時間は 4.1.1 項で述べた Nielsen の閾値よりも延長される。一方で、Tan らは HCI および認知心理学における確立された遅延閾値として、60 秒に達するとユーザは注意を完全に離脱させるか、システムの故障やコンテンツ品質の低さといった否定的な解釈を行う傾向があることを示している [18]。この知見に基づき、ボタン押下から楽曲再生開始までの応答遅延が 60 秒を超過した場合は異常として打ち切り、60 秒以内である場合を許容範囲内と定義する。

次に、ユーザの操作が正しく反映されるかについて評価する。評価方法としては、4.1.1 項で述べたボタン判定の制御テストおよび BPM・音量読み取りテストと同様に、各デバイスを結合した状態でボタン押下から中継機デバイスの動作反映までの成功率、および指揮棒の操作による BPM・

音量変更が実際に発音へ反映される成功率を計測する。MOP の目標値は、処理成功率 99.0%以上とする。この 99%（ツーナイン）という基準は、一般的な Web サービス等の開発環境において目標とされる稼働率の数値を参考としたものであり [15]，本プロジェクトで作成するシステムはブレッドボード上で稼働する一つのシステムであるため、テスト環境での運用に近いことから、この数値を目標として採用した。

次に、エンターテインメント性があるかについて評価する。評価方法としては、同様にアンケート調査を実施する。具体的には、「観覧車の回転と楽器が演奏をしている様子を見て、空間的な楽しさや驚きに満足したか?」、「指揮棒を振ったり、可変抵抗器を操作して、演奏のテンポなどがリアルタイムに変化する操作感到満足したか?」、および「光と音の動きが一体になったアトラクションとして、総合的な楽しさに満足したか?」という 3 つの質問を、1（不満）～5（大変満足）の 5 段階リッカート尺度で回答してもらう。評価指標には CSAT スコアを用いる。CSAT スコアとは、顧客満足度を測定するための指標であり、全回答のうち 5 段階評価で 4 以上の高評価を付けた回答者が占める割合として算出される。MOP の目標値は、一般的な企業の顧客満足度調査の平均値を参考として、CSAT スコアが 75%以上 [16] とする。また、CSAT スコアはアンケートという有限のサンプルから得られた比率の点推定値であり、サンプルサイズに依存した推定の不確実性を伴う。ここで点推定とは、母集団における真の値を標本から単一の数値として推定することである。そこで、点推定のみによる判定（非標準化）に加えて、95%Wilson 信頼区間 [19] の下限値による判定（標準化）を併用する。信頼区間とは、標本から推定した値が母集団において取り得る範囲を、一定の確からしさを持つ区間として示すものであり、Wilson 信頼区間とは比率に対する信頼区間の算出法の一つで、標本数が少ない場合や比率が 0%・100%に近い場合でも 0%から 100%の範囲に収まる妥当な区間が得られる特長を持つ。

結合テストで評価する項目と MOP の目標値を表 17 に示す。

表 17 結合テストで評価する性能指標（MOP）

評価項目	評価方法と MOP 目標値
輪唱タイミングの遅延	演奏開始から発音までの時間および楽器間の演奏ズレをミリ秒単位で計測。50 ms 未満 [14]
楽曲再生開始までの応答遅延	ボタン押下から楽器デバイスの楽曲再生開始までの時間を計測。60 s 以内 [18]
ユーザ操作の反映度	ボタン押下からの動作反映成功率、および BPM・音量変更の発音反映成功率を計測。処理成功率 99.0%以上 [15]
エンターテインメント性	空間的な楽しさ等に関するアンケート調査（5 段階リッカート尺度）。CSAT スコア 75%以上 [16]

4.2.2 結果

まず、輪唱のタイミングに対して不快に感じる程度の遅延がないかについて評価する。同期開始条件である、楽器間の同期が確立したと判定するために許容される同期ズレの閾値を 50 ms 以下とした場合、一度同期が確立した後の楽器間の演奏タイミングのズレは MOP の目標値である 50 ms 未満に収まり、輪唱のタイミングに関する MOP は満たされることが確認された。しかしながら、後述する楽曲再生開始までの応答遅延の測定において、同期開始条件を 50 ms 以下と厳格に設定すると、同期の確立自体に時間を要し、全 53 回の試行のうち 8 回が 60 s の閾値を超過するという結果が得られている。したがって、輪唱タイミングのズレそのものに関する MOP は満たされる一方、そのタイミングを実現するまでの過程においては応答遅延に関する MOP を満たせない場合があることに留意する必要がある。

次に、指揮デバイスのボタン押下から実際に楽器デバイスの PC で楽曲が再生されるまでの応答遅延について評価する。本測定は、同期開始条件を 60 ms 以下とした場合と 50 ms 以下とした場合の 2 条件で実施した。測定結果を表 18 および表 19 に示す。

同期開始条件を 60 ms 以下とした場合、全 25 回の試行すべてにおいて楽曲再生が正常に開始され、成功率は 100%であった。応答遅延は最小 4920 ms、最大 20130 ms の範囲に収まり、全体の平均応答遅延は 9054.0 ms（標準誤差 736.5 ms）であった。

表 18 同期開始条件を 60 ms 以下とした場合のボタン押下から楽曲再生開始までの応答遅延

BPM	試行回数	平均値 [ms]	標準誤差 [ms]	中央値 [ms]	最小値 [ms]	最大値 [ms]
60	5	7816.0	355.2	7680.0	6950	8950
90	5	6100.0	399.9	5870.0	5070	7250
120	5	9360.0	523.7	9700.0	7830	10910
150	5	9768.0	1906.5	8630.0	5000	16500
180	5	12226.0	2616.8	12210.0	4920	20130
全体	25	9054.0	736.5	8230.0	4920	20130

一方、同期開始条件を 50 ms 以下とした場合、全 53 回の試行のうち 8 回が（60 s）を超過しても楽曲再生が開始されず、全体の成功率は 84.9%にとどまった。特に BPM150 では成功率が 50.0%まで低下しており、同期条件を厳格化すると、60 s という応答遅延の MOP 目標値を満たせない試行が生じることが確認された。

これらの結果から、本システムでは同期開始条件を 60 ms 以下に設定することで、指揮デバイスのボタン押下から楽曲再生開始までの応答遅延が MOP の目標値を安定して満たすことが確認された一方、50 ms 以下という厳格な条件では、楽器間の同期の確立自体に長時間を要し、MOP を満たせない試行が生じることが明らかになった。

表 19 同期開始条件を 50 ms 以下とした場合のボタン押下から楽曲再生開始までの応答遅延

BPM	試行回数	60 s 超過	成功率	平均値 [ms]	標準誤差 [ms]	最小値 [ms]	最大値 [ms]
60	8	2	80.0%	21065.0	5802.5	8330	50110
90	9	1	90.0%	22417.8	5013.0	10140	58510
120	11	0	100.0%	15430.0	2837.3	7720	38640
150	5	5	50.0%	22872.0	7890.2	8570	52600
180	12	0	100.0%	15339.2	2842.5	6010	31930
全体	45	8	84.9%	18632.0	1933.4	6010	58510

次に、ユーザの操作が正しく反映されるかについて評価する。各デバイスを結合した状態で全 160 回のボタン押下試行を行ったところ、158 回で中継機デバイスが正常に動作し、成功率は 98.8% であった。また、指揮デバイスを振ることによる BPM・音量の変更についても、全ての試行において意図した段階が正しく反映され、多重検知や停止状態での誤反応は確認されなかった。以上より、ボタン押下による操作反映の成功率は、MOP の目標値である処理成功率 99.0% 以上をわずかに下回ったものの、指揮棒操作による BPM・音量変更の反映については誤動作なく機能しており、全体としてはユーザの操作がほぼ意図通りに反映されることが確認された。

続いて、エンターテインメント性があるかについて評価する。被験者 18 名に対する回答の度数分布を表 20 に、各質問・各評価スコアの回答人数を示すヒストグラムを図 41 に示す。

表 20 エンターテインメント性アンケートに対する回答の度数分布 (N = 18)

質問項目	1	2	3	4	5
	不満	(未使用)	どちらともいえない	満足	大変満足
空間的な楽しさや驚き	0	0	0	5	13
操作感	1	0	1	9	7
総合的な楽しさ	0	0	1	4	13

エンターテインメント性アンケート — 評価スコア別の回答人数分布

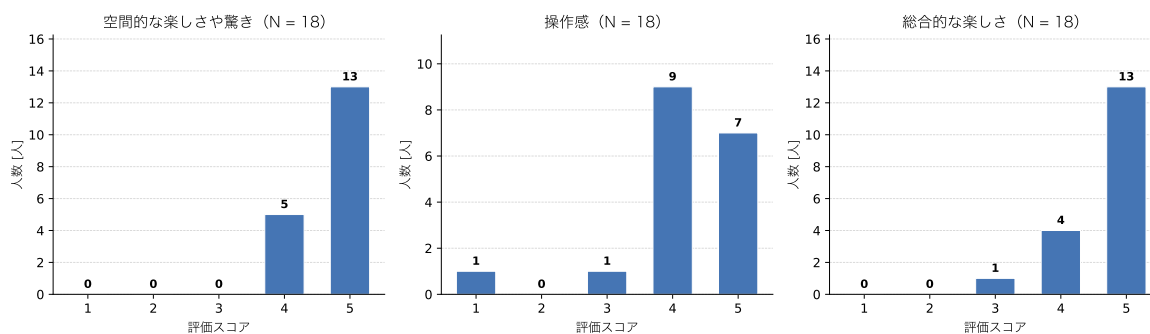


図 41 エンターテインメント性アンケートの評価スコア別の回答人数分布

CSAT スコアは、非標準化および標準化の両方で算出した。算出結果を表 21 に、非標準化の結果を図 42 に、標準化の結果を図 43 に示す。表中の Top2Box 人数とは、5 段階評価のうち上位 2 段階である 4 または 5 を選択した回答者の人数を指す。

表 21 エンターテインメント性アンケートの CSAT スコアと MOP 判定結果 (MOP 目標値 75%)

質問項目	Top2Box 人数	CSAT スコア [%]	95%Wilson 信頼区間 [%]	MOP 判定 (非標準化)	MOP 判定 (標準化)
空間的な楽しさや驚き	18/18	100.0	[82.4, 100.0]	達成	達成
操作感	16/18	88.9	[67.2, 96.9]	達成	未達成
総合的な楽しさ	17/18	94.4	[74.2, 99.0]	達成	未達成

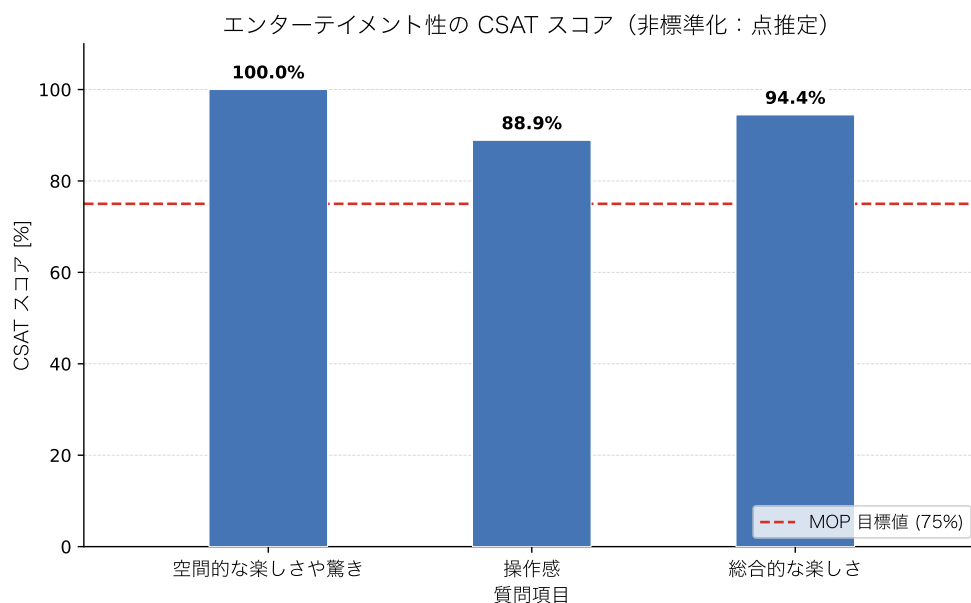


図 42 エンターテインメント性の CSAT スコア (非標準化：点推定) と MOP 目標値 (75%) の比較

表 21 に示す通り、非標準化 (点推定) で判定した場合、3 項目すべてが MOP の目標値である 75%以上を達成した。特に「空間的な楽しさや驚き」については 18 名全員が 4 (満足) または 5 (大変満足) と回答し、CSAT スコアは 100.0%であった。

一方、標準化で判定した場合、MOP を達成したのは「空間的な楽しさや驚き」の 1 項目のみであった。「操作感」および「総合的な楽しさ」は、点推定ではそれぞれ 88.9%、94.4%であったが、95%信頼区間の下限値はそれぞれ 67.2%、74.2%であり、いずれも 75%の目標値を下回った。

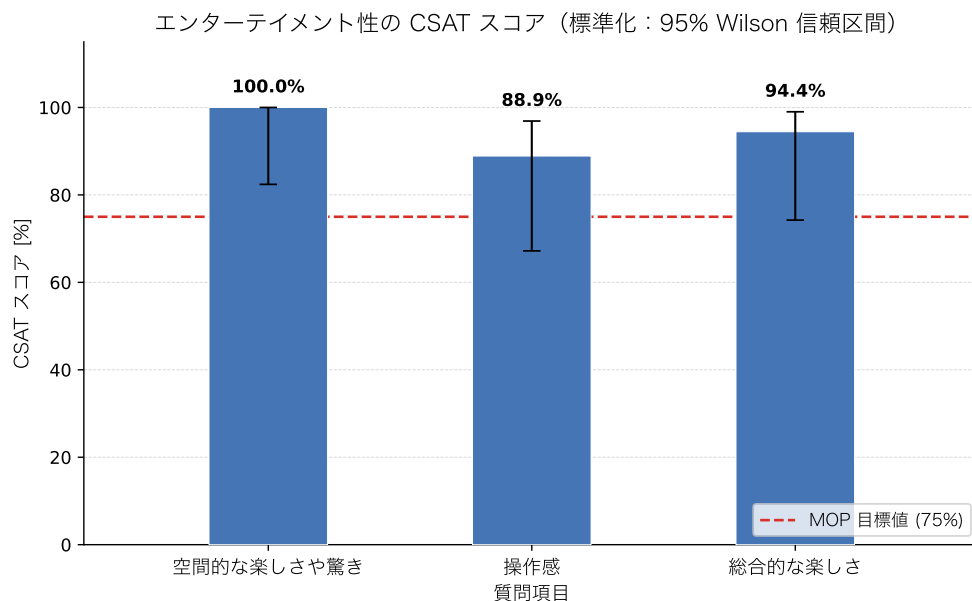


図 43 エンターテインメント性の CSAT スコア（標準化：95%Wilson 信頼区間）と MOP 目標値（75%）の比較

最後に、ハッカソン参加者に共通で配布されたアンケートの結果を示す。本アンケートは本チームの評価方法とは独立に実施されたものであり、「没入感・臨場感」, 「一体感・同期の心地よさ」, 「演出の華やかさ」, 「楽しさ・高揚感」の4項目を、7段階評価（7が最高評価）で問うものである。本チーム（チーム 40）に対する回答数は $n = 26$ 、全教室の総回答数は $n = 1671$ である。本アンケートは本チームが設定した MOP の判定には用いず、結果の統計的な性質と全教室平均との比較を示したうえで、これらに基づく考察は 5.5 項で述べる。

回答分布の統計的性質 チーム 40 に対する回答の生データから算出した記述統計量および Shapiro-Wilk 検定 [21] の結果を表 22 に示す。記述統計量とは、平均値や中央値のように、データの分布の特徴を要約して示す統計量の総称である。表 22 は、各評価項目について、分布の位置（平均値・中央値・最頻値）、散らばり（SD・四分位数・IQR）、および分布の形状をまとめたものである。SD とは標準偏差のことであり、データが平均値の周りにどの程度散らばっているかを表す指標である。形状の指標のうち、歪度は分布の左右非対称性を表す指標であり、負の値は低評価側に裾が長いことを示す。尖度は分布の中央への集中度と裾の重さを表す指標であり、正の値は正規分布より尖った分布、負の値は平坦な分布を示す。

表 22 参加者共通アンケートの記述統計量と Shapiro-Wilk 検定の結果（チーム 40, $n = 26$ ）

評価項目	平均値	SD	中央値	最頻値	Q_1	Q_3	IQR	歪度	尖度	W	p 値
没入感・臨場感	5.54	1.14	5.5	5・7	5	6.75	1.75	-0.01	-1.39	0.859	0.002
一体感・同期の心地よさ	5.00	1.60	6	6	4	6	2	-0.89	0.08	0.877	0.005
演出の華やかさ	6.35	0.80	6.5	7	6	7	1	-1.25	1.57	0.759	< 0.001
楽しさ・高揚感	6.27	1.00	7	7	6	7	1	-1.37	0.95	0.716	< 0.001

また、回答分布を視覚化した箱ひげ図を図 44 に示す。図中の凡例に示す通り、箱は四分位範囲 ($Q_1 \sim Q_3$ 、回答の中央 50%が収まる範囲)、太線は中央値、ひげは外れ値を除く最小値・最大値、点 は外れ値を表す。図 44 からは、「演出の華やかさ」と「楽しさ・高揚感」の箱が尺度上限の 7 に接して高評価側に密集していること、「一体感・同期の心地よさ」の箱とひげが 4 項目中最も低評価側まで広がっていることが読み取れる。

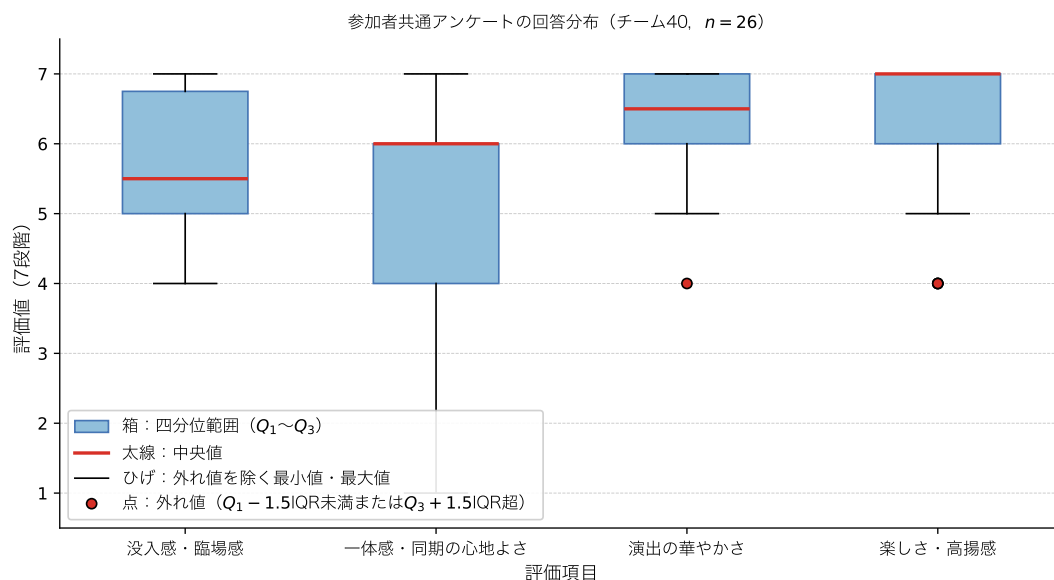


図 44 参加者共通アンケートの回答分布の箱ひげ図（チーム 40、 $n = 26$ ）

ここで、Shapiro-Wilk 検定とは、「標本が正規分布に従う母集団から抽出された」という帰無仮説を検定する正規性検定であり、1 に近いほど正規分布が高い検定統計量 W （1 に近いほど正規分布に近い）と p 値によって判定する [21]。帰無仮説とは、差や効果が存在しないことを想定して立てられ、検定によって棄却されるか否かが判断される仮説である。また p 値とは、帰無仮説が正しいと仮定したとき、実際に得られたデータと同等かそれ以上に極端な結果が偶然生じる確率であり、あらかじめ定めた有意水準（本稿では 5%）を下回る場合に帰無仮説は棄却される。本項でこの検定を用いる理由は 2 つある。第一に、 $n = 26$ という小標本ではヒストグラム等の目視による分布形状の判断が主観的になりやすいのに対し、Shapiro-Wilk 検定は小標本に対しても比較的高い検出力を持つ客観的な判定手段だからである。検出力とは、実際に差や分布の逸脱が存在するときに、検定がそれを正しく検出できる確率である。第二に、後述する群間比較・項目間比較において、平均値に基づくパラメトリック検定と順位に基づくノンパラメトリック検定のどちらが適切かを選択する根拠となるからである。パラメトリック検定とは、母集団が正規分布などの特定の分布に従うことを仮定して行う検定手法であり、ノンパラメトリック検定とは、データの値そのものではなく順位に基づいて検定を行うことで、母集団の分布形状の仮定を必要としない検定手法の総称である。

検定の結果、表 22 に示す通り 4 項目すべてにおいて $p < 0.05$ であり、回答分布が正規分布に従うという帰無仮説は棄却された。歪度・尖度と対応づけると、各項目の分布の崩れ方は異なってい

る。「演出の華やかさ」と「楽しさ・高揚感」は歪度がそれぞれ -1.25 , -1.37 と強い負の歪みを示している。これは天井効果、すなわち回答が尺度の上限付近に集中し、上限を超える評価を表現できないために分布が上限で頭打ちとなり、低評価側にのみ裾が伸びる現象が生じていることを示す。天井効果が生じた項目では、尺度上限を超える満足度の差を測定できないため、項目が持つ本来の評価の高さを平均値が過小に表現する可能性がある。実際に、これらの項目では評価4の回答が箱ひげ図上の外れ値として検出されている。「没入感・臨場感」は歪度がほぼ0である一方、尖度が -1.39 と大きく負であり、5と7に最頻値を持つ平坦な分布である。「一体感・同期の心地よさ」は中央値6に対して Q_1 が4、最小値が1であり、低評価側への裾の広がり（歪度 -0.89 ）が大きい。

この分布形状は、Q-Q プロットによっても確認できる。Q-Q プロット（Quantile-Quantile Plot：分位数-分位数プロット）とは、手元のデータが特定の確率分布（ここでは正規分布）にどの程度従っているかを視覚的に確認するためのグラフである。縦軸には実測値を小さい順に並べ替えた値をとり、横軸には「データが完全に正規分布に従っていた場合に、その順位の値が理論的に位置するはずの値」（理論分位数）をとる。ここで累積分布関数とは、確率変数がある値以下の値をとる確率を表す関数である。 n 個のデータのうち i 番目に小さい実測値に対応する理論分位数 q_i は、標準正規分布の累積分布関数の逆関数 Φ^{-1} を用いて式4で求められる。

$$q_i = \Phi^{-1}\left(\frac{i - 0.5}{n}\right) \quad (4)$$

データが正規分布に従っていればプロットされた点は直線上に並び、直線からの系統的な逸脱は分布の歪みの存在とその方向を示す。各項目の Q-Q プロットを図45に示す。図中の直線は、各項目の平均値と標準偏差を持つ正規分布に従う場合の期待直線である。

図45より、「演出の華やかさ」と「楽しさ・高揚感」は高評価側で実測値が期待直線から下方に折れ曲がっており、尺度上限の7で頭打ちとなる天井効果の形状が明瞭である。「没入感・臨場感」は同一評価値の点が横に並ぶ階段状の平坦な区間が広く、5と7に最頻値を持つ平坦な分布（尖度 -1.39 ）に対応する。「一体感・同期の心地よさ」は低評価側で実測値が期待直線から下方へ大きく逸脱しており、低評価側への裾の広がり（最小値1）を反映している。

以上のように正規性が棄却され、かつ回答が7段階の順序尺度であることから、以降の分析では代表値に中央値・最頻値を併記し、群間・項目間の比較にはノンパラメトリック検定を用いる。

全教室平均との比較 チーム40と全教室の平均値の比較を図46に示す。図46は、評価項目ごとにチーム40（ $n = 26$ ）と全教室（ $n = 1671$ ）の平均値を棒の高さで示し、回答のばらつき（平均 $\pm 1SD$ ）をエラーバーで表したものである。「演出の華やかさ」と「楽しさ・高揚感」ではチーム40の棒が全教室を大きく上回るとともにエラーバーが短く、高い評価が回答者間で安定していたことが読み取れる。

群間の差の検定には、Welch の t 検定 [22] と Mann-Whitney の U 検定を併用した。 t 検定とは、2つの群の平均値の差を、その差の推定誤差で割った統計量 t が t 分布に従うことを利用して、平均値に差があるかを調べる検定手法である。そのうち Welch の t 検定は、2群の分散が等しいという仮定を置かない方式であり、本比較のように2群のサンプル数（26と1671）とSDが大きく異なる

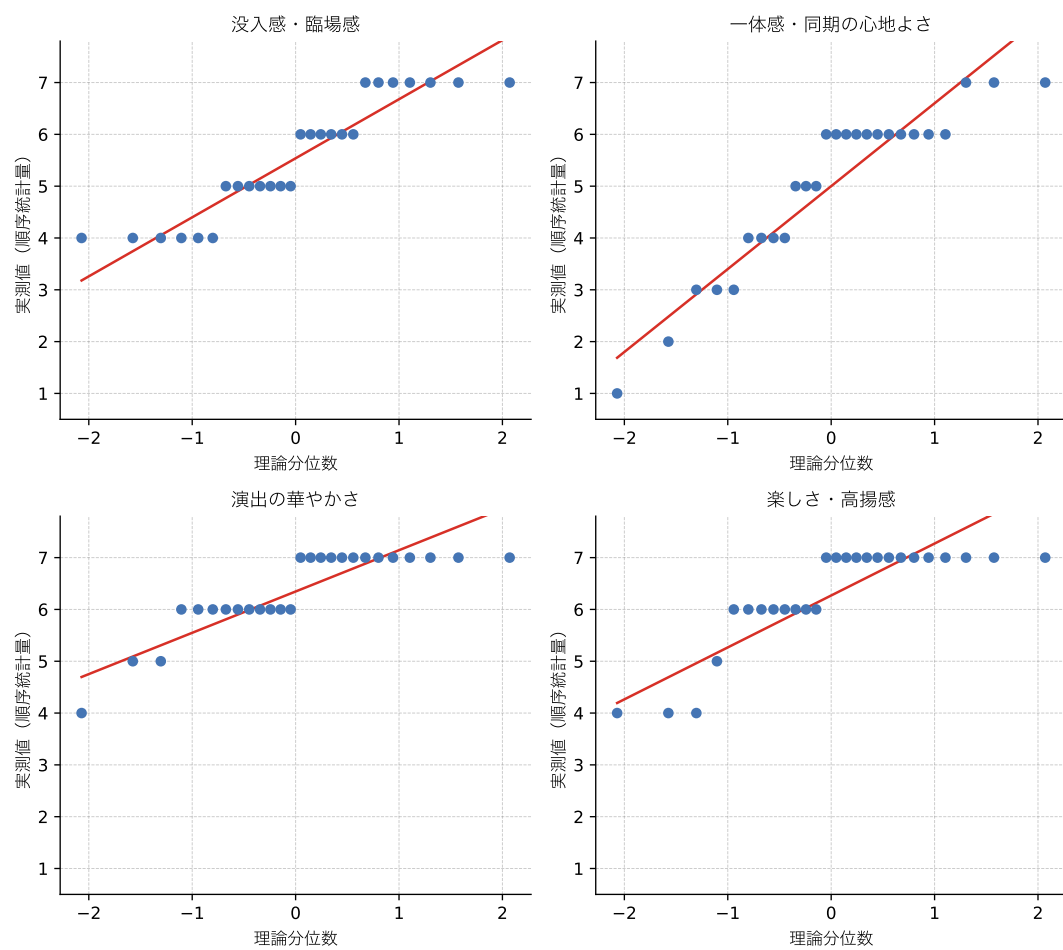


図 45 参加者共通アンケート各項目の Q-Q プロット (チーム 40, $n = 26$)

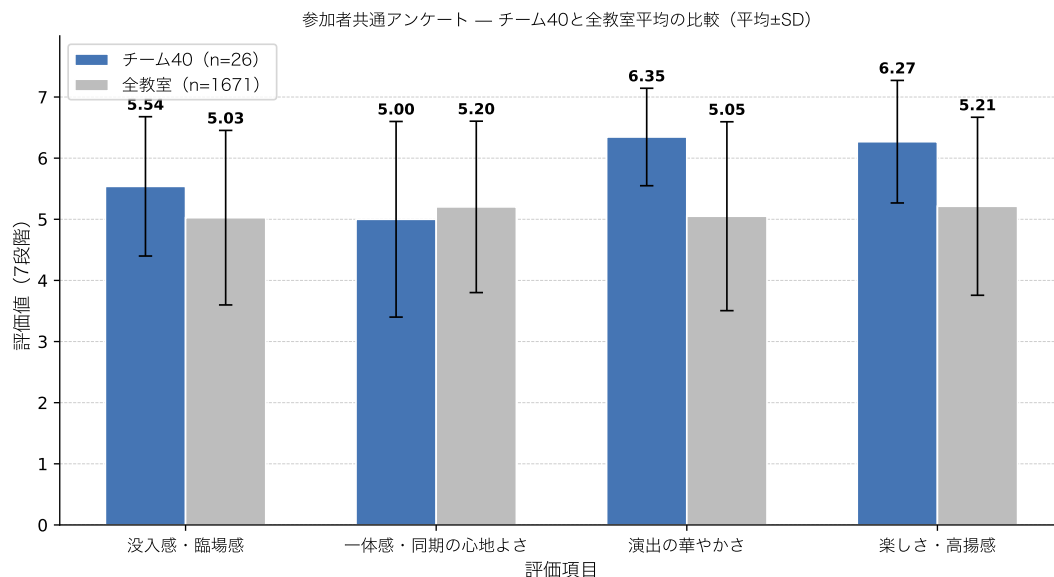


図 46 参加者共通アンケートにおけるチーム 40 と全教室平均の比較（平均±SD）

場合に適する。ただし、t 検定は平均値に基づくパラメトリック検定であり、前述の通り正規性が棄却されているうえ、本アンケートは順序尺度であるリッカート尺度を用いているため、群間差の有無の判定には Mann-Whitney の U 検定（対応のない 2 群の全データを順位に変換し、一方の群の順位が系統的に高いか低いかを調べるノンパラメトリック検定）の結果を主たる根拠とし、Welch の t 検定の結果は補助的な参考値として併記する。

また、効果量として Cohen の d [23] を算出した。Cohen の d とは、2 群の平均値の差を標準偏差で割って標準化した効果量であり、サンプル数に依存する p 値とは独立に、差の大きさそのものを表す指標である。本比較では、チーム 40 の平均値 $\bar{x}_{40}$ 、全教室の平均値 $\bar{x}_{all}$ 、および全教室の SD である s_{all} を用いて式 5 で算出した。

$$d = \frac{\bar{x}_{40} - \bar{x}_{all}}{s_{all}} \quad (5)$$

Cohen[23] は解釈の目安として、 $d = 0.2$ を小程度、 $d = 0.5$ を中程度、 $d = 0.8$ を大程度の効果としている。なお、効果量が多いことは有意差の存在を意味するものではなく、本稿では効果量を、検定で判定される差の有無とは区別された「差の大きさ」を示す補足資料として扱う。検定および効果量の結果を表 23 に示す。表 23 の各行は評価項目に対応し、両群の平均値（SD）に続けて、Welch の t 検定の統計量・自由度・ p 値、Cohen の d 、および Mann-Whitney の U 検定の統計量・ p 値を並べたものである。なお、チーム 40 の 26 件は全教室の 1671 件に含まれる（全体の約 1.6%）が、比率が小さいため比較結果への影響は限定的である。

「演出の華やかさ」および「楽しさ・高揚感」は、主たる根拠である Mann-Whitney の U 検定において全教室平均を有意に上回り（いずれも $p < 0.001$ ）、補助的な Welch の t 検定の結果もこれと一致した（いずれも $p < 0.001$ ）。効果量はそれぞれ $d = 0.84$ 、 $d = 0.73$ であり、Cohen の基準で

表 23 チーム 40 と全教室の群間比較 (Welch の t 検定, Cohen の d , Mann-Whitney の U 検定)

評価項目	チーム 40 平均 (SD)	全教室 平均 (SD)	t 値	自由度	p 値 (Welch)	Cohen の d	U	p 値 (U 検定)
没入感・臨場感	5.54 (1.14)	5.03 (1.43)	2.26	26.2	0.032	0.36	25656.0	0.104
一体感・同期の心地よさ	5.00 (1.60)	5.20 (1.40)	-0.64	25.6	0.526	-0.14	20684.5	0.668
演出の華やかさ	6.35 (0.80)	5.05 (1.54)	8.06	28.0	< 0.001	0.84	32746.0	< 0.001
楽しさ・高揚感	6.27 (1.00)	5.21 (1.46)	5.29	26.7	< 0.001	0.73	31207.0	< 0.001

大～中程度の差の大きさに相当する。一方、「没入感・臨場感」は、補助的な Welch の t 検定では $p = 0.032$ と有意であったが、主たる根拠である Mann-Whitney の U 検定では $p = 0.104$ と有意水準に達せず、全教室平均との有意差は確認されなかった ($d = 0.36$)。「一体感・同期の心地よさ」は 4 項目中唯一、平均値が全教室を下回り ($d = -0.14$)、いずれの検定でも有意差は認められなかった (Welch : $p = 0.526$, U 検定 : $p = 0.668$)。また、チーム 40 内の 4 項目間で比較すると、本項目は平均値・第 1 四分位数がともに最も低く、SD が最も大きい (回答のばらつきが最も大きい) 項目である。

項目間の比較 チーム 40 内の 4 項目間の評価の差について検定する。本アンケートは同一回答者が 4 項目すべてを評価した対応のあるデータであるため、Friedman 検定を適用した。Friedman 検定とは、対応のある 3 条件以上の比較に用いるノンパラメトリック検定であり、回答者ごとに各項目の評定へ順位をつけ、項目間で順位の合計に偏りがあるかを調べる手法である (反復測定分散分析のノンパラメトリック版に相当する)。検定の結果、 $\chi^2(3) = 16.67$, $p < 0.001$ であり、4 項目の評価の間に有意な差が認められた。

Friedman 検定は「どこかに差がある」ことを示すのみであるため、どの 2 項目間に差があるかを特定する事後検定として、Wilcoxon の符号付き順位検定を全 6 ペアに適用した。Wilcoxon の符号付き順位検定とは、対応のある 2 条件について、回答者ごとの評定の差の絶対値に順位をつけ、差の正負に偏りがあるかを調べるノンパラメトリック検定である。また、6 回の検定を繰り返すことによる第一種の過誤 (実際には差がないにもかかわらず有意と誤判定する誤り) の確率の増大を防ぐため、Holm 法 (p 値の小さい順に有意水準を段階的に厳しく調整する多重比較補正) による補正を行った。結果を表 24 に示す。表 24 は、項目の全ペアについて検定統計量 W , 補正前の p 値, Holm 補正後の p 値, および有意水準 5%での判定を並べたものである。

表 24 項目間の事後検定の結果 (Wilcoxon の符号付き順位検定, Holm 補正, $n = 26$)

ペア	W	p 値	Holm 補正後 p 値	判定 (5%水準)
没入感・臨場感, 一体感・同期の心地よさ	54.0	0.163	0.326	有意差なし
没入感・臨場感, 演出の華やかさ	9.0	0.002	0.011	有意
没入感・臨場感, 楽しさ・高揚感	18.0	0.005	0.015	有意
一体感・同期の心地よさ, 演出の華やかさ	27.0	0.002	0.011	有意
一体感・同期の心地よさ, 楽しさ・高揚感	38.5	0.004	0.015	有意
演出の華やかさ, 楽しさ・高揚感	24.5	0.751	0.751	有意差なし

表 24 に示す通り、Holm 補正後も「演出の華やかさ」および「楽しさ・高揚感」は、「没入感・臨

場感」および「一体感・同期の心地よさ」のいずれに対しても有意に高く評価されていた（補正後 $p < 0.05$ ）。一方、「演出の華やかさ」と「楽しさ・高揚感」の間、および「没入感・臨場感」と「一体感・同期の心地よさ」の間には有意差が認められなかった。

5 考察

本システムは、高い没入感を得るために「ユーザの能動的操作」があり、かつ「空間的なフィードバック」があるシステムの作成を目的として設計・評価を実施した。本節では、4節で示した単体テストおよび結合テストの結果に基づく考察を行い、実装・評価を通じて確認された本システムのハードウェア依存性を整理した後、評価指標そのものがシステムの有意性を議論する指標として妥当であるかを検討する。続いて、既存システムとの比較および本システムの優位点を整理し、4.2.2 項の末尾で示したハッカソン参加者共通アンケートの結果に基づく考察を行う。本稿の評価は、システムが設計通り動作することを確認する技術的評価（結合テスト）と、利用者からどのように評価されたかを示す主観評価（参加者共通アンケート）の二段階で構成されており、本節ではこの2つを統合して本システムの有効性を検討したうえで、最後に改善案と今後の展望について述べる。

5.1 単体テストおよび結合テストの結果に関する考察

5.1.1 同期確立時間の増大とその要因分析

結合テストにおいて、同期開始条件を 50 ms 以下とした場合、一度同期が確立した後の楽器間の演奏ズレはハース効果 [14] に基づく MOP の目標値である 50 ms 未満に収まった。一方、同期の確立自体に要する時間（ボタン押下から楽曲再生開始までの応答遅延）は、全 53 回の試行のうち 8 回が Tan らの示す 60 s [18] の閾値を超過し、成功率は 84.9%にとどまった。

この2つの同期開始条件の差を比較するため、表 18 および表 19 の測定値を BPM 別に図示したものを図 47 に示す。図 47(a) より、50 ms 条件の平均応答遅延は全ての BPM で 60 ms 条件を上回り、標準誤差（エラーバー）も大きい。すなわち、同期開始条件の厳格化は応答遅延を平均的に増大させるだけでなく、試行ごとのばらつきも拡大させている。また、図 47(b) より、60 s 以内に楽曲再生が開始された試行の割合は 60 ms 条件では全 BPM で 100%である一方、50 ms 条件では BPM60・90・150 で低下しており、特に BPM150 では 50.0%まで低下した。ただし、同条件の試行数は 10 回と少なく、BPM に対して単調な傾向も見られないことから、BPM に固有の要因であるかは判別できない。この未達成の要因として、技術的観点から次の2点が考えられる。

第一に、同期補正ループの収束特性と許容閾値の関係である。2.3.3 項で述べた通り、本システムの同期確立は、マスターと各スレーブが'R'通信で演奏位置のずれ（diff）を交換し、補正を繰り返した結果、「接続中の全楽器のずれが閾値以内」という条件が成立した時点で完了する。このずれの測定には、往復遅延の半分を片道遅延とみなす近似が用いられているため、測定値には無線通信の遅延変動に由来する誤差が含まれる。許容閾値を 50 ms に狭めると、この測定誤差やパケット

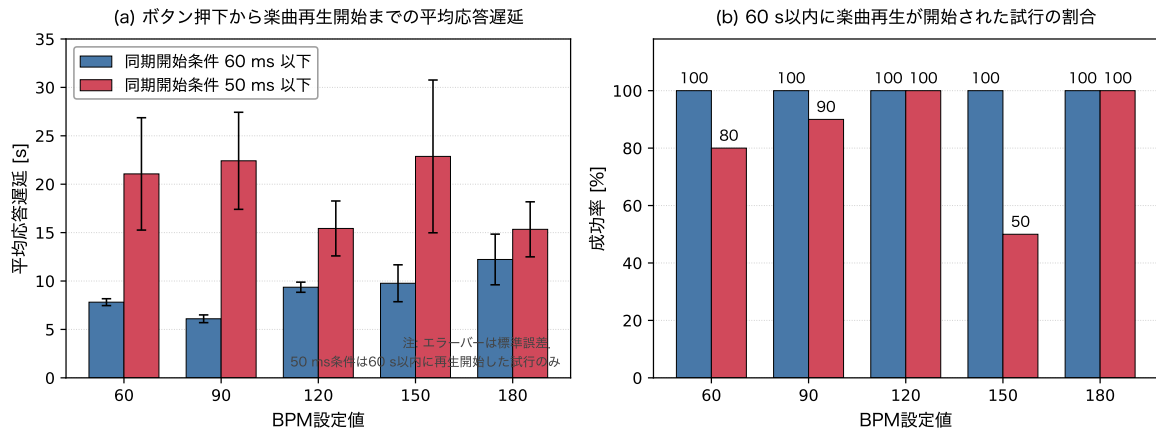


図 47 同期開始条件（60 ms 以下・50 ms 以下）別のボタン押下から楽曲再生開始までの応答遅延と成功率の比較

ロスによるずれの揺らぎが閾値幅に対して相対的に大きくなり、全楽器のずれが同時に閾値内へ収まる事象が生じにくくなるため、条件成立までの時間が延びたと考えられる。また、2.4GHz 帯の混雑による UDP パケットロスは、'R'・'O' 通信の欠落として補正の機会そのものを減少させる。本テストの実施環境は実演を行ったハッカソン会場そのものではないが、いずれも大学構内という多数の Wi-Fi 機器が 2.4GHz 帯を共有する電波環境である点は共通しており、パケットロスが同期確立を遅延させた要因として十分に考えられる。なお、指揮デバイスが指令送信時に行う 3 周の冗長送信（周間 20 ms, 2.1.3 項）は、演奏開始・終了等の指令を送る一度きりの処理であり、同期確立の補正ループには関与しないため、同期確立時間の増大要因ではない。

第二に、中継機デバイスの回転速度の変動である。2.3.3 項で述べた通り、各楽器デバイスはレーザー光の受光間隔から BPM を推定しており、同期確立の前提として受光間隔が安定している必要がある。しかし、3.2.2 項で述べた通り、同一の PWM 設定値に対するモータの回転速度は電源電圧の変動等により試行ごとに変動し、さらに 5.2 項で述べる光学系の物理的条件によって受光の成立自体が変動する。受光間隔が揺らぐと BPM 推定の安定判定に時間を要するため、同期確立時間の試行間のばらつきに寄与したと考えられる。

以上より、50 ms という同期開始条件下での応答遅延の未達成は、ずれ測定の誤差・パケットロスの下で狭い許容閾値を全楽器同時に満たすことの難しさ、および回転速度・受光の物理的な変動に起因するものと考えられる。なお、同期開始条件を 60 ms 以下に緩和した場合は全 25 回の試行で成功率 100%、平均応答遅延 9054.0 ms であり、許容ズレをわずか 10 ms 緩和するだけで同期確立が安定することから、本システムの同期性能は 50 ms という閾値の近傍で動作していると言える。

5.1.2 ユーザの能動的操作を支える処理成功率の要因分析

ボタン押下による操作反映の成功率は、全 160 回の試行のうち 158 回の成功で 98.8%であり、MOP の目標値である 99.0%以上にわずかに届かなかった。失敗した 2 回の試行はいずれも、モータドライバは動作していたにもかかわらず観覧車が回転しなかった事例である。この要因として、ブ

レッドボード上の配線の接触不安定が挙げられる。モータドライバまでの信号伝達は正常であったことから、ドライバ出力からモータまでの経路、すなわちブレッドボードのコンタクト部における接触抵抗の一時的な増大により、モータの起動トルクに必要な電流が供給されなかった可能性が高い。また、チャタリング防止のために設けた押下検知後の 300 ms の待機処理（2.1.3 項）は、待機中の再押下を検出しないため、利用者が短い間隔で操作した場合に微小な検出漏れを生じさせる余地がある。

一方、指揮デバイスの振り動作による BPM・音量変更の反映成功率は 100%であり、全試行において意図した段階が正しく反映され、多重検知や停止状態での誤反応も確認されなかった。これは、3 軸加速度センサの差分算出・多重検知防止・状態反転という段階的な検知ロジック（AccManager クラス）、および可変抵抗器のアナログ値に対する移動平均処理（PotManager クラス）が、設計通りに機能したことを示している。すなわち、ソフトウェアによる信号処理系は目標を満たす精度で動作しており、未達成の要因はブレッドボード実装というハードウェアの試作段階に固有の物理的要因に集約される。

5.1.3 音色再現度のばらつきと合成方式の限界

4.1.2 項で示した通り、MOS 評価に基づく音色再現度の MOP（中央値および最頻値がともに 4.0 以上）は全 6 楽器で達成された。一方、回答分布の散らばりに着目すると、ストリングス（評価 4 以上が 90%）に比べ、ピアノおよびキックドラム（同 71%, 74%）は第 1 四分位数が 3 であり、回答の 4 分の 1 以上が評価 3 以下に分布していた。この差の要因は、加算合成を基本とする本システムの音色合成方式の特性から説明できる。

ストリングスやフルートのような持続音系の楽器は、定常的な倍音構造の再現が音色の類似度を支配するため、正弦波の加算合成とビブラート・ノイズ成分の付加によって比較的高い再現度が得られる。これに対しピアノは、打鍵直後の急峻な減衰と、複数弦のわずかなピッチ差に起因するうなり（デチューン）が音色の特徴を決定づける減衰音系の楽器であり、本システムの合成方式ではこれらの時間変化の表現が不足していたと考えられる。また、キックドラムについては、音色の迫力を支配する低域（およそ 100 Hz 以下）の再生能力が使用したスピーカーの口径・筐体では不足しており、合成波形自体の問題に加えて出力系の物理的制約が評価を低下させたと考えられる。すなわち、ピアノは合成アルゴリズム上の限界、キックドラムは再生装置上の限界という、異なる技術的要因が低評価側への分布の広がりをもたらしたと整理できる。

5.2 ハードウェア依存性に関する考察

本節では、実装および評価の過程で確認された、本システムのハードウェア依存性について考察する。ここでハードウェア依存性とは、システムの動作や性能が、プログラムや設定値のみでは再現されず、個々のハードウェアの状態（部品の個体差、物理的な位置関係、電源の残量等）に依存する性質を指す。本システムがハードウェア依存であると考察する根拠は、次の 5 点である。

第一に、検知条件の設定を実施のたびにやり直す必要があったことである。2.2.3 項で述べた回転速度実測プログラム（LaserIntervalTest）は、周囲光量や取り付け誤差に対応するための適応し

きい値方式を採用しているにもかかわらず、実施するごとに検知条件の設定を行う必要があった。これは、環境光・レーザーの照射位置・CdSセルの受光位置といった物理的条件が実施のたびに变化し、一度定めた設定値が再利用できないことを意味する。

第二に、レーザー光の方向と受光素子の位置関係に対する敏感さである。3.2.3 項で述べた通り、レーザー光の方向や受光素子の微細な位置関係によって受光のしやすさが変化するため、デバイスを組み立て直すたびに発泡スチロールによる位置の微調整が必要であり、この調整の結果が同期の成立しやすさを左右した。この位置関係の敏感さを増幅させた要因が、回転部品の歪みである。3D プリンタで製作した回転リングにはわずかな反り・歪みがあり、リングに取り付けられたレーザーモジュールの照射方向が、リングの回転位置によって変化する。このため、レーザー光の到達位置は 8 個のモジュール間で一意に定まらず、図 48 に示すように、受光素子側の向きを個別に傾けて合わせ込む対応が必要であった。すなわち、同期性能が光学系の物理的な組み立て状態と部品の成形精度に依存している。

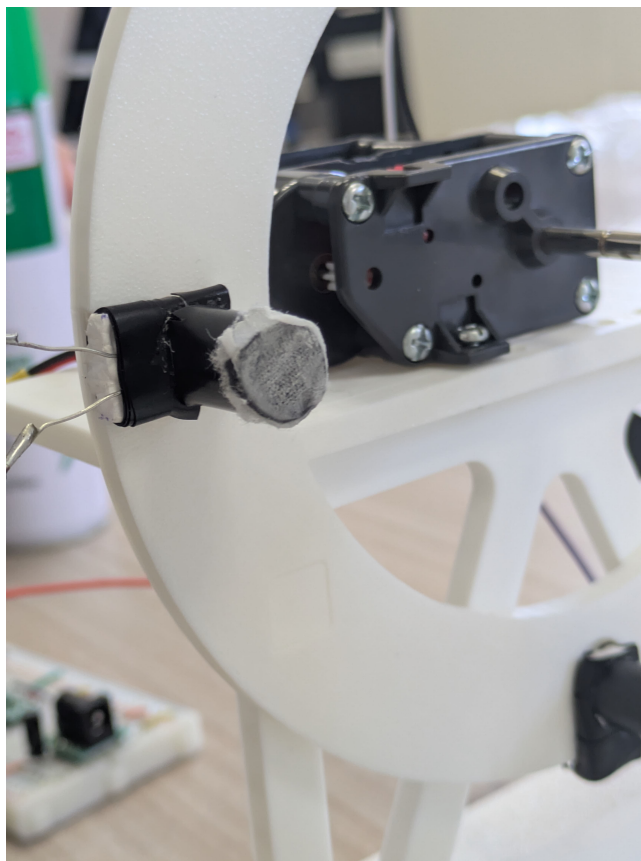


図 48 受光素子の取り付け角度の調整（回転部品の歪みによりレーザー光の到達方向が一定しないため、受光素子側の向きを個別に傾けて合わせている）

第三に、配線の保守性の低さである。本システムはブレッドボードとジャンパーワイヤーを多用した実装であるため、接触不良が発生した際に、多数の配線の中から断線箇所や接続ミスを特定す

ることに時間を要した。配線経路がはんだ付けやプリント基板によって固定されていないため、故障箇所の切り分けが個々の配線の状態確認に依存する。

第四に、レーザーモジュールの光量が電池の消耗に伴い低下することである。実装では、光量の不足への対策として外部抵抗を除去し、レーザーモジュールに内蔵された回路のみで駆動する構成としたため、電池電圧の低下がそのまま光量の低下として現れる。図 49 に示す通り、スイッチを ON にした直後は受光部に明瞭な輝点が確認できる一方、時間経過後は輝点が減光しており、両者の光量の差は目視でも判別できる。光量の低下は受光時と非受光時の明暗差を縮小させるため、受光検知の成立が電池の残量という時間変化するハードウェア状態に依存する。

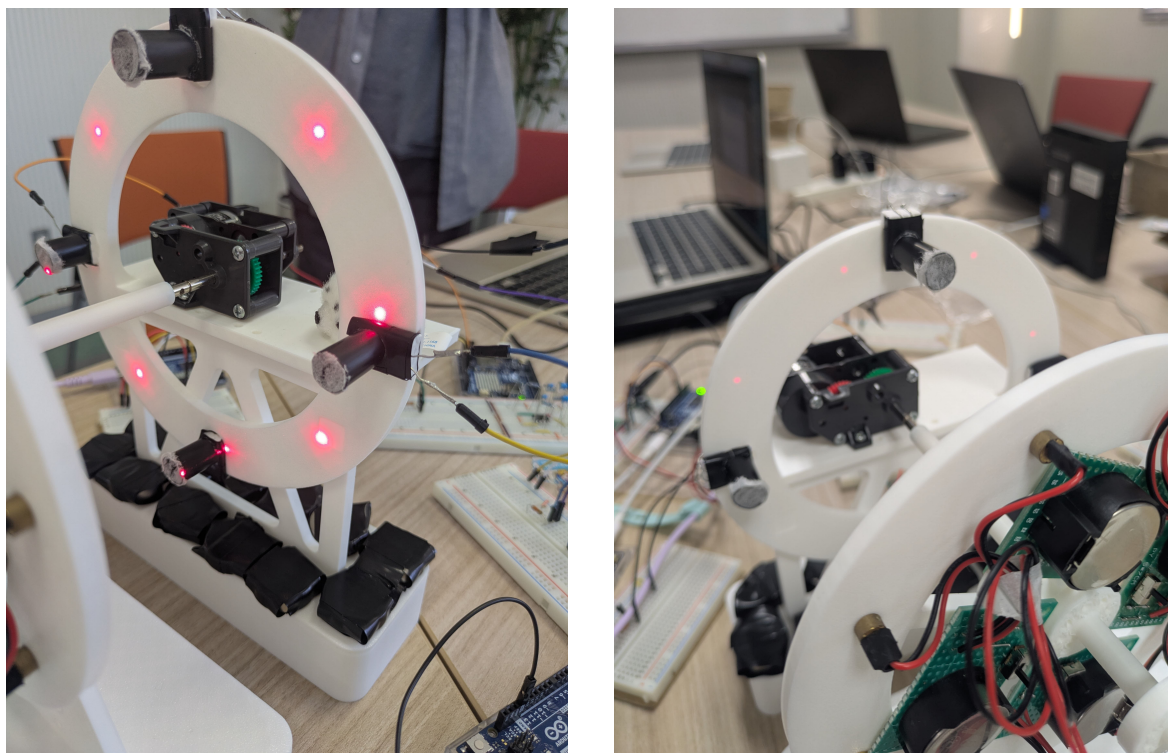


図 49 電池の消耗に伴うレーザー光量の変化（左：スイッチ ON 直後、右：時間経過後、受光部に投影された輝点の明るさが低下している）

第五に、ボタン電池の消耗の速さである。回転リング上の 8 個のレーザーモジュールを演奏中常時点灯させる構成であるため電池の消耗が早く、数時間規模の連続稼働は電池交換なしには維持できなかった。したがって、長時間の展示・実演には電池交換という人手の介入が前提となる。

以上の 5 点は、いずれも「同一のプログラム・同一の設定値を用いても、ハードウェアの状態が変われば同一の動作が再現されない」という共通の性質を持つ。この性質は、第一に、同一設計から複数の個体を製作した場合や、会場を移して再組み立てした場合に、個体・設置ごとの再調整を必須とするため、システムの再現性・可搬性を制約する。第二に、3.2.2 項で述べたモータ特性の試行間変動と同様に、評価テストの試行間で条件が完全には一致しないことを意味し、4 節で示した

測定値のばらつきの一因となった可能性がある。これらの依存性の低減策については 5.6 項で述べる。

5.3 評価指標の妥当性に関する考察

本節では、4 節で用いた評価指標が、本システムの有意性を議論するための指標として妥当であるかを検討する。

まず、音色再現度テストに MOS 評価を採用したことの妥当性について述べる。主観評価の指標としては、CSAT スコア（Customer Satisfaction Score：5 段階評価のうち 4 以上の高評価を付けた回答者の割合として算出される顧客満足度の指標）という評価方法もあるが、CSAT スコアは回答を「4 以上か否か」で二値化するため回答分布の情報の多くを捨てることになり、また目標値として一般に参照される 75%^[16] は企業の顧客満足度調査の平均値に由来する値であって、音の品質評価という本テストの文脈とは領域が異なる。これに対し MOS 評価^[20] は、音声・音響分野において主観品質を測定するために標準化された手法であり、評定値 4 が「Good」という品質水準に対応づけられているため、「中央値および最頻値が 4.0 以上」という目標値に領域固有の解釈可能な根拠を与えられる。したがって、評価対象（音色の類似性）と指標の由来する領域（音の主観品質評価）が一致する MOS 評価の採用は、CSAT スコアよりも妥当性が高いと判断できる。

次に、MOS 評価における代表値の選択について述べる。MOS 評価は本来、Mean Opinion Score（平均オピニオン評点）という名称の通り評定値の算術平均を代表値とする手法であり、本テストのように $N = 100$ の規模であれば、標本数が十分に大きいとき標本の分布が正規分布に近づく中心極限定理に基づいて平均値の信頼区間を構成し、平均値が目標値を上回るかを判定する方法も考えられる。しかし本テストでは、次の 3 つの理由から中央値・最頻値による判定を採用した。第一に、リッカート尺度は順序尺度であり、「3 と 4 の差」と「4 と 5 の差」が等しいことは保証されないため、算術平均の値そのものの解釈が困難である。第二に、回答分布は高評価側に偏った非対称な分布であり（例えばハイハットでは評価 5 が 62%）、このような分布では平均値が回答者の典型的な評定を代表しない。第三に、MOP の目標値 4.0 は「ある程度似ている」という回答カテゴリに対応しており、中央値・最頻値であれば「過半数の回答者が 4 以上と評定した」「最も多かった評定が 4 以上であった」という直接的な解釈が可能である。すなわち、中心極限定理は標本平均の推定精度（信頼区間の妥当性）を保証するものであって、順序尺度上の平均値の解釈可能性を与えるものではないため、本テストの判定には中央値・最頻値が適する。なお、リッカート尺度の順序尺度性を考慮して散らばりを四分位範囲で併記する構成も、尺度水準に整合した処理である。ただし、中央値・最頻値はカテゴリ値であるため楽器間の差異に対する分解能が低く、本稿では四分位数の併記によってこれを補ったが、サンプルサイズに基づく推定の不確実性の定量化（信頼区間に相当する評価）は行っていない点が限界として残る。

次に、応答遅延に関する閾値の妥当性について述べる。Nielsen の 10s^[17] および Tan らの 60s^[18] は、いずれもユーザが注意を維持できる限界を示す閾値であり、「これを超えると体験が破綻する」下限条件としては根拠がある。ただし、これらは注意の維持に関する閾値であって、快適と感じる応答時間の閾値ではない。ボタン押下から中継機デバイスの回転開始までの平均応答時間

は 1005.8 ms であり、10 s の約 10 分の 1 であり MOP は達成されたが、Nielsen が「思考の流れを維持できる限界」とする 1.0 s[17] とほぼ同水準である。したがって、本 MOP の達成は体験の破綻がないことを保証するものの、操作の快適性までを保証するものではなく、より厳しい閾値（例えば 1.0 s）を採用した場合の達成状況は境界上にあることに留意が必要である。一方、ハース効果に基づく 50 ms[14] は聴覚の知覚特性に直接根拠を持つ閾値であり、「複数の楽器デバイスの発音が 1 つの輪唱として知覚されるか」という本システムの MOE を判定する指標として妥当である。ただし、この指標は定常状態の演奏ズレのみを対象としており、5.1.1 項で述べた同期確立時間の増大を捉えられない。すなわち、同期の品質（ズレ）と同期の確立コスト（時間）は別個の指標で評価すべきであり、後者に対応する MOP が 60 s という粗い閾値しか持たなかったことは、評価設計上の課題である。

最後に、処理成功率 99.0%[15] という目標値について述べる。この値は Web サービスの稼働率（ツナイン）に由来するが、成功率 98.8%（158/160）という結果に対して統計的な観点から検討すると、成功率の 95% Wilson 信頼区間 [19] は [95.6%, 99.7%] であり、目標値の 99.0% を区間内に含む。すなわち、160 回という試行回数では、真の成功率が 99.0% 以上であるか否かを統計的に判別できず、「わずかに未達成」という点推定に基づく判定は、推定の不確実性を考慮すると確定的なものではない。99.0% という高い目標値の達成を有意に判定するには、数百回以上の試行が必要であり、試行回数の設計と目標値の水準が整合していなかった点は、評価指標の運用上の限界である。

5.4 既存システムとの比較および優位点

1 節で整理した通り、音楽に視覚的フィードバックを付加する既存システムには、それぞれ異なる限界がある。Yamaha Disklavier ENSPIRE シリーズ [8] に代表される自動演奏技術は、鍵盤動作という視覚的フィードバックを備えるが、その提示範囲は楽器本体の周辺に限られ、空間全体には行き渡らない。DMX 信号による照明制御 [9] は空間的な演出を実現するが、観客はあらかじめプログラムされた演出を受動的に鑑賞するのみであり、能動的な操作を持たない。ReacTable[10] のようなタンジブル・インターフェースは物理操作による能動的な演奏制御を実現するが、視覚的フィードバックは卓上ディスプレイの表示領域内で完結し、第三者や会場全体への共有を想定していない。

本システムは、指揮デバイスによる能動的操作（ボタンによる演奏開始・終了、振り動作と可変抵抗器による BPM・音量のリアルタイム制御）と、観覧車型デバイスの回転・レーザー光・分散配置された楽器デバイスの発音という空間的フィードバックを同時に備える点で、これらの既存システムのいずれとも構成が異なる。評価結果に基づく、この構成の優位点は次の 2 点に整理できる。第一に、操作の反映がボタン押下で 98.8%、振り動作による BPM・音量変更で 100% と高い成功率で機能しており、能動的操作が実効的に成立していることである。第二に、一度同期が確立した後の楽器間の演奏ズレが 50 ms 未満に収まっており、物理的に分散した複数の発音体を、聴覚上 1 つの演奏として成立する精度で協調動作させていることである。画面内の音源定位や単一スピーカーからの再生とは異なり、実空間に分散した音源と可動構造物が同期して動作する点は、既存システムでは提供されない体験であり、5.5 項で述べる参加者共通アンケートにおいて「演出の華や

かさ」および「楽しさ・高揚感」が全教室平均を有意に上回ったことは、この優位点が利用者の体験として知覚されたことを示唆している。

5.5 参加者共通アンケートに基づく考察

本節では、4.2.2 項の末尾で示したハッカソン参加者共通アンケートの結果（表 22、表 23、表 24）に基づき、本システムの体験としての到達点と課題を考察する。なお、本アンケートは他チームの作品との相対比較であり、本システムの有無による体験の差を比較したものではないため、本節の結果のみをもって本システムの有効性を示すものではない。本節では、本チームの評価とは独立に実施された第三者による客観的評価として、結合テストによるシステム評価を補強する結果と位置付けて考察する。

まず、全教室平均との比較（表 23）において、「演出の華やかさ」および「楽しさ・高揚感」は、主たる根拠である Mann-Whitney の U 検定で有意に全教室平均を上回り、補助的な Welch の t 検定の結果もこれと一致した。効果量（ $d = 0.84$, $d = 0.73$ ）も大～中程度の差の大きさを示しており、観覧車の回転・レーザー光・分散した発音体という空間的フィードバックが、演出面の体験として全参加チームの平均水準を上回って知覚されたことを示唆している。一方、「没入感・臨場感」は、平均値としては全教室を上回る向上傾向がみられたものの、主たる根拠である Mann-Whitney の U 検定では統計的な有意差は確認されなかった。補助的な Welch の t 検定では有意（ $p = 0.032$ ）であったが、正規性が棄却されたデータに対するパラメトリック検定の結果であるため、これのみを根拠に没入感が全参加チームの平均水準を上回ったと断定することはできない。

次に、「一体感・同期の心地よさ」が 4 項目中唯一、平均値が全教室を下回り、かつチーム 40 内でも平均値・第 1 四分位数がともに最も低く SD が最も大きい項目であったという結果は、5.1.1 項で述べた同期確立時間の課題と整合的である。すなわち、一度同期が確立した後の演奏ズレは 50 ms 未満に収まっていたものの、同期確立までに最大数十秒を要する場合があります。体験者が観覧した時点によっては同期が確立する前の状態（各楽器デバイスのタイミングが揃っていない状態）を観測した可能性がある。回答分布における低評価側（1～3）への裾の広がりには、このような体験タイミングの差によって説明できる。空間的な「演出」は高く評価される一方で「同期の心地よさ」が相対的に低いという乖離は、本システムの残された技術課題が同期確立の高速化・安定化にあることを、チーム外の評価者による独立したデータから裏付けるものである。

次に、項目間の比較（表 24）の結果から、チーム 40 の評価は「演出・楽しさ」が高く「没入感・一体感」が相対的に低いという 2 群の構造を持つことが分かる。演出面の刺激としての評価が、没入感・一体感という体験の深さに関する評価を有意に上回っているこの構造は、全教室比較の結果と合わせて、空間的フィードバックの演出効果は十分に機能した一方、同期の品質に関わる体験とそれを介した没入感には改善の余地があることを示している。

最後に、以上の結果を結合テストの結果と統合して本システムの有効性を検討する。技術的評価である結合テスト（4.2.2 項）では、一度同期が確立した後の演奏ズレが 50 ms 未満に収まる同期演奏、98.8%～100%の操作反映、および BPM・音量のリアルタイム制御が、設計通りに動作することを確認した。これに対し主観評価である参加者共通アンケートでは、第三者による客観的評価とし

て、「演出の華やかさ」および「楽しさ・高揚感」が全教室平均を有意に上回った。結合テストおよび参加者評価を総合すると、本システムは指揮動作に応じた空間演出を実現し、体験価値の向上に寄与するシステムとして有効であると考えられる。一方、「没入感・臨場感」および「一体感・同期の心地よさ」については統計的な有意差が確認されなかったため、本稿ではこれらに対する効果を断定せず、同期確立の高速化・安定化とあわせて今後の課題とする。

5.6 改善案と今後の展望

以上の考察に基づき、本システムの改善案を技術的課題ごとに述べる。

第一に、同期確立時間の短縮である。現行の一律3周の冗長送信は、パケットロスの有無にかかわらず固定のオーバーヘッドを発生させる。これに対し、受信側からの確認応答（ACK）に基づく選択的再送方式へ変更すれば、パケットが正常に到達している場合の送信回数を1回に削減でき、電波状況に応じて冗長度を動的に調整することも可能となる。また、同期確立を条件成立の待機ではなく、事前の時刻同期（各デバイス間のクロックオフセットを往復遅延から推定する方式）に基づく開始時刻の指定へ置き換えることで、同期確立時間をBPMや電波状況に依存しない一定値に近づけられると考えられる。物理層の対策としては、Arduino UNO R4 WiFiの無線部が2.4GHz帯のみの対応であることを踏まえ、混雑の少ないチャネルの事前調査・固定や、会場環境によっては有線（シリアルまたはRS-485等）による同期線の追加が有効である。

第二に、電気的な安定性の向上である。指揮デバイスで確認されたI2Cバスのロックアップに対しては、I2C配線の短縮・プルアップ抵抗の適正化・はんだ付けによる基板化により、ノイズや接触不良に起因するバス異常の発生自体を抑制することが望ましい。現行のソフトウェアによるバス復旧処理は有効に機能したが、復旧処理中は加速度検知が前回値で代替されるためである。また、中継機デバイスのモータがブラシ接点で発生させる電気ノイズおよび電源変動に対しては、既設の0.01 μ Fコンデンサに加えて、スナバ回路[13]の挿入やモータ系と制御系の電源分離が、回転速度の安定化（5.1.1項で述べた受光間隔の揺らぎの低減）に有効と考えられる。

第三に、ボタン操作の反映成功率の改善である。失敗事例の要因がブレッドボードの接触不安定にあることから、はんだ付けによるユニバーサル基板またはプリント基板への移行が最も直接的な対策である。また、300msの待機によるチャタリング防止を、RCフィルタとシュミットトリガによるハードウェアデバウンスと割り込み駆動の検知へ置き換えることで、待機中の検出漏れを排除できる。ここでRCフィルタとは、抵抗とコンデンサで構成され信号の急峻な変動を平滑化する回路であり、シュミットトリガとは、入力電圧の上昇時と下降時で異なる判定閾値を持たせることで、閾値付近の微小な電圧の揺らぎによる出力の反転を防ぐ回路である。ハードウェアデバウンスとは、これらの回路を組み合わせ、接点の機械的振動に起因する信号の乱れをソフトウェアの待機処理に頼らず電気的に除去する手法を指す。これらの対策のうえで、99.0%という目標値の達成を統計的に判定するためには、5.3項で述べた通り数百回規模の試行回数を確保する必要がある。

第四に、5.2項で述べたハードウェア依存性の低減である。配線の保守性に対しては、ボタン操作の対策と同様に、はんだ付けによる基板化で配線経路を固定することにより、接触不良の発生自体を抑えるとともに故障箇所の特定位を容易にできる。光学系の位置依存に対しては、レーザーと受

光素子の位置決めを手作業の微調整に委ねるのではなく、3D プリント部品にガイド（位置決め用の溝や穴）を一体成形し、組み立てのたびに同一の位置関係が機械的に再現される構造とすることが有効である。検知条件の設定については、起動時に一定時間の計測を行って閾値を自動決定する較正処理を組み込むことで、実施ごとの手動設定を不要にできる。電源に対しては、レーザーモジュールの駆動に昇圧・定電圧回路（電池電圧が低下しても一定の出力電圧を維持する回路）を追加して光量を電池残量から切り離すこと、および容量の大きい電池または充電式電池への変更により連続稼働時間を延長することが考えられる。

第五に、音色再現度の改善である。ピアノについては、打鍵直後の急峻な減衰を表現するエンベロープの精緻化と、複数弦のピッチ差を模擬したデチューン（わずかに周波数をずらした正弦波の重畳によるうなりの生成）の導入が有効と考えられる。キックドラムについては、合成波形の改善に加えて、低域再生能力の高い大口径スピーカーまたはエンクロージャの採用という出力系の改善が必要である。エンクロージャとは、スピーカーユニットを収める箱状の筐体であり、その容積や構造が低域の再生能力を大きく左右する部品である。

最後に、評価方法の展望である。本稿の評価は、音色再現度を MOS 評価により音の主観品質評価として標準的な枠組みに整合させ、参加者共通アンケートについては正規性検定の結果に基づきノンパラメトリック検定（Mann-Whitney の U 検定、Friedman 検定、Wilcoxon の符号付き順位検定）を適用した。一方、音色再現度テストにおけるサンプルサイズに基づく不確実性の定量化（中央値・最頻値に対する信頼区間に相当する評価）は今後の課題として残っている。また、参加者共通アンケートで示唆された「同期の心地よさ」の課題を検証するためには、同期確立前後の体験を区別した評価設計（体験タイミングの統制）が必要である。これらを整備することで、改善施策の効果を統計的に判定可能な形で追跡できると考えられる。

6 おわりに

本稿では、回転機構のレーザー受光と楽器間無線通信による同期・輪唱システムの設計・実装・評価について報告した。本節では、プロジェクトで実現できた内容と残された課題を整理し、1 節で示した目的との対応関係を述べる。

1 節では、デジタル配信では代替できないライブ体験の価値が、同じ空間を共有しているという感覚に由来するという知見に基づき、既存の自動演奏技術が持つ「多感覚的フィードバック」と「空間全体への共有」の不足という課題を指摘した。そのうえで、ユーザの能動的操作を受け付ける指揮デバイス、空間全体へ視覚的フィードバックを共有する観覧車型の中継機デバイス、および分散配置された楽器デバイスの 3 種で構成されるシステムにより、会場全体を巻き込む没入感の高い演奏体験を提供することを目的として設定した。

この目的に対し、本プロジェクトで実現できた内容は次の通りである。第一に、能動的操作の実現である。ボタン押下による演奏の開始・終了、指揮デバイスの振り動作および可変抵抗器による BPM・音量のリアルタイム変更を実装し、4 節の評価において、振り動作による BPM・音量変更は反映成功率 100%、ボタン押下は成功率 98.8%・平均応答時間約 1.0s で動作することを確認した。第

二に、空間的フィードバックの実現である。観覧車型デバイスの回転とレーザー受光による発音制御を実装し、物理的に分散した複数の楽器デバイスを、一度同期が確立した後はハース効果の許容限界である 50 ms 未満の演奏ズレで協調動作させた。これは、複数の発音体が聴覚上 1 つの輪唱として知覚される精度である。第三に、音色再現の実現である。6 種の楽器音を合成し、MOS 評価に基づく音色再現度の MOP（中央値および最頻値がともに 4.0 以上）を全楽器で達成した。さらに、本チームの評価とは独立に実施された参加者共通アンケートにおいて、「演出の華やかさ」および「楽しさ・高揚感」が全参加チームの平均を有意に上回った。本アンケートは他チームとの相対比較であり本システムの有無を比較したものではないが、第三者による客観的評価として、結合テストで確認した技術的な動作が利用者の体験としても高く評価されたことを補強する結果である。

一方、残された課題も評価を通じて明確になった。第一に、同期確立時間の課題である。同期開始条件を 50 ms 以下とした場合、同期の確立自体に長時間を要し、応答遅延の MOP を満たせない試行が生じた。この要因は、ずれ測定の誤差やパケットロスの下で狭い許容閾値を全楽器同時に満たすことの難しさ、および回転速度・受光の物理的な変動にあると分析した。第二に、体験としての「同期の心地よさ」の課題である。参加者共通アンケートにおいて「一体感・同期の心地よさ」は唯一全体平均を下回り、項目間の比較でも「演出・楽しさ」に対して有意に低い評価にとどまった。没入感についても、平均値としては全体を上回る傾向がみられたものの、順位に基づく検定では有意差を確認できなかった。すなわち、空間的フィードバックの演出面の効果が確認された一方、序論で目的とした没入感の向上については統計的な有意差が確認されなかったため断定できず、その隘路が同期確立の高速化・安定化にあることが、技術的評価と体験評価の両面から一貫して示唆された。第三に、実装・評価上の課題として、ブレッドボード実装に起因する操作反映の不安定さ、レーザー光量の電池残量への依存や光学系の位置調整に代表されるハードウェア依存性、ピアノ・キックドラムの音色再現のばらつき、および試行回数と目標値の水準の不整合といった評価設計の限界が残っている。

以上を総括すると、本プロジェクトは、能動的操作と空間全体で共有される物理的フィードバックを兼ね備えた演奏システムを実装し、結合テストによる技術的評価で設計通りの正常動作を確認するとともに、参加者共通アンケートによる主観評価で「演出の華やかさ」および「楽しさ・高揚感」が全体平均を上回る評価を得た。結合テストおよび参加者評価を総合すると、本システムは指揮動作に応じた空間演出を実現し、体験価値を向上させるシステムとして有効であると考えられる。一方、没入感・一体感については統計的な有意差が認められなかったため、本稿ではその効果を断定せず、今後の課題とする。その達成には、利用者がいつ体験しても同期した演奏に接することができる同期確立の即時性が不可欠である。5 節で述べた確認応答に基づく再送方式や時刻同期方式への変更、ノイズ対策、基板実装への移行といった改善を施したうえで、同期確立前後の体験を統制した評価を行うことが、会場全体を巻き込む没入感の高い演奏体験の完成に向けた次の段階である。

参考文献

- [1] 株式会社マーケットリサーチセンター, “音楽出版の日本市場(～2031年)、市場規模(パフォーマンス、同期、デジタル収益)・分析レポートを発表,” <https://www.atpress.ne.jp/news/4436457>, 2026/4/22 参照.
- [2] LP Information, “ライブコンサート市場占有率分析レポート,” <https://www.atpress.ne.jp/news/3304316>, 2026/4/22 参照.
- [3] 一般社団法人コンサートプロモーターズ協会, “ライブ市場調査 基礎推移表,” <https://www.acpc.or.jp/marketing/transition/>, 2026/5/18 参照.
- [4] 谷口由貴, “ストーリーミングサービスが生み出した新たな音楽消費サイクル,” 博報堂 The Hakuhodo, <https://www.hakuhodo.co.jp/magazine/61505/>, 2019/9/12, 2026/7/7 参照.
- [5] Martin Kreuzer, Melanie Wald-Fuhrmann, Christian Weining, Martin Tröndle, and Hauke Egermann, “Western Classical Music Concerts Are More Immersive, Intellectually Stimulating, and Social, When Experienced Live Rather Than in a Digital Stream: An Ecologically Valid Concert Study on Different Modes of Liveness,” *Music & Science*, vol.8, 2025.
- [6] Natalia Cooper, Dimitrios Mileounis, Patrick Williams, and Frank P. Brooks, “The effects of substitute multisensory feedback on task performance and the sense of presence in a virtual reality environment,” *PLOS ONE*, vol.13, no.2, p.e0191846, 2018.
- [7] R. Martin and N. Nielsen, “Enacting Musical Aesthetics: The Embodied Experience of Live Music,” *Music & Science*, vol.7, 2024.
- [8] Yamaha Music Japan, “ENSPIRE PRO,” https://jp.yamaha.com/products/musical_instruments/pianos/disklavier/enspire_pro/features.html#product-tabs, 2026/5/19 参照.
- [9] 関根 伸也, “DMX512 信号システムなどの現状技術,” 電気設備学会誌, vol.41, no.7, pp.382–385, 2021.
- [10] ReacTable Legacy, “Welcome|ReacTable Legacy,” <https://reactable.com/>, 2026/5/19 参照.
- [11] Control.com, “DC Motor Speed Control,” *Lessons In Electric Circuits*, <https://control.com/textbook/variable-speed-motor-controls/dc-motor-speed-control/>, 2026/5/4 参照.
- [12] 秋月電子通商, “fu650ad5-c6 データシート,” <https://akizukidenshi.com/goodsaffix/fu650ad5-c6.pdf>, 2026/6/30 参照.
- [13] 関根正興, “スナバ回路,” 電子情報通信学会 知識ベース, 8 群-5 編-2 章 2-4-2, 2019.
- [14] H. Haas, “Über den Einfluss eines Einfachechos auf die Hörsamkeit von Sprache,” *Acustica*, vol.1, no.2, pp.49–58, 1951.
- [15] KOBESOFT, “稼働率,” <https://kobesoft.co.jp/mikata/words/network-cloud/availability-rate/>, 2026/5/19 参照.
- [16] American Customer Satisfaction Index, “U.S. Overall Customer Satisfaction,” <https://theacsi.org/the-acsi-difference/us-overall-customer-satisfaction/>, 2026/5/17 参照.

- [17] J. Nielsen, “Response Times: The 3 Important Limits,” in *Usability Engineering*, Morgan Kaufmann, San Francisco, 1993, pp. 135–137. <https://www.nngroup.com/articles/response-times-3-important-limits/>
- [18] F. F.-Y. Tan and O. Nov, “Counting the Wait: Effects of Temporal Feedback on Downstream Task Performance and Perceived Wait-Time Experience during System-Imposed Delays,” in *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26)*, Barcelona, Spain, Apr. 2026, pp. 1–17. <https://doi.org/10.1145/3772318.3790475>
- [19] E. B. Wilson, “Probable Inference, the Law of Succession, and Statistical Inference,” *Journal of the American Statistical Association*, vol.22, no.158, pp.209–212, 1927.
- [20] ITU-T, “Methods for subjective determination of transmission quality,” Recommendation P.800, International Telecommunication Union, Geneva, 1996.
- [21] S. S. Shapiro and M. B. Wilk, “An Analysis of Variance Test for Normality (Complete Samples),” *Biometrika*, vol.52, no.3/4, pp.591–611, 1965.
- [22] B. L. Welch, “The Generalization of ‘Student’s’ Problem when Several Different Population Variances are Involved,” *Biometrika*, vol.34, no.1/2, pp.28–35, 1947.
- [23] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*, 2nd ed., Lawrence Erlbaum Associates, Hillsdale, NJ, 1988.

A 指揮デバイスのプログラムソース

本節では、指揮デバイスの制御プログラムの全文を示す。ファイル構成は 2.1.3 節の表 2 の通りである。

A.1 Controller_simultaneous.ino

コード 6 Controller_simultaneous.ino

```

1 #include "SyncManager.h"
2 #include "PotManager.h"
3 #include "AccManager.h"
4
5 #include <Wire.h>
6 #include <math.h>
7
8 // Wi-Fi のネットワーク名(hackathon001-WPA2)
9 const char SSID[] = "hackathon001-WPA2";
10 // その Wi-Fi のパスワード(hackathon001)
11 const char PASS[] = "hackathon001";
12
13 const uint8_t input_PIN = 3; // スイッチの状態を読み取るピン
14 bool switch_status; // ボタンの状態を保持
15 bool before_switch_status = 0; // 前回のボタンの状態を保持

```

```

16 bool isButtonPlaying = false; // 開始・停止信号を交互に切り替えるフラグ
17
18 SyncManager sync;
19 PotManager pot;
20 AccManager acc;
21
22 void setup() {
23     Serial.begin(115200);
24     while (!Serial) {}
25
26     Serial.println("---_Setup_Started_---");
27     pinMode(input_PIN, INPUT);
28
29     Serial.println("Attempting_to_connect_to_WiFi...");
30     sync.begin(SSID, PASS);
31
32     Serial.println("\nWiFi_Connected_successfully!");
33     Serial.println("Commands:_button_->_Start/End");
34
35     acc.setupADXL345();
36
37     before_switch_status = digitalRead(input_PIN);
38 }
39
40 void loop() {
41     button();
42     pot.update();
43     acc.update(sync, pot);
44 }
45
46 // 物理ボタンの押下を検知し、開始信号・停止信号のトグル送信と演奏状態フラグの反転を一
47 // 括管理する。
48 void button() {
49     switch_status = digitalRead(input_PIN);
50     if (switch_status == 0 && before_switch_status == 1) {
51         if (isButtonPlaying) {
52             Serial.println("Button:_[END]_sending_UDP...");
53             sync.sendEnd();
54             isButtonPlaying = false;
55         } else {
56             Serial.println("Button:_[START]_sending_UDP_to_all_simultaneously...");
57             uint8_t step = pot.getBpmStep();
58             sync.sendStart(step);
59             isButtonPlaying = true;
60         }
61         delay(300); // チャタリング防止
62     }
63     before_switch_status = switch_status;

```


63 }

A.2 AccManager.h

コード 7 AccManager.h

```
1 // 加速度センサー管理クラス
2 #ifndef ACC_MANAGER_H
3 #define ACC_MANAGER_H
4
5 #include <Arduino.h>
6 #include <Wire.h>
7 #include "SyncManager.h"
8 #include "PotManager.h"
9
10 class AccManager {
11 public:
12     AccManager();
13     // ADXL345（加速度センサー）のI2C初期化および計測開始の準備を行う。
14     void setupADXL345();
15     // 20ms周期の時間管理と、閾値判定による指揮棒のシェイク動作検知を行う。
16     void update(SyncManager& sync, PotManager& pot);
17     // potの予約フラグを確認し、変化があればsyncを通じてBPMや音量の段階番号を送信する。
18     void updateShakedInfo(SyncManager& sync, PotManager& pot);
19
20 private:
21     float readAccMagnitude(); // I2Cで読み取ったXYZの3軸生データから合成ベクトル(加速度)を算出
22     void recoverI2C(); // I2Cバスロックアップ(切断やノイズなどによりI2C通信が不能になったとき)からの復旧を行う。
23
24     const uint8_t ADXL345_ADDR = 0x53; // 加速度センサー(ADXL345)のI2Cアドレスを定義
25     const uint8_t REG_POWER_CTL = 0x2D; // 加速度センサの電源管理をするスイッチを定義
26     const uint8_t REG_DATA_FORMAT = 0x31; // 測定範囲設定用レジスタのアドレスを定義
27     const uint8_t REG_DATA0 = 0x32; // 測定した値が書き込まれる場所を定義
28
29     const float accThreshold = 110.0f; // 振ったと判断するための変化量の閾値を定義
30     const unsigned long shakeInterval = 750; // 連続検知を防止するための待機時間を定義
31     const unsigned long ACC_SAMPLE_INTERVAL = 20; // 加速度センサのサンプリング周期を定義
32
33     // 内部変数
34     float currentAccVal = 0.0f; // 加速度センサからの値を格納
35     float lastAccVal = 0.0f; // 変化量算出のための直前の加速度センサ値を格納
```

```

36     unsigned long lastDetectTime = 0; // 前回振ったことを検知した時間をms単位で格納
37     unsigned long lastSampleTime = 0; // 加速度センサの読み取り間隔を管理するための前
        回の時刻を格納
38     unsigned long lastRecoverMs = 0; // 復旧ログ処理の連続実行を防ぐための時刻を
        保持

39
40     // 振られたときに反転
41     bool isStarted = false;
42 };
43
44 #endif

```

A.3 AccManager.cpp

コード 8 AccManager.cpp

```

1  #include "AccManager.h"
2
3  AccManager::AccManager(){
4      // コンストラクタ（初期化処理なし）
5  }
6
7      // ADXL345（加速度センサー）のI2C初期化および計測開始の準備を行う。
8  void AccManager::setupADXL345() {
9      Wire.begin();
10
11      Wire.beginTransmission(ADXL345_ADDR);
12      Wire.write(REG_POWER_CTL);
13      Wire.write(0x08); // Measure bit ON
14      Wire.endTransmission();
15
16      Wire.beginTransmission(ADXL345_ADDR);
17      Wire.write(REG_DATA_FORMAT);
18      Wire.write(0x00);
19      Wire.endTransmission();
20
21      lastAccVal = readAccMagnitude();
22
23      Serial.println("GY-291_初期化完了");
24      Serial.print("閾値:");
25      Serial.print(accThreshold);
26      Serial.print("インターバル:");
27      Serial.print(shakeInterval);
28      Serial.println("_ms");
29  }
30
31      // 20ms周期の時間管理と、閾値判定による指揮棒のシェイク動作検知を行う。

```

```

32 void AccManager::update(SyncManager& sync, PotManager& pot) {
33     unsigned long now = millis();
34
35     if (now - lastSampleTime >= ACC_SAMPLE_INTERVAL) {
36         lastSampleTime = now;
37
38         currentAccVal = readAccMagnitude();
39         float diff = fabsf(currentAccVal - lastAccVal);
40
41         bool overThreshold = (diff > accThreshold);
42         bool intervalPassed = ((now - lastDetectTime) >= shakeInterval);
43
44         // 500msごとにdiff値を表示 (閾値チューニング用)
45         static unsigned long lastDebugTime = 0;
46         if (now - lastDebugTime >= 500) {
47             lastDebugTime = now;
48             //Serial.print("[ACC] diff="); Serial.print(diff, 1);
49             //Serial.print(" / 閾値="); Serial.println(accThreshold, 1);
50         }
51
52         if (overThreshold && intervalPassed) {
53             lastDetectTime = now;
54             isStarted = !isStarted;
55
56             Serial.print("[SHAKE検知]_diff=");
57             Serial.print(diff, 4);
58             Serial.println(isStarted ? "_|_START送信" : "_|_END送信");
59
60             updateShakedInfo(sync, pot);
61         }
62
63         lastAccVal = currentAccVal;
64     }
65 }
66
67 // I2Cで読み取ったXYZの3軸生データから合成ベクトル(加速度)を算出する.
68 float AccManager::readAccMagnitude() {
69     Wire.beginTransmission(ADXL345_ADDR);
70     Wire.write(REG_DATA_X0);
71     // endTransmission!=0 はNACK/バス異常。リピーテッドスタート前に検出する。
72     if (Wire.endTransmission(false) != 0) {
73         recoverI2C();
74         return lastAccVal;    // 直前値を返してニセshakeを防ぐ
75     }
76
77     uint8_t n = Wire.requestFrom(ADXL345_ADDR, (uint8_t)6, (uint8_t)true);
78     if (n < 6) {              // 6バイト読めない=ロックアップ等
79         recoverI2C();

```

```

80     return lastAccVal;
81 }
82
83 int16_t rawX = (int16_t)((Wire.read() | (Wire.read() << 8));
84 int16_t rawY = (int16_t)((Wire.read() | (Wire.read() << 8));
85 int16_t rawZ = (int16_t)((Wire.read() | (Wire.read() << 8));
86
87 float fx = (float)rawX;
88 float fy = (float)rawY;
89 float fz = (float)rawZ;
90
91 return sqrtf(fx * fx + fy * fy + fz * fz);
92 }
93
94 // I2Cバスロックアップ(切断やノイズなどによりI2C通信が不能になったとき)からの復旧を行
95 // う。
96 void AccManager::recoverI2C() {
97     unsigned long now = millis();
98     if (now - lastRecoverMs >= 1000) { // ログのスロットル (毎20msの連発を防ぐ)
99         lastRecoverMs = now;
100         Serial.println("[ACC]_I2C異常検出_→_バス復旧を実行");
101     }
102
103     Wire.end();
104
105     // バスクリア: SCLを9クロック叩いて握られたSDAを解放
106     pinMode(SDA, INPUT_PULLUP);
107     pinMode(SCL, OUTPUT);
108     for (int i = 0; i < 9; i++) {
109         digitalWrite(SCL, LOW); delayMicroseconds(5);
110         digitalWrite(SCL, HIGH); delayMicroseconds(5);
111     }
112     // STOP条件 (SDA: LOW→HIGH while SCL HIGH)
113     pinMode(SDA, OUTPUT);
114     digitalWrite(SDA, LOW); delayMicroseconds(5);
115     digitalWrite(SCL, HIGH); delayMicroseconds(5);
116     digitalWrite(SDA, HIGH); delayMicroseconds(5);
117
118     // Wire再初期化&センサ再設定 (setupADXL345の通信部と同じ)
119     Wire.begin();
120     Wire.beginTransmission(ADXL345_ADDR);
121     Wire.write(REG_POWER_CTL); Wire.write(0x08);
122     Wire.endTransmission();
123     Wire.beginTransmission(ADXL345_ADDR);
124     Wire.write(REG_DATA_FORMAT); Wire.write(0x00);
125     Wire.endTransmission();
126 }

```

```

127 // potの予約フラグを確認し、変化があればsyncを通じてBPMや音量の段階番号を送信する。
128 void AccManager::updateShakedInfo(SyncManager& sync, PotManager& pot) {
129     if (pot.bpmFrag) {
130         sync.sendBPM(pot.getBpmStep());
131         Serial.print("[SHAKE]_BPM送信_step="); Serial.println(pot.getBpmStep());
132         pot.bpmFrag = false;
133     }
134     if (pot.volFrag) {
135         sync.sendVolume(pot.getVolStep());
136         Serial.print("[SHAKE]_VOL送信_step="); Serial.println(pot.getVolStep());
137         pot.volFrag = false;
138     }
139 }

```

A.4 PotManager.h

コード 9 PotManager.h

```

1 // 可変抵抗器管理クラス
2 #ifndef POT_MANAGER_H
3 #define POT_MANAGER_H
4
5 #include <Arduino.h>
6 #include "SyncManager.h"
7
8 class PotManager {
9 public:
10     PotManager();
11
12     // 20ms周期の時間管理を行い、可変抵抗器の値のサンプリング処理を駆動する。
13     void update();
14     // カプセル化された現在のBPM段階番号（1～5）を外部に返す。
15     uint8_t getBpmStep() const { return currentBpmStep; }
16     // カプセル化された現在の音量段階番号（1～5）を外部に返す。
17     uint8_t getVolStep() const { return 6 - currentVolStep; }
18
19
20     bool bpmFrag = false; // 値の変化時に指揮棒の振りと連動させるためのBPM送信予約フラグ
21     bool volFrag = false; // 値の変化時に指揮棒の振りと連動させるための音量送信予約フラグ
22
23 private:
24     // 平滑化された平均アナログ値を、設計書に基づく境界値に照らし合わせて1～5の段階番号に変換する。
25     int calcStep(float val);

```

```

26 // BPM用可変抵抗器の値を移動平均で処理し、前回値からの変化を検知してフラグ（真偽
    値）を返す。
27 bool bpmUpdatePotentiometers();
28 // 音量用可変抵抗器の値を移動平均で処理し、前回値からの変化を検知してフラグ（真偽
    値）を返す。
29 bool volUpdatePotentiometers();
30
31 // ピン定義
32 const int PIN_BPM = A0; // BPM変更用の可変抵抗器を接続するアナログ入力ピン番号を
    定義
33 const int PIN_VOL = A1; // 音量変更用の可変抵抗器を接続するアナログ入力ピン番号を
    定義
34 const unsigned long SAMPLE_INTERVAL = 20; // 可変抵抗器の値を読み取るサンプリング
    周期
35 const int sampleSize = 10; // 移動平均算出のためのサンプル数を定義
36 const int STEP_BOUNDARIES[4] = {300, 500, 650, 818}; // アナログ入力値を5段階に変
    換するための閾値

37
38 // 内部変数
39 unsigned long lastSampleTime = 0; // 前回のサンプリング時刻
40
41 int bpmSamples[10] = {0}; // BPM用のサンプル値を保持するための配列
42 long bpmTotal = 0; // bpmSamples内の累積合計を格納
43 float averageBpmVal = 0.0f; // 算出されたBPM用のアナログ入力の平均値を格納
44
45 int volSamples[10] = {0}; // 音量用のサンプル値を保持するための配列
46 long volTotal = 0; // volSamples内の累積合計を格納
47 float averageVolVal = 0.0f; // 算出された音量用のアナログ入力の平均値を格納
48
49 int bpmReadIndex = 0; // BPMの配列内での現在の書き込み位置を管理するインデックス
50 int volReadIndex = 0; // Volumeの配列内での現在の書き込み位置を管理するインデック
    ス
51 bool bpmBufferFull = false; // BPM用の移動平均バッファが初回サンプルで満たされたか
    を判定するフラグ
52 bool volBufferFull = false; // 音量用の移動平均バッファが初回サンプルで満たされた
    かを判定するフラグ

53
54 // 算出されたBPMの5段階のレベル（1～5）を格納
55 uint8_t currentBpmStep = 0;
56 // 算出された音量の5段階のレベル（1～5）を格納
57 uint8_t currentVolStep = 0;
58
59 int prevBpmStep = 0; // 前回のBPM段階を保持
60 int prevVolStep = 0; // 前回の音量段階を保持
61 };
62

```

63 `#endif`

A.5 PotManager.cpp

コード 10 PotManager.cpp

```
1  #include "PotManager.h"
2
3  PotManager::PotManager(){
4    // コンストラクタ（初期化処理なし）
5  }
6
7  // 20ms周期の時間管理を行い、可変抵抗器の値のサンプリング処理を駆動する.
8  void PotManager::update() {
9    unsigned long now = micros();
10   if (now - lastSampleTime >= SAMPLE_INTERVAL) {
11     lastSampleTime = now;
12     if (bpmUpdatePotentiometers()) bpmFrag = true;
13     if (volUpdatePotentiometers()) volFrag = true;
14   }
15 }
16
17 // 平滑化された平均アナログ値を、設計書に基づく境界値に照らし合わせて1~5の段階番号に
18   変換する.
19 int PotManager::calcStep(float val) {
20   if (val <= STEP_BOUNDARIES[0]) return 1;
21   else if (val <= STEP_BOUNDARIES[1]) return 2;
22   else if (val <= STEP_BOUNDARIES[2]) return 3;
23   else if (val <= STEP_BOUNDARIES[3]) return 4;
24   else return 5;
25 }
26
27 // BPM用可変抵抗器の値を移動平均で処理し、前回値からの変化を検知してフラグ（真偽値）
28   を返す.
29 bool PotManager::bpmUpdatePotentiometers() {
30   bpmTotal -= bpmSamples[bpmReadIndex];
31   bpmSamples[bpmReadIndex] = analogRead(PIN_BPM);
32   bpmTotal += bpmSamples[bpmReadIndex];
33
34   bpmReadIndex = (bpmReadIndex + 1) % sampleSize;
35   if (bpmReadIndex == 0) bpmBufferFull = true;
36   if (!bpmBufferFull) return false;
37
38   averageBpmVal = (float)bpmTotal / sampleSize;
39   currentBpmStep = calcStep(averageBpmVal);
40
41   if (currentBpmStep != prevBpmStep) {
42     Serial.println("-----_BPMパラメータ更新_-----");
43   }
44 }
```



```

41     Serial.print("[BPM]_平均センサ値="); Serial.print(averageBpmVal, 1);
42     Serial.print("_段階=");          Serial.print(currentBpmStep);
43     prevBpmStep = currentBpmStep;
44     return true;
45 }
46 return false;
47 }
48
49 // 音量用可変抵抗器の値を移動平均で処理し、前回値からの変化を検知してフラグ（真偽値）
   を返す.
50 bool PotManager::volUpdatePotentiometers() {
51     volTotal -= volSamples[volReadIndex];
52     volSamples[volReadIndex] = analogRead(PIN_VOL);
53     volTotal += volSamples[volReadIndex];
54
55     volReadIndex = (volReadIndex + 1) % sampleSize;
56     if (volReadIndex == 0) volBufferFull = true;
57     if (!volBufferFull) return false;
58
59     averageVolVal = (float)volTotal / sampleSize;
60     currentVolStep = calcStep(averageVolVal);
61
62     if(currentVolStep != prevVolStep){
63         Serial.println("-----_VOLパラメータ更新_-----");
64         Serial.print("[VOL]_平均センサ値="); Serial.print(averageVolVal, 1);
65         Serial.print("_段階=");          Serial.print(6 - currentVolStep);
66         prevVolStep = currentVolStep;
67         return true;
68     }
69     return false;
70 }

```

A.6 SyncManager.h

コード 11 SyncManager.h

```

1  #ifndef SYNC_MANAGER_H
2  #define SYNC_MANAGER_H
3
4  #include <WiFiS3.h>
5  #include <WiFiUdp.h>
6
7  extern IPAddress fluteIP; // フルートのIPアドレス
8  extern IPAddress pianoIP; // ピアノのIPアドレス
9  extern IPAddress stringsIP; // スtringスのIPアドレス
10 extern IPAddress drumIP; // ドラムのIPアドレス
11 extern IPAddress ferrisWheelIP; // 観覧車（送信）のIPアドレス
12 extern IPAddress ferrisWheelGroup; // 観覧車（送信）のIPアドレス(マルチキャスト用)

```

```

13 extern IPAddress instrumentGroup; // 楽器全体のIPアドレス
14
15 class SyncManager {
16 public:
17     // コンストラクタ（ポート番号などを初期化する）
18     SyncManager();
19
20     // setup内で一度だけ呼び出される関数であり、Wi-FiとUDPを準備している。
21     void begin(const char ssid[], const char pass[]);
22
23     // 演奏開始合図（'S'）とBPMの段階番号をラウンドロビン(順次割り当て)で送信。
24     void sendStart(uint8_t stepNumber);
25
26     // BPM情報を観覧車へユニキャスト送信するための関数。
27     void sendBPM(uint8_t stepNumber);
28
29     // 音量情報を各楽器機へラウンドロビンでユニキャスト送信するための関数。
30     void sendVolume(uint8_t stepNumber);
31
32     // 演奏終了合図（'E'）を全機へラウンドロビンで送信する関数。
33     void sendEnd();
34
35 private:
36     // ラウンドロビン方式で到達時間差を無くし、かつパケットロス対策として3回冗長送信
        を行うための共通処理。
37     // （※ターゲットごとに3連射してから次のターゲットへ進む方式だと、後方の
38     // ターゲットほど到達が遅れて機器間に見えにくい時間差が生まれるため、
39     // 1周ごとに全ターゲットへ送ってから次の周に進む方式にしている。）
40     void packetRoundRobin(const uint8_t packet[], uint8_t b, const IPAddress targets
        [], uint8_t targetCount);
41 };
42
43 #endif

```

A.7 SyncManager.cpp

コード 12 SyncManager.cpp

```

1 #include "SyncManager.h"
2
3
4 const uint16_t TARGET_PORT = 5005; // ターゲットとなるポート番号
5 const uint16_t LOCAL_PORT = 8080; // 自身のポート番号
6 WiFiUDP Udp;
7
8
9 IPAddress fluteIP(192, 168, 11, 14); // フルートのIPアドレス
10 IPAddress pianoIP(192, 168, 11, 12); // ピアノのIPアドレス

```

```

11 IPAddress stringsIP(192, 168, 11, 13); // スtringsのIPアドレス
12 IPAddress drumIP(192, 168, 11, 11); // ドラムのIPアドレス
13 IPAddress ferrisWheelIP(192, 168, 11, 10); // 観覧車（送信）のIPアドレス
14 IPAddress ferrisWheelGroup(239, 255, 0, 1); // 観覧車（送信）のIPアドレス(マルチキャ
    スト用) ※未使用
15 IPAddress instrumentGroup(239, 255, 0, 2); // 楽器全体のIPアドレス
16
17 SyncManager::SyncManager() {
18     // コンストラクタ（今回は初期化処理なし）（インスタンスが作られた瞬間に、自動的に
        一番最初に1回だけ実行される特殊な関数）
19 }
20
21 // setup内で一度だけ呼び出される関数であり、Wi-FiとUDPを準備している.
22 void SyncManager::begin(const char ssid[], const char pass[]) {
23     WiFi.begin(ssid, pass); // Wi-Fi接続処理を開始
24     // 接続状態が「WL_CONNECTED（接続完了）」になるまでループ待機
25     while (WiFi.status() != WL_CONNECTED) {
26     }
27     Udp.begin(LOCAL_PORT); // 自身のポートを開放し、UDP通信を有効化
28 }
29
30 // 演奏開始合図（'S'）とBPMの段階番号をラウンドロビン(順次割り当て)で送信.
31 void SyncManager::sendStart(uint8_t stepNumber) {
32     // 設計書通りの2バイト固定長ペイロード（第1バイト:ヘッダ'S', 第2バイト:BPM段階番
        号）
33     uint8_t startPacket[2] = {'S', stepNumber};
34     // ドラムはisInstJoinを'S'受信時にリセットするため、他機より早く
35     // 届けたい。ただしラウンドロビン送信なら全ターゲットへの到達は
36     // ほぼ同時になるため、この並び自体はもう重要ではない。
37     IPAddress targets[] = {drumIP, ferrisWheelIP, pianoIP, stringsIP, fluteIP};
38     packetRoundRobin(startPacket, 2, targets, 5);
39 }
40
41 // BPM情報を観覧車へユニキャスト送信するための関数.
42 void SyncManager::sendBPM(uint8_t stepNumber){
43     uint8_t bpmPacket[2] = {'B', stepNumber};
44     IPAddress targets[] = {ferrisWheelIP};
45     packetRoundRobin(bpmPacket, 2, targets, 1);
46 }
47
48 // 音量情報を各楽器機へラウンドロビンでユニキャスト送信するための関数.
49 void SyncManager::sendVolume(uint8_t stepNumber){
50     uint8_t volumePacket[2] = {'V', (uint8_t)(6 - stepNumber)};
51     IPAddress targets[] = {fluteIP, pianoIP, stringsIP, drumIP};
52     packetRoundRobin(volumePacket, 2, targets, 4);
53 }
54
55 // 演奏終了合図（'E'）を全機へラウンドロビンで送信する関数.

```

```

56 void SyncManager::sendEnd(){
57     uint8_t endPacket[2] = {'E', 0x00};
58     IPAddress targets[] = {drumIP, ferrisWheelIP, pianoIP, stringsIP, fluteIP};
59     packetRoundRobin(endPacket, 2, targets, 5);
60 }
61
62 // ラウンドロビン方式で到達時間差を無くし、かつパケットロス対策として3回冗長送信を行
// うための共通処理。
63 // 1周目で全ターゲットに1回ずつ送り、20ms空けてから2周目、3周目と繰り返す。
64 // ターゲットごとに3連射してから次へ進む方式だと、後方のターゲットほど
65 // 到達が遅れて機器間で無視できない時間差が生まれるため、この方式にしている。
66 void SyncManager::packetRoundRobin(const uint8_t packet[], uint8_t b, const IPAddress
    targets[], uint8_t targetCount){
67     for (int round = 0; round < 3; round++) {
68         for (uint8_t t = 0; t < targetCount; t++) {
69             Udp.beginPacket(targets[t], TARGET_PORT); // 送信先IPとポートをセット
70             Udp.write(packet, b);
71             Udp.endPacket(); // パケットをパースして実際に送信
72         }
73         // 最後の周（3周目）以外は、20msのインターバル（遅延）を挟む
74         if (round < 2) {
75             delay(20);
76         }
77     }
78 }

```

B 中継機デバイス（FerrisWheel）のプログラムソース

本節では、中継機デバイスのモータ回転制御および LED マトリクス表示を行う制御プログラムの全文を示す。

B.1 FerrisWheel.ino

コード 13 FerrisWheel.ino

```

1  #include "MotorControl.h"
2  #include "Display.h"
3  #include <WiFiS3.h>
4  #include <WiFiUdp.h>
5
6  const char SSID[] = "hackathon001-WPA2"; // Wi-Fi のネットワーク名(hackathon001-WPA2)
7  const char PASS[] = "hackathon001"; // その Wi-Fi のパスワード(hackathon001)
8
9
10 IPAddress localIP(192, 168, 11, 10); // 観覧車の静的IPアドレスを設定
11 IPAddress gateway(192, 168, 11, 1); // 静的IP設定のために使用するデフォルトゲートウェ
    イのアドレスを設定

```

```

12 IPAddress subnet(255, 255, 255, 0); // 静的IP設定のために使用するサブネットマスク
13
14 const uint16_t LOCAL_PORT = 5005; // 自身のポート番号
15
16 MotorControl motor;
17
18 WiFiUDP Udp;
19
20 uint8_t packetBuffer[10]; // UDP通信で受信したパケットのデータを一時的に格納するバッファ
21
22 // 演奏中かどうかを判定('S'受信でtrue, 'E'でfalse)
23 // 'B'(BPM変更)は演奏中のみ受け付け、開始前の不意の'B'でモーターが回り出すのを防ぐ。
24 bool running = false;
25
26 void setup() {
27     Serial.begin(115200);
28     delay(2000);
29
30     Serial.println("---_FerrisWheel_Setup_Start_---");
31
32     motor.begin();
33     motor.createRpmTable();
34
35     WiFi.config(localIP, gateway, subnet);
36     WiFi.begin(SSID, PASS);
37     Serial.print("Connecting_to_WiFi...");
38     while (WiFi.status() != WL_CONNECTED) {
39         delay(500);
40         Serial.print(".");
41     }
42     Serial.println("\nWiFi_Connected!");
43     Serial.print("IP:_"); Serial.println(WiFi.localIP());
44
45     Udp.begin(LOCAL_PORT);
46     Serial.print("Listening_on_Unicast_Port_"); Serial.println(LOCAL_PORT);
47
48     displaySetup();
49 }
50
51 void loop() {
52     uint8_t packetSize = Udp.parsePacket();
53     if (packetSize <= 0) return;
54
55     if (packetSize != 2) {
56         Serial.print("想定外のパケットサイズ:_"); Serial.println(packetSize);
57         while (Udp.available()) Udp.read();

```

```

58     return;
59 }
60
61 Udp.read(packetBuffer, sizeof(packetBuffer));
62 char header = packetBuffer[0];
63 uint8_t payload = packetBuffer[1];
64
65 switch (header) {
66     case 'S':
67         Serial.print("演奏開始_('S')_payload="); Serial.println(payload);
68         // payload に BPM 段階が入っているのでそれで起動 (元コードの global 変数
           参照バグを修正)
69         running = true;
70         motor.startRamp(payload);
71         displayBPM(payload);
72         // 冗長送信の残りパケットを捨てる
73         while (Udp.parsePacket() > 0) {
74             while (Udp.available()) Udp.read();
75         }
76         break;
77
78     case 'B':
79         if (!running) {
80             Serial.println("演奏前の_'B'_を無視");
81             break;
82         }
83         Serial.print("BPM変更_('B')_段階="); Serial.println(payload);
84         motor.changeBPM(payload);
85         displayBPM(payload);
86         break;
87
88     case 'E':
89         Serial.println("演奏終了_('E')");
90         running = false;
91         motor.stopRamp();
92         clearDisplay();
93         break;
94
95     default:
96         Serial.print("想定外のヘッダ:"); Serial.println(header);
97         break;
98 }
99 }

```

B.2 Display.h

コード 14 Display.h

```

1  #ifndef DISPLAY_H
2  #define DISPLAY_H
3
4  #include <Arduino.h>
5  #include "Arduino_LED_Matrix.h"
6
7  #define BPM_STEP_1  60
8  #define BPM_STEP_2  90
9  #define BPM_STEP_3 120
10 #define BPM_STEP_4 150
11 #define BPM_STEP_5 180
12
13 #define FONT_WIDTH  3
14 #define FONT_HEIGHT 5
15 #define MATRIX_ROWS 8
16 #define MATRIX_COLS 12
17
18 // LEDマトリクスの初期化を行う.
19 void displaySetup();
20 // 指定された段階番号 (1~5) に対応するBPM値を算出し, BPM値をLEDマトリクスに表示する
   関数.
21 void displayBPM(uint8_t stepNumber);
22 // LEDマトリクスを全消灯する.
23 void clearDisplay();
24 // 指定された1桁の数字をフレームバッファから描画する.
25 void drawDigit(int digit, int x_offset);
26
27 extern const uint8_t font3x5[10][15];
28 extern ArduinoLEDMatrix matrix;
29
30 #endif

```

B.3 Display.cpp

コード 15 Display.cpp

```

1  #include "Display.h"
2  #include "MotorControl.h"
3  #include "Arduino_LED_Matrix.h"
4
5  const uint8_t font3x5[10][15] = {
6      // 0
7      {1,1,1, 1,0,1, 1,0,1, 1,0,1, 1,1,1},
8      // 1
9      {0,1,0, 1,1,0, 0,1,0, 0,1,0, 1,1,1},
10     // 2
11     {1,1,1, 0,0,1, 1,1,1, 1,0,0, 1,1,1},
12     // 3

```



```

13 {1,1,1, 0,0,1, 0,1,1, 0,0,1, 1,1,1},
14 // 4
15 {1,0,1, 1,0,1, 1,1,1, 0,0,1, 0,0,1},
16 // 5
17 {1,1,1, 1,0,0, 1,1,1, 0,0,1, 1,1,1},
18 // 6
19 {1,1,1, 1,0,0, 1,1,1, 1,0,1, 1,1,1},
20 // 7
21 {1,1,1, 0,0,1, 0,0,1, 0,0,1, 0,0,1},
22 // 8
23 {1,1,1, 1,0,1, 1,1,1, 1,0,1, 1,1,1},
24 // 9
25 {1,1,1, 1,0,1, 1,1,1, 0,0,1, 1,1,1}
26 };
27
28 static const int bpmTable[5] = {
29     BPM_STEP_1, BPM_STEP_2, BPM_STEP_3, BPM_STEP_4, BPM_STEP_5
30 };
31
32 ArduinoLEDMatrix matrix;
33
34 // LEDマトリクス初期化を行う.
35 void displaySetup() {
36     matrix.begin();
37     uint32_t warmUp[3] = {0, 0, 0};
38     matrix.loadFrame(warmUp);
39     delay(200);
40 }
41
42 static uint8_t frameBuffer[MATRIX_ROWS][MATRIX_COLS];
43 static int currentBPM = -1;
44
45 // 指定された段階番号 (1~5) に対応するBPM値を算出し、BPM値をLEDマトリクスに表示する
  関数.
46 void displayBPM(uint8_t stepNumber) {
47     if (stepNumber < 1 || stepNumber > 5) return;
48     uint8_t bpm = bpmTable[stepNumber - 1];
49
50     memset(frameBuffer, 0, sizeof(frameBuffer));
51
52     int digits[3];
53     int numDigits;
54     if (bpm >= 100) {
55         numDigits = 3;
56         digits[0] = bpm / 100;
57         digits[1] = (bpm / 10) % 10;
58         digits[2] = bpm % 10;
59     } else if (bpm >= 10) {

```

```

60         numDigits = 2;
61         digits[0] = bpm / 10;
62         digits[1] = bpm % 10;
63     } else {
64         numDigits = 1;
65         digits[0] = bpm;
66     }
67
68     int totalWidth = numDigits * FONT_WIDTH + (numDigits - 1) * 1;
69     int startX     = (MATRIX_COLS - totalWidth) / 2;
70
71     for (int i = 0; i < numDigits; i++) {
72         drawDigit(digits[i], startX + i * (FONT_WIDTH + 1));
73     }
74
75     uint32_t bitmap[3] = {0, 0, 0};
76     for (int row = 0; row < MATRIX_ROWS; row++) {
77         for (int col = 0; col < MATRIX_COLS; col++) {
78             if (frameBuffer[row][col]) {
79                 int n = row * MATRIX_COLS + col;
80                 bitmap[n / 32] |= (1UL << (31 - (n % 32)));
81             }
82         }
83     }
84     matrix.loadFrame(bitmap);
85 }
86
87 // 指定された1桁の数字をフレームバッファから描画する.
88 void drawDigit(int digit, int x_offset) {
89     if (digit < 0 || digit > 9) return;
90     const int y_offset = (MATRIX_ROWS - FONT_HEIGHT) / 2;
91     for (int y = 0; y < FONT_HEIGHT; y++) {
92         for (int x = 0; x < FONT_WIDTH; x++) {
93             int col = x_offset + x;
94             int row = y_offset + y;
95             if (col >= 0 && col < MATRIX_COLS && row >= 0 && row < MATRIX_ROWS) {
96                 frameBuffer[row][col] = font3x5[digit][y * FONT_WIDTH + x];
97             }
98         }
99     }
100 }
101
102 // LEDマトリクスを全消灯する.
103 void clearDisplay() {
104     currentBPM = -1;
105     uint32_t blank[3] = {0, 0, 0};
106     matrix.loadFrame(blank);
107 }

```

B.4 MotorControl.h

コード 16 MotorControl.h

```
1  #ifndef MOTOR_CONTROL_H
2  #define MOTOR_CONTROL_H
3
4  #include <Arduino.h>
5
6  class MotorControl {
7  public:
8      MotorControl();
9
10     // setup内で一度だけ呼び出される関数であり、モーターピンの初期化を行う。
11     void begin();
12     // rpmTableを作成する関数。
13     void createRpmTable();
14     // 演奏開始時の加速を制御する。初期のBPMに基づき一定時間をかけて徐々に目標速度
        (RPM) へ到達させる。
15     void startRamp(uint8_t stepNumber);
16     // 演奏中の BPM 変更を行う。
17     void changeBPM(uint8_t stepNumber);
18     // 演奏終了時の減速を制御する。現在のBPMに基づき、時間をかけて徐々に速度を0に近づ
        けていく。
19     void stopRamp();
20
21 private:
22     // 段階番号をBPMに変換する関数。
23     void stepToNumber(uint8_t stepNumber);
24     // omegaとV_averageを作成する関数。
25     void updateStatus();
26
27     const uint8_t PIN_MOTOR_PWM = 5; // モーター制御用の出力ピン
28     const uint8_t bpmTable[5] = {60, 90, 120, 150, 180}; // 5つの離散値 BPM 格納
29     const float VCC = 5.0; // Arduino への供給電圧 Vcc
30     const uint8_t pwmTable[5] = {26, 28, 30, 32, 37}; // 5つの離散値 PWM 格納
31
32     uint8_t currentBPM = 0; // 現在演奏中の BPM データ
33     float RPM = 0; // BPM から計算された目標回転数
34     float omega = 0; // 目標角速度 omega
35     uint8_t pwmValue = 0; // analogWrite 用の出力値
36     float V_average = 0; // 平均電圧 V_average
37     float rpmTable[5] = {}; // rpmを格納する（計算するため空にしている）
38 };
39
40 #endif
```

B.5 MotorControl.cpp

コード 17 MotorControl.cpp

```
1  #include "MotorControl.h"
2
3  MotorControl::MotorControl() {}
4
5  // setup内で一度だけ呼び出される関数であり、モーターピンの初期化を行う。
6  void MotorControl::begin() {
7      pinMode(PIN_MOTOR_PWM, OUTPUT);
8      analogWrite(PIN_MOTOR_PWM, 0);
9  }
10
11 // rpmTableを作成する関数。
12 void MotorControl::createRpmTable() {
13     for (uint8_t i = 0; i < 5; i++) {
14         rpmTable[i] = bpmTable[i] / 16.0f;
15     }
16     Serial.println("[Motor]_rpmTable_初期化完了");
17 }
18
19 // 演奏開始時の加速を制御する。初期のBPMに基づき一定時間をかけて徐々に目標速度 (RPM)
    // へ到達させる。
20 void MotorControl::startRamp(uint8_t stepNumber) {
21     stepToNumber(stepNumber);
22     Serial.println("[Motor]_Start_Ramp_Up...");
23     for (int i = 0; i <= pwmValue; i += 5) {
24         analogWrite(PIN_MOTOR_PWM, i);
25         delay(40);
26     }
27     analogWrite(PIN_MOTOR_PWM, pwmValue);
28     Serial.println("[Motor]_Target_speed_reached.");
29 }
30
31 // 演奏終了時の減速を制御する。現在のBPMに基づき、時間をかけて徐々に速度を0に近づけて
    // いく。
32 void MotorControl::stopRamp() {
33     Serial.println("[Motor]_Start_Ramp_Down...");
34     for (int i = pwmValue; i >= 0; i -= 5) {
35         analogWrite(PIN_MOTOR_PWM, i);
36         delay(40);
37     }
38     analogWrite(PIN_MOTOR_PWM, 0);
39     Serial.println("[Motor]_Stopped.");
40 }
41
42 // 演奏中の BPM 変更を行う。
```

```

43 void MotorControl::changeBPM(uint8_t stepNumber) {
44     stepToNumber(stepNumber);
45     updateStatus();
46     analogWrite(PIN_MOTOR_PWM, pwmValue);
47     Serial.print("[Motor]_Speed_Changed._PWM:_"); Serial.println(pwmValue);
48 }
49
50 // 段階番号をBPMに変換する関数.
51 void MotorControl::stepToNumber(uint8_t stepNumber) {
52     if (stepNumber < 1 || stepNumber > 5) return;
53     currentBPM = bpmTable[stepNumber - 1];
54     pwmValue    = pwmTable[stepNumber - 1];
55     RPM         = rpmTable[stepNumber - 1];
56 }
57
58 // omegaとV_averageを作成する関数.
59 void MotorControl::updateStatus() {
60     omega      = 2.0 * PI * RPM / 60.0f;
61     V_average = VCC * (pwmValue / 256.0f);
62 }

```

C 回転速度実測プログラム (LaserIntervalTest) のプログラムソース

本節では、中継機デバイスの回転速度実測に用いた LaserIntervalTest.ino の全文を示す。

C.1 LaserIntervalTest.ino

コード 18 LaserIntervalTest.ino

```

1 // CdSセルでのレーザー検知間隔を計測するスケッチ
2 // 連続する2回のパルス検知の時間差(interval)をシリアルモニタに表示し続ける。
3 //
4 // 検知ロジックは HCK_inst_0701 の drum_0701.ino / notDrum_0701.ino で使っている
5 // SyncManager::detectLight() をそのまま流用（起動時キャリブレーション+
6 // 直近ピーク履歴による適応しきい値）。BPM段階への変換は行わず、生のinterval[ms]
7 // だけを出す。
8
9 const int CDS_PIN = A0;
10
11 const int CDS_ON_THRESHOLD = 800; // 起動直後だけ使う仮しきい値
12 const int CDS_OFF_THRESHOLD = 780; // 起動直後だけ使う仮しきい値
13 const int CDS_MIN_DELTA = 25; // 暗い状態との差がこれ未満ならレーザー扱いしない
14 const uint8_t CDS_PEAK_HISTORY = 4; // 直近何回分のピークからしきい値を決めるか
15 const uint8_t CDS_ON_RATIO = 65; // baselineからピークまでの65%をONしきい値にする
16 const uint8_t CDS_OFF_RATIO = 35; // baselineからピークまでの35%をOFFしきい値にする
17 const int CDS_THRESHOLD_STEP_LIMIT = 25; // 1回の検出でしきい値が動きすぎない上限
18 const unsigned long CDS_STARTUP_CALIBRATION_MS = 1000; // 起動直後の誤検出防止時間

```

```

19 const unsigned long DEBOUNCE_MS = 200;
20
21 class LaserDetector {
22 public:
23     LaserDetector()
24         : lastDetectTime(0), laserOn(false),
25           baseline(0), baselineReady(false),
26           onThreshold(CDS_ON_THRESHOLD), offThreshold(CDS_OFF_THRESHOLD),
27           pulsePeak(0), peakIndex(0), peakCount(0),
28           calibrated(false), startupCalibrated(false), waitingForRelease(false),
29           calibrationStartMs(0), startupMin(1023), releaseThreshold(0) {
30         clearPeakHistory();
31     }
32
33     // 新しいパルスを検知したら true を返す。interval[ms] を out に書く（前回検知が無ければ0）。
34     bool update(int cdsValue, unsigned long nowMs, unsigned long &intervalOut) {
35         if (calibrationStartMs == 0) calibrationStartMs = nowMs;
36         if (!detectLight(cdsValue, nowMs)) return false;
37         if (nowMs - lastDetectTime < DEBOUNCE_MS) return false;
38
39         intervalOut = (lastDetectTime != 0) ? (nowMs - lastDetectTime) : 0;
40         bool hasInterval = (lastDetectTime != 0);
41         lastDetectTime = nowMs;
42         return hasInterval;
43     }
44
45 private:
46     unsigned long lastDetectTime;
47     bool laserOn;
48     int baseline;
49     bool baselineReady;
50     int onThreshold;
51     int offThreshold;
52     int pulsePeak;
53     int peakHistory[CDS_PEAK_HISTORY];
54     uint8_t peakIndex;
55     uint8_t peakCount;
56     bool calibrated;
57     bool startupCalibrated;
58     bool waitingForRelease;
59     unsigned long calibrationStartMs;
60     int startupMin;
61     int releaseThreshold;
62
63     bool detectLight(int v, unsigned long nowMs) {
64         if (!baselineReady) {
65             baseline = v;

```

```

66     startupMin = v;
67     baselineReady = true;
68     updateFallbackThresholds();
69 }
70
71 // 起動直後にレーザーが当たっていても、最初の1秒は検出せず暗い側の値を探す
72 if (!startupCalibrated) {
73     if (v < startupMin) startupMin = v;
74     baseline = startupMin;
75     updateFallbackThresholds();
76     if (nowMs - calibrationStartMs < CDS_STARTUP_CALIBRATION_MS) return false;
77     startupCalibrated = true;
78     waitingForRelease = (v >= onThreshold);
79     releaseThreshold = (baseline > 1023 - CDS_MIN_DELTA) ? baseline - CDS_MIN_DELTA
80         : offThreshold;
81     return false;
82 }
83
84 if (waitingForRelease) {
85     if (v <= releaseThreshold) {
86         baseline = v;
87         updateFallbackThresholds();
88         waitingForRelease = false;
89     }
90     return false;
91 }
92
93 if (!laserOn) {
94     if (v < baseline) baseline = v;
95     else baseline = (baseline * 31 + v) / 32;
96     if (!calibrated) updateFallbackThresholds();
97 }
98
99 if (!laserOn && v >= onThreshold) {
100     laserOn = true;
101     pulsePeak = v;
102     return true;
103 }
104
105 if (laserOn) {
106     if (v > pulsePeak) pulsePeak = v;
107     if (v <= offThreshold) {
108         laserOn = false;
109         registerPeak(pulsePeak);
110     }
111 }
112 return false;
113 }

```



```

113
114 void registerPeak(int peak) {
115     if (peak - baseline < CDS_MIN_DELTA) return;
116
117     peakHistory[peakIndex] = peak;
118     peakIndex = (peakIndex + 1) % CDS_PEAK_HISTORY;
119     if (peakCount < CDS_PEAK_HISTORY) peakCount++;
120     if (peakCount < 3) return;
121
122     long sum = 0;
123     for (uint8_t i = 0; i < peakCount; i++) sum += peakHistory[i];
124     int peakAvg = sum / peakCount;
125     int amplitude = peakAvg - baseline;
126     if (amplitude < CDS_MIN_DELTA) return;
127
128     int targetOn = constrain(baseline + (amplitude * CDS_ON_RATIO) / 100, 0, 1023);
129     int targetOff = constrain(baseline + (amplitude * CDS_OFF_RATIO) / 100, 0,
        targetOn - 5);

130
131     if (!calibrated) {
132         onThreshold = targetOn;
133         offThreshold = targetOff;
134         calibrated = true;
135     } else {
136         onThreshold = limitStep(onThreshold, targetOn, CDS_THRESHOLD_STEP_LIMIT);
137         offThreshold = limitStep(offThreshold, targetOff, CDS_THRESHOLD_STEP_LIMIT);
138     }
139     if (offThreshold >= onThreshold) offThreshold = max(0, onThreshold - 5);
140 }
141
142 void updateFallbackThresholds() {
143     onThreshold = constrain(baseline + CDS_MIN_DELTA, 0, 1023);
144     offThreshold = constrain(baseline + CDS_MIN_DELTA / 2, 0, max(0, onThreshold - 5)
        );
145 }
146
147 int limitStep(int current, int target, int limit) {
148     if (target > current + limit) return current + limit;
149     if (target < current - limit) return current - limit;
150     return target;
151 }
152
153 void clearPeakHistory() {
154     for (uint8_t i = 0; i < CDS_PEAK_HISTORY; i++) peakHistory[i] = 0;
155 }
156 };
157

```

```

158 LaserDetector detector;
159
160 void setup() {
161     Serial.begin(115200);
162     Serial.println("---レーザー間隔計測_Setup_Started_---");
163 }
164
165 void loop() {
166     unsigned long now = millis();
167     int cdsValue = analogRead(CDS_PIN);
168     unsigned long interval = 0;
169
170     if (detector.update(cdsValue, now, interval)) {
171         Serial.print("[interval]_");
172         Serial.print(interval);
173         Serial.println("_ms");
174     }
175 }

```

D 楽器デバイス（Processing）のプログラムソース

本節では、楽器デバイスにおける音色表現を担う Processing プログラムの全文を示す。

D.1 SoundProcessingCode.pde

コード 19 SoundProcessingCode.pde

```

1  import ddf.minim.*;
2  import ddf.minim.analysis.*;
3  import ddf.minim.effects.*;
4  import ddf.minim.ugens.*;
5
6  import java.util.Arrays;
7  import java.util.Comparator;
8
9  import processing.serial.*;
10 Serial myPort;
11
12 Minim minim;
13 AudioOutput out;
14 FFT fft;
15
16 // 受信した値を保持する
17 int musicBPM = 120;
18 float currentMasterVolume = 1.0;
19
20

```

```

21 // デバック発音用
22 String timbre = "Flute";
23 byte soundOctave = 60;
24 byte soundDoReMi = 9;
25 byte soundStartVelocity = 96;
26 byte soundEndVelocity = 96;
27
28 boolean isDebugMode = true;
29
30
31 // 自身の担当するArduinoの番号
32 // 0:なし 1:フルート 2:ストリングス 3:ピアノ 4:ドラム
33 final byte arduinoNumber = 3;
34
35
36 // 空間の残響音をまとめるミキサー
37 Summer longOut;
38 Summer shortOut;
39
40
41 InstrumentPlayer player;
42 DataUtils utils;
43
44
45
46
47
48
49
50
51 void setup()
52 {
53     size(1024, 850);
54     minim = new Minim(this);
55     out = minim.getLineOut(Minim.MONO, 1024);
56
57     // --- リバーブ（残響）の初期化 ---
58     // 長い残響用（ピアノ、ストリングスなど）
59     longOut = new Summer();
60     AddReverb longReverb = new AddReverb(15000, 0.20f);
61     longOut.patch(longReverb);
62     longReverb.patch(out);
63
64     // 短い残響用（ドラムなど）
65     shortOut = new Summer();
66     AddReverb shortReverb = new AddReverb(2000, 0.25f);
67     shortOut.patch(shortReverb);
68     shortReverb.patch(out);

```

```

69
70
71 player = new InstrumentPlayer(out);
72 utils = new DataUtils();
73
74 // 演奏用楽譜データの取得
75 float[][] Note_f = utils.GetLUT("Note");
76 Note = new int[Note_f.length][Note_f[0].length];
77 for (int i = 0; i < Note_f.length; i++) {
78     for (int j = 0; j < Note_f[i].length; j++) {
79         Note[i][j] = int(Note_f[i][j]);
80     }
81 }
82
83 out.setGain(-6.0f); // 音割れ防止のためマスター音量を下げる
84
85 fft = new FFT(out.bufferSize(), out.sampleRate());
86
87
88 // --- Processing内演奏用の別スレッド ---
89 audioThread = new Thread(new Runnable() {
90     public void run() {
91         while (true) {
92             if (isPlaying && isDebugMode) {
93                 long getTime = System.nanoTime();
94                 // 設定したBPMに基づくインターバルで発音処理を呼び出す
95                 while (getTime - lastTime >= interval) {
96                     SoundPlay();
97                     lastTime += interval;
98                     tick++;
99                 }
100             }
101
102             else {
103                 // 停止中はCPUを休ませる
104                 try { Thread.sleep(100); } catch (InterruptedException e) {}
105             }
106
107             if (isPlaying) {
108                 // 演奏中のwhileの回転速度を抑制
109                 Thread.onSpinWait();
110             }
111         }
112     }
113 });
114 audioThread.start();
115
116

```

```

117 // --- シリアル通信の開始処理 ---
118 println("===_利用可能なシリアルポート_===");
119 String portList[] = Serial.list();
120 println(portList); // ポート一覧を確認
121 myPort = new Serial(this, portList[2], 115200); // ポートは適切に変更
122 myPort.bufferUntil('\n'); // 改行コードまでバッファリング
123 println("受信待機中・・・");
124 println("=====");
125
126 }
127
128
129
130 void draw()
131 {
132     background(0);
133
134     // --- オシロスコープの描画（時間領域の波形表示） ---
135     stroke(255);
136     for (int i=0; i < out.bufferSize() - 1; i++) {
137         float x1 = map(i, 0, out.bufferSize(), 0, width);
138         float x2 = map(i+1, 0, out.bufferSize(), 0, width);
139         line(x1, 150 - out.left.get(i)*250, x2, 150 - out.left.get(i+1)*250);
140     }
141
142     // --- スペクトルアナライザの描画（周波数領域の振幅表示） ---
143     fft.forward(out.mix); // FFT解析の実行
144     stroke(255, 0, 255);
145
146     for(int i = 0; i < fft.specSize() - 1; i++) {
147         // FFTのインデックスが何Hzに相当するか取得
148         float f1 = fft.indexToFreq(i);
149         float f2 = fft.indexToFreq(i+1);
150
151         // 可聴域（20～20000Hz）のみを描画
152         // 表示範囲から外れる場合は線を描画しない
153         if (f2 < 20 || f1 > 20000) continue;
154
155         float amp1 = fft.getBand(i);
156         float amp2 = fft.getBand(i+1);
157
158         float y1 = 800 - amp1 * 4;
159         float y2 = 800 - amp2 * 4;
160
161         // 専用関数を使ってX座標を計算
162         float x1 = FreqToX(f1);
163         float x2 = FreqToX(f2);
164

```

```

165     line(x1, y1, x2, y2);
166 }
167
168 // --- 周波数の目盛りとラベルの描画 (X軸・対数スケール) ---
169 fill(255);
170 textSize(12);
171 textAlign(CENTER, TOP);
172
173 // 添付目盛りとラベルの配列
174 float[] labelFreqsX = {20, 30, 40, 50, 60, 80, 100, 200, 300, 400, 500,
175                        800, 1000, 2000, 3000, 4000, 6000, 8000, 10000, 20000};
176 String[] labelStrsX = {"20", "30", "40", "50", "60", "80", "100", "200", "300", "400", "500",
177                        "800", "1k", "2k", "3k", "4k", "6k", "8k", "10k", "20k"};
178
179 for (int i = 0; i < labelFreqsX.length; i++) {
180     float x = FreqToX(labelFreqsX[i]);
181
182     stroke(100); // 縦線
183     line(x, 300, x, 820);
184
185     fill(200);
186     text(labelStrsX[i], x, 830);
187 }
188
189 // --- 振幅の目盛りとラベルの描画 (Y軸) ---
190 textAlign(RIGHT, CENTER); // テキストを右揃えにして数値を綺麗に並べる
191
192 // 振幅(amp)を0から120まで、20刻みで横線を描画します (上限を160から120に変更)
193 for (int amp = 20; amp <= 120; amp += 20) {
194     // スペクトルアナライザの描画と同じ計算式を使用
195     float y = 800 - amp * 4;
196
197     // オシロスコープの描画領域 (Y=300より上) に被らないように描画
198     if (y >= 320) {
199         stroke(50); // 横線
200         line(40, y, width, y); // 数値ラベルと被らないようにX=40から開始
201
202         fill(200);
203         text(amp, 35, y); // 左端に振幅の数値を描画
204     }
205 }
206
207 // デバック用操作説明
208 fill(200);
209 textAlign(LEFT, TOP);
210 text("octave:1~7___C~B:A~J___C#~A#:W,E,T~U_____type:Z,C~M_____StartVelocity:0~>___
      EndVelocity:^^~_____MusicStart:shift+A~J___MusicStop:shift+Q", 20, 20);

```

```

211 }
212
213
214 // --- 周波数を対数スケールのX座標に変換する関数 ---
215 float FreqToX(float f) {
216     float minFreq = 20.0f;
217     float maxFreq = 20000.0f;
218
219     // 範囲外の値を制限する
220     if (f < minFreq) f = minFreq;
221     if (f > maxFreq) f = maxFreq;
222
223     // log() を使って対数スケールにマッピングする
224     return map(log(f), log(minFreq), log(maxFreq), 0, width);
225 };
226
227
228
229
230 // --- シリアル通信の受信イベント ---
231 void serialEvent(Serial p) {
232     //println("受信");
233
234     String inString = p.readStringUntil('\n'); // 改行まで文字列を読み込む
235     if (inString == null) return; // 何も読み込めなかった場合は処理停止
236
237     inString = trim(inString); // 末尾の改行コードや余分な空白を削除
238     String[] list = split(inString, ','); // カンマで分割して配列にする
239     if (list.length <= 1) return; // データが空の場合は処理停止
240
241     // データの識別用ヘッダとデータ数
242     String header = list[0];
243     int listLength = list.length - 1;
244
245     switch (header) {
246
247         // 発音指示 (形式: N, 音程, 長さ, 開始音量, 終了音量, 楽器タイプ...)
248         case "N" :
249             if (listLength % 5 != 0) break;
250             for (int i = 5; i <= listLength; i += 5) {
251
252                 float pitch, duration, startVel, endVel, type;
253                 try {
254                     pitch = float(list[i-4]);
255                     duration = float(list[i-3]) * (2.5f / musicBPM); // 長さをtickからミリ秒に
256                                     変換
257                     startVel = float(list[i-2]);
258                     endVel = float(list[i-1]);

```



```

258         type      = int(list[i]);
259         println("N受信(" + list[i-4] + ",_" + list[i-3] + ",_" + list[i-2] + ",_" +
                list[i-1] + ",_" + list[i] + ")");
260     } catch (NumberFormatException e) {
261         continue;
262     }
263     // 数値の場合のみ処理続行
264
265     // 自身のデバイス番号に合致する楽器を鳴らす
266     switch (arduinoNumber) {
267         case 1 :
268             player.PlayFlute(pitch, duration, startVel, endVel, currentMasterVolume);
269             break;
270         case 2 :
271             player.PlayStrings(pitch, duration, startVel, endVel, currentMasterVolume
                );
272             break;
273         case 3 :
274             if (type == 0) player.PlayPiano(pitch, duration, startVel,
                currentMasterVolume);
275             else if (type == 1) player.PlayTrumpet(pitch, duration, startVel, endVel,
                currentMasterVolume);
276             else if (type == 2) player.PlayHorn(pitch, duration, startVel, endVel,
                currentMasterVolume);
277             break;
278         case 4 :
279             int a = int(list[i-4]) % 12; // MIDIノートからドラム種類を判別
280             if (a == 0) player.PlayKick(currentMasterVolume, startVel);
281             else if (a == 3) player.PlaySnare(currentMasterVolume, startVel, endVel);
282             else if (a == 4) player.PlayHiHat(currentMasterVolume, startVel);
283             break;
284             default: break;
285     }
286 }
287 break;
288
289 // BPM変更指示 (形式: B, 更新BPM)
290 case "B" :
291     if (listLength != 1) break;
292     int getBPM;
293     try {
294         getBPM = int(list[1]);
295         println("B受信_" + list[1] + ");");
296     } catch (NumberFormatException e) {
297         break;
298     }
299     // 数値の場合のみ処理続行
300     int minimumBPM = 60;

```

```

301     int maximumBPM = 180;
302     musicBPM = constrain(getBPM, minimumBPM, maximumBPM);
303     break;
304
305     // マスターボリューム変更指示 (形式: V, 更新ボリューム)
306     case "V" :
307         if (listLength != 1) break;
308         float getVolume;
309         try {
310             getVolume = float(list[1]);
311             println("V受信_(" + list[1] + ")");
312         } catch (NumberFormatException e) {
313             break;
314         }
315         // 数値の場合のみ処理続行
316         float minimumVolume = 10.0f;
317         float maximumVolume = 50.0f;
318         currentMasterVolume = map( constrain( getVolume, minimumVolume, maximumVolume),
319                                     minimumVolume, maximumVolume, 0.2, 1.0);
319         break;
320
321     default: break;
322 }
323 }
324
325
326
327
328
329
330
331 // キーボードによるデバッグ発音用
332 void Play()
333 {
334     switch(timbre){
335         case "Flute":
336             //long a = System.nanoTime();
337             player.PlayFlute(soundOctave + soundDoReMi, 2.0f, soundStartVelocity,
338                             soundEndVelocity, 1.0f);
339             //a = millis()-a;
340             //println("経過時間: " + a);
341             break;
342         case "Strings":
343             //long a = System.nanoTime();
344             player.PlayStrings(soundOctave + soundDoReMi, 1.0f, soundStartVelocity,
345                               soundEndVelocity, 1.0f);
346             //a = millis()-a;
347             //println("経過時間: " + a);

```

```

346     break;
347 case "Drum":
348     //long a = System.nanoTime();
349     if (soundDoReMi == 0) {
350         player.PlayKick(1.0f, soundStartVelocity);
351     } else if (soundDoReMi == 3) {
352         player.PlaySnare(1.0f, soundStartVelocity, soundEndVelocity);
353     } else if (soundDoReMi == 4) {
354         player.PlayHiHat(1.0f, soundStartVelocity);
355     }
356     //a = millis()-a;
357     //println("経過時間: " + a);
358     break;
359 case "Piano":
360     //long a = System.nanoTime();
361     player.PlayPiano(soundOctave + soundDoReMi, 10.0f, soundStartVelocity, 1.0f);
362     //a = millis()-a;
363     //println("経過時間: " + a);
364     break;
365 case "Horn":
366     //long a = System.nanoTime();
367     player.PlayHorn(soundOctave + soundDoReMi, 1.0f, soundStartVelocity,
368         soundEndVelocity, 1.0f);
369     //a = millis()-a;
370     //println("経過時間: " + a);
371     break;
372 case "Trumpet":
373     //long a = System.nanoTime();
374     player.PlayTrumpet(soundOctave + soundDoReMi, 1.0f, soundStartVelocity,
375         soundEndVelocity, 1.0f);
376     //a = millis()-a;
377     //println("経過時間: " + a);
378     break;
379 }
380 }
381
382 // キー入力
383 void keyPressed() {
384     if (isDebugMode == false) return;
385     switch (key)
386     {
387
388     // オクターブ変更
389     case '1':
390         soundOctave = 24;
391         break;

```

```

392 case '2':
393     soundOctave = 36;
394     break;
395 case '3':
396     soundOctave = 48;
397     break;
398 case '4':
399     soundOctave = 60;
400     break;
401 case '5':
402     soundOctave = 72;
403     break;
404 case '6':
405     soundOctave = 84;
406     break;
407 case '7':
408     soundOctave = 96;
409     break;
410
411 // 強弱変更
412 case '0':
413     soundStartVelocity = 127;
414     break;
415 case '-':
416     soundStartVelocity = 112;
417     break;
418 case 'o':
419     soundStartVelocity = 96;
420     break;
421 case 'p':
422     soundStartVelocity = 80;
423     break;
424 case 'l':
425     soundStartVelocity = 64;
426     break;
427 case ';':
428     soundStartVelocity = 48;
429     break;
430 case ',':
431     soundStartVelocity = 32;
432     break;
433 case '.':
434     soundStartVelocity = 16;
435     break;
436 case '^':
437     soundEndVelocity = 127;
438     break;
439 case '≠':

```

```

440     soundEndVelocity = 112;
441     break;
442 case '@':
443     soundEndVelocity = 96;
444     break;
445 case '[':
446     soundEndVelocity = 80;
447     break;
448 case ':':
449     soundEndVelocity = 64;
450     break;
451 case ']':
452     soundEndVelocity = 48;
453     break;
454 case '/':
455     soundEndVelocity = 32;
456     break;
457 case '_':
458     soundEndVelocity = 16;
459     break;
460
461 // 発音
462 case 'a':
463     soundDoReMi = 0;
464     Play();
465     break;
466 case 'w':
467     soundDoReMi = 1;
468     Play();
469     break;
470 case 's':
471     soundDoReMi = 2;
472     Play();
473     break;
474 case 'e':
475     soundDoReMi = 3;
476     Play();
477     break;
478 case 'd':
479     soundDoReMi = 4;
480     Play();
481     break;
482 case 'f':
483     soundDoReMi = 5;
484     Play();
485     break;
486 case 't':
487     soundDoReMi = 6;

```

```
488     Play();
489     break;
490 case 'g':
491     soundDoReMi = 7;
492     Play();
493     break;
494 case 'y':
495     soundDoReMi = 8;
496     Play();
497     break;
498 case 'h':
499     soundDoReMi = 9;
500     Play();
501     break;
502 case 'u':
503     soundDoReMi = 10;
504     Play();
505     break;
506 case 'j':
507     soundDoReMi = 11;
508     Play();
509     break;
510
511 // 楽器変更
512 case 'z':
513 case 'Z':
514     timbre = "Flute";
515     break;
516 case 'c':
517 case 'C':
518     timbre = "Strings";
519     break;
520 case 'v':
521 case 'V':
522     timbre = "Drum";
523     break;
524 case 'b':
525 case 'B':
526     timbre = "Piano";
527     break;
528 case 'n':
529 case 'N':
530     timbre = "Horn";
531     break;
532 case 'm':
533 case 'M':
534     timbre = "Trumpet";
535     break;
```

```

536
537
538
539
540 // 演奏
541 case 'A' : // 酒場でブギウギ
542     tick = 1536;
543     returnTick = new int[][] {{2880, 1728, 48}, {2784, 2880, 281}, {4512, 1728,
544         48}};
545     returnIndex = 0;
546     noteIndex = 29;
547     interval = GetInterval( 139 );
548     lastTime = System.nanoTime();
549     isPlaying = true;
550     break;
551
552 case 'S' : // かえるのうた
553     tick = 0;
554     returnTick = new int[][] {{1536, 0, 0}};
555     returnIndex = 0;
556     noteIndex = 0;
557     interval = GetInterval( 120 );
558     lastTime = System.nanoTime();
559     isPlaying = true;
560     break;
561
562 case 'D' : // 王宮のメヌエット（強弱なし）
563     tick = 4608;
564     returnTick = new int[][] {{5760, 4608, 651}, {5736, 5832, 850}, {6432, 5856,
565         856}, {6216, 6456, 993}, {6816, 4608, 651}, {5760, 6816, 1046}, {8832,
566         4608, 651}};
567     returnIndex = 0;
568     noteIndex = 651;
569     interval = GetInterval( 122 );
570     lastTime = System.nanoTime();
571     isPlaying = true;
572     break;
573
574 case 'F' : // トルコ行進曲
575     tick = 8856;
576     returnTick = new int[][] {{9240, 8856, 1399}, {10008, 9240, 1499}, {10392,
577         10008, 1669}, {10776, 10392, 1783}, {11544, 10776, 1895}, {11544, 10008,
578         1669}, {10392, 10008, 1669}, {10392, 8856, 1399}, {9240, 8856, 1399},
579         {10008, 11544, 2111}, {11928, 11544, 2111}, {11904, 11952, 2225}, {13632,
580         8856, 1399}};
581     returnIndex = 0;
582     noteIndex = 1399;
583     interval = GetInterval( 120 );

```

```

577     lastTime = System.nanoTime();
578     isPlaying = true;
579     break;
580
581     case 'G' : // 序曲_中盤（製作中用）
582         tick = 96;
583         returnTick = new int[][] {{1608, 72, 0}};
584         returnIndex = 0;
585         noteIndex = 0;
586         interval = GetInterval( 120 );
587         lastTime = System.nanoTime();
588         isPlaying = true;
589         break;
590
591     case 'H' : // 序曲_終盤なし
592         tick = 13632;
593         returnTick = new int[][] {{16488, 14952, 2899}}; // ピアノ用
594         //returnTick = new int[][] {{16488, 14952, 3053}}; // スtringス用
595         //returnTick = new int[][] {{16488, 14952, 2985}}; // フルート用
596         returnIndex = 0;
597         noteIndex = 2638;
598         interval = GetInterval( 120 );
599         lastTime = System.nanoTime();
600         isPlaying = true;
601         break;
602
603     case 'J' : // 序曲_ドラム用
604         tick = 768;
605         returnTick = new int[][] {{3624, 2088, 637}};
606         returnIndex = 0;
607         noteIndex = 92;
608         interval = GetInterval( 120 );
609         lastTime = System.nanoTime();
610         isPlaying = true;
611         break;
612
613     case 'Q' : // 演奏停止
614         isPlaying = false;
615         break;
616
617     default:
618         break;
619 }
620 }

```

D.2 Utils.pde

コード 20 Utils.pde

```

1 // =====
2 // 波形データ(JSON)のロードと、
3 // ピッチ・ベロシティに合わせたプロファイル生成を担うクラス
4 // =====
5 class DataUtils {
6
7     // デバック用の楽譜
8     private float[][] note;
9
10    // 各楽器の強度別 (FF/MF/PP) スペクトルプロファイル
11    private float[][][] fluteFFLUT, fluteMFLUT, flutePPLUT;
12
13    private float[][][] violinFFLUT, violinMFLUT, violinPPLUT;
14    private float[][][] violaFFLUT, violaMFLUT, violaPPLUT;
15    private float[][][] celloFFLUT, celloMFLUT, celloPPLUT;
16    private float[][][] bassFFLUT, bassMFLUT, bassPPLUT;
17
18    private float[][][] pianoFFLUT, pianoMFLUT, pianoPPLUT;
19    private float[][] pianoDecayFFLUT, pianoWobbleRateFFLUT, pianoWobbleDepthFFLUT;
20    private float[][] pianoDecayMFLUT, pianoWobbleRateMFLUT, pianoWobbleDepthMFLUT;
21    private float[][] pianoDecayPPLUT, pianoWobbleRatePPLUT, pianoWobbleDepthPPLUT;
22
23    private float[][] kickLUT, snareLUT, hihatLUT;
24
25
26
27    private float[][][] hornFFLUT, hornMFLUT, hornPPLUT;
28    private float[][][] trumpetFFLUT, trumpetMFLUT, trumpetPPLUT;
29
30
31    DataUtils() {
32        //println("Loading JSON data...");
33
34        // JSONファイルからデータをロードする
35
36        // --- 楽譜・演奏データの読み込み ---
37        //note = Load2LUT("序曲_フルート冒頭_processing.json");
38        //note = Load2LUT("序曲_フルート中盤_processing.json");
39        //note = Load2LUT("序曲_ピアノ冒頭_processing.json");
40        //note = Load2LUT("序曲_ピアノ中盤_processing.json");
41        //note = Load2LUT("序曲_ストリングス冒頭_processing.json");
42        //note = Load2LUT("序曲_ストリングス中盤_processing.json");
43        //note = Load2LUT("序曲_ドラム冒頭_processing.json");
44        //note = Load2LUT("序曲_ドラム中盤_processing.json");
45

```

```

46 //note = Load2LUT("かえるのうた8小節ver「酒場でブギウギ(強弱付き)」 「王宮のメヌ
    エット(強弱なし)」 「トルコ行進曲」_processing.json");
47 //note = Load2LUT("共通部分0706_processing.json");
48 //note = Load2LUT("ドラム0706_1_processing.json");
49 note = Load2LUT("ピアノ0706_1_processing.json");
50 //note = Load2LUT("ストリングス0706_1_processing.json");
51 //note = Load2LUT("フルート0706_1_processing.json");
52
53
54 // --- 各楽器のLUTの読み込み ---
55 fluteFFLUT = Load3LUT("fluteFFLUT.json");
56 fluteMFLUT = Load3LUT("fluteMFLUT.json");
57 flutePPLUT = Load3LUT("flutePPLUT.json");
58
59 violinFFLUT = Load3LUT("violinFFLUT.json");
60 violinMFLUT = Load3LUT("violinMFLUT.json");
61 violinPPLUT = Load3LUT("violinPPLUT.json");
62 violaFFLUT = Load3LUT("violaFFLUT.json");
63 violaMFLUT = Load3LUT("violaMFLUT.json");
64 violaPPLUT = Load3LUT("violaPPLUT.json");
65 celloFFLUT = Load3LUT("celloFFLUT.json");
66 celloMFLUT = Load3LUT("celloMFLUT.json");
67 celloPPLUT = Load3LUT("celloPPLUT.json");
68 bassFFLUT = Load3LUT("bassFFLUT.json");
69 bassMFLUT = Load3LUT("bassMFLUT.json");
70 bassPPLUT = Load3LUT("bassPPLUT.json");
71
72 pianoFFLUT = Load3LUT("pianoFFLUT.json");
73 pianoMFLUT = Load3LUT("pianoMFLUT.json");
74 pianoPPLUT = Load3LUT("pianoPPLUT.json");
75 pianoDecayFFLUT = Load2LUT("pianoDecaysFF.json");
76 pianoDecayMFLUT = Load2LUT("pianoDecaysMF.json");
77 pianoDecayPPLUT = Load2LUT("pianoDecaysPP.json");
78 pianoWobbleRateFFLUT = Load2LUT("pianoWobbleRatesFF.json");
79 pianoWobbleRateMFLUT = Load2LUT("pianoWobbleRatesMF.json");
80 pianoWobbleRatePPLUT = Load2LUT("pianoWobbleRatesPP.json");
81 pianoWobbleDepthFFLUT = Load2LUT("pianoWobbleDepthsFF.json");
82 pianoWobbleDepthMFLUT = Load2LUT("pianoWobbleDepthsMF.json");
83 pianoWobbleDepthPPLUT = Load2LUT("pianoWobbleDepthsPP.json");
84
85 kickLUT = Load2LUT("kickLUT.json");
86 snareLUT = Load2LUT("snareLUT.json");
87 hihatLUT = Load2LUT("hihatLUT.json");
88
89 hornFFLUT = Load3LUT("hornFFLUT.json");
90 hornMFLUT = Load3LUT("hornMFLUT.json");
91 hornPPLUT = Load3LUT("hornPPLUT.json");
92

```

```

93     trumpetFFLUT = Load3LUT("trumpetFFLUT.json");
94     trumpetMFLUT = Load3LUT("trumpetMFLUT.json");
95     trumpetPPLUT = Load3LUT("trumpetPPLUT.json");
96
97     //println("Data loaded successfully!");
98 }
99
100
101 // JSONファイルを読み込んで float[][] に変換する関数
102 float[][] Load2LUT(String fileName) {
103     JSONArray jsonArray = loadJSONArray(fileName);
104     float[][] lut = new float[jsonArray.size()][];
105
106     for (int i = 0; i < jsonArray.size(); i++) {
107         if (jsonArray.isNull(i)) {
108             lut[i] = null; // 存在しない音程は null にする
109             continue;
110         }
111         JSONArray row = jsonArray.getJSONArray(i);
112         if (row.size() > 0) {
113             lut[i] = new float[row.size()];
114             for (int j = 0; j < row.size(); j++) {
115                 lut[i][j] = row.getFloat(j);
116             }
117         } else {
118             lut[i] = null; // データが欠損している部分は null にする
119         }
120     }
121     return lut;
122 }
123
124
125 // JSONファイルを読み込んで float[音程][ピーク][周波数・振幅] に変換する関数
126 float[][][] Load3LUT(String fileName) {
127     JSONArray jsonArray = loadJSONArray(fileName);
128     // 第1次元（音程の数）を確定
129     float[][][] lut = new float[jsonArray.size()][][];
130
131     for (int i = 0; i < jsonArray.size(); i++) {
132         // JSON上で null になっている、または要素が欠損している場合の処理
133         if (jsonArray.isNull(i)) {
134             lut[i] = null; // 存在しない音程は null にする
135             continue;
136         }
137
138         JSONArray peaksArray = jsonArray.getJSONArray(i);
139         int numPeaks = peaksArray.size();
140

```

```

141     if (numPeaks > 0) {
142         // 第2次元（その音程が持つピークの数）を動的に確定
143         lut[i] = new float[numPeaks][2];
144
145         for (int j = 0; j < numPeaks; j++) {
146             JSONArray peakPair = peaksArray.getJSONArray(j);
147
148             // [周波数, 振幅] のペア（要素数2）を確実に取得
149             if (peakPair.size() == 2) {
150                 lut[i][j][0] = peakPair.getFloat(0); // 周波数 (Hz)
151                 lut[i][j][1] = peakPair.getFloat(1); // 相対振幅 (0.0 ~ 1.0)
152             }
153         }
154     } else {
155         lut[i] = null; // ピーク数が0個の場合もデータなしとして null を代入
156     }
157 }
158 return lut;
159 }
160
161
162
163
164
165 // 特定の3次元LUTから、指定したピッチに「最も近い有効データ」を取り出し、正確なピッ
    チへシフトして返す関数
166 // 戻り値：float[ピーク数][2] -> [h][0]:周波数(Hz), [h][1]:振幅
167 float[][] GetProfileForPitch(float pitch, float[][][] lut, int baseMidiNote) {
168
169     // 要求した本来のピッチ（小数ビブラートや範囲外のC3/A7など）を保持
170     float originalPitch = pitch;
171
172     // LUTの要素数から自動的に最高音を算出し、インデックス探索用のみ安全にクランプす
        る
173     float maxMidiNote = baseMidiNote + lut.length - 1;
174     float searchPitch = constrain(pitch, baseMidiNote, maxMidiNote);
175
176     // 四捨五入(round)で、最もピッチが近いサンプルのインデックスを狙う
177     int targetIndex = round(searchPitch) - baseMidiNote;
178     targetIndex = constrain(targetIndex, 0, lut.length - 1);
179
180     // データが欠損している場合は隣接する有効データを探索する
181     int finalIndex = targetIndex;
182     //println(finalIndex);
183     if (lut[finalIndex] == null) {
184         int lowerIndex = finalIndex;
185         while (lowerIndex >= 0 && lut[lowerIndex] == null) lowerIndex--;
186

```

```

187     int upperIndex = finalIndex;
188     while (upperIndex < lut.length && lut[upperIndex] == null) upperIndex++;
189
190     // 上下のうち、存在する方、あるいは要求された元のピッチ (searchPitch) により近い方
    // を採用する
191     if (lowerIndex >= 0 && upperIndex < lut.length) {
192         if (abs(searchPitch - (lowerIndex + baseMidiNote)) <= abs((upperIndex +
            baseMidiNote) - searchPitch)) {
193             finalIndex = lowerIndex;
194         } else {
195             finalIndex = upperIndex;
196         }
197     } else if (lowerIndex >= 0) {
198         finalIndex = lowerIndex;
199     } else if (upperIndex < lut.length) {
200         finalIndex = upperIndex;
201     } else {
202         // 万が一、LUT全体が空だった場合：基音のみ生成
203         float f0 = 440.0f * pow(2.0f, (originalPitch - 69.0f) / 12.0f);
204         float[][] fallback = new float[1][2];
205         fallback[0][0] = f0;    // 周波数
206         fallback[0][1] = 1.0f;  // 振幅
207         return fallback;
208     }
209 }
210 //println(finalIndex);
211
212 // 見つかったサンプルのデータを取得
213 float[][] sourceProfile = lut[finalIndex];
214 int numPeaks = sourceProfile.length;
215 float[][] result = new float[numPeaks][2];
216
217 // =====
218 // 流用元データから要求ピッチへのシフト倍率を計算
219 // =====
220 // 流用元のサンプルが本来持っている「正しい基準MIDIノート番号」を逆算
221 float sourceMidi = finalIndex + baseMidiNote;
222
223 // 制限のない本来の要求ピッチ (originalPitch) と、流用元 (sourceMidi) の差分（半
    // 音単位）を計算
224 float pitchShiftFactor = pow(2.0f, (originalPitch - sourceMidi) / 12.0f);
225 // =====
226
227 for (int h = 0; h < numPeaks; h++) {
228     result[h][0] = sourceProfile[h][0] * pitchShiftFactor;
229     result[h][1] = sourceProfile[h][1];
230 }
231

```

```

232     return result;
233 }
234
235
236
237
238 // ピッチとベロシティの2軸から、最終的な「周波数と音量のペア」の2次元配列を生成する
// 関数
239 // 戻り値: float[ピーク数][2] -> [h][0]:周波数(Hz), [h][1]:最終音量
240 float[][] GetDynamicProfile(float pitch, float velocity, String type, int
    baseMidiNote, float basePower) {

241
242 // 3つの3次元LUTから、現在のピッチに最も近い実測スペクトルをそれぞれ取得
243 float[][] profilePP, profileMF, profileFF;
244
245 switch (type) {
246     case "Flute":
247         profilePP = GetProfileForPitch(pitch, flutePPLUT, baseMidiNote);
248         profileMF = GetProfileForPitch(pitch, fluteMFLUT, baseMidiNote);
249         profileFF = GetProfileForPitch(pitch, fluteFFLUT, baseMidiNote);
250         break;
251     case "Violin":
252         profilePP = GetProfileForPitch(pitch, violinPPLUT, baseMidiNote);
253         profileMF = GetProfileForPitch(pitch, violinMFLUT, baseMidiNote);
254         profileFF = GetProfileForPitch(pitch, violinFFLUT, baseMidiNote);
255         break;
256     case "Viola":
257         profilePP = GetProfileForPitch(pitch, violaPPLUT, baseMidiNote);
258         profileMF = GetProfileForPitch(pitch, violaMFLUT, baseMidiNote);
259         profileFF = GetProfileForPitch(pitch, violaFFLUT, baseMidiNote);
260         break;
261     case "Cello":
262         profilePP = GetProfileForPitch(pitch, celloPPLUT, baseMidiNote);
263         profileMF = GetProfileForPitch(pitch, celloMFLUT, baseMidiNote);
264         profileFF = GetProfileForPitch(pitch, celloFFLUT, baseMidiNote);
265         break;
266     case "Bass":
267         profilePP = GetProfileForPitch(pitch, bassPPLUT, baseMidiNote);
268         profileMF = GetProfileForPitch(pitch, bassMFLUT, baseMidiNote);
269         profileFF = GetProfileForPitch(pitch, bassFFLUT, baseMidiNote);
270         break;
271     case "Piano":
272         profilePP = GetProfileForPitch(pitch, pianoPPLUT, baseMidiNote);
273         profileMF = GetProfileForPitch(pitch, pianoMFLUT, baseMidiNote);
274         profileFF = GetProfileForPitch(pitch, pianoFFLUT, baseMidiNote);
275         break;
276     case "Horn":

```

```

277     profilePP = GetProfileForPitch(pitch, hornPPLUT, baseMidiNote);
278     profileMF = GetProfileForPitch(pitch, hornMFLUT, baseMidiNote);
279     profileFF = GetProfileForPitch(pitch, hornFFLUT, baseMidiNote);
280     break;
281 case "Trumpet":
282     profilePP = GetProfileForPitch(pitch, trumpetPPLUT, baseMidiNote);
283     profileMF = GetProfileForPitch(pitch, trumpetMFLUT, baseMidiNote);
284     profileFF = GetProfileForPitch(pitch, trumpetFFLUT, baseMidiNote);
285     break;
286 default:
287     println("Error:_LUT_type_' + type + "'が見つかりません!!_ピアノを適用しまし
        た。");
288     profilePP = GetProfileForPitch(pitch, pianoPPLUT, baseMidiNote);
289     profileMF = GetProfileForPitch(pitch, pianoMFLUT, baseMidiNote);
290     profileFF = GetProfileForPitch(pitch, pianoFFLUT, baseMidiNote);
291     break;
292 }
293
294 // ベロシティ (0~127) を 0.0~1.0 に正規化
295 float normVel = constrain(velocity, 0, 127) / 127.0f;
296
297 // ベロシティ 閾値
298 float thresholdPP = 32.0f / 127.0f;
299 float thresholdMF = 80.0f / 127.0f;
300 float thresholdFF = 112.0f / 127.0f;
301
302 // 現在のベロシティ層に応じて、ベースとなる波形プロファイル（周波数・振幅の骨組
        み）を決定
303 float[][] baseProfile;
304 float t = 0;
305
306 if (normVel <= thresholdPP) {
307     baseProfile = profilePP;
308 } else if (normVel <= thresholdMF) {
309     // pp ~ mf の間：構造に近い方のスペクトル形状を選択
310     t = map(normVel, thresholdPP, thresholdMF, 0.0f, 1.0f);
311     baseProfile = (t < 0.5f) ? profilePP : profileMF;
312 } else if (normVel <= thresholdFF) {
313     // mf ~ ff の間
314     t = map(normVel, thresholdMF, thresholdFF, 0.0f, 1.0f);
315     baseProfile = (t < 0.5f) ? profileMF : profileFF;
316 } else {
317     baseProfile = profileFF;
318 }
319
320
321 // 振幅が閾値以上の有効なピークのみを抽出してエネルギー計算
322 int numPeaks = baseProfile.length;

```

```

323     int validCount = 0;
324     float ampThreshold = 0.05f;
325     // 有効なデータの数をカウント
326     for (int h = 0; h < numPeaks; h++) {
327         if (baseProfile[h][1] >= ampThreshold) validCount++;
328     }
329     // 判明した有効なサイズの配列
330     float[][] finalProfile = new float[validCount][2];
331
332     int validPeakCount = 0;
333     float energy = 0;
334
335     for (int h = 0; h < numPeaks; h++) {
336         // 0.05未満のデータは完全に無視してスキップ
337         if(baseProfile[h][1] < ampThreshold) continue;
338
339         // 有効なデータだけを finalProfile の前から順に詰める
340         finalProfile[validPeakCount][0] = baseProfile[h][0]; // 周波数 (Hz) 固定
341         finalProfile[validPeakCount][1] = baseProfile[h][1]; // 振幅の初期値
342
343         // 除外されなかった「有効なピークだけ」でエネルギーを計算
344         energy += baseProfile[h][1] * baseProfile[h][1];
345
346         validPeakCount++;
347     }
348
349     // エネルギー総量をもとにゲインを正規化
350     float targetVolume = basePower * pow(normVel, 0.9f);
351     float gain = targetVolume / sqrt(energy + 1e-9f);
352
353     for (int i = 0; i < validPeakCount; i++) {
354         finalProfile[i][1] *= gain;
355     }
356
357     return finalProfile;
358 }
359
360
361
362
363 // クラスの外部からJSONデータを使用したいときの関数
364 float[][] GetLUT(String type) {
365     switch (type) {
366         case "Note":
367             return note;
368
369         case "PianoDecayFF":
370             return pianoDecayFFLUT;

```



```

371     case "PianoWobbleRateFF":
372         return pianoWobbleRateFFLUT;
373     case "PianoWobbleDepthFF":
374         return pianoWobbleDepthFFLUT;
375
376     case "PianoDecayMF":
377         return pianoDecayMFLUT;
378     case "PianoWobbleRateMF":
379         return pianoWobbleRateMFLUT;
380     case "PianoWobbleDepthMF":
381         return pianoWobbleDepthMFLUT;
382
383     case "PianoDecayPP":
384         return pianoDecayPPLUT;
385     case "PianoWobbleRatePP":
386         return pianoWobbleRatePPLUT;
387     case "PianoWobbleDepthPP":
388         return pianoWobbleDepthPPLUT;
389
390     case "Kick":
391         return kickLUT;
392     case "Snare":
393         return snareLUT;
394     case "HiHat":
395         return hihatLUT;
396
397     default:
398         println("Error:_LUT_type_" + type + "'が見つかりません!!");
399         return null; // 存在しない名前が指定された場合は null を返す
400 }
401 }
402 }

```

D.3 InstrumentPlayer.pde

コード 21 InstrumentPlayer.pde

```

1  // =====
2  // 楽器の発音を管理するクラス
3  // 各楽器のインスタンスを生成し、AudioOutputへ割り当てる
4  // =====
5  class InstrumentPlayer {
6      AudioOutput out;
7
8      InstrumentPlayer(AudioOutput out) {
9          this.out = out; // Mainから渡された出力を保持する
10     }
11 }

```

```

12 // フルート
13 void PlayFlute(float midiNote, float duration, float startVel, float endVel, float
    masterVol) {
14     out.playNote(0.0f, duration,
15         new FluteInstrument(midiNote, duration, startVel, endVel, masterVol));
16 }
17
18 // スtringス
19 void PlayStrings(float midiNote, float duration, float startVel, float endVel,
    float masterVol) {
20     out.playNote(0.0f, duration,
21         new StringsInstrument(midiNote, duration, startVel, endVel, masterVol));
22 }
23
24 // ピアノ
25 void PlayPiano(float midiNote, float duration, float velocity, float masterVol) {
26     out.playNote(0.0f, duration,
27         new PianoInstrument(midiNote, velocity, masterVol));
28 }
29
30 // キック
31 void PlayKick(float masterVol, float velocity) {
32     out.playNote(0, 0.1, new KickInstrument(masterVol, velocity));
33 }
34
35 // スネア
36 void PlaySnare(float masterVol, float velocity, float sinAmp) {
37     out.playNote(0, 0.3, new SnareInstrument(masterVol, velocity, sinAmp));
38 }
39
40 // クローズハイハット
41 void PlayHiHat(float masterVol, float velocity) {
42     out.playNote(0, 0.15, new HiHatInstrument(masterVol, velocity));
43 }
44
45
46 // ホルン
47 void PlayHorn(float midiNote, float duration, float startVel, float endVel, float
    masterVol) {
48     out.playNote(0.0f, duration,
49         new HornInstrument(midiNote, duration, startVel, endVel, masterVol));
50 }
51
52 // トランペット
53 void PlayTrumpet(float midiNote, float duration, float startVel, float endVel,
    float masterVol) {
54     out.playNote(0.0f, duration,
55         new TrumpetInstrument(midiNote, duration, startVel, endVel, masterVol));

```

```
56 }
57 }
```

D.4 NoteSampler.pde

コード 22 NoteSampler.pde

```
1 // 既存の変数に volatile をつける
2 volatile int tick;
3 volatile long interval;
4 volatile long lastTime = 0;
5 volatile int[][] returnTick; // {トリガーtick, ジャンプ先tick, ジャンプ先noteIndex}
6 volatile int returnIndex;
7 volatile int noteIndex;
8 volatile boolean isPlaying = false;
9
10 // 音楽専用スレッド用の変数を追加
11 Thread audioThread;
12
13
14
15
16 long GetInterval (int playBPM) {
17     return 60000000000L / (playBPM * 24);
18 }
19
20
21
22 void SoundPlay() {
23
24     // 楽譜の読み取りをスキップしたり戻ったりする
25     if (tick == returnTick[returnIndex][0]) {
26         tick = returnTick[returnIndex][1];
27         noteIndex = returnTick[returnIndex][2];
28         returnIndex ++;
29         if (returnIndex == returnTick.length) returnIndex = 0;
30     }
31
32     // 楽譜データを参照して発音する音符があれば音を鳴らす
33     if (noteIndex < Note.length) {
34         float durationTuning = interval / 1000000000.0f; // tickから秒への変換
35         while (Note[noteIndex][0] == tick) {
36             float duration = float(Note[noteIndex][1]) * durationTuning;
37             float pitch = float(Note[noteIndex][2]);
38             float startVelocity = float(Note[noteIndex][3]);
39             float endVelocity = float(Note[noteIndex][4]);
40             float type = (Note[noteIndex].length == 6) ? float(Note[noteIndex][5]) : 0; //
                typeがない楽譜データにも対応
```

```

41     switch (timbre) {
42         case "Flute": case "Flute2":
43             player.PlayFlute(pitch, duration, startVelocity, endVelocity, 1.0f);
44             break;
45         case "Strings":
46             player.PlayStrings(pitch, duration, startVelocity, endVelocity, 1.0f);
47             break;
48         case "Piano":
49             if (type == 0) player.PlayPiano(pitch, duration, startVelocity, 1.0f);
50             else if (type == 1) player.PlayTrumpet(pitch, duration, startVelocity,
51                 endVelocity, 1.0f);
52             else if (type == 2) player.PlayHorn(pitch, duration, startVelocity,
53                 endVelocity, 1.0f);
54             break;
55         case "Drum":
56             int a = int(pitch) % 12; // MIDIノートからドラム種類を判別
57             if (a == 0) player.PlayKick(1.0f, startVelocity);
58             else if (a == 3) player.PlaySnare(1.0f, startVelocity, endVelocity);
59             else if (a == 4) player.PlayHiHat(1.0f, startVelocity);
60             break;
61         case "Horn":
62             player.PlayHorn(pitch, duration, startVelocity, endVelocity, 1.0f);
63             break;
64         case "Trumpet":
65             player.PlayTrumpet(pitch, duration, startVelocity, endVelocity, 1.0f);
66             break;
67         default:
68             break;
69     }
70     noteIndex ++;
71     if (noteIndex == Note.length) break;
72 }
73
74
75
76
77
78 int[][] Note = {
79     // (setupでの上書きをコメント文にすれば演奏可能)
80
81
82     // 【1小節目】
83     {0, 480, 60, 96, 96, 0}, // [0] C4
84
85     // 【6小節目】
86     {480, 480, 62, 96, 96, 0}, // [1] D4

```

```

87
88 // 【11小節目】
89 {960, 480, 64, 96, 96, 0}, // [2] E4
90
91 // 【16小節目】
92 {1440, 480, 65, 96, 96, 0}, // [3] F4
93
94 // 【21小節目】
95 {1920, 480, 67, 96, 96, 0}, // [4] G4
96
97 // 【26小節目】
98 {2400, 480, 69, 96, 96, 0}, // [5] A4
99
100 // 【31小節目】
101 {2880, 480, 71, 96, 96, 0}, // [6] B4
102
103
104 };

```

D.5 AddEffect.pde

コード 23 AddEffect.pde

```

1 // =====
2 // 空間反響音（リバーブ）を生成するクラス
3 // =====
4 class AddReverb extends UGen {
5     public UGenInput audio;
6
7     float[] impulseResponse;
8
9     // 左右ごとの記憶バッファ
10    float[] historyBufferL;
11    float[] historyBufferR;
12
13    int bufferIndex = 0;
14    int kernelSize;
15
16    AddReverb(int size, float level) {
17        audio = new UGenInput(InputType.AUDIO);
18        this.kernelSize = size;
19
20        impulseResponse = new float[kernelSize];
21        historyBufferL = new float[kernelSize];
22        historyBufferR = new float[kernelSize];
23
24        // インパルス応答の生成
25        for (int i = 0; i < kernelSize; i++) {

```

```

26     float decay = exp(-3.8f * i / kernelSize);
27     impulseResponse[i] = random(-1.0f, 1.0f) * decay * level;
28 }
29 }
30
31 // 音声の1サンプルフレームごとに関呼び出される処理
32 protected void uGenerate(float[] channels) {
33     // 入力音を「配列」として取得し、左右の音を別々に取り出す
34     float[] inputs = audio.getLastValues();
35     float inL = inputs[0];
36     float inR = (inputs.length > 1) ? inputs[1] : inL; // モノラル時はL/R共通化
37
38     // 最新の入力を左右それぞれの記憶バッファに格納
39     historyBufferL[bufferIndex] = inL;
40     historyBufferR[bufferIndex] = inR;
41
42     // たたみ込み積分の計算
43     float outL = 0;
44     float outR = 0;
45     for (int j = 0; j < kernelSize; j++) {
46         int idx = (bufferIndex - j + kernelSize) % kernelSize;
47         outL += historyBufferL[idx] * impulseResponse[j];
48         outR += historyBufferR[idx] * impulseResponse[j];
49     }
50
51     // 記憶バッファの書き込み位置を1つ進める（左右共通の時間が進む）
52     bufferIndex = (bufferIndex + 1) % kernelSize;
53
54     // 結果を出力先のチャンネルに合成
55     if (channels.length == 1) {
56         channels[0] = inL + outL; // モノラルの出力
57     } else if (channels.length >= 2) {
58         channels[0] = inL + outL; // 左の出力
59         channels[1] = inR + outR; // 右の出力
60     }
61 }
62 }

```

D.6 Flute.pde

コード 24 Flute.pde

```

1 // =====
2 // フルート (Flute) のクラス
3 // =====
4 class FluteInstrument implements Instrument {
5     byte numHarmonics;
6

```

```

7   ADSR masterEnv;
8   ADSR[] harmonicEnvs;
9
10  Line[] wobbleLines;
11  float[] wobbleFactors;
12
13  Line[] ampLines;
14  float[] startAmps;
15  float[] endAmps;
16
17  ADSR breathEnv;
18
19
20
21  FluteInstrument(float midiNote, float duration, float startVel, float endVel, float
    masterVol) {
22      Summer mixer = new Summer();
23
24      float masterAttackTime = 0.18f;
25      masterAttackTime = min(masterAttackTime, duration - 0.1f);
26      if (masterAttackTime < 0.035f) masterAttackTime = 0.035f;
27
28      masterEnv = new ADSR(1.0f, masterAttackTime, 0.1f, 0.9f, 0.18f);
29      mixer.patch(masterEnv);
30
31
32      float profileVel = (startVel > 0) ? startVel : endVel;
33      if (profileVel <= 0) profileVel = 64.0f; // どちらも0の場合のセーフティ
34
35      float startGainFactor = pow(startVel / profileVel, 0.9f);
36      float endGainFactor   = pow(endVel / profileVel, 0.9f);
37
38      // プロファイルを取得
39      float[][] startProfile = utils.GetDynamicProfile(midiNote, profileVel, "Flute",
        59, 1.3f);
40
41
42      float volFactor = masterVol * 0.022f;
43
44      numHarmonics = (byte)startProfile.length;
45
46      harmonicEnvs = new ADSR[numHarmonics];
47      ampLines     = new Line[numHarmonics];
48      wobbleLines  = new Line[numHarmonics];
49      wobbleFactors = new float[numHarmonics];
50      startAmps    = new float[numHarmonics];
51      endAmps      = new float[numHarmonics];

```

```

52
53
54 byte baseWaveNum = 0;
55 for (byte i = 1; i < numHarmonics; i++) {
56     if (startProfile[i-1][0] < startProfile[i][0]) baseWaveNum = i;
57 }
58
59
60 // 倍音ごとの自然な振幅揺らぎ深度
61 float[] wobbleDepths = {0.1250f, 0.1616f, 0.2078f, 0.2340f, 0.2114f, 0.2742f,
    0.3716f, 0.2946f, 0.7420f, 0.8796f, 0.7850f, 0.5070f, 0.6242f, 0.7654f,
    0.6702f, 0.3898f, 0.8850f, 0.5942f, 0.3152f, 0.5030f};

62
63 // 息遣い全体のスピード（全倍音で同期）
64 float globalBreathSpeed = random(2.5f, 3.5f);
65
66
67
68 // --- 1. 加算合成パート ---
69 for (int i = 0; i < numHarmonics; i++) {
70
71     float targetFreq = startProfile[i][0];
72
73     Oscil osc = new Oscil(targetFreq, 0.0f, Waves.SINE);
74     osc.setPhase(random(1.0f));
75
76     // 開始・終了音量を算出
77     float baseAmp = startProfile[i][1] * volFactor;
78     startAmps[i] = baseAmp * startGainFactor;
79     endAmps[i] = baseAmp * endGainFactor;
80
81     // ① ビブラート (FM変調)
82     float vibDepthHz = targetFreq * 0.0015f;
83     Oscil freqController = new Oscil(globalBreathSpeed, vibDepthHz/10, Waves.SINE);
84     freqController.offset.setLastValue(targetFreq);
85     freqController.patch(osc.frequency);
86
87     // ② 音量揺らぎ (AM変調)
88     wobbleFactors[i] = random(0.1f, 0.2f);
89     wobbleFactors[i] = wobbleDepths[i];
90     wobbleLines[i] = new Line();
91     Oscil ampLFO = new Oscil(globalBreathSpeed, 0.0f, Waves.SINE);
92     ampLFO.setPhase(random(1.0f));
93     wobbleLines[i].patch(ampLFO.amplitude);
94     ampLFO.patch(osc.amplitude);
95
96     ampLines[i] = new Line();

```



```

97     ampLines[i].patch(osc.amplitude);
98
99     float attackTime = (i == baseWaveNum) ? 0.058f : 0.395f;
100     harmonicEnvs[i] = new ADSR(1.0f, attackTime/3, 1.0f, 0.7f, 10.0f);
101     osc.patch(harmonicEnvs[i]);
102     harmonicEnvs[i].patch(mixer);
103 }
104
105
106 // --- 2. 息のノイズパート ---
107 float startNorm = constrain(startVel, 0, 127) / 127.0f;
108 float attackNoiseVol = masterVol * 0.008f * pow(startNorm, 0.9f);
109 Noise breathNoise = new Noise(attackNoiseVol, Noise.Tint.WHITE);
110
111 MoogFilter breathFilter = new MoogFilter(581.4f, 0.15f);
112 breathFilter.type = MoogFilter.Type.BP;
113
114 breathEnv = new ADSR(1.0f, 0.001f, 0.2f, 0.0f, 0.0f);
115
116 breathNoise.patch(breathFilter).patch(breathEnv).patch(mixer);
117 }
118
119
120 void noteOn(float dur) {
121     masterEnv.noteOn();
122     float safeDur = dur + 0.5f;
123
124     for(int i = 0; i < numHarmonics; i++) {
125         harmonicEnvs[i].noteOn();
126
127         ampLines[i].activate(safeDur, startAmps[i], endAmps[i]);
128
129         if (wobbleLines[i] != null) {
130             float factor = wobbleFactors[i];
131             wobbleLines[i].activate(safeDur, startAmps[i] * factor, endAmps[i] * factor);
132         }
133     }
134
135     breathEnv.noteOn();
136
137     masterEnv.patch(longOut);
138 }
139
140
141
142 void noteOff() {
143     masterEnv.noteOff();
144 }

```

```

145     for(int i=0; i<numHarmonics; i++) harmonicEnvs[i].noteOff();
146
147     masterEnv.unpatchAfterRelease(longOut);
148 }
149 }

```

D.7 Strings.pde

コード 25 Strings.pde

```

1  // =====
2  // スtringス (Strings) のクラス
3  // (ヴァイオリン・ビオラ・チェロ・コントラバス)
4  // =====
5  class StringsInstrument implements Instrument {
6
7      ADSR masterEnv;
8
9      byte maxTotalOscils = 40;
10
11      Line[] ampLines = new Line[maxTotalOscils];
12      Line[] vibDepthLines = new Line[maxTotalOscils];
13
14      float[] startAmps = new float[maxTotalOscils];
15      float[] endAmps = new float[maxTotalOscils];
16      float[] targetVibDepths = new float[maxTotalOscils];
17
18      byte totalActiveOscils = 0;
19
20      Line[] wobbleLines = new Line[maxTotalOscils];
21
22
23
24      StringsInstrument(float midiNote, float duration, float startVel, float endVel,
25                          float masterVol) {
26          Summer mixer = new Summer();
27
28          // 音符の長さに応じた動的アタック (弓を弾くスピード)
29          float attackTime = 0.3f - constrain(startVel, 0, 127) / 127.0f * 0.25f;
30          attackTime = min(attackTime, duration - 0.1f);
31          if (attackTime < 0.035f) attackTime = 0.035f;
32
33          masterEnv = new ADSR(1.0f, attackTime, 0.5f, 0.9f, 0.12f);
34          mixer.patch(masterEnv);
35
36          float volFactor = masterVol * 0.020f;
37

```

```

38
39
40 // --- 自然なビブラートの数式 ---
41 float baseVibSpeed = map(midiNote, 48, 100, 5.2f, 6.5f);
42 float globalVibSpeed = baseVibSpeed + random(-0.2f, 0.2f);
43
44 float fmDepthCents = map(midiNote, 48, 100, 10.0f, 20.0f) * random(0.9f, 1.0f) *
    0.2f;

45
46
47
48
49 // --- 楽器セクションのクロスフェード比率 ---
50 float baseBassRatio = 0.0f;
51 float baseCelloRatio = 0.0f;
52 float baseViolaRatio = 0.0f;
53 float baseViolinRatio = 0.0f;
54
55 if (midiNote <= 36) {
56     baseBassRatio = 1.0f;
57 } else if (midiNote <= 48) {
58     baseBassRatio = map(midiNote, 36, 48, 1.0f, 0.0f);
59     baseCelloRatio = 1.0f - baseBassRatio;
60 } else if (midiNote <= 60) {
61     baseCelloRatio = map(midiNote, 48, 60, 1.0f, 0.0f);
62     baseViolaRatio = 1.0f - baseCelloRatio;
63 } else if (midiNote <= 72) {
64     baseViolaRatio = map(midiNote, 60, 72, 1.0f, 0.0f);
65     baseViolinRatio = 1.0f - baseViolaRatio;
66 } else {
67     baseViolinRatio = 1.0f;
68 }
69
70 float bassBlend = baseBassRatio;
71 float celloBlend = baseCelloRatio;
72 float violaBlend = baseViolaRatio;
73 float violinBlend = baseViolinRatio;
74
75
76
77
78 // 開始・終了時ベロシティによる音量変化
79 float profileVel = (startVel > 0) ? startVel : endVel;
80 if (profileVel <= 0) profileVel = 64.0f; // 両方0の場合の完全セーフティ
81
82 float startGainFactor = pow(startVel / profileVel, 0.9f);
83 float endGainFactor = pow(endVel / profileVel, 0.9f);

```

```

84
85 // 各楽器のプロファイルごとにオシレーター群を生成
86 totalActiveOscils = 0; // カウンターリセット
87 // GetDynamicProfileにはプロファイル、createInstrumentOscillatorsにはファクターを
   渡す
88 if (bassBlend > 0.0f) createInstrumentOscillators(utils.GetDynamicProfile(
   midiNote, profileVel, "Bass", 24, 0.9f), bassBlend, (bassBlend < 1.0f) ? 20 :
   30, volFactor, globalVibSpeed, fmDepthCents, mixer, startGainFactor,
   endGainFactor);
89 if (celloBlend > 0.0f) createInstrumentOscillators(utils.GetDynamicProfile(
   midiNote, profileVel, "Cello", 36, 0.9f), celloBlend, (celloBlend < 1.0f) ?
   20 : 30, volFactor, globalVibSpeed, fmDepthCents, mixer, startGainFactor,
   endGainFactor);
90 if (violaBlend > 0.0f) createInstrumentOscillators(utils.GetDynamicProfile(
   midiNote, profileVel, "Viola", 48, 0.9f), violaBlend, (violaBlend < 1.0f) ?
   20 : 30, volFactor, globalVibSpeed, fmDepthCents, mixer, startGainFactor,
   endGainFactor);
91 if (violinBlend > 0.0f) createInstrumentOscillators(utils.GetDynamicProfile(
   midiNote, profileVel, "Violin", 55, 0.9f), violinBlend, (violinBlend < 1.0f)
   ? 20 : 30, volFactor, globalVibSpeed, fmDepthCents, mixer, startGainFactor,
   endGainFactor);

92
93 }
94
95
96
97 private void createInstrumentOscillators(final float[][] profile, float blendRatio,
   int maxPeaks, float volFactor, float globalVibSpeed, float fmDepthCents,
   Summer mixer, float startGainFactor, float endGainFactor) {
98     if (profile == null || profile.length == 0) return;
99
100     int rawLength = profile.length;
101     final int limitPeaks = min(maxPeaks, rawLength);
102
103     // 振幅の大きい成分から優先してピックアップ
104     Integer[] indices = new Integer[rawLength];
105     for (int i = 0; i < rawLength; i++) indices[i] = i;
106     Arrays.sort(indices, new Comparator<Integer>() {
107         public int compare(Integer a, Integer b) {
108             return Float.compare(profile[b][1], profile[a][1]);
109         }
110     });
111
112     float[][] filteredProfile = new float[limitPeaks][2];
113     for (int i = 0; i < limitPeaks; i++) {
114         filteredProfile[i][0] = profile[indices[i]][0];
115         filteredProfile[i][1] = profile[indices[i]][1];

```

```

116     }
117
118     // 抽出後、周波数順にソートし直す
119     Arrays.sort(filteredProfile, new Comparator<float[]>() {
120         public int compare(float[] a, float[] b) {
121             return Float.compare(a[0], b[0]);
122         }
123     });
124
125     for (int h = 0; h < limitPeaks; h++) {
126         if (totalActiveOscils >= maxTotalOscils) break;
127
128         float targetFreq = filteredProfile[h][0];
129         float rawAmp      = filteredProfile[h][1];
130
131         if (targetFreq > 16000.0f) continue;
132
133         // 数学的高域フィルター：耳障りな超高域を削る
134         if (targetFreq > 6000.0f) {
135             float filterFactor = map(targetFreq, 6000.0f, 16000.0f, 1.0f, 0.0f);
136             rawAmp *= constrain(filterFactor, 0.0f, 1.0f);
137         }
138
139         // 実際の開始音量と終了音量を決定
140         float baseAmp = rawAmp * volFactor * blendRatio;
141         float finalStartAmp = baseAmp * startGainFactor;
142         float finalEndAmp   = baseAmp * endGainFactor;
143
144         int idx = totalActiveOscils;
145         startAmps[idx] = finalStartAmp;
146         endAmps[idx]   = finalEndAmp;
147
148         Oscil osc = new Oscil(targetFreq, 0.0f, Waves.SINE);
149         osc.setPhase(random(1.0f));
150
151
152         // --- ビブラート (FM変調) ---
153         float vibDepthRatio = pow(2.0f, fmDepthCents / 1200.0f) - 1.0f;
154         targetVibDepths[idx] = targetFreq * vibDepthRatio;
155
156         Oscil freqController = new Oscil(globalVibSpeed, 0.0f, Waves.SINE);
157         freqController.offset.setLastValue(targetFreq);
158         freqController.patch(osc.frequency);
159
160         vibDepthLines[idx] = new Line();
161         vibDepthLines[idx].patch(freqController.amplitude);
162
163

```

```

164 // --- 音量揺らぎ (AM変調) ---
165 wobbleLines[idx] = new Line();
166 Oscil ampLFO = new Oscil(globalVibSpeed, 0.0f, Waves.SINE);
167 ampLFO.setPhase(random(1.0f));
168
169 wobbleLines[idx].patch(ampLFO.amplitude);
170 ampLFO.patch(osc.amplitude);
171
172
173 // ベース音量の適用
174 ampLines[idx] = new Line();
175 ampLines[idx].patch(osc.amplitude);
176
177 osc.patch(mixer);
178 totalActiveOscils++;
179 }
180 }
181
182
183
184
185 void noteOn(float dur) {
186
187     masterEnv.noteOn();
188     float safeDur = dur + 0.7f;
189     float vibDelayTime = min(0.5f, dur);
190
191     for(int i = 0; i < totalActiveOscils; i++) {
192         if (ampLines[i] == null || vibDepthLines[i] == null || wobbleLines[i] == null)
193             continue;
194
195         // 音量のフェード
196         ampLines[i].activate(safeDur, startAmps[i], endAmps[i]);
197
198         // ウナリのフェードダウン
199         float wobbleDepth = map(i, 0, maxTotalOscils - 1, 0.15f, 0.60f);
200         wobbleLines[i].activate(safeDur, startAmps[i] * wobbleDepth, endAmps[i] *
201             wobbleDepth);
202
203         // ビブラートの立ち上がり
204         vibDepthLines[i].activate(vibDelayTime, 0.0f, targetVibDepths[i]);
205     }
206
207     masterEnv.patch(longOut);
208 }

```

```

208
209 void noteOff() {
210     masterEnv.noteOff();
211     masterEnv.unpatchAfterRelease(longOut);
212 }
213 }

```

D.8 Piano.pde

コード 26 Piano.pde

```

1 // =====
2 // ピアノ (Piano) のクラス
3 // =====
4 class PianoInstrument implements Instrument {
5     byte numHarmonics;
6
7     ADSR masterEnv; // ダンパー (ミュート) の役割を担うマスター
8     ADSR hammerEnv; // ハンマーが弦を叩くノイズ用
9
10    // 個別の減衰 (Damp) を管理するリスト
11    // 芯の弦と共鳴弦でDampの数変動するため、配列ではなくArrayListを使用
12    ArrayList<Damp> damp = new ArrayList<Damp>();
13
14    PianoInstrument(float midiNote, float velocity, float masterVol) {
15        Summer mixer = new Summer();
16
17        // ユーザー定義のペロシティ閾値
18        float thresholdPP = 32.0f / 127.0f;
19        float thresholdMF = 80.0f / 127.0f;
20        float thresholdFF = 112.0f / 127.0f;
21
22        // ペロシティによる強弱層の判定
23        String baseVelocity;
24        float t = 0;
25        if (velocity <= thresholdPP) {
26            baseVelocity = "PP";
27        } else if (velocity <= thresholdMF) {
28            // pp ~ mf の間：構造に近い方のスペクトル形状を選択
29            t = map(velocity, thresholdPP, thresholdMF, 0.0f, 1.0f);
30            baseVelocity = (t < 0.5f) ? "PP" : "MF";
31        } else if (velocity <= thresholdFF) {
32            // mf ~ ff の間
33            t = map(velocity, thresholdMF, thresholdFF, 0.0f, 1.0f);
34            baseVelocity = (t < 0.5f) ? "MF" : "FF";
35        } else {
36            baseVelocity = "FF";
37        }

```

```

38
39 float[][]pianoDecayLUT = utils.GetLUT("PianoDecay" + baseVelocity);
40 float[][]pianoWobbleRateLUT = utils.GetLUT("PianoWobbleRate" + baseVelocity);
41 float[][]pianoWobbleDepthLUT = utils.GetLUT("PianoWobbleDepth" + baseVelocity);
42
43
44
45 // =====
46 // 1. 全体のエンベロープ設定（ダンパーの挙動）
47 // =====
48 masterEnv = new ADSR(1.0f, 0.001f, 0.4f, 0.5f, 0.09f);
49 mixer.patch(masterEnv);
50
51 float normVel = pow(constrain(velocity, 0, 127) / 127.0f, 0.9f);
52 float volFactor = masterVol * 0.026f;
53
54
55
56 // =====
57 // 2. LUTデータの取得とインハーモニシティ設定
58 // =====
59 // 強弱に応じた倍音構成（音色レシピ）を取得
60 float[][] profile = utils.GetDynamicProfile(midiNote, velocity, "Piano", 21, 0.9f
    );

61
62 // 実測ピークデータから、今回のループ回数を動的に確定させる！
63 numHarmonics = (byte)profile.length;
64
65 int lutIndex = constrain(round(midiNote) - 21, 0, 87); // A0(MIDI:21)基点
66
67
68 // --- データ欠損時用（数式モデル）の事前計算 ---
69 float mathBaseDecay = 29.0f * pow(0.9757f, midiNote);
70 if (midiNote < 40) mathBaseDecay = min(mathBaseDecay, 20.0f); // 低音
    の物理的限界（頭打ち）
71 if (midiNote >= 89) mathBaseDecay *= map(midiNote, 89, 108, 1.0f, 3.0f); // 超高
    音のダンパーレス構造による伸び

72
73 float mathBaseWobbleRate = map(abs(midiNote - 66.0f), 0, 45, 0.5f, 4.5f); // うな
    りの速さ(MIDI66を底としたU字型)
74 float mathBaseDepth = map(midiNote, 21, 108, 0.03f, 0.15f); // うな
    りの深さ(高音ほど深い)

75
76 // リバース（共鳴）が最大音量になるまでの時間。低音はゆっくり、高音は速く立ち上がる。

```



```

77 float buildUpTime = map(midiNote, 21, 108, 3.0f, 0.5f);
78
79
80
81 // =====
82 // 3. 弦のモデリング (デュアル・ストリング構造)
83 // =====
84 for (int i = 0; i < numHarmonics; i++) {
85     float amp = profile[i][1] * volFactor;
86     if (amp <= 0.0001f) continue; // 音量が極端に小さい倍音は負荷軽減のためスキップ
87
88
89     float targetFreq = profile[i][0]; // 実測された上擦り済みの正確な周波数
90     if (targetFreq > 22050.0f) continue; // 人間の可聴域を超えたらスキップ
91
92     float decayTime, wobbleRate, wobbleDepth;
93
94     // JSONから減衰率・うなり速度を取得
95     // なければ数式モデル補完
96     if (pianoDecayLUT != null && pianoWobbleRateLUT != null && pianoWobbleDepthLUT
97         != null
98         && pianoDecayLUT[lutIndex] != null && i < pianoDecayLUT[lutIndex].
99             length) {
100         decayTime = pianoDecayLUT[lutIndex][i];
101         wobbleRate = pianoWobbleRateLUT[lutIndex][i];
102         wobbleDepth = pianoWobbleDepthLUT[lutIndex][i];
103
104         // もしデータが0、または異常値だった場合は数式モデルで安全に保護する
105         if (decayTime <= 0.1f) {
106             float harmonicDecayCurve = (i <= 14) ? map(i, 0, 14, 1.0f, 0.3f) : map(i,
107                 14, 80, 0.3f, 1.5f);
108             decayTime = max(mathBaseDecay * harmonicDecayCurve, 0.1f);
109         }
110     } else {
111         // データがない場合
112         float harmonicDecayCurve = (i <= 14) ? map(i, 0, 14, 1.0f, 0.3f) : map(i, 14,
113             80, 0.3f, 1.5f);
114         decayTime = max(mathBaseDecay * harmonicDecayCurve, 0.1f);
115         wobbleRate = mathBaseWobbleRate * (1.0f + i * 0.02f);
116         wobbleDepth = mathBaseDepth * pow(0.9f, i);
117     }
118
119     decayTime *= 0.8;
120
121     // -----
122     // レイヤーA：アタックの芯となる弦

```

```

120 // -----
121 Oscil coreOsc = new Oscil(targetFreq, amp, Waves.SINE);
122 // アタックのインパクトを最大化するため、位相(波のスタート位置)は0付近に固定
123 coreOsc.setPhase(random(1.0f));
124
125 // 0.001秒で素早く立ち上がり、それぞれの寿命(decayTime)で消えていく
126 Damp coreDamp = new Damp(0.001f, decayTime);
127 coreOsc.patch(coreDamp).patch(mixer);
128 damp.add(coreDamp);
129
130
131 // -----
132 // レイヤーB：時間差で来る響き（共鳴弦）
133 // -----
134 float reverbAmp = amp * wobbleDepth; // 共鳴の音量は、芯の音の数%~十数%程度に
    抑える

135
136 // うなりのスピード(wobbleRate)が設定されており、かつ音量が十分ある場合のみ共鳴
    弦を生成
137 if (reverbAmp > 0.0001f && wobbleRate > 0.0f) {
138     // 周波数をわずかにズラす（Detune）ことで、芯の弦と干渉して自然な「うなり(
        Beat)」を生む
139     Oscil resOsc = new Oscil(targetFreq + wobbleRate, reverbAmp, Waves.SINE);
140     resOsc.setPhase(random(1.0f));
141
142     // 数秒かけてゆっくり音量を上げる
143     Damp resDamp = new Damp(buildUpTime, decayTime);
144     resOsc.patch(resDamp).patch(mixer);
145     damp.add(resDamp);
146 }
147 }
148
149
150 // =====
151 // 4. ハンマーの打撃ノイズ（アタック）
152 // =====
153 float hammerVol = masterVol * 0.003f * normVel;
154 Noise hammerNoise = new Noise(hammerVol, Noise.Tint.WHITE);
155
156 // 高音ほど硬い音にフィルタリング
157 float hammerCutoff = map(midiNote, 21, 108, 600f, 4000f);
158 MoogFilter hammerFilter = new MoogFilter(hammerCutoff, 0.1f);
159 hammerFilter.type = MoogFilter.Type.LP;
160
161 hammerEnv = new ADSR(1.0f, 0.001f, 0.02f, 0.0f, 0.01f);
162
163 hammerNoise.patch(hammerFilter).patch(hammerEnv);

```

```

164 }
165
166
167 void noteOn(float dur) {
168     masterEnv.noteOn();
169     hammerEnv.noteOn();
170
171     // 弦のDampを一斉にスタート
172     for (Damp d : damps) d.activate();
173
174     masterEnv.patch(longOut);
175     hammerEnv.patch(shortOut); // ハンマーノイズは独立して出力
176 }
177
178 void noteOff() {
179     masterEnv.noteOff(); // ダンパーペダルを降ろす
180     hammerEnv.noteOff();
181     masterEnv.unpatchAfterRelease(longOut);
182     hammerEnv.unpatchAfterRelease(shortOut);
183 }
184 }

```

D.9 Drum.pde

コード 27 Drum.pde

```

1 // =====
2 // 1. キック (Kick) のクラス
3 // =====
4 class KickInstrument implements Instrument {
5
6     ADSR masterEnv;
7     ADSR[] harmonicEnvs;
8
9     KickInstrument(float masterVol, float velocity) {
10         Summer mixer = new Summer();
11
12         float[][]kickLUT = utils.GetLUT("Kick");
13
14         masterEnv = new ADSR(1.0f, 0.001f, 0.0f, 1.0f, 0.1f);
15         mixer.patch(masterEnv);
16
17         int numComponents = 50;
18         harmonicEnvs = new ADSR[numComponents];
19         Line[] pitchSweeps = new Line[numComponents];
20
21         float baseAmp = masterVol * pow(constrain(velocity, 0, 127) / 127.0f, 0.9f) *
            14.0f;

```

```

22
23 // LUTの成分から加算合成
24 for (int i = 0; i < numComponents; i++) {
25     if (kickLUT[i] == null || kickLUT[i].length < 3) continue;
26
27     float freq = kickLUT[i][0] * random(0.95f, 1.05f);
28     float mag = kickLUT[i][1];
29     float decayTime = kickLUT[i][2];
30     float amp = (baseAmp * mag) / numComponents;
31
32     Oscil tone = new Oscil(freq, amp, Waves.SINE);
33     tone.setPhase(random(1.0f)); // 初期位相を散らしてアタックのピーク割れを防ぐ
34
35     // 個別の成分のエンベロープ
36     harmonicEnvs[i] = new ADSR(1.0f, 0.001f, decayTime*0.5f, 0.0f, 0.1f);
37
38     // ピッチスイープ
39     pitchSweeps[i] = new Line(0.01f, freq * 1.5f, freq);
40     pitchSweeps[i].patch(tone.frequency);
41
42     // 結線: Oscil -> 個別ADSR -> Mixer
43     tone.patch(harmonicEnvs[i]);
44     harmonicEnvs[i].patch(mixer);
45 }
46 }
47
48 void noteOn(float duration) {
49     masterEnv.noteOn();
50     for (int i = 0; i < harmonicEnvs.length; i++) {
51         if (harmonicEnvs[i] != null) harmonicEnvs[i].noteOn();
52     }
53     masterEnv.patch(shortOut);
54 }
55
56 void noteOff() {
57     masterEnv.noteOff();
58     for (int i = 0; i < harmonicEnvs.length; i++) {
59         if (harmonicEnvs[i] != null) harmonicEnvs[i].noteOff();
60     }
61     masterEnv.unpatchAfterRelease(shortOut);
62 }
63 }
64
65
66
67
68

```

```

69 // =====
70 // 2. スネア (Snare) のクラス
71 // =====
72 class SnareInstrument implements Instrument {
73
74     ADSR masterEnv;
75     ADSR bodyAmpEnv;
76     ADSR overtoneAmpEnv;
77     ADSR snappyAmpEnv;
78
79     SnareInstrument(float masterVol, float velocity, float sinAmp) {
80         Summer mixer = new Summer();
81
82         // 全体の音量調整
83         float baseAmp = masterVol * pow(constrain(velocity, 0, 127) / 127.0f, 0.9f) *
84             0.15f;
85         float sinSet = constrain(sinAmp, 0, 127) / 127.0f;
86
87         // =====
88         // パラメータ設定
89         // =====
90
91         // --- 1. 胴鳴り (ベースとなる太さ) ---
92         float bodyFreq = 172.3f * random(0.99f, 1.01f);
93         float bodyPitchMod = 3.0f;
94         float bodySweepTime = 0.025f;
95         float bodyDecay = 0.07f;
96         float bodyVolRatio = 0.63f * sinSet;
97
98         // --- 2. 倍音 (カンカンしたヘッドの芯) ---
99         float overtoneFreq = 323.0f * random(0.98f, 1.02f);
100         float overtoneDecay = 0.048f;
101         float overtoneVolRatio = 0.55f * sinSet;
102
103         // --- 3. スナッピー (ホワイトノイズ) ---
104         float noiseDecay = 0.22f;
105         float noiseVolRatio = 3.5f;
106         float noiseHPCutoff = 2500.0f * random(0.9f, 1.1f);
107         float noiseLPCutoff1 = 2800.0f * random(0.9f, 1.1f);
108         float noiseLPCutoff2 = 4000.0f * random(0.9f, 1.1f);
109         float noiseLPCutoff3 = 7000.0f * random(0.9f, 1.1f);
110
111         // =====
112         // 各モジュールの構築とパッチング
113         // =====
114
115         // --- 1. 胴鳴りパートの構築 ---

```

```

116 Oscil bodyOsc = new Oscil(bodyFreq, baseAmp * bodyVolRatio, Waves.SINE);
117 // 叩いた瞬間に音程を急激に下げる (アタック感の演出)
118 Line bodyPitchEnv = new Line(bodySweepTime, bodyFreq * bodyPitchMod, bodyFreq);
119 bodyPitchEnv.patch(bodyOsc.frequency);
120 // 音量エンベロープ (Attack, Decay, Sustain, Release)
121 bodyAmpEnv = new ADSR(1.0f, 0.001f, bodyDecay, 0.0f, 0.05f);
122
123 bodyOsc.patch(bodyAmpEnv).patch(mixer);
124
125
126
127 // --- 2. 倍音パートの構築 ---
128 Oscil overtoneOsc = new Oscil(overtoneFreq, baseAmp * overtoneVolRatio, Waves.
    SINE);
129 overtoneAmpEnv = new ADSR(1.0f, 0.002f, overtoneDecay, 0.0f, 0.05f);
130
131 overtoneOsc.patch(overtoneAmpEnv).patch(mixer);
132
133
134
135 // --- 3. スナッピー (Noise) パートの構築 ---
136 Noise snappyNoise = new Noise(baseAmp * noiseVolRatio, Noise.Tint.WHITE);
137
138 // フィルター
139 HighPassSP snappyHP = new HighPassSP(noiseHPCutoff, out.sampleRate());
140 LowPassSP snappyLP1 = new LowPassSP(noiseLPCutoff1, out.sampleRate());
141 LowPassSP snappyLP2 = new LowPassSP(noiseLPCutoff2, out.sampleRate());
142 LowPassSP snappyLP3 = new LowPassSP(noiseLPCutoff3, out.sampleRate());
143
144 snappyAmpEnv = new ADSR(1.0f, 0.003f, noiseDecay, 0.0f, 0.05f);
145
146 // ホワイトノイズ -> フィルター -> エンベロープ -> ミキサー
147 snappyNoise.patch(snappyHP).patch(snappyLP1).patch(snappyLP2).patch(snappyLP3).
    patch(snappyAmpEnv).patch(mixer);

148
149
150
151 // --- 4. マスターエンベロープ ---
152 masterEnv = new ADSR(1.0f, 0.0001f, 0.0f, 1.0f, 0.01f);
153 mixer.patch(masterEnv);
154 }
155
156 void noteOn(float duration) {
157     masterEnv.noteOn();
158     bodyAmpEnv.noteOn();
159     overtoneAmpEnv.noteOn();
160     snappyAmpEnv.noteOn();

```

```

161
162     masterEnv.patch(shortOut);
163 }
164
165 void noteOff() {
166     masterEnv.noteOff();
167     bodyAmpEnv.noteOff();
168     overtoneAmpEnv.noteOff();
169     snappyAmpEnv.noteOff();
170
171     masterEnv.unpatchAfterRelease(shortOut);
172 }
173 }
174
175
176
177 // =====
178 // 3. ハイハット (HiHat) のクラス
179 // =====
180 class HiHatInstrument implements Instrument {
181
182     ADSR masterEnv;
183     ADSR[] toneEnvList;
184     ADSR     noiseEnv;
185
186     HiHatInstrument(float masterVol, float velocity) {
187         Summer mixer = new Summer();
188
189         // 全体の音量調整
190         float baseAmp = masterVol * pow(constrain(velocity, 0, 127) / 127.0f, 0.9f) *
            0.04f;
191
192
193         // =====
194         // パラメータ設定
195         // =====
196
197         // --- 1. 金属共鳴 (チキチキ・カンカンした芯の成分) ---
198         float[][] toneList = {
199             {21.53f, 0.0619f, 0.131f},
200             {7149.0f, 1.00f, 0.109f},
201             {7644.3f, 0.62f, 0.040f},
202             {5964.7f, 0.61f, 0.060f},
203             {13006.1f, 0.52f, 0.015f},
204             {5555.57f, 0.4585f, 0.118f},
205             {14610.28f, 0.1893f, 0.091f},
206             {4931.1f, 0.1834f, 0.109f}

```

```

207         };
208
209         // --- 2. 打撃摩擦 ---
210         float noiseVolRatio = 6.00f * random(0.99f, 1.01f);
211         float noiseHPCutoff1 = 7000.0f * random(0.9f, 1.1f);
212         float noiseHPCutoff2 = 4000.0f * random(0.9f, 1.1f);
213         float noiseLPCutoff1 = 5000.0f * random(0.9f, 1.1f);
214         float noiseLPCutoff2 = 30000.0f * random(0.9f, 1.1f);
215         float noiseDecay      = 0.080f;
216
217
218         // =====
219         // 各モジュールの構築とパッチング
220         // =====
221
222         // --- 1. 金属共鳴パートの構築 ---
223         //toneOscList = new Oscil[toneList.length];
224         toneEnvList = new ADSR[toneList.length];
225
226         for (int i = 0; i < toneList.length; i++){
227             Oscil toneOsc = new Oscil(toneList[i][0] * random(0.98f, 1.02f), baseAmp *
                toneList[i][1], Waves.SINE);
228             toneOsc.setPhase(random(1.0f));
229             toneEnvList[i] = new ADSR(1.0f, 0.001f, toneList[i][2]*0.5, 0.0f, 0.01f);
230             toneOsc.patch(toneEnvList[i]).patch(mixer);
231         }
232
233         // --- 2. ノイズパートの構築 ---
234         Noise hatNoise = new Noise(baseAmp * noiseVolRatio, Noise.Tint.WHITE);
235
236         // フィルター
237         HighPassSP hatHP1      = new HighPassSP(noiseHPCutoff1, out.sampleRate());
238         HighPassSP hatHP2      = new HighPassSP(noiseHPCutoff2, out.sampleRate());
239         LowPassSP hatLP1       = new LowPassSP(noiseLPCutoff1, out.sampleRate());
240         LowPassSP hatLP2       = new LowPassSP(noiseLPCutoff2, out.sampleRate());
241
242         noiseEnv = new ADSR(1.0f, 0.001f, noiseDecay, 0.0f, 0.02f);
243
244         // ホワイトノイズ -> フィルター -> エンベロープ -> ミキサー
245         hatNoise.patch(hatHP1).patch(hatHP2).patch(hatLP1).patch(hatLP2).patch(noiseEnv).
            patch(mixer);
246
247         // --- 3. 全体の結合とマスターエンベロープ ---
248         masterEnv = new ADSR(1.0f, 0.0001f, 0.0f, 1.0f, 0.02f);
249         mixer.patch(masterEnv);
250     }
251

```



```

252 void noteOn(float duration) {
253     masterEnv.noteOn();
254     for (int i = 0; i < toneEnvList.length; i++ ) toneEnvList[i].noteOn();
255     noiseEnv.noteOn();
256     masterEnv.patch(shortOut);
257 }
258
259 void noteOff() {
260     masterEnv.noteOff();
261     for (int i = 0; i < toneEnvList.length; i++) {if (toneEnvList[i] != null)
262         toneEnvList[i].noteOff();}
263     noiseEnv.noteOff();
264     masterEnv.unpatchAfterRelease(shortOut);
265 }

```

D.10 Horn.pde

コード 28 Horn.pde

```

1 // =====
2 // ホルン (Horn) のクラス
3 // =====
4 class HornInstrument implements Instrument {
5     byte numHarmonics;
6
7     ADSR masterEnv;
8     ADSR[] harmonicEnvs;
9
10    Line[] wobbleLines;
11    float[] wobbleFactors;
12
13    Line[] pitchEnvLines;
14    float[] targetFreqs;
15
16    Line[] ampLines;
17    float[] startAmps;
18    float[] endAmps;
19
20    ADSR breathEnv;
21
22
23
24
25    HornInstrument(float midiNote, float duration, float startVel, float endVel, float
26        masterVol) {
27        Summer mixer = new Summer();

```

```

28     float masterAttackTime = 0.1f;
29     masterAttackTime = min(masterAttackTime, duration - 0.1f);
30     if (masterAttackTime < 0.035f) masterAttackTime = 0.035f;
31
32     masterEnv = new ADSR(1.0f, masterAttackTime, 0.0f, 1.0f, 0.13f);
33     mixer.patch(masterEnv);
34
35
36     float profileVel = (startVel > 0) ? startVel : endVel;
37     if (profileVel <= 0) profileVel = 64.0f;
38
39     float startGainFactor = pow(startVel / profileVel, 0.9f);
40     float endGainFactor   = pow(endVel / profileVel, 0.9f);
41
42     float[][] startProfile = utils.GetDynamicProfile(midiNote, profileVel, "Horn",
        34, 1.0f);
43
44
45     float volFactor = masterVol * 0.017f;
46
47     numHarmonics = (byte)startProfile.length;
48
49     harmonicEnvs = new ADSR[numHarmonics];
50     ampLines     = new Line[numHarmonics];
51     wobbleLines  = new Line[numHarmonics];
52     wobbleFactors = new float[numHarmonics];
53     startAmps     = new float[numHarmonics];
54     endAmps       = new float[numHarmonics];
55     pitchEnvLines = new Line[numHarmonics];
56     targetFreqs   = new float[numHarmonics];
57
58
59     byte baseWaveNum = 0;
60     for (byte i = 1; i < numHarmonics; i++) {
61         if (startProfile[i-1][0] < startProfile[i][0]) baseWaveNum = i;
62     }
63
64     float globalBreathSpeed = random(2.5f, 3.5f);
65
66
67     // --- 1. 加算合成パート ---
68     for (int i = 0; i < numHarmonics; i++) {
69
70         float targetFreq = startProfile[i][0];
71
72         Oscil osc = new Oscil(targetFreq, 0.0f, Waves.SINE);
73         osc.setPhase(random(1.0f));

```

```

74
75     float baseAmp = startProfile[i][1] * volFactor;
76     startAmps[i] = baseAmp * startGainFactor;
77     endAmps[i]   = baseAmp * endGainFactor;
78
79     float vibDepthHz = targetFreq * 0.001f;
80     Oscil freqController = new Oscil(globalBreathSpeed, vibDepthHz, Waves.SINE);
81     targetFreqs[i] = targetFreq;
82
83     pitchEnvLines[i] = new Line();
84     pitchEnvLines[i].patch(freqController.offset).patch(osc.frequency);
85
86     wobbleFactors[i] = random(0.12f, 0.17f);
87     wobbleLines[i] = new Line();
88     Oscil ampLFO = new Oscil(globalBreathSpeed, 0.0f, Waves.SINE);
89     ampLFO.setPhase(random(1.0f));
90
91     wobbleLines[i].patch(ampLFO.amplitude).patch(osc.amplitude);
92
93
94     ampLines[i] = new Line();
95     ampLines[i].patch(osc.amplitude);
96
97     float attackTime = (i == baseWaveNum) ? 0.058f : 0.395f;
98     harmonicEnvs[i] = new ADSR(1.0f, attackTime/3, 1.0f, 0.7f, 10.0f);
99     osc.patch(harmonicEnvs[i]).patch(mixer);
100 }
101
102
103
104 // --- 2. 息のノイズパート ---
105 float startNorm = constrain(startVel, 0, 127) / 127.0f;
106 float attackNoiseVol = masterVol * 0.003f * pow(startNorm, 0.9f);
107 Noise breathNoise = new Noise(attackNoiseVol, Noise.Tint.WHITE);
108
109 MoogFilter breathFilter = new MoogFilter(381.4f, 0.15f);
110 breathFilter.type = MoogFilter.Type.BP;
111 breathEnv = new ADSR(1.0f, 0.01f, 0.08f, 0.0f, 0.0f);
112
113 breathNoise.patch(breathFilter).patch(breathEnv).patch(mixer);
114 }
115
116
117
118 void noteOn(float dur) {
119     masterEnv.noteOn();
120     float safeDur = dur + 0.5f;
121

```

```

122     float sweepTime = 0.12f;
123     float pitchMod  = 0.05f;
124
125     for(int i = 0; i < numHarmonics; i++) {
126         harmonicEnvs[i].noteOn();
127         ampLines[i].activate(safeDur, startAmps[i], endAmps[i]);
128
129         if (wobbleLines[i] != null) {
130             float factor = wobbleFactors[i];
131             wobbleLines[i].activate(safeDur, startAmps[i] * factor, endAmps[i] * factor);
132         }
133
134         if (pitchEnvLines[i] != null) {
135             float baseFreq = targetFreqs[i];
136             float startFreq = baseFreq * (1.0f + pitchMod);
137             pitchEnvLines[i].activate(sweepTime, startFreq, baseFreq);
138         }
139     }
140
141     breathEnv.noteOn();
142     masterEnv.patch(longOut);
143 }
144
145
146
147 void noteOff() {
148     masterEnv.noteOff();
149     for(int i=0; i<numHarmonics; i++) harmonicEnvs[i].noteOff();
150     masterEnv.unpatchAfterRelease(longOut);
151 }
152 }

```

D.11 Trumpet.pde

コード 29 Trumpet.pde

```

1  // =====
2  // トランペット (Trumpet) のクラス
3  // =====
4  class TrumpetInstrument implements Instrument {
5      byte numHarmonics;
6
7      ADSR masterEnv;
8      ADSR[] harmonicEnvs;
9
10     Line[] wobbleLines;
11     float[] wobbleFactors;
12

```

```

13 Line[] pitchEnvLines;
14 float[] targetFreqs;
15
16 Line[] ampLines;
17 float[] startAmps;
18 float[] endAmps;
19
20 ADSR breathEnv;
21
22
23
24
25 TrumpetInstrument(float midiNote, float duration, float startVel, float endVel,
    float masterVol) {
26     Summer mixer = new Summer();
27
28     float masterAttackTime = 0.1f - constrain(startVel, 0, 127) / 127.0f * 0.8f;
29     masterAttackTime = min(masterAttackTime, duration - 0.05f);
30     if (masterAttackTime < 0.035f) masterAttackTime = 0.035f;
31
32     masterEnv = new ADSR(1.0f, masterAttackTime, 0.0f, 1.0f, 0.17f);
33     mixer.patch(masterEnv);
34
35
36     float profileVel = (startVel > 0) ? startVel : endVel;
37     if (profileVel <= 0) profileVel = 64.0f;
38
39     float startGainFactor = pow(startVel / profileVel, 0.9f);
40     float endGainFactor    = pow(endVel / profileVel, 0.9f);
41
42     float[][] startProfile = utils.GetDynamicProfile(midiNote, profileVel, "Trumpet",
        52, 1.0f);
43
44
45     float volFactor = masterVol * 0.018f;
46
47     numHarmonics = (byte)startProfile.length;
48
49     harmonicEnvs = new ADSR[numHarmonics];
50     ampLines     = new Line[numHarmonics];
51     wobbleLines  = new Line[numHarmonics];
52     wobbleFactors = new float[numHarmonics];
53     startAmps    = new float[numHarmonics];
54     endAmps      = new float[numHarmonics];
55     pitchEnvLines = new Line[numHarmonics];
56     targetFreqs  = new float[numHarmonics];
57

```

```

58
59 byte baseWaveNum = 0;
60 for (byte i = 1; i < numHarmonics; i++) {
61     if (startProfile[i-1][0] < startProfile[i][0]) baseWaveNum = i;
62 }
63
64 float globalBreathSpeed = random(4.0f, 5.0f);
65
66
67 // 1. 加算合成パート (7つのサイン波)
68 for (int i = 0; i < numHarmonics; i++) {
69
70     float targetFreq = startProfile[i][0];
71
72     Oscil osc = new Oscil(targetFreq, 0.0f, Waves.SINE);
73     osc.setPhase(random(1.0f));
74
75     float baseAmp = startProfile[i][1] * volFactor;
76     startAmps[i] = baseAmp * startGainFactor;
77     endAmps[i] = baseAmp * endGainFactor;
78
79     // アタック時にピッチが急降下する特有のエンベロープ準備
80     float vibDepthHz = targetFreq * 0.0007f;
81     Oscil freqController = new Oscil(globalBreathSpeed, vibDepthHz, Waves.SINE);
82     targetFreqs[i] = targetFreq;
83
84     pitchEnvLines[i] = new Line();
85     pitchEnvLines[i].patch(freqController.offset).patch(osc.frequency);
86
87     wobbleFactors[i] = random(0.10f, 0.20f);
88     wobbleLines[i] = new Line();
89     Oscil ampLFO = new Oscil(globalBreathSpeed, 0.0f, Waves.SINE);
90     ampLFO.setPhase(random(1.0f));
91
92     wobbleLines[i].patch(ampLFO.amplitude).patch(osc.amplitude);
93
94
95     ampLines[i] = new Line();
96     ampLines[i].patch(osc.amplitude);
97
98     float attackTime = (i == baseWaveNum) ? 0.058f : 0.395f;
99     harmonicEnvs[i] = new ADSR(1.0f, attackTime/3, 1.0f, 0.7f, 10.0f);
100     osc.patch(harmonicEnvs[i]).patch(mixer);
101 }
102
103
104 // --- 2. 息のノイズパート ---
105 float startNorm = constrain(startVel, 0, 127) / 127.0f;

```

```

106     float attackNoiseVol = masterVol * 0.01f * pow(startNorm, 0.9f);
107     Noise breathNoise = new Noise(attackNoiseVol, Noise.Tint.WHITE);
108
109     MoogFilter breathFilter = new MoogFilter(581.4f, 0.15f);
110     breathFilter.type = MoogFilter.Type.BP;
111     breathEnv = new ADSR(1.0f, 0.01f, 0.08f, 0.0f, 0.0f);
112
113     breathNoise.patch(breathFilter).patch(breathEnv).patch(mixer);
114 }
115
116
117 void noteOn(float dur) {
118     masterEnv.noteOn();
119     float safeDur = dur + 0.5f;
120
121     float sweepTime = 0.07f; // ピッチ変化にかかる時間
122     float pitchMod = 0.03f; // アタック時に高くなる割合
123
124     for(int i = 0; i < numHarmonics; i++) {
125         harmonicEnvs[i].noteOn();
126
127         ampLines[i].activate(safeDur, startAmps[i], endAmps[i]);
128
129         if (wobbleLines[i] != null) {
130             float factor = wobbleFactors[i];
131             wobbleLines[i].activate(safeDur, startAmps[i] * factor, endAmps[i] * factor);
132         }
133
134         // 金管楽器特有のピッチアタック（高いピッチから本来のピッチへ下げる）
135         if (pitchEnvLines[i] != null) {
136             float baseFreq = targetFreqs[i];
137
138             // スタート地点の周波数：本来の周波数に pitchMod(%) を上乗せした値
139             float startFreq = baseFreq * (1.0f + pitchMod);
140
141             // startFreq から baseFreq(本来の周波数) へ、一直線に下げる
142             pitchEnvLines[i].activate(sweepTime, startFreq, baseFreq);
143         }
144     }
145
146     breathEnv.noteOn();
147     masterEnv.patch(longOut);
148 }
149
150
151
152 void noteOff() {
153     masterEnv.noteOff();

```

```

154     for(int i=0; i<numHarmonics; i++) harmonicEnvs[i].noteOff();
155     masterEnv.unpatchAfterRelease(longOut);
156 }
157 }

```

E 楽器デバイスの同期プログラムのプログラムソース

本節では、楽器デバイスの同期方式を担う Arduino プログラムの全文を示す。掲載にあたり、次の2点の省略を行っている。第一に、埋め込みの楽譜データ（Note[] 配列）は、各小節ごとに同一形式のデータが多数繰り返されるのみであるため、冒頭2小節分のみを掲載し、以降は省略している。第二に、スレーブ機であるピアノ・ストリングス・フルートの3つのスケッチは、自機のIPアドレスと楽器ID（MY_INST_ID）の2定数および楽譜データを除いて完全に同一であるため、ピアノ（piano_0706.ino）のみ全文を掲載し、ストリングス・フルートについては相違箇所のみを示す。

E.1 drum_0706.ino（マスター機）

コード 30 drum_0706.ino（楽譜データは冒頭2小節のみ掲載）

```

1  #include <WiFiS3.h>
2  #include <WiFiUdp.h>
3  #include <Arduino.h>
4
5  const char SSID[] = "hackathon001-WPA2";
6  const char PASS[] = "hackathon001";
7
8  IPAddress localIP(192, 168, 11, 11); // ドラム（マスター）のIPアドレス
9  const uint16_t LOCAL_PORT = 5005; // 自身のポート番号
10
11  WiFiUDP Udp;
12  // UDP通信で受信したパケットのデータを一時的に格納する
13  uint8_t packetBuffer[32];
14
15  const int CDS_PIN = A0; // CdSセルを接続するアナログ入力ピン番号を定義
16  const int CDS_ON_THRESHOLD = 800; // 起動直後だけ使う仮しきい値
17  const int CDS_OFF_THRESHOLD = 780; // 起動直後だけ使う仮しきい値
18  const int CDS_MIN_DELTA = 25; // 暗い状態との差がこれ未満ならレーザー扱いしない
19  const uint8_t CDS_PEAK_HISTORY = 4; // 直近何回分のピークからしきい値を決めるか
20  const uint8_t CDS_ON_RATIO = 65; // baselineからピークまでの65%をONしきい値にする
21  const uint8_t CDS_OFF_RATIO = 35; // baselineからピークまでの35%をOFFしきい値にする
22  const int CDS_THRESHOLD_STEP_LIMIT = 25; // 1回の検出でしきい値が動きすぎない上限
23  const unsigned long CDS_STARTUP_CALIBRATION_MS = 1000; // 起動直後の誤検出防止時間
24  const unsigned long DEBOUNCE_MS = 20; // 直前の検知からこの時間未満の再検知を無視する
    デバウンス時間を定義
25  const int BPM_LIST[] = { 60, 90, 120, 150, 180 }; // 5段階の目標BPM設定値（60～180）
    を定義

```



```

26  const float ROTATION_PERIOD_LIST[] = { 330, 240, 190, 150, 100 }; // 各BPMに対応する
    観覧車の実測回転周期を定義
27  const int MIN_STABLE_COUNT = 3; // BPM確定（安定）とみなす連続一致回数（3回）を定義
28
29  struct NoteData {
30      uint16_t timing; // 発音タイミング（楽曲開始からの累積Tick数）
31      uint16_t duration; // 発音の長さ（Tick単位）
32      uint8_t pitch; // 音高を示すMIDIノート番号（0～127）
33      uint8_t startVelocity; // 発音開始時の強さ（0～127）
34      uint8_t endVelocity; // 発音終了時の強さ（0～127）
35      uint8_t type; // 発音の種類を指定するフラグ
36  };
37
38  // 楽譜データ
39  const NoteData Note[] = {
40
41      // 【1小節目】
42      {0, 1, 60, 80, 96, 0}, // [0] C4
43      {12, 1, 64, 96, 80, 0}, // [1] E4
44      {24, 1, 63, 80, 127, 0}, // [2] D#4
45      {24, 1, 60, 80, 96, 0}, // [3] C4
46      {36, 1, 64, 96, 80, 0}, // [4] E4
47      {48, 1, 60, 80, 96, 0}, // [5] C4
48      {60, 1, 64, 96, 80, 0}, // [6] E4
49      {72, 1, 63, 80, 127, 0}, // [7] D#4
50      {72, 1, 60, 80, 96, 0}, // [8] C4
51      {84, 1, 64, 96, 80, 0}, // [9] E4
52
53      // 【2小節目】
54      {96, 1, 60, 80, 96, 0}, // [10] C4
55      {108, 1, 64, 96, 80, 0}, // [11] E4
56      {120, 1, 63, 80, 127, 0}, // [12] D#4
57      {120, 1, 60, 80, 96, 0}, // [13] C4
58      {132, 1, 64, 96, 80, 0}, // [14] E4
59      {144, 1, 60, 80, 96, 0}, // [15] C4
60      {156, 1, 64, 96, 80, 0}, // [16] E4
61      {168, 1, 63, 80, 127, 0}, // [17] D#4
62      {168, 1, 60, 80, 96, 0}, // [18] C4
63      {180, 1, 64, 96, 80, 0}, // [19] E4
64      {186, 1, 64, 48, 80, 0}, // [20] E4
65
66
67      // …（以降、末尾まで同一形式の楽譜データが続くため省略）…
68
69  };
70
71  // NoteDataに含まれる音符の数
72  const uint16_t note_count = sizeof(Note) / sizeof(Note[0]);

```

```

73
74 // =====
75 // Tick・演奏管理変数
76 // =====
77 uint32_t globalTick    = 0; // 全体基準Tick
78 uint32_t curTick      = 0; // ドラム楽譜用Tick
79 uint32_t lastTick     = 0; // updateTick用タイムスタンプμ[s]
80 uint32_t tickIval     = 0; // 1Tickあたりμのs
81 uint16_t scoreIdx     = 0; // 次に発音する予定のデータのインデックス
82
83 uint8_t curBpm = 0; //現在のBPM
84 uint8_t curVol = 50; //現在の音量
85
86 // =====
87 // 楽譜シーケンス定義
88 //   startSequence[曲ID][イベント][0=演奏開始elapsedPlayTick, 1=楽器ID]
89 //   楽器ID: 0=ドラム, 1=ピアノ, 2=ストリングス, 3=フルート   終端={-1, -1}
90 //
91 //   ★ 0通信送信タイミング = 演奏開始elapsedPlayTick - 0_ADVANCE_TICKS
92 //   ★ state2移行後の最初のイベントがelapsedPlayTick=0 (ドラム以外) の場合、
93 //     そのイベントの0通信は elapsedPlayTick=0 では間に合わないため、
94 //     実際の演奏開始は 0_ADVANCE_TICKS 分だけ後ろにずれる
95 // =====
96 const int32_t 0_ADVANCE_TICKS = 96; // 0通信を演奏開始の何Tick前に送るか
97 const int      MAX_SONGS      = 5; // 最大の曲数
98 const int      MAX_EVENTS     = 10; // 最大の発音開始指示の上限
99 const int      drumInstId     = 0; // ドラムのID
100
101 // ★実際の曲構成に合わせて書き換える★
102 // elapsedPlayTick=0 からドラム以外の楽器が始まる場合、
103 // その楽器のtriggerTickを 0 ではなく 0_ADVANCE_TICKS(=96) 以上に設定するか、
104 // もしくはドラム自身のtriggerTickを0にして、他楽器を96以降にする
105 int startSequence[MAX_SONGS][MAX_EVENTS][2] = {
106     { // 曲ID: 0 かえるのうた
107         {0,      drumInstId}, // elapsed=0でドラム自身が演奏開始
108         {192,   1},           // elapsed=192でピアノ開始 (0通信はelapsed=96)
109         {384,   2},           // elapsed=384でストリングス (0通信はelapsed=288)
110         {576,   3},           // elapsed=576でフルート (0通信はelapsed=480)
111         {-1, -1}
112     },
113     { // 曲ID: 1 酒場でブギウギ
114         //{0,      drumInstId}, // elapsed=0でドラム自身が演奏開始
115         {0,    1},           // elapsed=192でピアノ開始 (0通信はelapsed=96)
116         //{0,    2},           // elapsed=384でストリングス (0通信はelapsed=288)
117         //{0,    3},           // elapsed=576でフルート (0通信はelapsed=480)
118         {-1, -1}
119     },
120     { // 曲ID: 2 王宮のメヌエット

```

```

121 // {0, drumInstId}, // elapsed=0でドラム自身が演奏開始
122 // {0, 1}, // elapsed=192でピアノ開始 (0通信はelapsed=96)
123 {0, 2}, // elapsed=384でストリングス (0通信はelapsed=288)
124 // {0, 3}, // elapsed=576でフルート (0通信はelapsed=480)
125 {-1, -1}
126 },
127 { // 曲ID: 3 トルコ行進曲
128 // {0, drumInstId}, // elapsed=0でドラム自身が演奏開始
129 {0, 1}, // elapsed=192でピアノ開始 (0通信はelapsed=96)
130 // {0, 2}, // elapsed=384でストリングス (0通信はelapsed=288)
131 // {0, 3}, // elapsed=576でフルート (0通信はelapsed=480)
132 {-1, -1}
133 },
134 { // 曲ID: 4 序曲
135 {0, drumInstId}, // elapsed=0でドラム自身が演奏開始
136 {0, 1}, // elapsed=192でピアノ開始 (0通信はelapsed=96)
137 {0, 2}, // elapsed=384でストリングス (0通信はelapsed=288)
138 {0, 3}, // elapsed=576でフルート (0通信はelapsed=480)
139 {-1, -1}
140 }
141 };
142
143 int currentSongId = 0; // 現在の曲のID
144 int32_t elapsedPlayTick = 0; // state3移行後 (演奏開始後) の経過Tick
145
146 bool isDrumPlaying = false; // ドラムが演奏中でないかどうか
147
148 const uint8_t MAX_INSTRUMENTS = 4; // 最大楽器数
149 uint8_t oSentCount[MAX_EVENTS] = {0}; // 各イベントの0通信の送信回数を保存
150 bool isInstConnected[MAX_INSTRUMENTS] = {false}; // 同期に参加してるかどうかv
151 bool isInstJoin[MAX_INSTRUMENTS] = {false}; // 演奏同期に参加する予告が来た
152 // か
153 int32_t lastReceivedDiff[MAX_INSTRUMENTS] = {0}; // 他の楽器からT通信で送られるdiffを
154 // 格納
155
156 uint8_t receivedBpmLevels[MAX_INSTRUMENTS] = {0}; // 参加している楽器のBPM段階を集約
157 // する配列 (ドラム自身も含む)
158 uint8_t currentBpmLevel = 1; // 現在確定しているBPM段階 (前回値)
159
160 // SyncManager (レーザー検知・BPM安定判定)
161 class SyncManager {
162 public:
163 SyncManager()
164 : lastDetectTime(0), correctedBPM(0),
165 stableCount(0), prevCorrectedBPM(0), laserOn(false),
166 baseline(0), baselineReady(false),
167 onThreshold(CDS_ON_THRESHOLD), offThreshold(CDS_OFF_THRESHOLD),

```

```

165     pulsePeak(0), peakIndex(0), peakCount(0),
166     calibrated(false), startupCalibrated(false), waitingForRelease(false),
167     calibrationStartMs(0), startupMin(1023), releaseThreshold(0) {
168     clearPeakHistory();
169 }
170
171 // 全ての内部状態を初期値に戻す
172 void reset() {
173     lastDetectTime = 0;
174     correctedBPM = 0;
175     stableCount = 0;
176     prevCorrectedBPM = 0;
177     laserOn = false;
178     baseline = 0;
179     baselineReady = false;
180     onThreshold = CDS_ON_THRESHOLD;
181     offThreshold = CDS_OFF_THRESHOLD;
182     pulsePeak = 0;
183     peakIndex = 0;
184     peakCount = 0;
185     calibrated = false;
186     startupCalibrated = false;
187     waitingForRelease = false;
188     calibrationStartMs = 0;
189     startupMin = 1023;
190     releaseThreshold = 0;
191     clearPeakHistory();
192 }
193
194 // Cdsの読み取り値と現在時刻から、しきい値較正・エッジ検出・デバウンスを行い、レー
    ザー検知時にtrueを返す.
195 bool update(int cdsValue, unsigned long nowMs) {
196     if (calibrationStartMs == 0) calibrationStartMs = nowMs;
197     if (!detectLight(cdsValue, nowMs)) return false;
198     if (nowMs - lastDetectTime < DEBOUNCE_MS) return false;
199     if (lastDetectTime != 0) {
200         unsigned long interval = nowMs - lastDetectTime;
201         int newBPM = intervalToBpm((float)interval);
202         if (newBPM == prevCorrectedBPM) stableCount++;
203         else stableCount = 1;
204         prevCorrectedBPM = newBPM;
205         correctedBPM = newBPM;
206     }
207     lastDetectTime = nowMs;
208     return true;
209 }
210
211 // 補正後の値を取得する窓口関数.

```

```

212 int getCorrectedBPM() const { return correctedBPM; }
213 int getStableCount() const { return stableCount; }
214
215 private:
216 unsigned long lastDetectTime; // 直前にレーザー検知が確定した時刻を格納
217 int correctedBPM; // 今回レーザーが通過したときの間隔 (BPM)
218 int stableCount; // 同じBPMが何回連続で出たか
219 int prevCorrectedBPM; // 前回レーザーが通過したときの間隔(BPM)
220 bool laserOn; // 現在レーザーがCdSセルに当たっているかを格納
221 int baseline; // 暗状態 (レーザー非通過時) の読み取り値の追従値を格納
222 bool baselineReady; // baselineの初期値取得が完了したかを格納
223 int onThreshold; // 現在有効なON判定しきい値を格納
224 int offThreshold; // 現在有効なOFF判定しきい値を格納
225 int pulsePeak; // 検知中のパルスにおける読み取り値の最大値を一時的に格納
226 int peakHistory[CDS_PEAK_HISTORY]; // しきい値較正に用いる直近ピーク値を保持する配
    列
227 uint8_t peakIndex; // peakHistory内の現在の書き込み位置を管理するインデックス
228 uint8_t peakCount; // peakHistoryに記録済みのピーク数を格納
229 bool calibrated; // しきい値の較正が一度でも行われたかを格納
230 bool startupCalibrated; // 起動直後のキャリブレーション期間が終了したかを格納
231 bool waitingForRelease; // 起動時に暗転待ち状態かを格納
232 unsigned long calibrationStartMs; // 起動キャリブレーションを開始した時刻を格納
233 int startupMin; // 起動キャリブレーション中に観測された最小値を格納
234 int releaseThreshold; // waitingForRelease状態を解除する判定値を格納
235
236 // レーザーのON/OFF判定, 起動時キャリブレーション, ベースライン更新を行う.
237 bool detectLight(int v, unsigned long nowMs) {
238     if (!baselineReady) {
239         baseline = v;
240         startupMin = v;
241         baselineReady = true;
242         updateFallbackThresholds();
243     }
244
245     // 起動直後にレーザーが当たっていても、最初の1秒は検出せず暗い側の値を探す
246     if (!startupCalibrated) {
247         if (v < startupMin) startupMin = v;
248         baseline = startupMin;
249         updateFallbackThresholds();
250         if (nowMs - calibrationStartMs < CDS_STARTUP_CALIBRATION_MS) return false;
251         startupCalibrated = true;
252         waitingForRelease = (v >= onThreshold);
253         releaseThreshold = (baseline > 1023 - CDS_MIN_DELTA) ? baseline - CDS_MIN_DELTA
            : offThreshold;
254         return false;
255     }
256
257     if (waitingForRelease) {

```

```

258     if (v <= releaseThreshold) {
259         baseline = v;
260         updateFallbackThresholds();
261         waitingForRelease = false;
262     }
263     return false;
264 }
265
266 if (!laserOn) {
267     if (v < baseline) baseline = v;
268     else baseline = (baseline * 31 + v) / 32;
269     if (!calibrated) updateFallbackThresholds();
270 }
271
272 if (!laserOn && v >= onThreshold) {
273     laserOn = true;
274     pulsePeak = v;
275     return true;
276 }
277
278 if (laserOn) {
279     if (v > pulsePeak) pulsePeak = v;
280     if (v <= offThreshold) {
281         laserOn = false;
282         registerPeak(pulsePeak);
283     }
284 }
285 return false;
286 }
287
288 // 検知したピーク値を記録し, ON/OFF判定しきい値を再校正する.
289 void registerPeak(int peak) {
290     if (peak - baseline < CDS_MIN_DELTA) return;
291
292     peakHistory[peakIndex] = peak;
293     peakIndex = (peakIndex + 1) % CDS_PEAK_HISTORY;
294     if (peakCount < CDS_PEAK_HISTORY) peakCount++;
295     if (peakCount < 3) return;
296
297     long sum = 0;
298     for (uint8_t i = 0; i < peakCount; i++) sum += peakHistory[i];
299     int peakAvg = sum / peakCount;
300     int amplitude = peakAvg - baseline;
301     if (amplitude < CDS_MIN_DELTA) return;
302
303     int targetOn = constrain(baseline + (amplitude * CDS_ON_RATIO) / 100, 0, 1023);
304     int targetOff = constrain(baseline + (amplitude * CDS_OFF_RATIO) / 100, 0,
        targetOn - 5);

```

```

305
306     if (!calibrated) {
307         onThreshold = targetOn;
308         offThreshold = targetOff;
309         calibrated = true;
310     } else {
311         onThreshold = limitStep(onThreshold, targetOn, CDS_THRESHOLD_STEP_LIMIT);
312         offThreshold = limitStep(offThreshold, targetOff, CDS_THRESHOLD_STEP_LIMIT);
313     }
314     if (offThreshold >= onThreshold) offThreshold = max(0, onThreshold - 5);
315 }
316
317 // 較正前に使用する仮のしきい値を更新する.
318 void updateFallbackThresholds() {
319     onThreshold = constrain(baseline + CDS_MIN_DELTA, 0, 1023);
320     offThreshold = constrain(baseline + CDS_MIN_DELTA / 2, 0, max(0, onThreshold - 5)
321 );
322 }
323
324 // しきい値が一度に変化する最大量を制限する.
325 int limitStep(int current, int target, int limit) {
326     if (target > current + limit) return current + limit;
327     if (target < current - limit) return current - limit;
328     return target;
329 }
330
331 // ピーク履歴を初期化する.
332 void clearPeakHistory() {
333     for (uint8_t i = 0; i < CDS_PEAK_HISTORY; i++) peakHistory[i] = 0;
334 }
335
336 // 実測の1周時間テーブルの中から最も近いものを探し、対応するBPM_LISTの値を返す
337 int intervalToBpm(float intervalMs) {
338     int best = BPM_LIST[0]; float bestD = fabs(intervalMs - ROTATION_PERIOD_LIST[0]);
339     for (int i = 1; i < 5; i++) {
340         float d = fabs(intervalMs - ROTATION_PERIOD_LIST[i]);
341         if (d < bestD) { bestD = d; best = BPM_LIST[i]; }
342     }
343     return best;
344 };
345
346 SyncManager syncManager;
347
348 // =====
349 // state:
350 // 0 = 待機 (BPM安定待ち)

```

```

351 // 1 = BPM安定、楽器と同期中 (diff収束待ち)
352 // 2 = 同期完了・演奏開始前 (0通信準備 / state3への96tick待機)
353 // 3 = 演奏中 (elapsedPlayTick が進む、ドラムも演奏する場合は isDrumPlaying=true)
354 // 4 = システム停止
355 // =====
356 int state = 4;
357
358 // 楽譜遷移 (ジャンプ) 定義
359 // jumpTable[曲ID][ジャンプ番号][0=トリガーtick, 1=ジャンプ先tick, 2=ジャンプ先
    scoreIdx]
360 const int MAX_JUMPS = 20;
361 const int32_t jumpTable[MAX_SONGS][MAX_JUMPS][3] = {
362     { // 曲ID: 0 かえるのうた
363         {768, 0, 0},
364         {-1, -1, -1}
365     },
366     {
367         {768, 0, 0},
368         {-1, -1, -1}
369     },
370     {
371         {768, 0, 0},
372         {-1, -1, -1}
373     },
374     {
375         {768, 0, 0},
376         {-1, -1, -1}
377     },
378     {
379         {3624, 2088, 637},
380         {-1, -1, -1}
381     }
382 };
383
384 uint8_t currentJumpIdx = 0; // 現在のジャンプインデックス
385
386 // 曲ごとの初期設定データ
387 struct SongStartData {
388     uint32_t startTick;
389     uint16_t startNoteIdx;
390 };
391
392 // Processingの設定に合わせた初期値の定義
393 const SongStartData songStartParams[MAX_SONGS] = {
394     {0, 0}, // 曲ID: 0 (かえるのうた)
395     {0, 0}, // 曲ID: 1 (酒場でブギウギ風)
396     {0, 0}, // 曲ID: 2 (王宮のメヌエット)
397     {0, 0}, // 曲ID: 3 (トルコ行進曲)

```



```

398     {768, 92} // 曲ID: 4 (序曲)
399 };
400
401
402
403
404
405 // 関数宣言
406 void updateTick();
407 void setBpm(uint8_t bpm);
408 void sendVolumeValue(uint8_t vol);
409 void sendPacket();
410 void processOCommands();
411 void sendOPacket(uint8_t instId, uint8_t songId, uint32_t startGlobalTick);
412 bool checkSyncReady();
413 void sendNPacket(uint8_t bpm);
414 void sendRPacket(uint8_t instId, int32_t diff);
415
416 // IPアドレス (楽器IDからIPを引く)
417 IPAddress instIP(uint8_t instId) {
418     // ID=1→192.168.11.12, ID=2→.13, ID=3→.14
419     return IPAddress(192, 168, 11, 11 + instId);
420 }
421
422 // セットアップ
423 void setup() {
424     Serial.begin(115200);
425     WiFi.config(localIP);
426     WiFi.begin(SSID, PASS);
427     Serial.print("Connecting_WiFi...");
428     while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
429     Serial.println("\nWiFi_Connected!_IP:_ " + WiFi.localIP().toString());
430     Udp.begin(LOCAL_PORT);
431     Serial.println("state=0_BPM安定待ち");
432 }
433
434 // メインループ
435 void loop() {
436     unsigned long now = millis();
437     int cdsValue = analogRead(CDS_PIN);
438
439     // ---- UDP受信 ----
440     int pktSize = Udp.parsePacket();
441     while (pktSize > 0) {
442         if (pktSize >= 2) {
443             Udp.read(packetBuffer, sizeof(packetBuffer));
444             char header = (char)packetBuffer[0];
445

```

```

446 switch (header) {
447     //楽器からの同期パケット
448     case 'T':
449         if (pktSize >= 6) {
450             uint32_t slaveTick =
451                 ((uint32_t)packetBuffer[1] << 24) |
452                 ((uint32_t)packetBuffer[2] << 16) |
453                 ((uint32_t)packetBuffer[3] << 8) |
454                 (uint32_t)packetBuffer[4];
455             uint8_t id = packetBuffer[5];
456
457             if (id < MAX_INSTRUMENTS && id != drumInstId) {
458                 isInstConnected[id] = true;
459                 if (isInstJoin[id] != true) isInstJoin[id] = true;
460                 int32_t diff = (int32_t)(globalTick - slaveTick);
461                 lastReceivedDiff[id] = diff;
462
463                 sendRPacket(id, diff);
464
465                 Serial.print(F("[T]_id=")); Serial.print(id);
466                 Serial.print(F("_diff=")); Serial.println(diff);
467             }
468         }
469         break;
470
471     case 'V': {
472         uint8_t step = packetBuffer[1];
473         uint8_t vol = (step >= 1 && step <= 5) ? (step * 10) : 30;
474         sendVolumeValue(vol);
475         break;
476     }
477     //指揮機からのブロードキャスト終了指示. 演奏を停止し、state4にして初期化する
478     case 'E':
479         isDrumPlaying = false;
480         curTick = 0;
481         scoreIdx = 0;
482         globalTick = 0;
483         elapsedPlayTick = 0;
484         state = 4;
485         memset(oSentCount, 0, sizeof(oSentCount));
486         memset(isInstConnected, 0, sizeof(isInstConnected));
487         memset(isInstJoin, 0, sizeof(isInstJoin));
488         curBpm = 0;
489         tickIval = 0;
490
491         // BPM関連の初期化
492         memset(receivedBpmLevels, 0, sizeof(receivedBpmLevels));
493         currentBpmLevel = 0;

```

```

494
495     Serial.println(F("[E]_演奏終了。state=4_に初期化"));
496     break;
497
498     // 指揮機からのブロードキャスト開始指示。state0に戻して初期化する
499     case 'S':
500     {
501         static unsigned long lastSReceivedTime = 0;
502         unsigned long nowMs = millis();
503
504         // 1秒以内の連続したS通信（UDP連送）は無視する
505         if (nowMs - lastSReceivedTime < 1000) {
506             Serial.println(F("[S]_連続受信のためスキップ"));
507             break;
508         }
509         lastSReceivedTime = nowMs;
510
511         // 楽譜IDのローテーション
512         currentSongId = (currentSongId + 1) % MAX_SONGS;
513
514         // 初期化处理
515         isDrumPlaying      = false;
516         curTick            = 0;
517         scoreIdx           = 0;
518         globalTick         = 0;
519         elapsedPlayTick    = 0;
520         currentJumpIdx     = 0; // ジャンプインデックスも初期化
521         state              = 0;
522         memset(oSentCount, 0, sizeof(oSentCount));
523         memset(isInstConnected, 0, sizeof(isInstConnected));
524         memset(isInstJoin, 0, sizeof(isInstJoin));
525         curBpm = 0;
526         tickIval = 0;
527
528         memset(receivedBpmLevels, 0, sizeof(receivedBpmLevels));
529         currentBpmLevel = 0;
530
531         syncManager.reset();
532
533         Serial.print(F("[S]_演奏開始。state=0_に初期化。曲変更。SongID:"));
534         Serial.println(currentSongId);
535         break;
536     }
537
538     // 'J': 楽器からの演奏参加予告。同期に参加する楽器を読み取る
539     case 'J':
540     if(pktSize >= 2){
541         uint8_t instId = packetBuffer[1];

```

```

542     if (instId < MAX_INSTRUMENTS) {
543         isInstJoin[instId] = true;
544         Serial.print(F("[J]_参加予告受信:_instId=")); Serial.println(instId);
545     }
546 }
547 break;
548
549 // 子機からのBPM提案を受信
550 case 'M':
551     if (pktSize >= 3) {
552         uint8_t bpmVal = packetBuffer[1];
553         uint8_t instId = packetBuffer[2];
554         if (instId < MAX_INSTRUMENTS) {
555             receivedBpmLevels[instId] = bpmVal;
556             Serial.print(F("[M]_BPM提案受信_instId=")); Serial.print(instId);
557             Serial.print(F("_BPM=")); Serial.println(bpmVal);
558         }
559     }
560
561     break;
562
563
564     default: break;
565 }
566 } else {
567     Udp.read(); // 空読みして廃棄
568 }
569 pktSize = Udp.parsePacket(); // 次のパケットがあるか確認
570 }
571
572 if (state == 4) return;
573
574 // ---- レーザー検知 ----
575 bool laser = syncManager.update(cdsValue, now);
576
577 // ---- ステートマシン ----
578 switch (state) {
579
580     // --- state0: BPM安定待ち ---
581     case 0:
582         if (laser) {
583             int s = syncManager.getStableCount();
584             uint8_t curLaserBpm = syncManager.getCorrectedBPM();
585             Serial.print(F("[S0]_BPM=")); Serial.print(curLaserBpm);
586             Serial.print(F("_stable=")); Serial.println(s);
587
588             // ドラム自身のレーザーBPMを配列に上書き
589             receivedBpmLevels[drumInstId] = curLaserBpm;

```

```

590
591     if (s >= MIN_STABLE_COUNT) {
592         state = 1;
593         lastTick = micros();
594
595         // 初回確定時に自身へ適用 & 即座にN通信で子機に送信
596         currentBpmLevel = curLaserBpm;
597         setBpm(currentBpmLevel);
598         sendNPacket(currentBpmLevel);
599
600         Serial.println(F("[S0→1]_BPM安定"));
601     }
602 }
603 break;
604
605 // --- state1: 楽器と同期待ち ---
606 case 1:
607     updateTick();
608     if (laser) {
609         receivedBpmLevels[drumInstId] = syncManager.getCorrectedBPM(); // 配列上書き
610                                     // のみ
611         if (checkSyncReady()) {
612             // state2へ移行。elapsedPlayTick は state3移行後に開始するが、0通信の送信タ
613             // イミング算出のためにカウントはここから始める
614             state = 2;
615             elapsedPlayTick = -O_ADVANCE_TICKS;
616             memset(oSentCount, 0, sizeof(oSentCount));
617             Serial.println(F("[S1→2]_同期完了"));
618         } else {
619             Serial.println(F("[S1]_同期待ち..."));
620         }
621     }
622     break;
623
624 // --- state2: 同期完了・演奏開始待ち ---
625 // elapsedPlayTick を進めながら 0通信タイミングを監視する。state3への移行は
626 // processOCommands() 内でドラム自身の開始タイミングが来たとき。
627 case 2:
628     updateTick();
629     if (laser) receivedBpmLevels[drumInstId] = syncManager.getCorrectedBPM();
630     processOCommands();
631     break;
632
633 // --- state3: 演奏中 ---
634 case 3:
635     updateTick();
636     if (laser) receivedBpmLevels[drumInstId] = syncManager.getCorrectedBPM();
637     processOCommands();

```

```

635         break;
636
637         // --- state4: システム停止中 ---
638         // case 4: break;
639     }
640 }
641
642 // 同期完了チェック
643 // 接続済み楽器が1台以上 かつ 全員のdiffが SYNC_READY_THRESHOLD以内
644 bool checkSyncReady() {
645     bool expectedToJoin = false;
646
647     for (uint8_t i = 1; i < MAX_INSTRUMENTS; i++) {
648         if (isInstJoin[i]) {
649             expectedToJoin = true; // 参加予定の楽器が少なくとも1台いる
650
651             float SYNC_READY_THRESHOLD = 60 * 1000 / tickIval; // 60ms超えてたら同期ができて
                いないとみなすから50
652
653             // 参加予定だが未接続、または同期のズレが大きい場合は「まだ準備未完了」
654             if (!isInstConnected[i] || abs(lastReceivedDiff[i]) > SYNC_READY_THRESHOLD) {
655                 return false;
656             }
657         }
658     }
659
660     // 参加予定の全楽器が条件をクリアしていれば true。
661     // ※もし「他楽器が0台（ドラムソロ）でも開始させたい」場合は、return true;
662     return expectedToJoin;
663 }
664
665 // =====
666 // 0通信管理
667 // 各イベントについて「演奏開始elapsedTick - 0_ADVANCE_TICKS」になったら送信。
668 // ドラム自身のイベント（ID=0）は演奏開始処理のみ行う（0通信不要）。
669 //
670 // ・elapsedPlayTick が負（=state2の最初96tick）の間も処理するため、
671 //   startSequenceの最初のイベントが elapsed=0 であっても対応できる。
672 // ・受信側(notDrum)が演奏開始予定globalTick をもとにcurTickを計算するため、
673 //   「ドラムの globalTick + 0_ADVANCE_TICKS」を startGlobalTick として送る。
674 // =====
675 void processOCommands() {
676     if (currentSongId < 0 || currentSongId >= MAX_SONGS) return;
677
678     for (int i = 0; i < MAX_EVENTS; i++) {
679         int32_t evTick = startSequence[currentSongId][i][0];
680         int     instId = startSequence[currentSongId][i][1];

```

```

681
682     if (evTick == -1) break;
683     if (oSentCount[i] >= 3) continue; // 3回送信し終わっていたらスキップ
684
685     int32_t ticksUntilStart = evTick - elapsedPlayTick;
686
687     if (instId == drumInstId) {
688         if (elapsedPlayTick >= evTick) {
689             if (!isDrumPlaying) {
690                 isDrumPlaying = true;
691                 curTick = songStartParams[currentSongId].startTick;
692                 scoreIdx = songStartParams[currentSongId].startNoteIdx;
693                 state = 3;
694                 Serial.println(F("[DRUM]_演奏開始"));
695             }
696             oSentCount[i] = 3; // ドラム自身は通信不要なので完了扱い
697         }
698     } else {
699         if (isInstJoin[instId] && isInstConnected[instId]) {
700             uint32_t startGlobalTick = (uint32_t)((int32_t)globalTick + ticksUntilStart);
701
702             // 0_ADVANCE_TICKS(96)を切っていて、まだ0回目なら 1回目の送信
703             if (ticksUntilStart <= 0_ADVANCE_TICKS && oSentCount[i] == 0) {
704                 sendOPacket_Single(instId, currentSongId, startGlobalTick);
705                 oSentCount[i] = 1;
706             }
707             // 2/3 (64Tick前) を切っていて、まだ1回目なら 2回目の送信
708             else if (ticksUntilStart <= 0_ADVANCE_TICKS * 2 / 3 && oSentCount[i] == 1) {
709                 sendOPacket_Single(instId, currentSongId, startGlobalTick);
710                 oSentCount[i] = 2;
711             }
712             // 1/3 (32Tick前) を切っていて、まだ2回目なら 3回目の送信
713             else if (ticksUntilStart <= 0_ADVANCE_TICKS / 3 && oSentCount[i] == 2) {
714                 sendOPacket_Single(instId, currentSongId, startGlobalTick);
715                 oSentCount[i] = 3;
716             }
717         }
718     }
719 }
720 if (state == 2 && !isDrumPlaying) {
721     int32_t earliestEvent = INT32_MAX;
722     for (int i = 0; i < MAX_EVENTS; i++) {
723         int32_t evTick = startSequence[currentSongId][i][0];
724         int instId = startSequence[currentSongId][i][1];
725         if (evTick == -1) break;
726
727         // 修正点：ドラム自身 (drumInstId) も開始時間の計算に含める

```

```

728     if ((instId == drumInstId || isInstJoin[instId]) && evTick < earliestEvent) {
729         earliestEvent = evTick;
730     }
731 }
732
733 if (earliestEvent == INT32_MAX || earliestEvent > 0_ADVANCE_TICKS * 2) {
734     if (elapsedPlayTick < 0) {
735         elapsedPlayTick = 0;
736         Serial.println(F("[SKIP]_近接楽器なし:_elapsedPlayTick=0_に即進め"));
737     }
738     else {
739         int32_t targetElapsed = earliestEvent - 0_ADVANCE_TICKS;
740         if (elapsedPlayTick < targetElapsed) {
741             elapsedPlayTick = targetElapsed;
742             Serial.print(F("[SKIP]_elapsedPlayTick_を_")); Serial.print(targetElapsed);
743             Serial.println(F("_に調整"));
744         }
745     }
746 }
747 }
748
749 // 0通信を「1回だけ(delayなしで)」送る専用の関数
750 void sendOPacket_Single(uint8_t instId, uint8_t songId, uint32_t startGlobalTick) {
751     Udp.beginPacket(instIP(instId), LOCAL_PORT);
752     Udp.write('0');
753     Udp.write(songId);
754     Udp.write((uint8_t)((startGlobalTick >> 24) & 0xFF));
755     Udp.write((uint8_t)((startGlobalTick >> 16) & 0xFF));
756     Udp.write((uint8_t)((startGlobalTick >> 8) & 0xFF));
757     Udp.write((uint8_t)(startGlobalTick & 0xFF));
758     Udp.endPacket();
759 }
760
761 // N通信パケット送信 (確定BPMを全参加楽器へブロードキャスト)
762 void sendNPacket(uint8_t bpm) {
763     for (int i = 1; i < MAX_INSTRUMENTS; i++) {
764         Udp.beginPacket(instIP(i), LOCAL_PORT);
765         Udp.write('N');
766         Udp.write(bpm);
767         Udp.endPacket();
768         // }
769         delay(1); // 2msならok
770     }
771     // }
772 }
773
774 void sendRPacket(uint8_t instId, int32_t diff){
775     // R通信: diffのみ返信 (0通信は別途 sendOPacket で送る)

```



```

776   Udp.beginPacket(instIP(instId), LOCAL_PORT);
777   Udp.write('R');
778   Udp.write((uint8_t)((diff >> 24) & 0xFF));
779   Udp.write((uint8_t)((diff >> 16) & 0xFF));
780   Udp.write((uint8_t)((diff >> 8) & 0xFF));
781   Udp.write((uint8_t)(diff & 0xFF));
782   Udp.endPacket();
783 }
784
785 // =====
786 // Tick管理
787 // =====
788 void updateTick() {
789   if (tickIval == 0) return;
790   uint32_t now = micros();
791   while (now - lastTick >= tickIval) {
792     uint32_t currentIval = tickIval; // ループ突入時の値を保存
793
794     globalTick++;
795
796     // --- 48TickごとにBPM段階の多数決とN通信送信 ---
797     if (state >= 1 && globalTick % 48 == 0) {
798       uint8_t newLevel = determineSystemBpmLevel();
799       if (newLevel != currentBpmLevel && newLevel > 0) {
800         currentBpmLevel = newLevel;
801         setBpm(currentBpmLevel);
802       }
803       sendNPacket(currentBpmLevel);
804       // Serial.print(F("[SYNC] 48Tick周期 N通信送信: 確定BPM = "));
805       // Serial.println(currentBpmLevel);
806     }
807
808     if (state >= 2) elapsedPlayTick++;
809     if (state == 3 && isDrumPlaying) {
810       if (jumpTable[currentSongId][currentJumpIdx][0] != -1 && curTick == jumpTable[
         currentSongId][currentJumpIdx][0]) {
811         curTick = jumpTable[currentSongId][currentJumpIdx][1];
812         scoreIdx = jumpTable[currentSongId][currentJumpIdx][2];
813         currentJumpIdx++;
814
815         if (currentJumpIdx >= MAX_JUMPS || jumpTable[currentSongId][currentJumpIdx
           ][0] == -1) {
816           currentJumpIdx = 0;
817         }
818         Serial.println(F("[JUMP]_楽譜をジャンプしました!"));
819       }
820
821       sendPacket();

```

```

822     curTick++;
823 }
824
825 lastTick += currentIval; // 書き換えられていても元の値を足す
826 }
827 }
828
829 // =====
830 // BPM更新 → tickIval再計算
831 // =====
832 void setBpm(uint8_t bpm) {
833     if (bpm == curBpm) return;
834     curBpm = bpm;
835     tickIval = 60000000UL / (uint32_t)bpm / 24;
836     Serial.print('B'); Serial.print(','); Serial.print(bpm); Serial.print('\n');
837 }
838
839 void sendVolumeValue(uint8_t vol) {
840     if (vol == curVol) return;
841     curVol = vol;
842     Serial.print('V'); Serial.print(','); Serial.print(vol); Serial.print('\n');
843 }
844
845 // N通信で送信するためのBPM段階を決定する関数
846 uint8_t determineSystemBpmLevel() {
847     uint8_t validLevels[MAX_INSTRUMENTS];
848     uint8_t validCount = 0;
849
850     // 有効なデータ（参加中かつデータがある楽器）だけを抽出
851     for (int i = 0; i < MAX_INSTRUMENTS; i++) {
852         if ((i == drumInstId || isInstJoin[i]) && receivedBpmLevels[i] > 0) {
853             validLevels[validCount] = receivedBpmLevels[i];
854             validCount++;
855         }
856     }
857
858     if (validCount == 0) return currentBpmLevel; // データが一つもなければ前回維持
859     if (validCount == 1) return validLevels[0]; // 1台だけならそれを採用
860
861     // 2. 配列を小さい順にバブルソート
862     for (int i = 0; i < validCount - 1; i++) {
863         for (int j = 0; j < validCount - i - 1; j++) {
864             if (validLevels[j] > validLevels[j + 1]) {
865                 uint8_t temp = validLevels[j];
866                 validLevels[j] = validLevels[j + 1];
867                 validLevels[j + 1] = temp;
868             }
869         }
870     }

```

```

870 }
871
872 // 3. 中央値の取得と、偶数時の「ユーザー提案ロジック（丸め）」
873 if (validCount % 2 != 0) {
874     // 奇数個なら、ど真ん中がそのまま中央値
875     return validLevels[validCount / 2];
876 } else {
877     // 偶数個なら、中央を挟む2つの値を取得
878     uint8_t mid1 = validLevels[validCount / 2 - 1];
879     uint8_t mid2 = validLevels[validCount / 2];
880
881     // もし2つの値が同じなら、それが中央値
882     if (mid1 == mid2) return mid1;
883
884     // 異なる場合、前回値（currentBpmLevel）と差が小さい方を採用
885     int diff1 = abs((int)mid1 - (int)currentBpmLevel);
886     int diff2 = abs((int)mid2 - (int)currentBpmLevel);
887
888     if (diff1 <= diff2) {
889         return mid1;
890     } else {
891         return mid2;
892     }
893 }
894 }
895
896 // =====
897 // ノート送信（Processingへ）
898 // =====
899 void sendPacket() {
900     while (scoreIdx < note_count && Note[scoreIdx].timing <= curTick) {
901         //uint8_t vs = (uint8_t)((uint16_t)Note[scoreIdx].startVelocity * curVol / 50);
902         //uint8_t ve = (uint8_t)((uint16_t)Note[scoreIdx].endVelocity * curVol / 50);
903         Serial.print('N'); Serial.print(',');
904         Serial.print(Note[scoreIdx].pitch); Serial.print(',');
905         Serial.print(Note[scoreIdx].duration); Serial.print(',');
906         Serial.print(Note[scoreIdx].startVelocity); Serial.print(',');
907         Serial.print(Note[scoreIdx].endVelocity); Serial.print(',');
908         Serial.println(Note[scoreIdx].type);
909         scoreIdx++;
910     }
911 }

```

E.2 piano_0706.ino（スレーブ機）

コード 31 piano_0706.ino（楽譜データは冒頭2小節のみ掲載）

```

1 #include <Wi-FiS3.h>

```

```

2  #include <WiFiUdp.h>
3
4  const char SSID[] = "hackathon001-WPA2";
5  const char PASS[] = "hackathon001";
6
7  // ★楽器ごとに変更（ピアノ=12, スtringス=13, フルート=14）
8  IPAddress localIP(192, 168, 11, 12);
9  IPAddress drumIP (192, 168, 11, 11);
10 const uint16_t LOCAL_PORT = 5005;
11
12 // ★楽器ID (0=ドラム, 1=ピアノ, 2=Stringス, 3=フルート)
13 const uint8_t MY_INST_ID = 1;
14
15 WiFiUDP Udp;
16 // UDP通信で受信したパケットのデータを一時的に格納
17 uint8_t packetBuffer[32];
18
19 const int CDS_PIN          = A0; //CdSセルを接続するアナログ入力ピン番号を定義
20 const int CDS_ON_THRESHOLD = 820; // 起動直後だけ使う仮しきい値
21 const int CDS_OFF_THRESHOLD = 800; // 起動直後だけ使う仮しきい値
22 const int CDS_MIN_DELTA = 25;      // 暗い状態との差がこれ未満ならレーザー扱いしない
23 const uint8_t CDS_PEAK_HISTORY = 4; // 直近何回分のピークからしきい値を決めるか
24 const uint8_t CDS_ON_RATIO = 65;   // baselineからピークまでの65%をONしきい値にする
25 const uint8_t CDS_OFF_RATIO = 35;  // baselineからピークまでの35%をOFFしきい値にする
26 const int CDS_THRESHOLD_STEP_LIMIT = 25; // 1回の検出でしきい値が動きすぎない上限
27 const unsigned long CDS_STARTUP_CALIBRATION_MS = 1000; // 起動直後の誤検出防止時間
28 const unsigned long DEBOUNCE_MS = 20; // 直前の検知からこの時間未満の再検知を無視する
    デバウンス時間を定義
29 const int BPM_LIST[] = { 60, 90, 120, 150, 180 };
30 // 段階1~5(BPM_LISTと対応)の実測の1周あたり時間[ms]。理論値(30000/BPM)では観覧車の
31 // 実際の回転速度と合わないことが判明したため、実測値でBPMを判定する。
32 const float ROTATION_PERIOD_LIST[] = {380, 280, 210, 160, 100}; //pwmと大体対応させて
    いる
33 const int MIN_STABLE_COUNT = 3; //BPM確定（安定）とみなす連続一致回数を定義
34
35 // 楽譜データ
36 struct NoteData {
37     uint16_t timing; // 発音タイミング
38     uint16_t duration; // 発音の長さ (Tick単位)
39     uint8_t pitch; // 音高を示すMIDIノート番号 (0~127)
40     uint8_t startVelocity; // 発音開始時の強さ (0~127)
41     uint8_t endVelocity; // 発音開始時の強さ (0~127)
42     uint8_t type; // 発音開始時の強さ (0~127)
43 };
44
45 const NoteData Note[] = {
46
47     // 【1小節目】

```

```

48 {0, 24, 60, 96, 80, 0}, // [0] C4
49 {24, 24, 62, 96, 80, 0}, // [1] D4
50 {48, 24, 64, 96, 80, 0}, // [2] E4
51 {72, 24, 65, 96, 80, 0}, // [3] F4
52
53 // 【2小節目】
54 {96, 24, 64, 96, 80, 0}, // [4] E4
55 {120, 24, 62, 96, 80, 0}, // [5] D4
56 {144, 36, 60, 96, 80, 0}, // [6] C4
57
58
59 // …（以降、末尾まで同一形式の楽譜データが続くため省略）…
60
61 };
62
63 const uint16_t note_count = sizeof(Note) / sizeof(Note[0]); // 音符の数(配列に含まれて
    いる合計/音符1つあたりの要素数)
64 uint16_t scoreIdx = 0; // 次に発音する予定のデータのインデックス
65
66 // =====
67 // Tick管理変数
68 // =====
69 uint32_t globalTick = 0; // 全体基準のTick
70 uint32_t curTick = 0; // 楽譜用Tick
71 uint32_t lastTick = 0; // updateTick用タイムスタンプ
72
73 uint32_t baseTickIval = 0; // BPMから計算した基本interval μ[s/tick]
74 uint32_t tickIval = 0; // 実際に使うinterval（同期補正あり）
75 int32_t ticksToCorrect = 0; // 補正残りTick数
76 int32_t lastDiff = 0; // 直近のdiff（BPM変化時の再計算用）
77 uint32_t lastTPacketMicroTime = 0; // 直近のTパケット送信時の時刻Micro
78 uint32_t lastTPacketTick = 0; // 直近のTパケット送信時のglobalTick（drumNotStarted判
    定用）

79
80 bool drumNotStarted = true; // ドラム起動前同期フラグ
81
82 uint8_t curBpm = 0; // 現在のBPM
83 uint8_t curVol = 50; // 現在の音量
84
85 // 演奏管理
86 bool waitingForStart = false; // 0通信を受信して演奏開始globalTickを待っている
87 uint32_t startGlobalTick = 0; // ドラムから指定された演奏開始globalTick
88 uint8_t currentSongId = 0; // 現在の曲のID
89
90 // SyncManager（レーザー検知・BPM安定判定）
91 class SyncManager {
92 public:

```

```

93 SyncManager()
94 : lastDetectTime(0), correctedBPM(0),
95   stableCount(0), prevCorrectedBPM(0), laserOn(false),
96   baseline(0), baselineReady(false),
97   onThreshold(CDS_ON_THRESHOLD), offThreshold(CDS_OFF_THRESHOLD),
98   pulsePeak(0), peakIndex(0), peakCount(0),
99   calibrated(false), startupCalibrated(false), waitingForRelease(false),
100   calibrationStartMs(0), startupMin(1023), releaseThreshold(0) {
101   clearPeakHistory();
102 }
103
104 // 全ての内部状態を初期値に戻す
105 void reset() {
106     lastDetectTime = 0;
107     correctedBPM = 0;
108     stableCount = 0;
109     prevCorrectedBPM = 0;
110     laserOn = false;
111     baseline = 0;
112     baselineReady = false;
113     onThreshold = CDS_ON_THRESHOLD;
114     offThreshold = CDS_OFF_THRESHOLD;
115     pulsePeak = 0;
116     peakIndex = 0;
117     peakCount = 0;
118     calibrated = false;
119     startupCalibrated = false;
120     waitingForRelease = false;
121     calibrationStartMs = 0;
122     startupMin = 1023;
123     releaseThreshold = 0;
124     clearPeakHistory();
125 }
126
127 // Cdsの読み取り値と現在時刻から、しきい値較正・エッジ検出・デバウンスを行い、レー
    ザー検知時にtrueを返す。
128 bool update(int cdsValue, unsigned long nowMs) {
129     if (calibrationStartMs == 0) calibrationStartMs = nowMs;
130     if (!detectLight(cdsValue, nowMs)) return false;
131     if (nowMs - lastDetectTime < DEBOUNCE_MS) return false;
132     if (lastDetectTime != 0) {
133         unsigned long interval = nowMs - lastDetectTime;
134         int newBPM = intervalToBpm((float)interval);
135         if (newBPM == prevCorrectedBPM) stableCount++;
136         else stableCount = 1;
137         prevCorrectedBPM = newBPM;
138         correctedBPM = newBPM;
139     }

```

```

140     lastDetectTime = nowMs;
141     return true;
142 }
143
144 // 補正後の値を取得する窓口関数.
145 int getCorrectedBPM() const { return correctedBPM; }
146 int getStableCount() const { return stableCount; }
147
148 private:
149     unsigned long lastDetectTime; // 直前にレーザー検知が確定した時刻を格納
150     int correctedBPM; // 今回レーザーが通過したときの間隔 (BPM)
151     int stableCount; // 同じBPMが何回連続で出たか
152     int prevCorrectedBPM; // 前回レーザーが通過したときの間隔(BPM)
153     bool laserOn; // 現在レーザーがCdSセルに当たっているかを格納
154     int baseline; // 暗状態 (レーザー非通過時) の読み取り値の追従値を格納
155     bool baselineReady; // baselineの初期値取得が完了したかを格納
156     int onThreshold; // 現在有効なON判定しきい値を格納
157     int offThreshold; // 現在有効なOFF判定しきい値を格納
158     int pulsePeak; // 検知中のパルスにおける読み取り値の最大値を一時的に格納
159     int peakHistory[CDS_PEAK_HISTORY]; // しきい値校正に用いる直近ピーク値を保持する配
    列
160     uint8_t peakIndex; // peakHistory内の現在の書き込み位置を管理するインデックス
161     uint8_t peakCount; // peakHistoryに記録済みのピーク数を格納
162     bool calibrated; // しきい値の校正が一度でも行われたかを格納
163     bool startupCalibrated; // 起動直後のキャリブレーション期間が終了したかを格納
164     bool waitingForRelease; // 起動時に暗転待ち状態かを格納
165     unsigned long calibrationStartMs; // 起動キャリブレーションを開始した時刻を格納
166     int startupMin; // 起動キャリブレーション中に観測された最小値を格納
167     int releaseThreshold; // waitingForRelease状態を解除する判定値を格納
168
169     // レーザーのON/OFF判定, 起動時キャリブレーション, ベースライン更新を行う.
170     bool detectLight(int v, unsigned long nowMs) {
171         if (!baselineReady) {
172             baseline = v;
173             startupMin = v;
174             baselineReady = true;
175             updateFallbackThresholds();
176         }
177
178         // 起動直後にレーザーが当たっていても、最初の1秒は検出せず暗い側の値を探す
179         if (!startupCalibrated) {
180             if (v < startupMin) startupMin = v;
181             baseline = startupMin;
182             updateFallbackThresholds();
183             if (nowMs - calibrationStartMs < CDS_STARTUP_CALIBRATION_MS) return false;
184             startupCalibrated = true;
185             waitingForRelease = (v >= onThreshold);

```

```

186         releaseThreshold = (baseline > 1023 - CDS_MIN_DELTA) ? baseline - CDS_MIN_DELTA
187             : offThreshold;
188         return false;
189     }
190     if (waitingForRelease) {
191         if (v <= releaseThreshold) {
192             baseline = v;
193             updateFallbackThresholds();
194             waitingForRelease = false;
195         }
196         return false;
197     }
198
199     if (!laserOn) {
200         if (v < baseline) baseline = v;
201         else baseline = (baseline * 31 + v) / 32;
202         if (!calibrated) updateFallbackThresholds();
203     }
204
205     if (!laserOn && v >= onThreshold) {
206         laserOn = true;
207         pulsePeak = v;
208         return true;
209     }
210
211     if (laserOn) {
212         if (v > pulsePeak) pulsePeak = v;
213         if (v <= offThreshold) {
214             laserOn = false;
215             registerPeak(pulsePeak);
216         }
217     }
218     return false;
219 }
220
221 // 検知したピーク値を記録し，ON/OFF判定しきい値を再校正する.
222 void registerPeak(int peak) {
223     if (peak - baseline < CDS_MIN_DELTA) return;
224
225     peakHistory[peakIndex] = peak;
226     peakIndex = (peakIndex + 1) % CDS_PEAK_HISTORY;
227     if (peakCount < CDS_PEAK_HISTORY) peakCount++;
228     if (peakCount < 3) return;
229
230     long sum = 0;
231     for (uint8_t i = 0; i < peakCount; i++) sum += peakHistory[i];
232     int peakAvg = sum / peakCount;

```



```

233     int amplitude = peakAvg - baseline;
234     if (amplitude < CDS_MIN_DELTA) return;
235
236     int targetOn = constrain(baseline + (amplitude * CDS_ON_RATIO) / 100, 0, 1023);
237     int targetOff = constrain(baseline + (amplitude * CDS_OFF_RATIO) / 100, 0,
        targetOn - 5);
238
239     if (!calibrated) {
240         onThreshold = targetOn;
241         offThreshold = targetOff;
242         calibrated = true;
243     } else {
244         onThreshold = limitStep(onThreshold, targetOn, CDS_THRESHOLD_STEP_LIMIT);
245         offThreshold = limitStep(offThreshold, targetOff, CDS_THRESHOLD_STEP_LIMIT);
246     }
247     if (offThreshold >= onThreshold) offThreshold = max(0, onThreshold - 5);
248 }
249
250 // 較正前に使用する仮のしきい値を更新する.
251 void updateFallbackThresholds() {
252     onThreshold = constrain(baseline + CDS_MIN_DELTA, 0, 1023);
253     offThreshold = constrain(baseline + CDS_MIN_DELTA / 2, 0, max(0, onThreshold - 5)
        );
254 }
255
256 // しきい値が一度に変化する最大量を制限する.
257 int limitStep(int current, int target, int limit) {
258     if (target > current + limit) return current + limit;
259     if (target < current - limit) return current - limit;
260     return target;
261 }
262
263 // ピーク履歴を初期化する.
264 void clearPeakHistory() {
265     for (uint8_t i = 0; i < CDS_PEAK_HISTORY; i++) peakHistory[i] = 0;
266 }
267
268 // 実測の1周時間テーブルの中から最も近いものを探し、対応するBPM_LISTの値を返す
269 int intervalToBpm(float intervalMs) {
270     int best = BPM_LIST[0]; float bestD = fabs(intervalMs - ROTATION_PERIOD_LIST[0]);
271     for (int i = 1; i < 5; i++) {
272         float d = fabs(intervalMs - ROTATION_PERIOD_LIST[i]);
273         if (d < bestD) { bestD = d; best = BPM_LIST[i]; }
274     }
275     return best;
276 }
277 };

```

```

278
279 SyncManager syncManager;
280
281 // =====
282 // state:
283 // 0 = 待機 (BPM安定待ち)
284 // 1 = BPM安定、ドラムと同期中 (Tパケット送信・diff収束待ち)
285 // 2 = 演奏中
286 // 4 = システム停止中
287 // =====
288 int state = 4;
289
290
291 // 楽譜遷移 (ジャンプ) 定義
292 // jumpTable[曲ID][ジャンプ番号][0=トリガーtick, 1=ジャンプ先tick, 2=ジャンプ先
    scoreIdx]
293 const int MAX_SONGS = 5;
294 const int MAX_JUMPS = 20;
295 const int32_t jumpTable[MAX_SONGS][MAX_JUMPS][3] = {
296     { // 曲ID: 0 かえるのうた
297         {1536, 0, 0},
298         {-1, -1, -1}
299     },
300     {
301         {2880, 1728, 48}, {2784, 2880, 281}, {4512, 1728, 48},
302         {-1, -1, -1}
303     },
304     {
305         {5760, 4608, 651}, {5736, 5832, 850}, {6432, 5856, 856}, {6216, 6456, 993},
306         {6816, 4608, 651}, {5760, 6816, 1046}, {8832, 4608, 651},
307         {-1, -1, -1}
308     },
309     {
310         {9240, 8856, 1399}, {10008, 9240, 1499}, {10392, 10008, 1669}, {10776, 10392,
311         1783}, {11544, 10776, 1895}, {11544, 10008, 1669}, {10392, 10008, 1669},
312         {10392, 8856, 1399}, {9240, 8856, 1399}, {10008, 11544, 2111}, {11928, 11544,
313         2111}, {11904, 11952, 2225}, {13632, 8856, 1399},
314         {-1, -1, -1}
315     },
316     {
317         {16488, 14952, 2973},
318         {-1, -1, -1}
319     }
320 };
321
322 uint8_t currentJumpIdx = 0; // 現在のジャンプインデックス
323
324 // 曲ごとの初期設定データ

```

```

321 struct SongStartData {
322     uint32_t startTick;
323     uint16_t startNoteIdx;
324 };
325
326 // Processingの設定に合わせた初期値の定義
327 // インデックスがそのまま currentSongId に対応
328 const SongStartData songStartParams[MAX_SONGS] = {
329     {0, 0}, // 曲ID: 0 (かえるのうた)
330     {1536, 29}, // 曲ID: 1 (酒場でブギウギ)
331     {4608, 651}, // 曲ID: 2 (王宮のメヌエット)
332     {8856, 1399}, // 曲ID: 3 (トルコ行進曲)
333     {13632, 2638} // 曲ID: 4 (序曲)
334 };
335
336 // =====
337 // 関数宣言
338 // =====
339 void updateTick();
340 void setBpm(uint8_t bpm);
341 void applyDiffCorrection(int32_t diff);
342 void sendTPacket();
343 void sendPacket();
344 void sendMPacket(uint8_t bpm);
345
346 // =====
347 // セットアップ
348 // =====
349 void setup() {
350     Serial.begin(115200);
351     WiFi.config(localIP);
352     WiFi.begin(SSID, PASS);
353     Serial.print("Connecting_WiFi...");
354     while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
355     Serial.println("\nWiFi_Connected!");
356     Udp.begin(LOCAL_PORT);
357     Serial.println("state=0_BPM安定待ち");
358 }
359
360 // =====
361 // メインループ
362 // =====
363 void loop() {
364     unsigned long now = millis();
365     int cdsValue = analogRead(CDS_PIN);
366
367     // ---- UDP受信 ----
368     int pktSize = Udp.parsePacket();

```

```

369 while (pktSize > 0) {
370   if (pktSize >= 2) {
371     Udp.read(packetBuffer, sizeof(packetBuffer));
372     char header = (char)packetBuffer[0];
373     uint8_t payload = packetBuffer[1];
374
375     switch (header) {
376       // 'R': ドラムからのdiff返信
377       case 'R':
378         // 停止中 (4) や安定待ち (0) に届いた過去のR通信は遅延ゴミなので無視
379         if (state == 0 || state == 4) break;
380
381         if (pktSize >= 5) {
382           int32_t rawDiff =
383             ((int32_t)(int8_t)packetBuffer[1] << 24) |
384             ((int32_t)packetBuffer[2] << 16) |
385             ((int32_t)packetBuffer[3] << 8) |
386             (int32_t)packetBuffer[4];
387
388           // 遅延 (レイテンシ) 補正の計算
389           uint32_t rtt = micros() - lastTPacketMicroTime;
390           // 【安全対策】 往復時間が長すぎる場合は古いパケットとみなして破棄
391           if (rtt > 1000 * 1000) {
392             Serial.println(F("[R]_パケット遅延過大のためスキップ"));
393             break;
394           }
395
396           uint32_t latencyMicros = rtt / 2; // 片道遅延
397           int32_t latencyTicks = 0;
398
399           if (baseTickIval > 0) {
400             // 四捨五入してTick数に変換: (値 + 割る数の半分) / 割る数
401             latencyTicks = (int32_t)((latencyMicros + (baseTickIval / 2)) /
402                                   baseTickIval);
403           }
404
405           // ドラム機の勘違い (遅延分) を差し引いて真のズレを計算
406           int32_t diff = rawDiff - latencyTicks;
407
408           Serial.print(F("[R]_rawDiff=")); Serial.print(rawDiff);
409           Serial.print(F("_latencyTicks=")); Serial.print(latencyTicks);
410           Serial.print(F("_trueDiff=")); Serial.println(diff);
411
412           // Serial.print(F("[R] diff=")); Serial.println(diff);
413
414           if (drumNotStarted) {
415             if ((int32_t)lastTPacketTick + diff == 0) {
416               // Tパケット送信時点でドラムはまだ0。自身もリセットして足並みを揃える

```

```

416         globalTick = 0;
417         lastTick    = micros(); // Tick計測の基点もリセット
418         Serial.println(F("[R]_ドラム未起動:_globalTick_リセット"));
419     } else {
420         // ドラムが動き出した初回：即時同期
421         drumNotStarted = false;
422         globalTick = (uint32_t)((int32_t)globalTick + diff);
423         lastDiff    = diff;
424         Serial.print(F("[R]_ドラム起動検知:_即時同期_globalTick="));
425         Serial.println(globalTick);
426         // 以降は通常補正も適用
427         if (state >= 1) {
428             // 「補正が完了している」または「緊急事態（ズレが24Tick以上）」の時だけ補正を更新
429             if (ticksToCorrect == 0 || abs(diff) >= 24) {
430                 applyDiffCorrection(diff);
431             } else {
432                 Serial.println(F("[R]_補正中のため新規R通信の適用をスキップ"));
433             }
434         }
435     }
436 } else {
437     // 通常の同期補正
438     lastDiff = diff;
439     if (state >= 1) {
440         // 「補正が完了している」または「緊急事態（ズレが24Tick以上）」の時だけ補正を更新
441         if (ticksToCorrect == 0 || abs(diff) >= 24) {
442             applyDiffCorrection(diff);
443         } else {
444             Serial.println(F("[R]_補正中のため新規R通信の適用をスキップ"));
445         }
446     }
447 }
448 }
449 break;
450
451 // '0': ドラムからの演奏開始指示
452 case '0':
453     if (pktSize >= 6) {
454         if(waitingForStart || state == 2) return;
455
456         uint8_t songId = packetBuffer[1];
457         uint32_t sgt =
458             ((uint32_t)packetBuffer[2] << 24) |
459             ((uint32_t)packetBuffer[3] << 16) |
460             ((uint32_t)packetBuffer[4] << 8) |
461             (uint32_t)packetBuffer[5];

```

```

462
463     currentSongId    = songId;
464     startGlobalTick = sgt;
465
466     Serial.print(F("[0]_songId=")); Serial.print(songId);
467     Serial.print(F("_startGlobalTick=")); Serial.println(sgt);
468
469     // 既に超過していたらスキップして即開始
470     if ((int32_t)(globalTick - sgt) >= 0) {
471         // 超過分だけ curTick を進める
472         uint32_t overrun = globalTick - sgt;
473         curTick    = songStartParams[currentSongId].startTick + overrun;
474         scoreIdx   = songStartParams[currentSongId].startNoteIdx;
475         // overrunより前のノートスキップ
476         while (scoreIdx < note_count && Note[scoreIdx].timing < curTick) scoreIdx
            ++;
477         state = 2;
478         Serial.print(F("[0]_超過済み。overrun=")); Serial.print(overrun);
479         Serial.println(F("_Tick_→_即開始"));
480     } else {
481         waitingForStart = true;
482         Serial.println(F("[0]_開始待機中..."));
483     }
484 }
485 break;
486
487 case 'V': {
488     uint8_t vol = (payload >= 1 && payload <= 5) ? (payload * 10) : 30;
489     if (vol != curVol) {
490         curVol = vol;
491         Serial.print('V'); Serial.print(','); Serial.print(vol); Serial.print('\n')
            ;
492     }
493     break;
494 }
495
496 // 'E': 演奏終了通知 (指揮機から)
497 case 'E':
498     drumNotStarted = true;
499     waitingForStart = false;
500     state          = 4; // BPMは確定済みのままstate1に戻す 観覧車も停止するため
        1ではなく0
501     curTick        = 0;
502     scoreIdx       = 0;
503     globalTick     = 0;
504     curBpm         = 0; // 追加: BPMリセット
505     tickIval       = 0; // 追加
506     baseTickIval   = 0; // 追加

```

```

507     Serial.println(F("[E]演奏終了。drumNotStarted_リセット"));
508     break;
509
510     // -----
511     // 'S': 演奏開始通知（指揮機から）。ドラムに対して演奏への参加を申請
512     case 'S':
513         drumNotStarted = true;
514         waitingForStart = false;
515         state = 0; // BPMは確定済みのままstate1に戻す 観覧車も停止するため
                    // 1ではなく0
516         curTick = 0;
517         scoreIdx = 0;
518         globalTick = 0;
519         currentJumpIdx = 0; // ジャンプインデックスも初期化
520         Serial.println(F("[S]演奏開始。drumNotStarted_リセット"));
521
522         syncManager.reset();
523
524         sendJPacket();
525
526         Serial.println(F("[S]ドラムに同期予告を送信。"));
527
528         break;
529
530         // マスターからの確定BPMを受信
531     case 'N':
532         if (pktSize >= 2) {
533             uint8_t confirmedBpm = packetBuffer[1];
534             setBpm(confirmedBpm);
535             Serial.print(F("[N]確定BPM受信:")); Serial.println(confirmedBpm);
536         }
537         break;
538
539
540     default: break;
541 }
542 } else {
543     Udp.read(); // 空読みして破棄
544 }
545 pktSize = Udp.parsePacket(); // 次のパケットがあるか確認
546 }
547
548 if (state == 4) return;
549
550 // ---- レーザー検知 ----
551 bool laser = syncManager.update(cdsValue, now);
552
553 // ---- ステートマシン ----

```

```

554 switch (state) {
555
556     // --- state0: BPM安定待ち ---
557     case 0:
558         if (laser) {
559             int s = syncManager.getStableCount();
560             uint8_t curLaserBpm = syncManager.getCorrectedBPM();
561             Serial.print(F("[S0]_BPM=")); Serial.print(curLaserBpm);
562             Serial.print(F("_stable=")); Serial.println(s);
563
564             sendMPacket(curLaserBpm); // M通信でドラムに提案
565
566             if (s >= MIN_STABLE_COUNT) {
567                 state = 1;
568                 lastTick = micros();
569                 // setBpm(syncManager.getCorrectedBPM());
570
571                 Serial.println(F("[S0→1]_BPM安定(N通信待ち)"));
572             }
573         }
574         break;
575
576     // --- state1: 同期中 ---
577     case 1:
578         updateTick();
579         if (laser) {
580             static unsigned long lastSentTime = 0;
581             if (now - lastSentTime > 200) { // 200ms以内の連続送信（チャタリング）をブ
                ロック
582             // setBpm(syncManager.getCorrectedBPM());
583             // レーザー通過のたびにTパケットを送信してdiff返信を要求
584             sendMPacket(syncManager.getCorrectedBPM()); // M通信で提案
585             lastTPacketTick = globalTick;
586             sendTPacket();
587             lastSentTime = now;
588         }
589     }
590
591     // waitingForStart かつ startGlobalTick に到達したら演奏開始
592     if (waitingForStart && (int32_t)(globalTick - startGlobalTick) >= 0) {
593         waitingForStart = false;
594         curTick = songStartParams[currentSongId].startTick;
595         scoreIdx = songStartParams[currentSongId].startNoteIdx;
596         state = 2;
597         Serial.println(F("[S1→2]_演奏開始！"));
598     }
599     break;
600

```



```

601 // --- state2: 演奏中 ---
602 case 2:
603     updateTick();
604     if (laser) {
605         static unsigned long lastSentTime = 0;
606         if (now - lastSentTime > 200) { //200ms以内の連続送信（チャタリング）をブロッ
            ク
607             // setBpm(syncManager.getCorrectedBPM());
608             // レーザー通過のたびにTパケットを送信してdiff返信を要求
609             sendMPacket(syncManager.getCorrectedBPM()); // M通信で提案
610             lastTPacketTick = globalTick;
611             sendTPacket();
612             lastSentTime = now;
613         }
614     }
615     break;
616
617 // --- state4: システム停止中 ---
618 // case 4: break;
619 }
620 }
621
622 // =====
623 // diff補正によるtickIval算出
624 // diff = drumGlobalTick - myGlobalTick
625 // diff > 0: 自分が遅れている → 速く進む (tickIvalを小さく)
626 // diff < 0: 自分が早すぎる → 遅く進む (tickIvalを大きく)
627 // =====
628 void applyDiffCorrection(int32_t diff) {
629     if (baseTickIval == 0) return;
630
631     float SYNC_READY_THRESHOLD = 60 * 1000 / baseTickIval;
632
633     if (abs(diff) <= SYNC_READY_THRESHOLD) {
634         // ズレ±が2 Tick以内なら何もしない
635         tickIval = baseTickIval;
636         ticksToCorrect = 0;
637     } else if (diff <= -24) {
638         // 大幅に早い → ×2 で -diff Tick間遅らせる
639         tickIval = baseTickIval * 2;
640         ticksToCorrect = -diff;
641         Serial.println(F("[SYNC]_大幅早い:_×2"));
642     } else if (diff >= 24) {
643         // 大幅に遅い → ×2/3 で ×diff3 Tick間速める
644         tickIval = baseTickIval * 2 / 3;
645         ticksToCorrect = diff * 3;
646         Serial.println(F("[SYNC]_大幅遅い:_×2/3"));
647     } else {

```

```

648 // 小さいズレ → 次の48Tickでdiffを吸収する比率補正
649 tickIval = (uint32_t)((48UL * baseTickIval) / (48 + diff));
650 ticksToCorrect = 48 + diff;
651 Serial.print(F("[SYNC]_微補正_diff=")); Serial.println(diff);
652 }
653
654 uint8_t sendProcessingBpm = (uint8_t)(60000000UL / (tickIval * 24));
655 Serial.print('B'); Serial.print(','); Serial.print(sendProcessingBpm); Serial.print
    ('\n');
656 }
657
658 // =====
659 // BPM更新
660 // =====
661 void setBpm(uint8_t bpm) {
662     if (bpm == curBpm) return;
663     curBpm = bpm;
664     uint32_t oldBaseTickIval = baseTickIval;
665     baseTickIval = 60000000UL / (uint32_t)bpm / 24;
666
667     if (oldBaseTickIval > 0 && ticksToCorrect > 0) {
668         // 現在の補正スピードを新しいBPMの比率に合わせてスケールさせる
669         tickIval = (tickIval * baseTickIval) / oldBaseTickIval;
670         Serial.println(F("[BPM変化]_補正スピードを新BPMにスケール"));
671     } else {
672         tickIval = baseTickIval;
673     }
674
675     uint8_t sendProcessingBpm = (uint8_t)(60000000UL / (tickIval * 24));
676     Serial.print('B'); Serial.print(','); Serial.print(sendProcessingBpm); Serial.print
        ('\n');
677 }
678
679 // Tパケット送信 (自身のglobalTickをドラムへ)
680 void sendTPacket() {
681     Udp.beginPacket(drumIP, LOCAL_PORT);
682     Udp.write('T');
683     Udp.write((uint8_t)((globalTick >> 24) & 0xFF));
684     Udp.write((uint8_t)((globalTick >> 16) & 0xFF));
685     Udp.write((uint8_t)((globalTick >> 8) & 0xFF));
686     Udp.write((uint8_t)(globalTick & 0xFF));
687     Udp.write(MY_INST_ID);
688     Udp.endPacket();
689     lastTPacketMicroTime = micros();
690 }
691
692 void sendJPacket() {
693     for(int i=0;i<3;i++){

```

```

694     Udp.beginPacket(drumIP, LOCAL_PORT);
695     Udp.write('J');
696     Udp.write(MY_INST_ID);
697     Udp.endPacket();
698     delay(3); // Wi-Fiのバッファ溢れを防ぐための必須の遅延
699 }
700 }
701
702 // M通信送信（自身のレーザーBPMをドラムへ提案）
703 void sendMPacket(uint8_t bpm) {
704     Udp.beginPacket(drumIP, LOCAL_PORT);
705     Udp.write('M');
706     Udp.write(bpm);
707     Udp.write(MY_INST_ID);
708     Udp.endPacket();
709 }
710
711 // =====
712 // Tick管理
713 // =====
714 void updateTick() {
715     if (tickIval == 0) return;
716     uint32_t now = micros();
717     while (now - lastTick >= tickIval) {
718         uint32_t currentIval = tickIval; // ループ突入時の値を保存
719
720         globalTick++;
721
722         // 補正カウントダウン
723         if (ticksToCorrect > 0) {
724             ticksToCorrect--;
725             if (ticksToCorrect == 0) {
726                 tickIval = baseTickIval;
727                 Serial.println(F("[SYNC]_補正完了"));
728             }
729         }
730
731         if (state == 2) {
732             // ジャンプ（ループ）処理の判定
733             if (jumpTable[currentSongId][currentJumpIdx][0] != -1 && curTick == jumpTable[
                currentSongId][currentJumpIdx][0]) {
734                 curTick = jumpTable[currentSongId][currentJumpIdx][1];
735                 scoreIdx = jumpTable[currentSongId][currentJumpIdx][2];
736                 currentJumpIdx++;
737
738                 if (currentJumpIdx >= MAX_JUMPS || jumpTable[currentSongId][currentJumpIdx
                    ][0] == -1) {
739                     currentJumpIdx = 0;

```

```

740     }
741 }
742
743     sendPacket();
744     curTick++;
745 }
746
747     lastTick += currentIval; // ★書き換えられていても元の値を足す
748 }
749 }
750
751 // =====
752 // ノート送信 (Processingへ)
753 // =====
754 void sendPacket() {
755     while (scoreIdx < note_count && Note[scoreIdx].timing <= curTick) {
756         //uint8_t vs = (uint8_t)((uint16_t)Note[scoreIdx].startVelocity * curVol / 50);
757         //uint8_t ve = (uint8_t)((uint16_t)Note[scoreIdx].endVelocity * curVol / 50);
758         Serial.print('N'); Serial.print(',');
759         Serial.print(Note[scoreIdx].pitch); Serial.print(',');
760         Serial.print(Note[scoreIdx].duration); Serial.print(',');
761         Serial.print(Note[scoreIdx].startVelocity); Serial.print(',');
762         Serial.print(Note[scoreIdx].endVelocity); Serial.print(',');
763         Serial.println(Note[scoreIdx].type);
764         scoreIdx++;
765     }
766 }

```

E.3 strings_0706.ino (スレーブ機・相違箇所のみ)

コード 32 strings_0706.ino (piano_0706.ino との相違箇所のみ)

```

1 // strings_0706.ino: 以下の2定数と楽譜データ (Note[]配列) を除き,
2 // piano_0706.ino と完全に同一である.
3
4 IPAddress localIP(192, 168, 11, 13);
5
6 // 楽器ID (0=ドラム, 1=ピアノ, 2=ストリングス, 3=フルート)
7 const uint8_t MY_INST_ID = 2; // ストリングス

```

E.4 flute_0706.ino (スレーブ機・相違箇所のみ)

コード 33 flute_0706.ino (piano_0706.ino との相違箇所のみ)

```

1 // flute_0706.ino: 以下の2定数と楽譜データ (Note[]配列) を除き,
2 // piano_0706.ino と完全に同一である.
3
4 IPAddress localIP(192, 168, 11, 14);
5

```

```
6 // 楽器ID (0=ドラム, 1=ピアノ, 2=ストリングス, 3=フルート)
7 const uint8_t MY_INST_ID = 3; // フルート
```